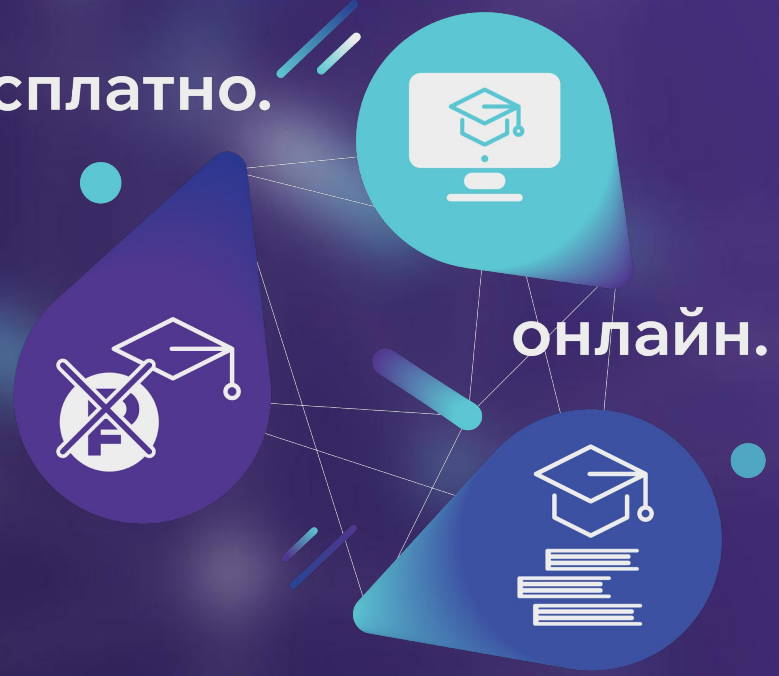
Deep Learning School



2026

DLS

Nina Konovalova

@AfeliaN

@ReadingAI

Diffusion models

From Stable Diffusion to FLUX

@READING_AI

DLS

PART 1

Modern architectures

From Stable Diffusion to FLUX

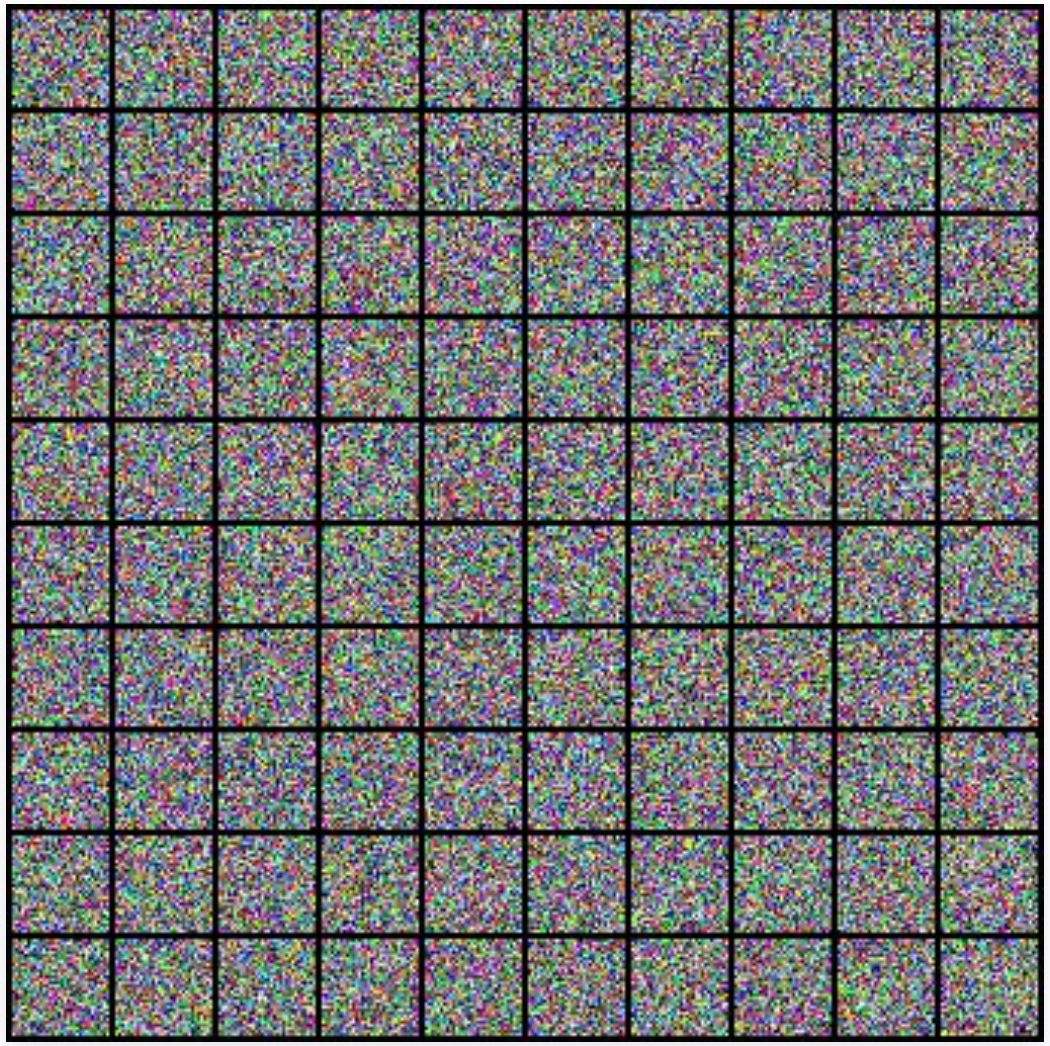
DLS

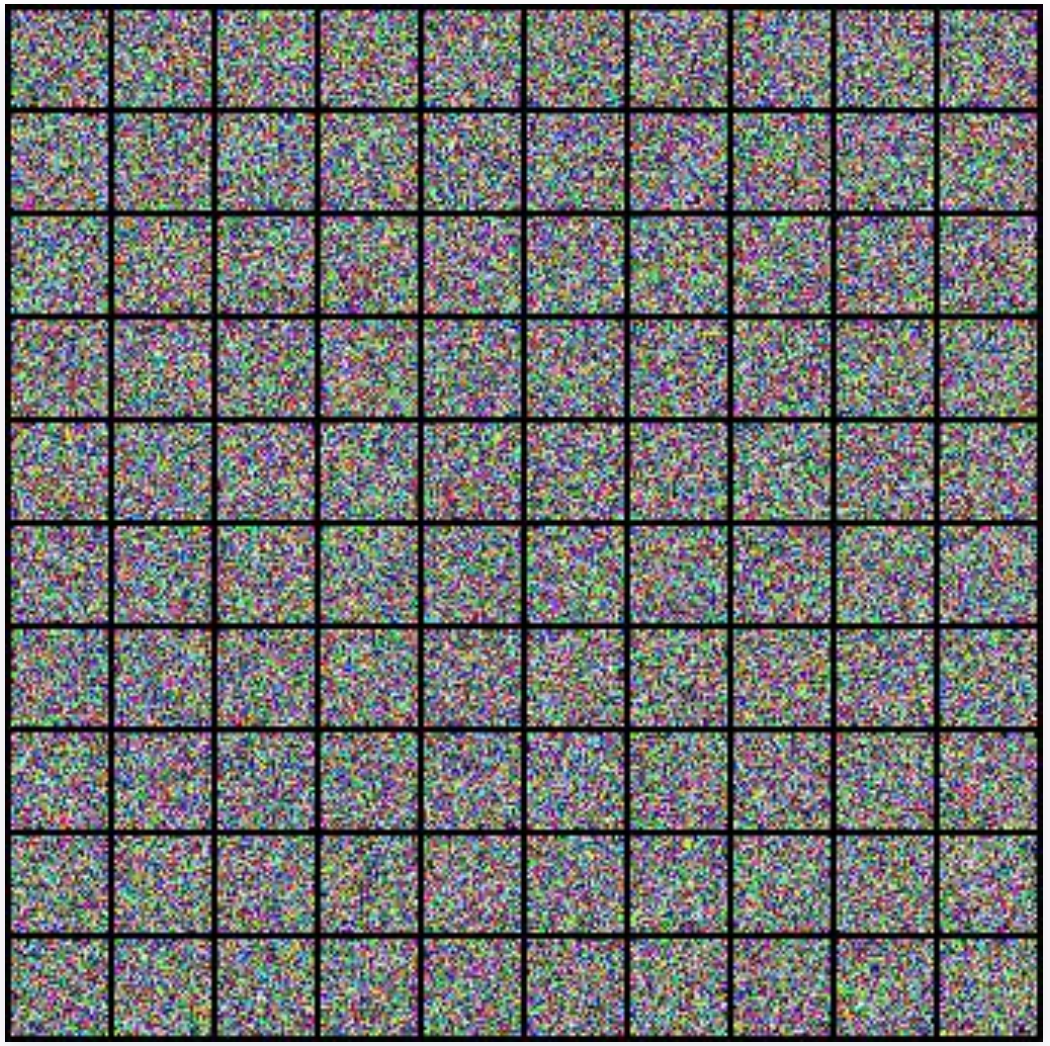
01

Pixel-space diffusion

DDPM, GLIDE, Imagen, Dalle2 and the cost of high resolution

Diffusion models





Diffusion models in pixel space

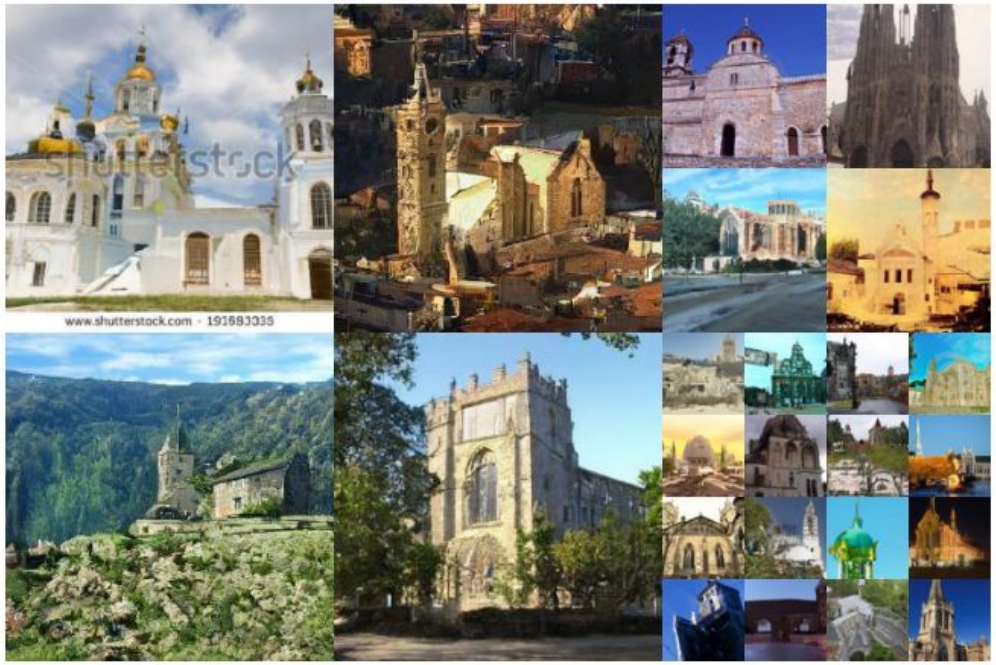


Figure 3: LSUN Church samples. FID=7.89

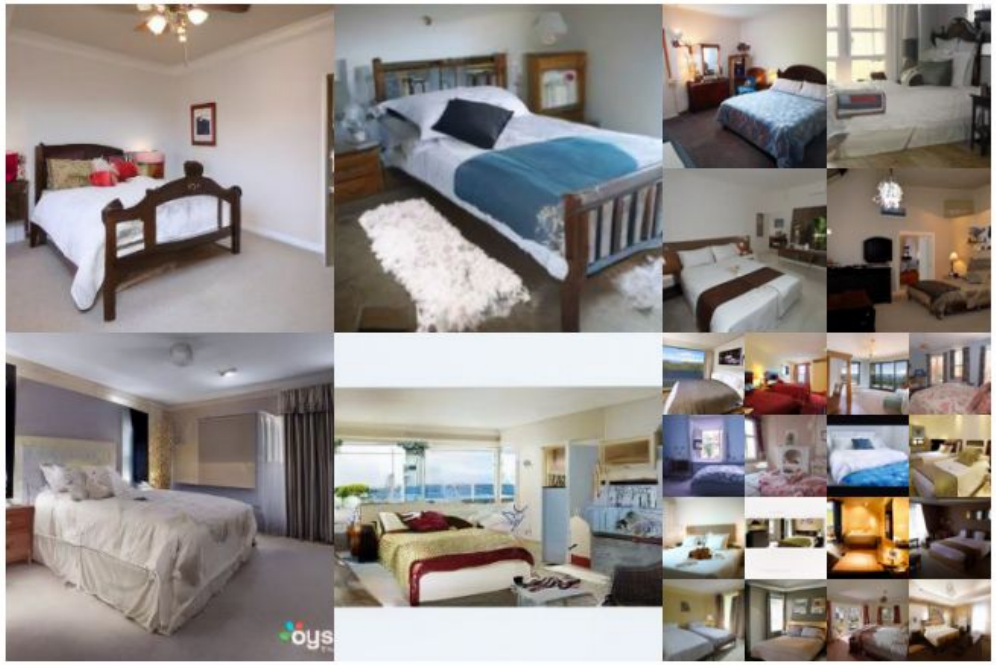


Figure 4: LSUN Bedroom samples. FID=4.90

Source

Diffusion models in pixel space

CelebA-HQ 256x256 samples

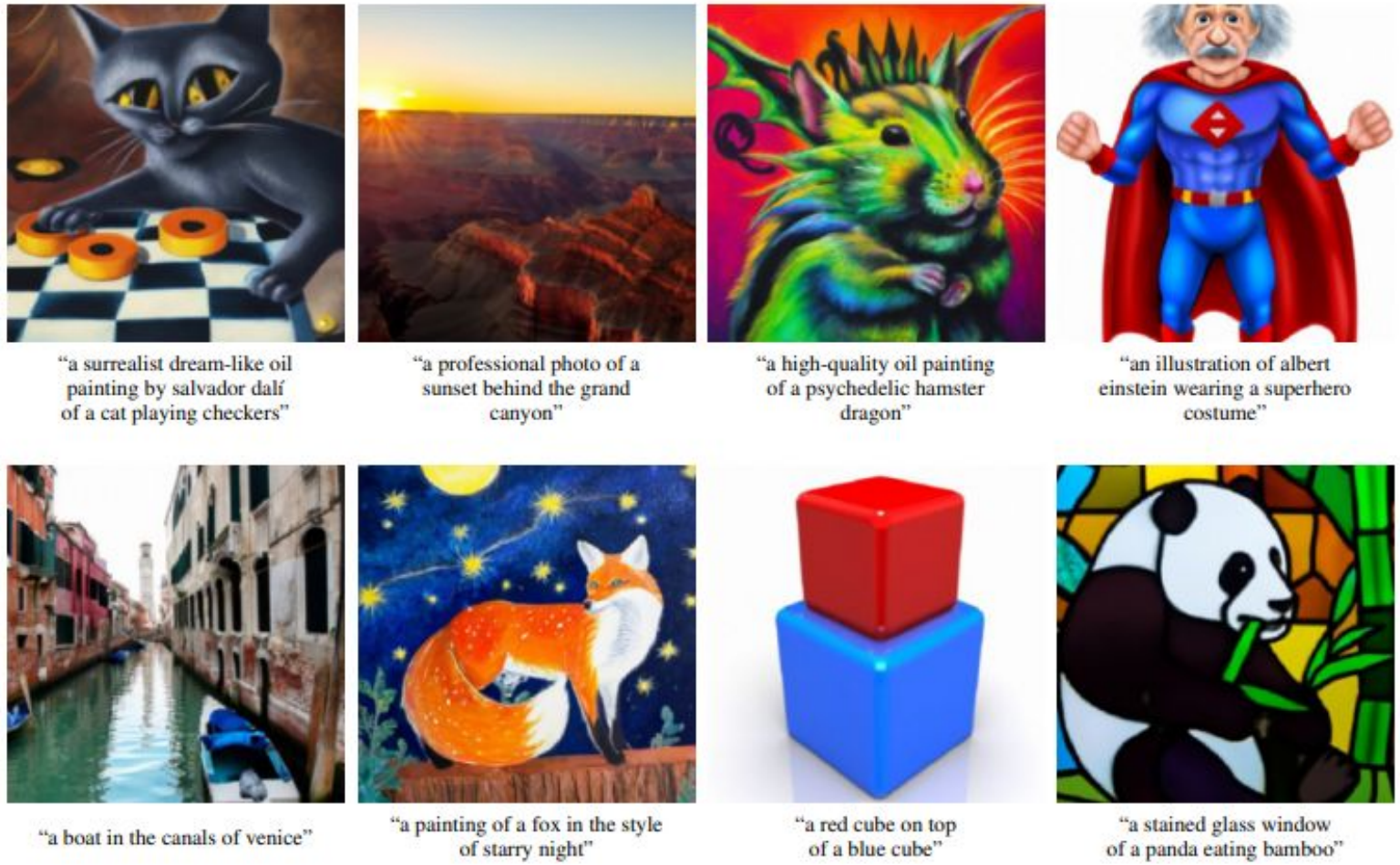
Source

Diffusion models in pixel space

LSUN 256x256 Church, Bedroom, and Cat samples. Notice that our models occasionally generate dataset watermarks.

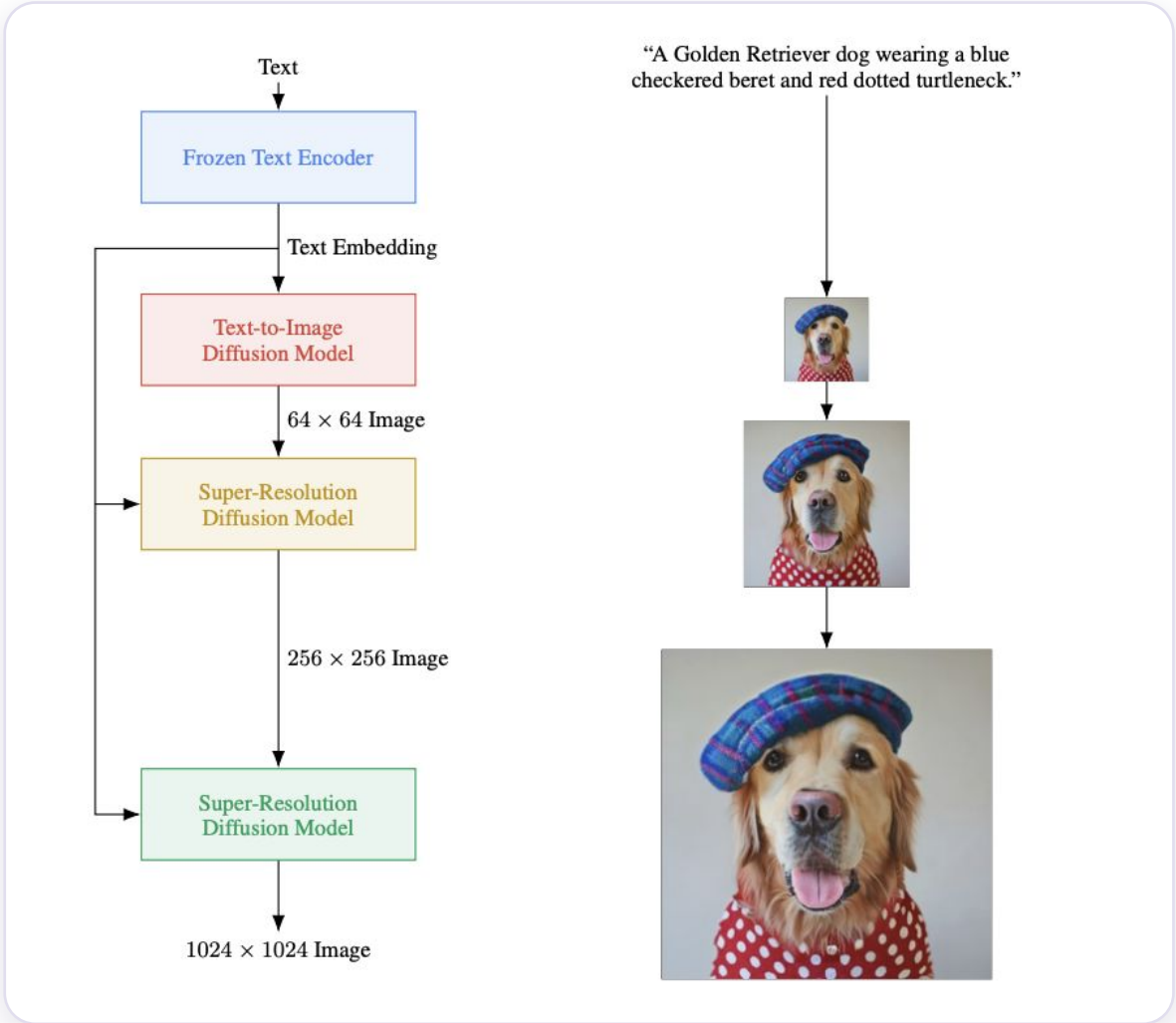
Source

GLIDE: генерация по тексту



Source

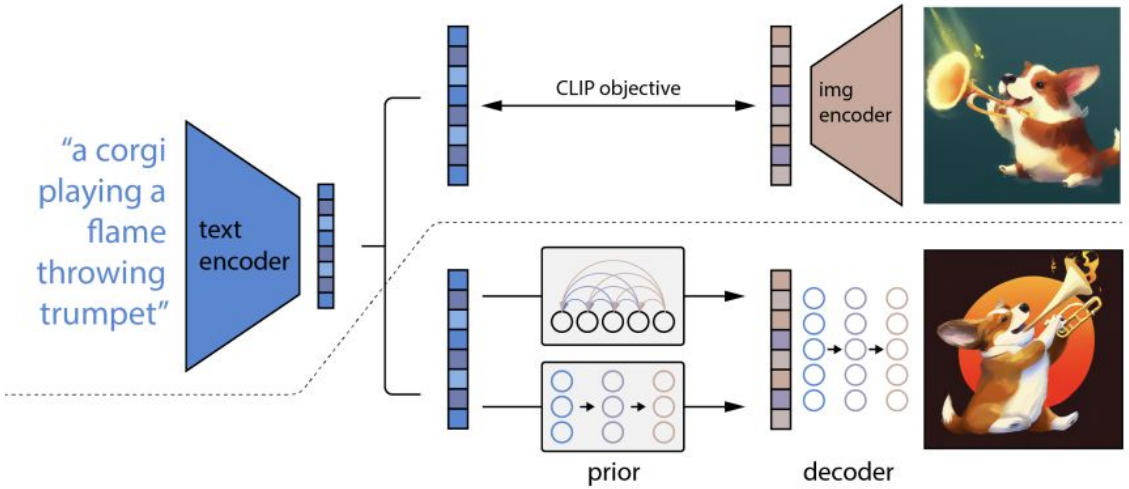
Imagen: каскад диффузий



Source

paper →

DALL·E 2 (unCLIP)

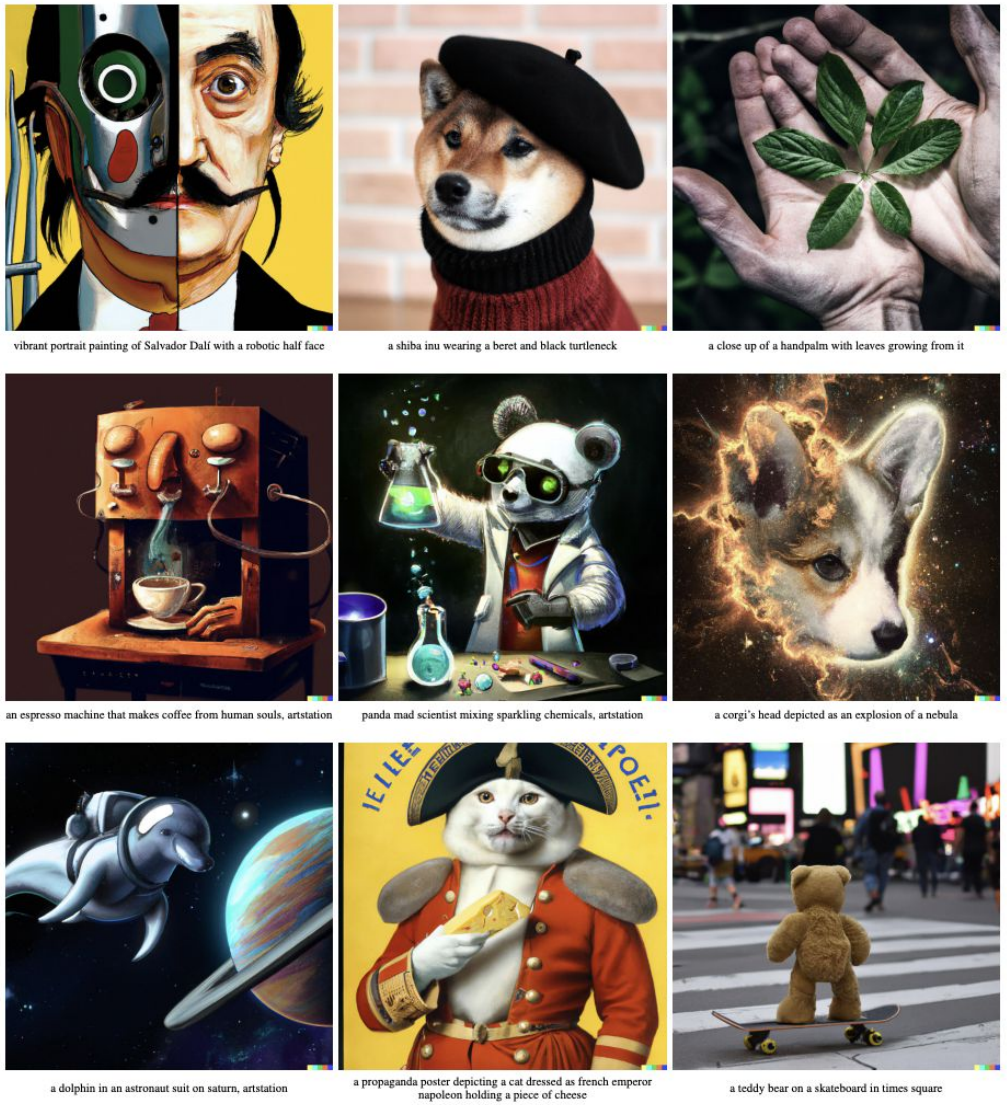


Why a prior?

- Generating the image embedding improves diversity with minimal loss in photorealism
- Guidance only on the decoder

Editing in CLIP space

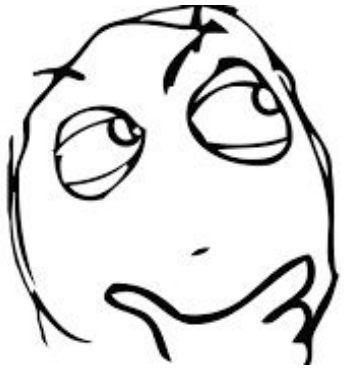
- Variations of an input image
- Interpolations
- Text diffs



Diffusion models in pixel space

Bigger resolution

- More compute for training
- More compute for generation



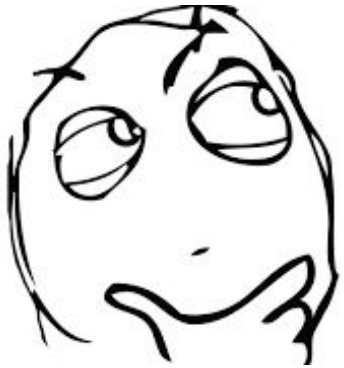
SPOILER

?

Diffusion models in pixel space

Bigger resolution

- More compute for training
- More compute for generation



SPOILER

Sometimes pixel space can work well…

DLS

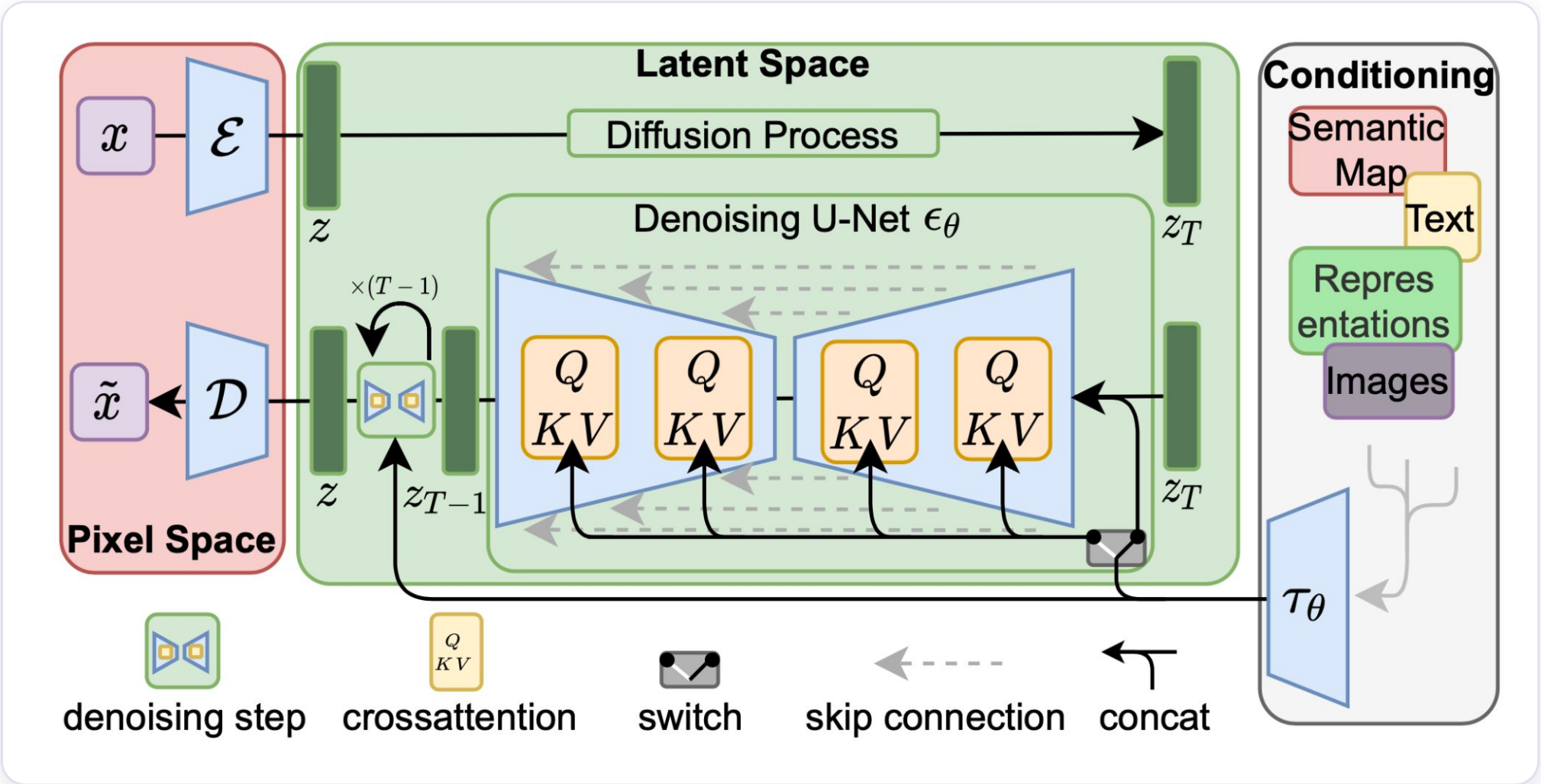
02

Latent diffusion

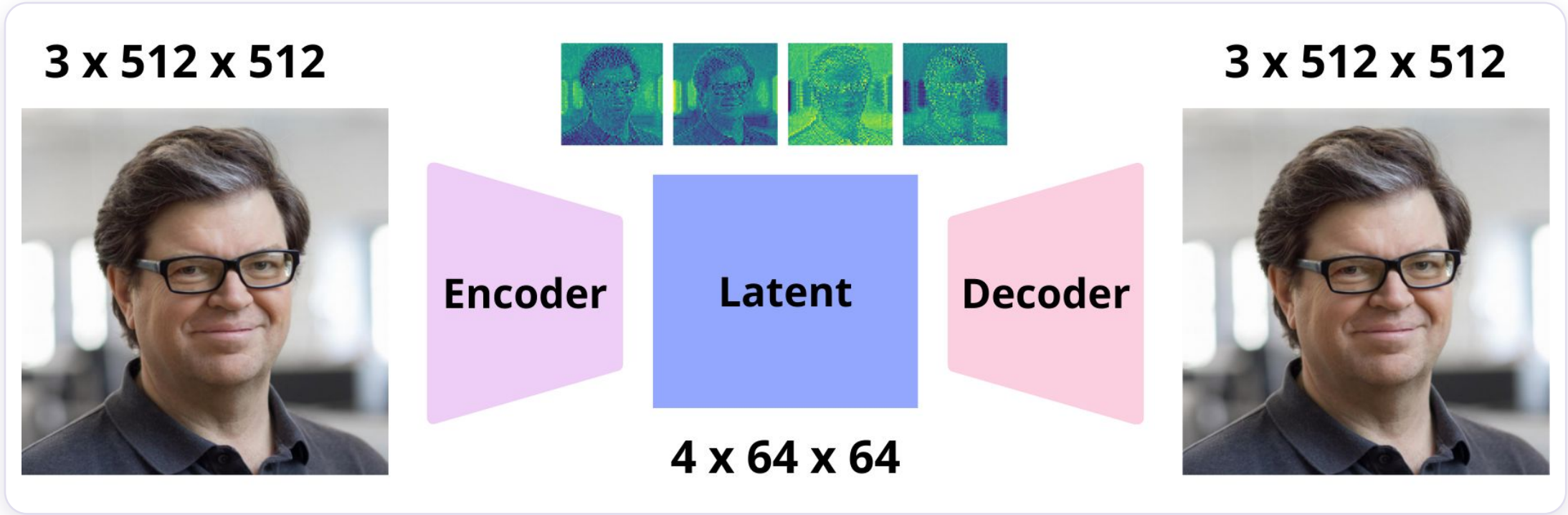
Diffusion in the compressed latent space of an autoencoder

paper →

Latent diffusion



Latent diffusion

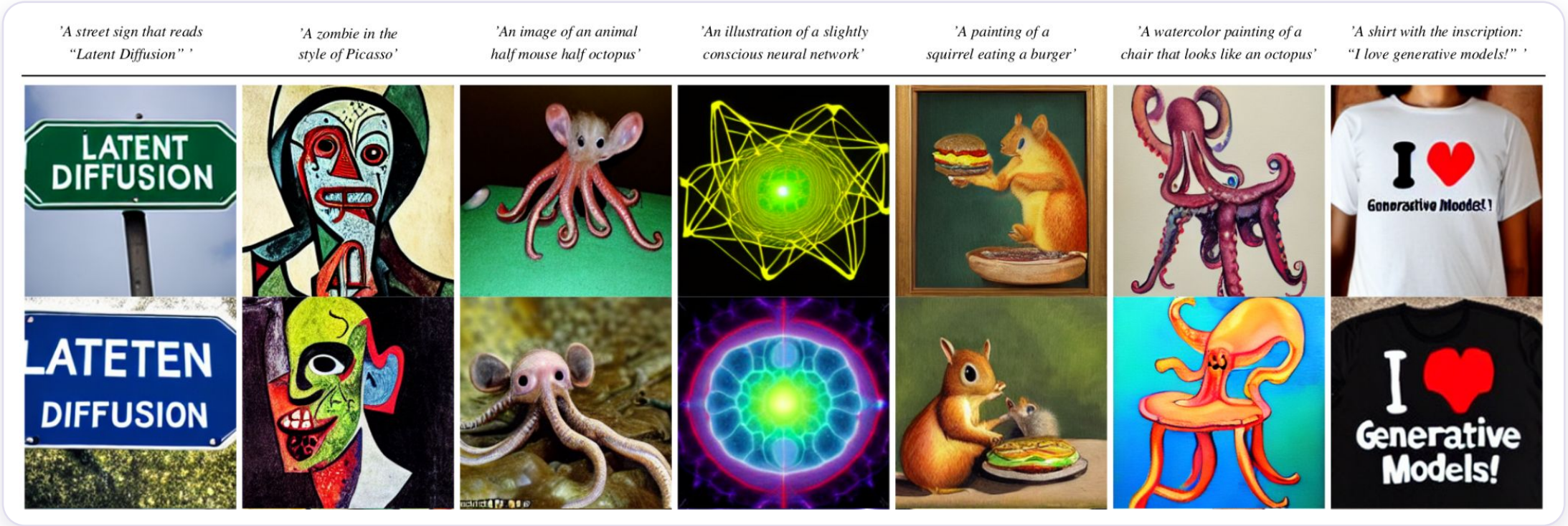


paper →

Latent diffusion

paper →

Latent diffusion



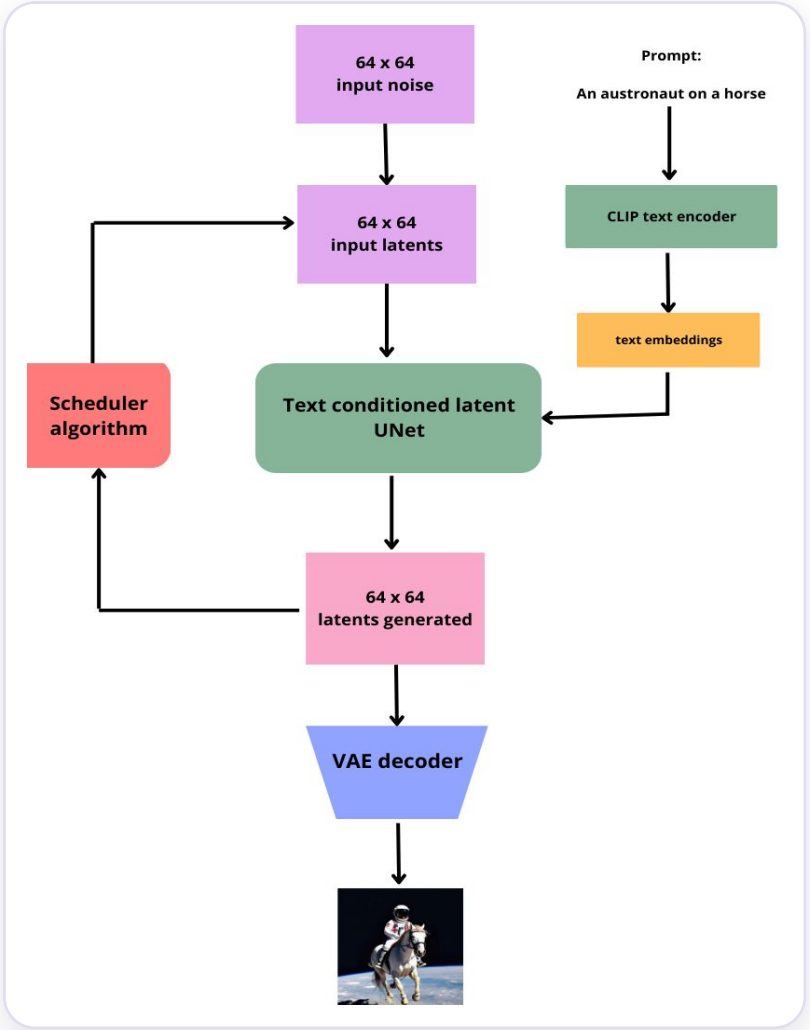
DLS

03

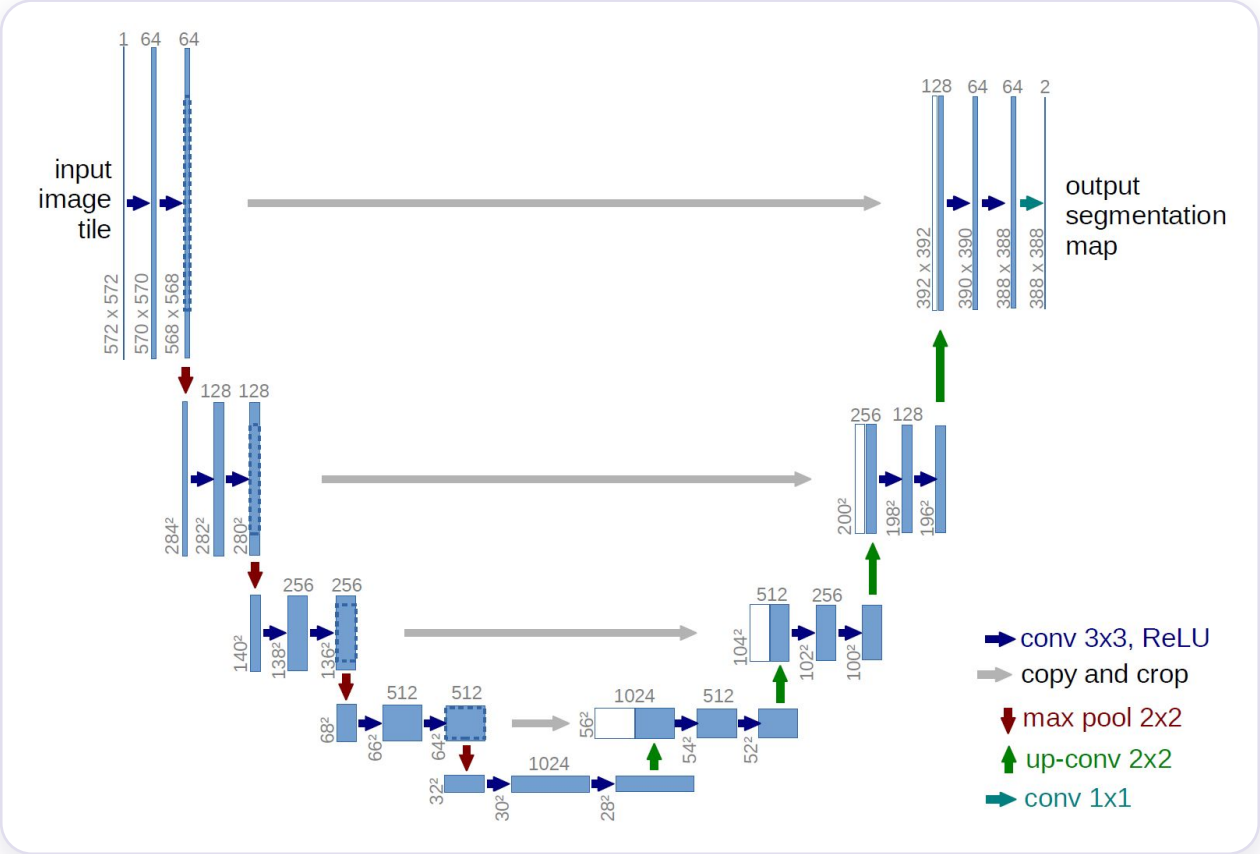
UNet text-to-image

Stable Diffusion 1.x, attention maps and SDXL

Text-to-image UNet-based diffusion



Text-to-image UNet-based diffusion

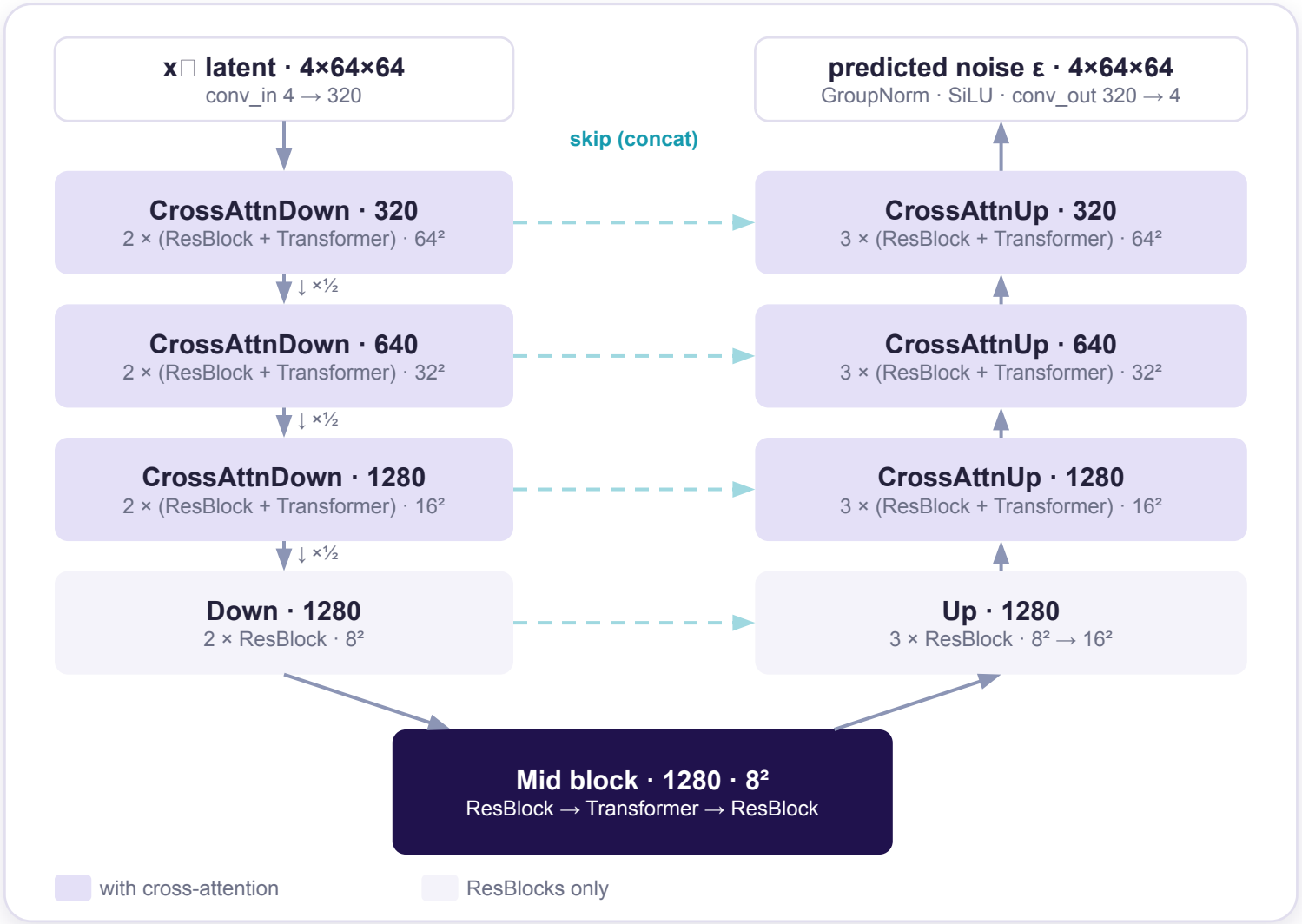


The most popular architecture is old fellow **UNet**, familiar from the segmentation task

UNet is a stack of Residual Convolutional Blocks operating on different resolutions

Deeper layers have more channels and operate on smaller features maps

SD 1.x UNet: overview



TIMESTEP

t → sinusoidal embedding · 320

MLP → temb · 1280

added inside every ResBlock

TEXT PROMPT

CLIP ViT-L/14 text encoder · frozen

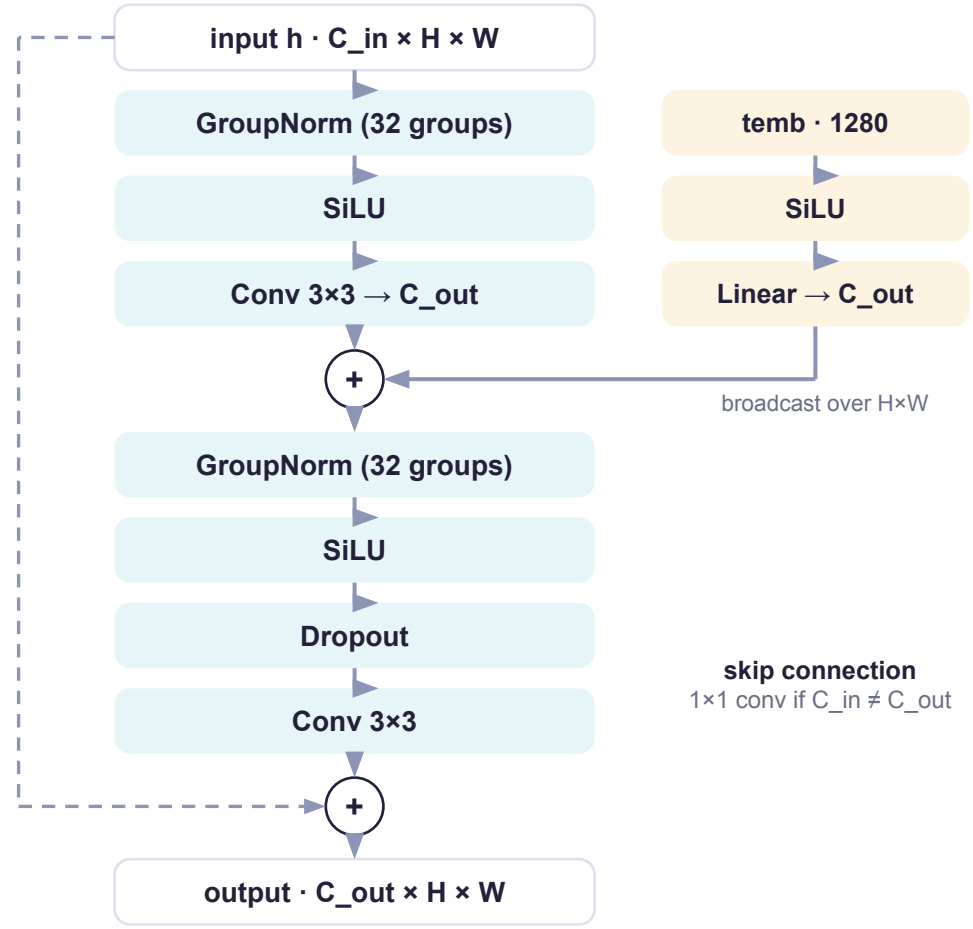
77 tokens × 768

keys & values of every cross-attention

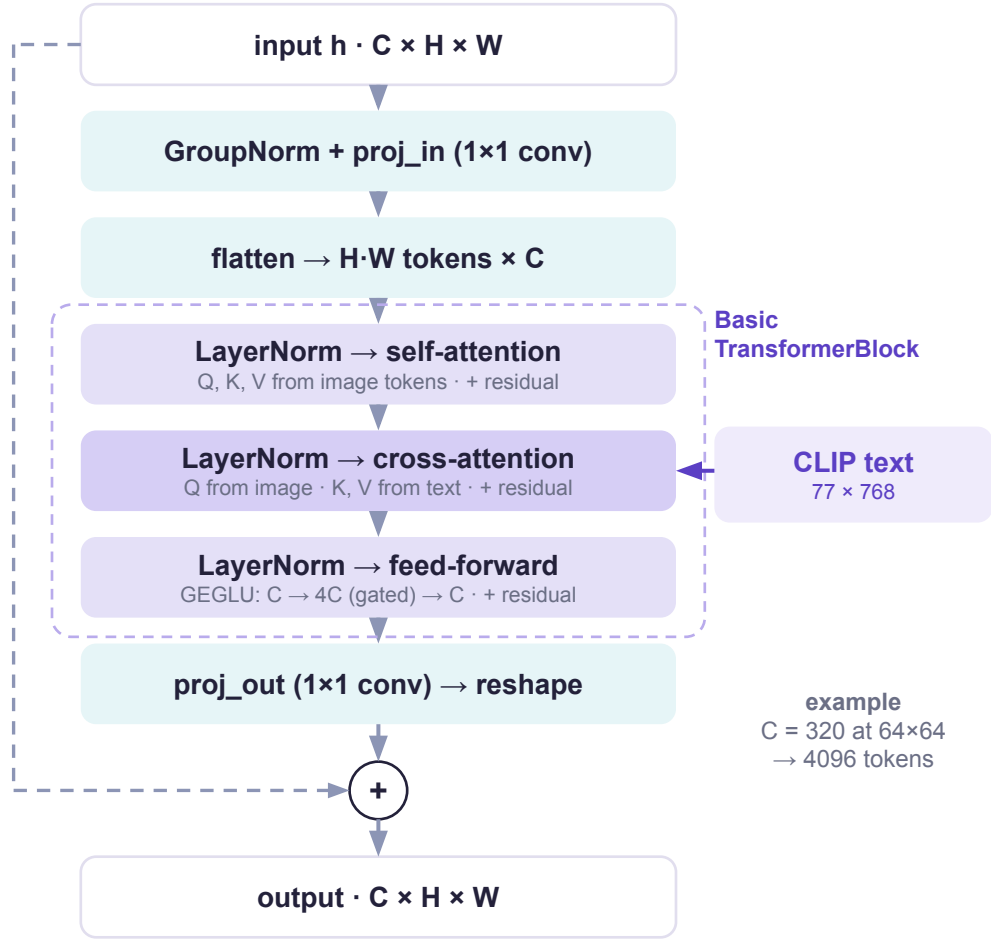
~860M params · 512² image → 64² latent

SD 1.x UNet: building blocks

RESNETBLOCK2D

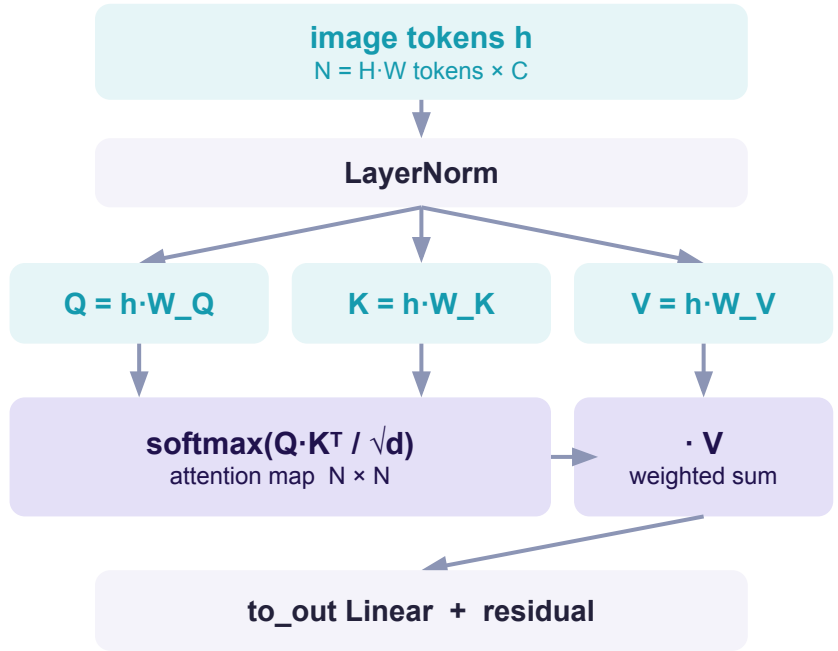


TRANSFORMER2DMODEL (SPATIAL TRANSFORMER)



Self-attention

INSIDE THE SPATIAL TRANSFORMER

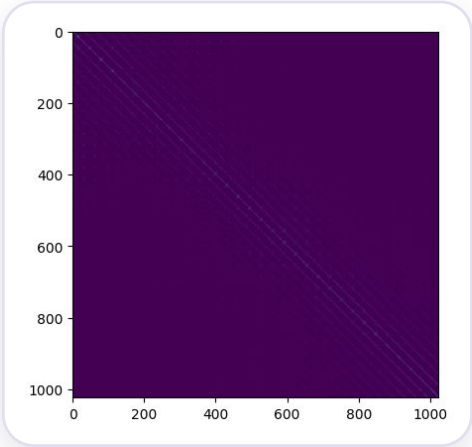


$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V$$

Every pixel can look at every other pixel: global context at the cost of O(N²)

generated image

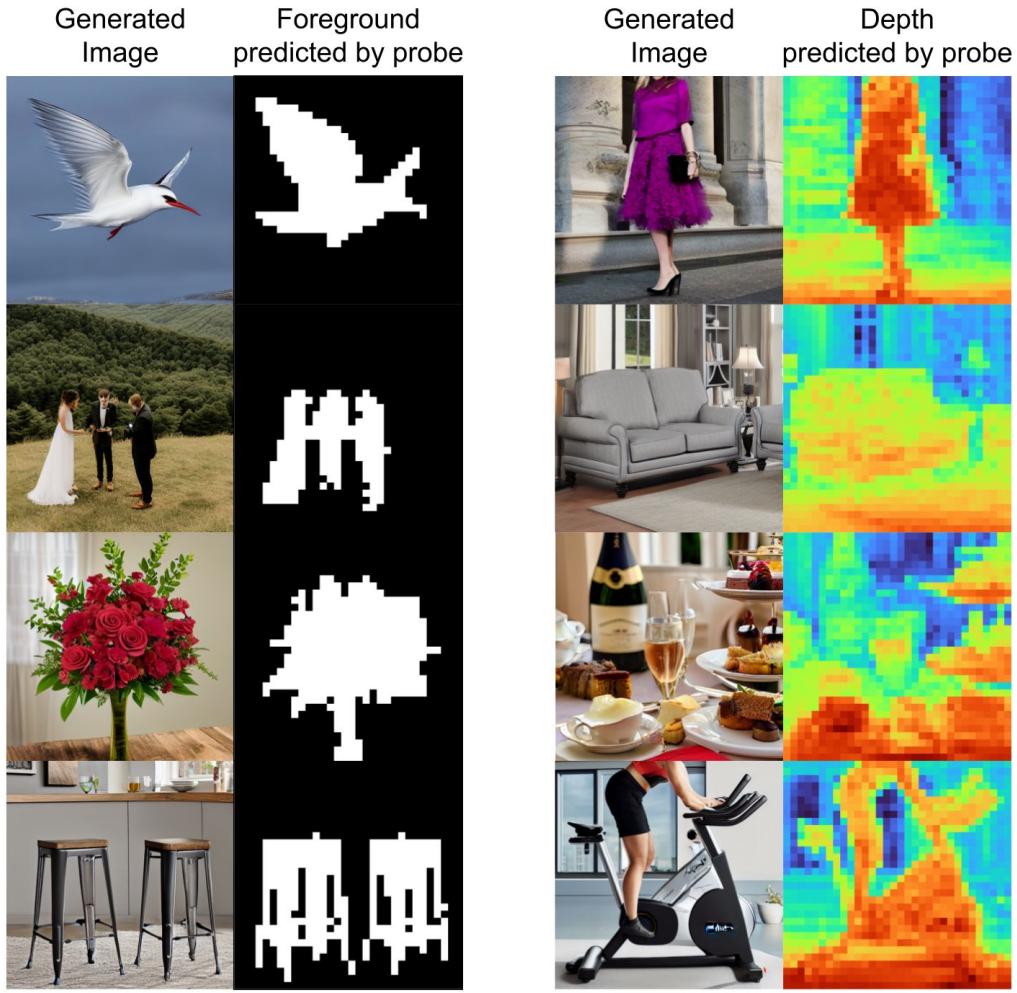
token grid 32×32 = 1024 tokens

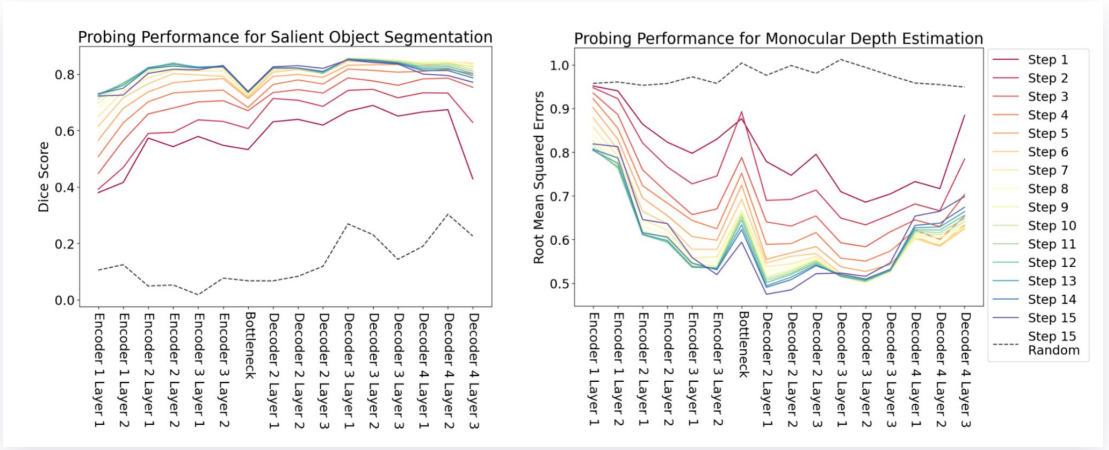


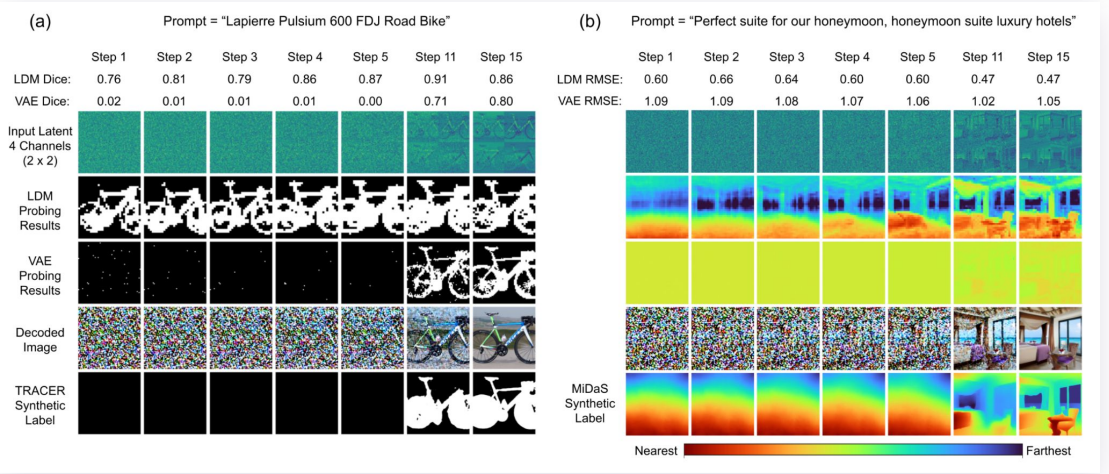
real map: 1024 × 1024

paper →

Beyond Surface Statistics

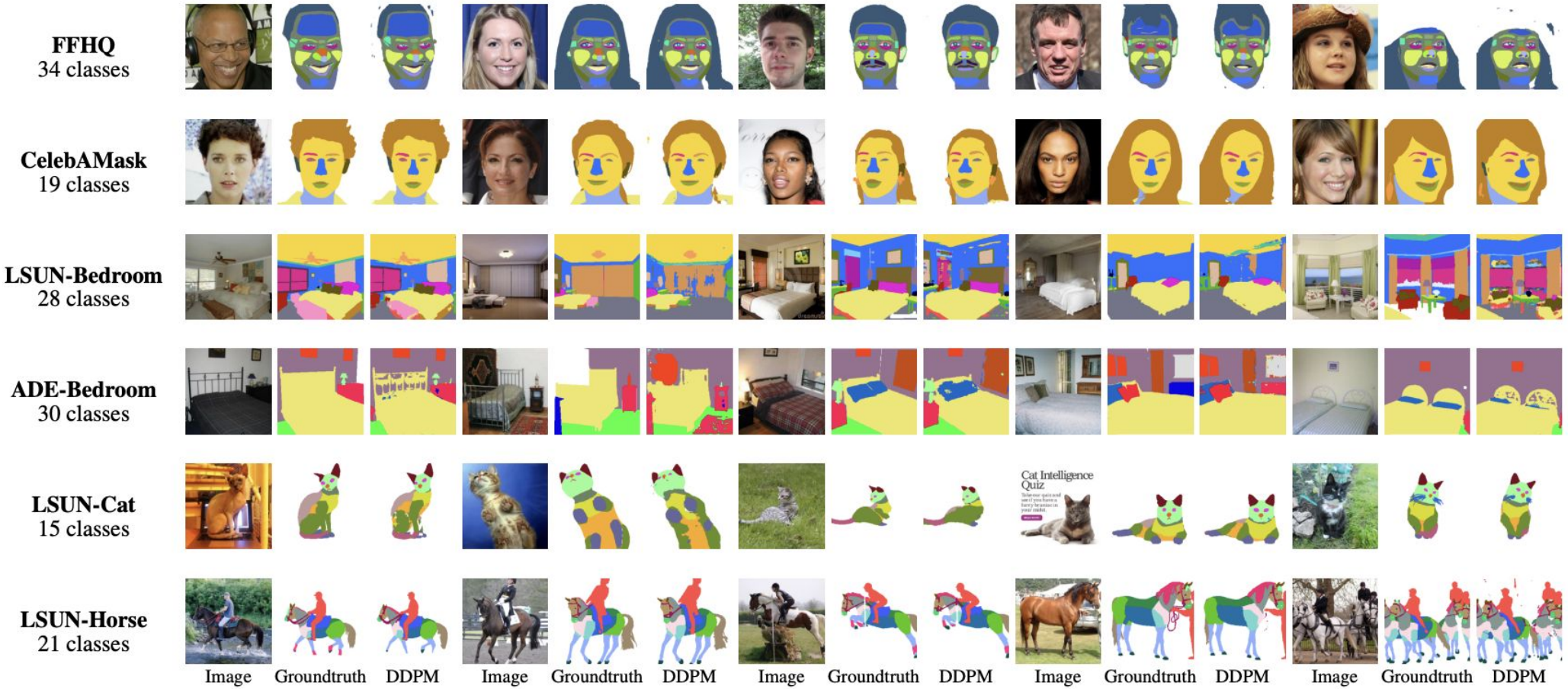






paper →

LABEL-EFFICIENT SEMANTIC SEGMENTATION



Cross-attention

TEXT ENTERS THE UNET HERE

image tokens h
N = H·W × C

text tokens c
CLIP · 77 × 768

LayerNorm

Q = h·W_Q

K = c·W_K

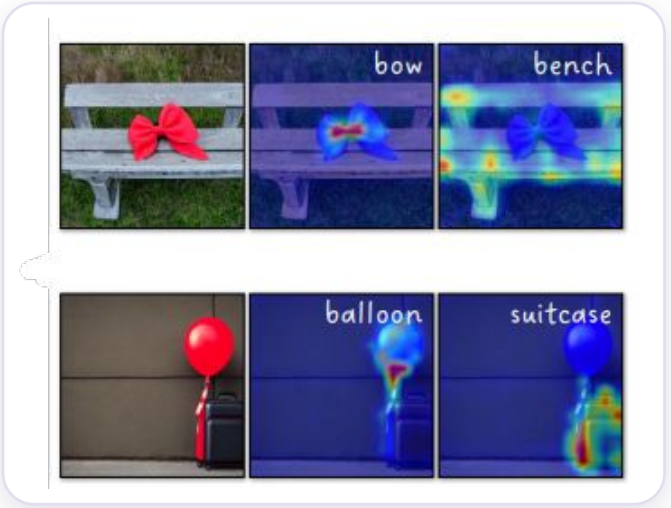
V = c·W_V

softmax(Q·Kᵀ / √d) · V
attention map N × 77 (pixels × words)

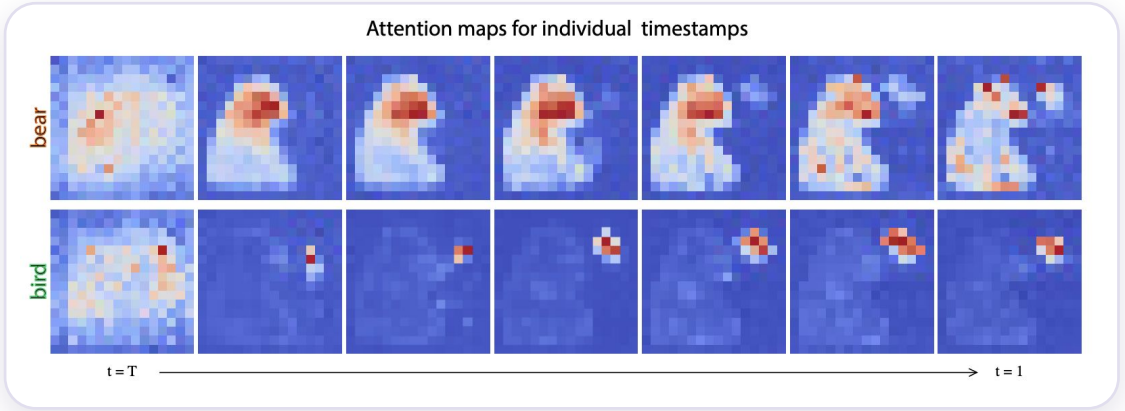
to_out Linear + residual

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V$$

Each pixel gathers information from the words; the map for one word shows where it is drawn



cross-attention maps for individual words

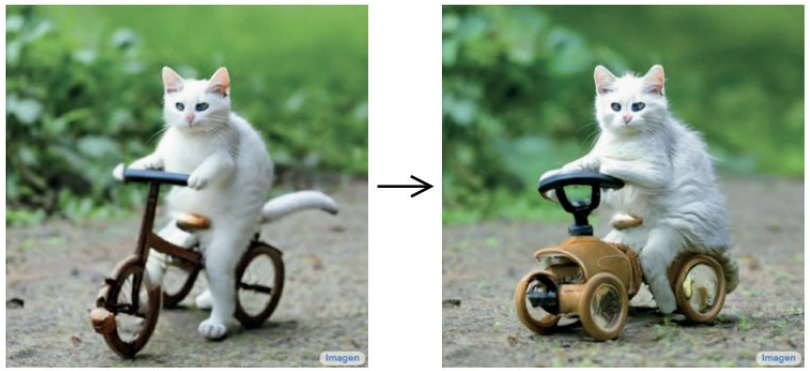


maps for “bear” and “bird” across denoising steps

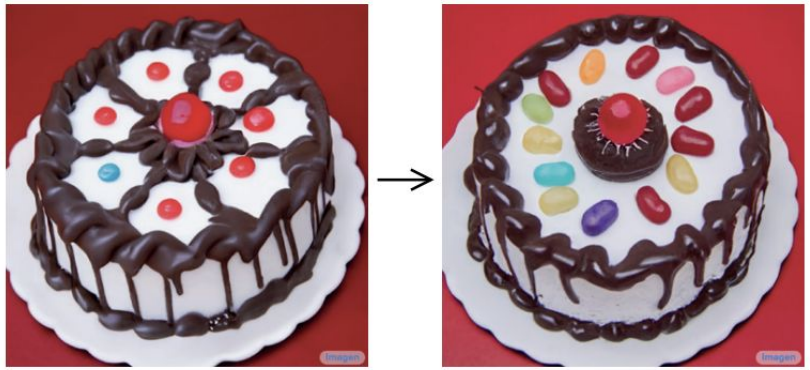
arxiv.org/pdf/2404.04650 · arxiv.org/pdf/2208.01626

paper →

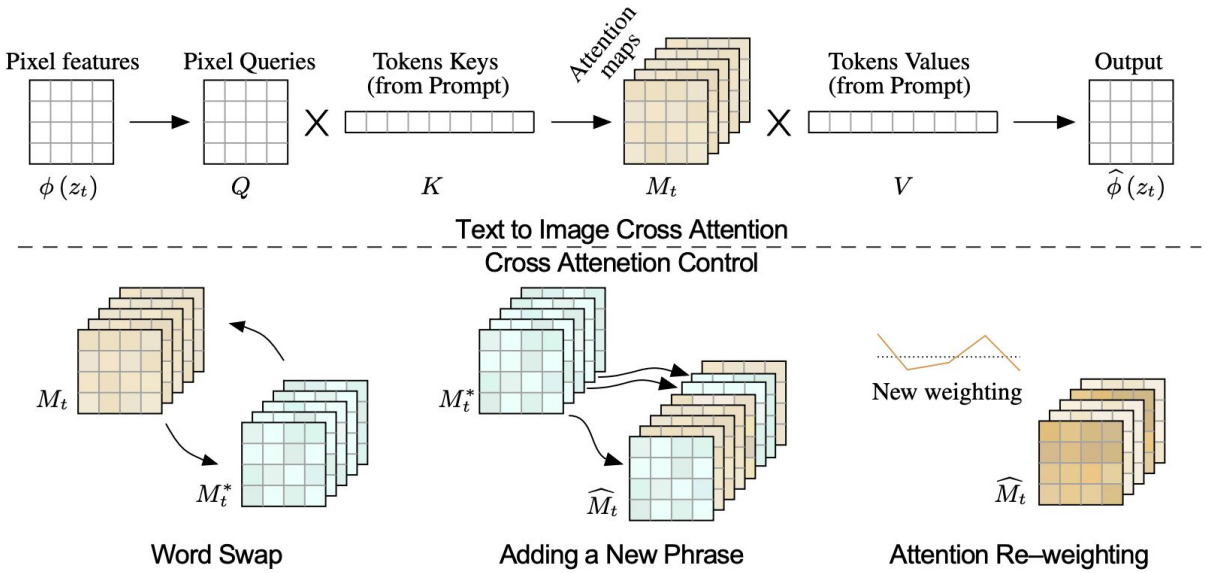
Prompt-to-prompt



"Photo of a cat riding on a ~~bicycle~~ car."

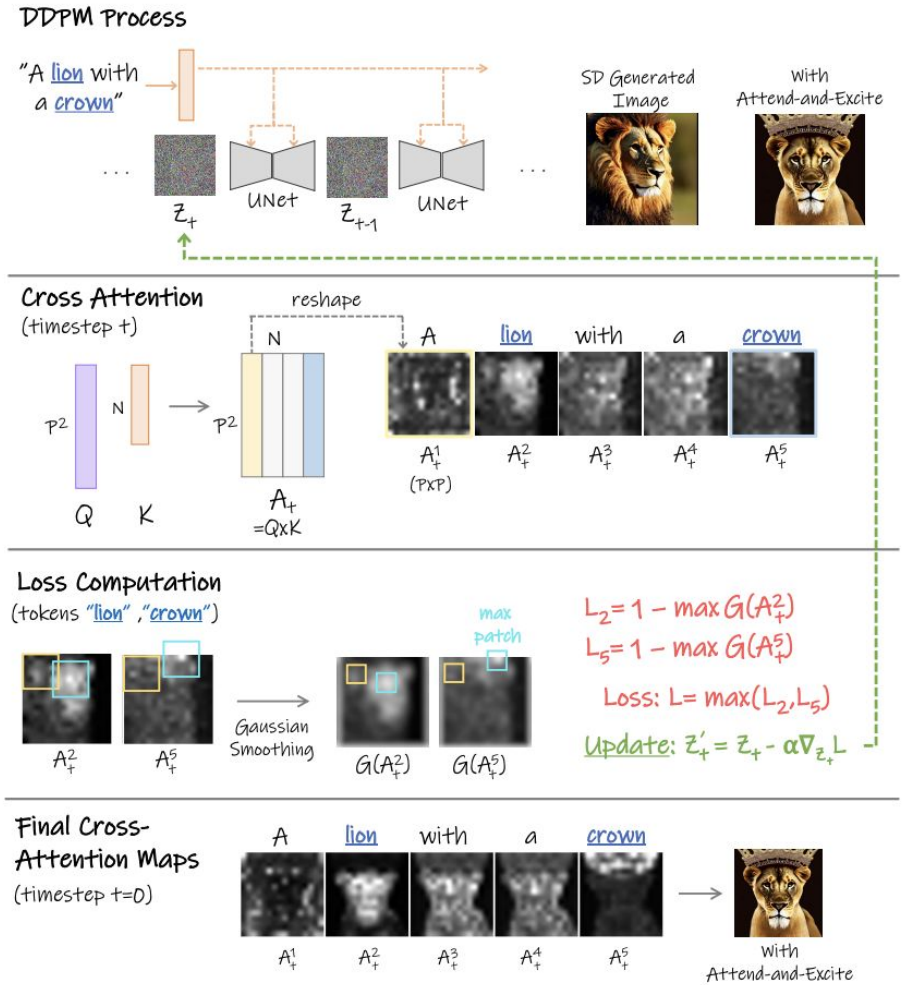


"a cake with decorations."
jelly beans



paper →

Attend-and-Excite



SD1.4 - SD1.5 results

SDXL

CLIP ViT-L — the same encoder from SD1.5, producing a 768-dim per-token embedding.

OpenCLIP ViT-bigG — a much larger encoder trained on LAION-2B, producing a 1280-dim per-token embedding. It also outputs a pooled embedding.

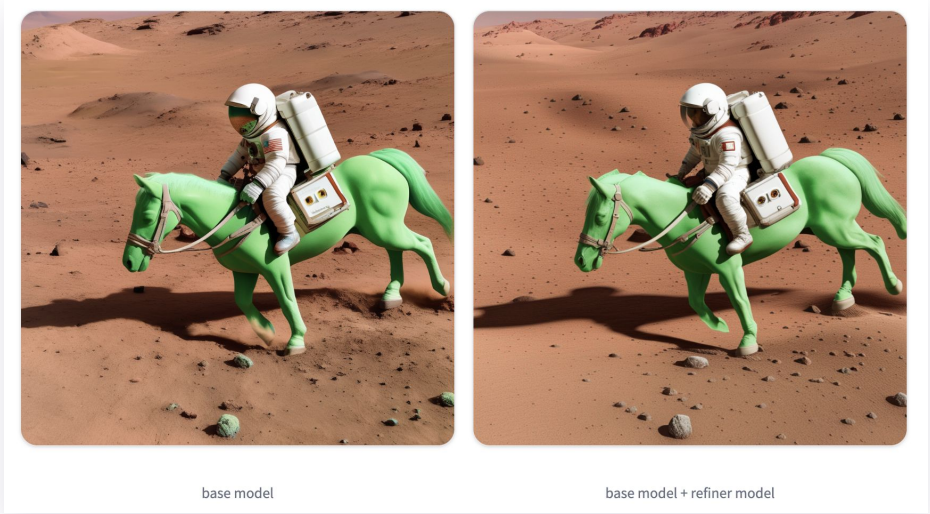
The two per-token embeddings are concatenated along the channel dimension. The pooled embedding from OpenCLIP ViT-bigG gets projected and **added directly to the timestep embedding**.

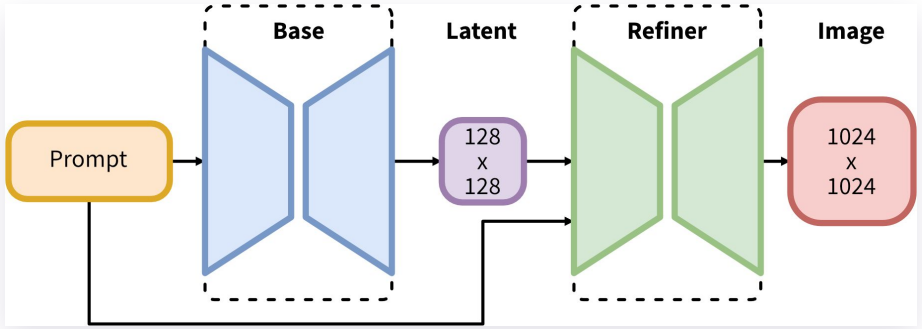
Multi-aspect training

Can generate images not only squared but also rectangle

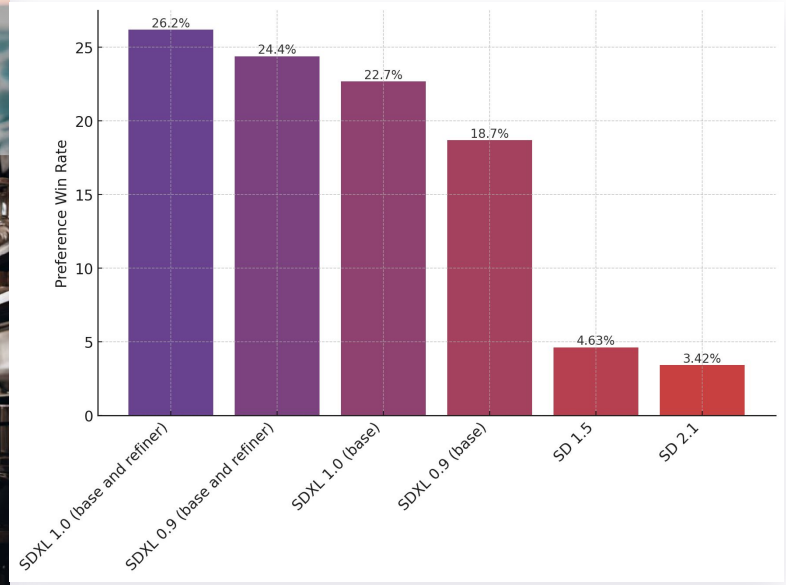
Refiner model

SDXL introduces two U-Net models — the base model and a refiner that functions as an image-to-image model

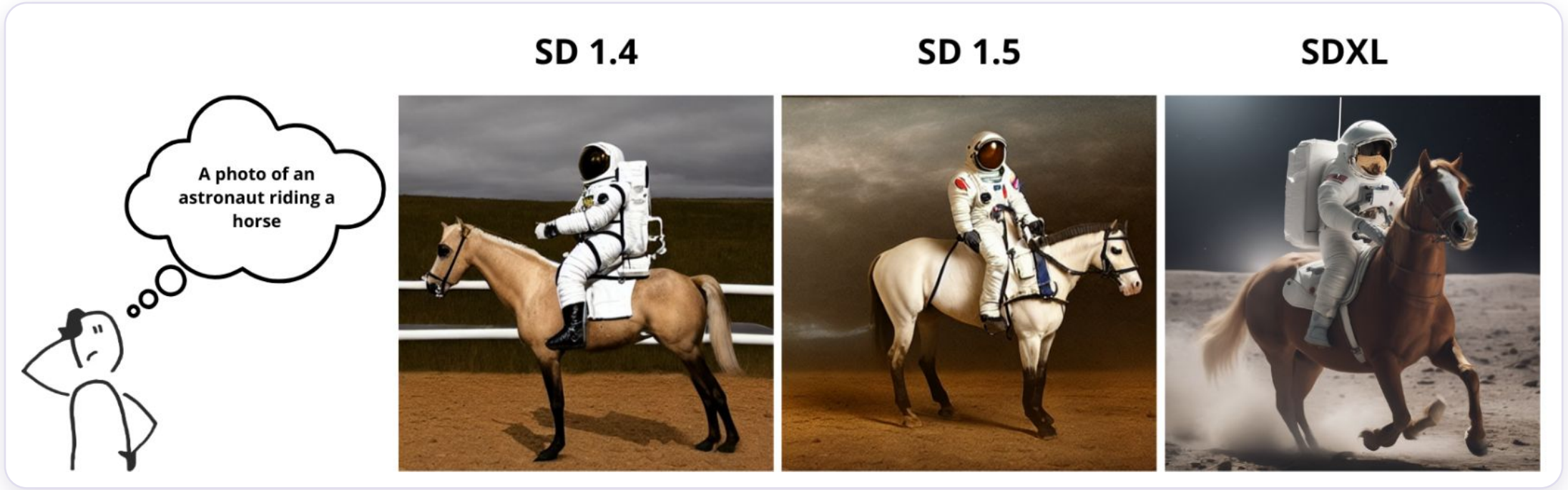




SDXL some results



Stable diffusion UNet-based history



DLS

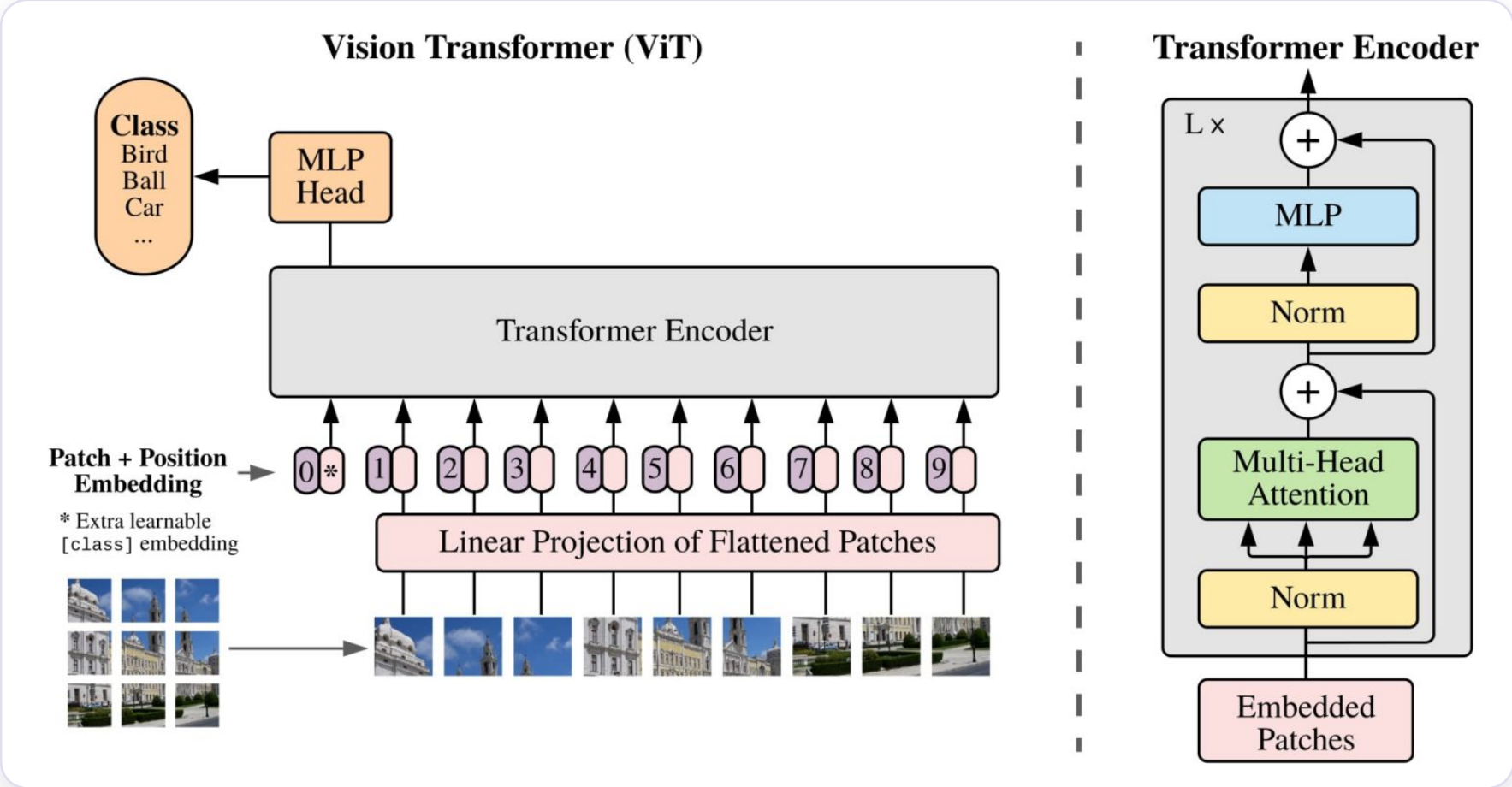
04

Diffusion transformers

ViT, DiT, AdaLN and scaling

paper →

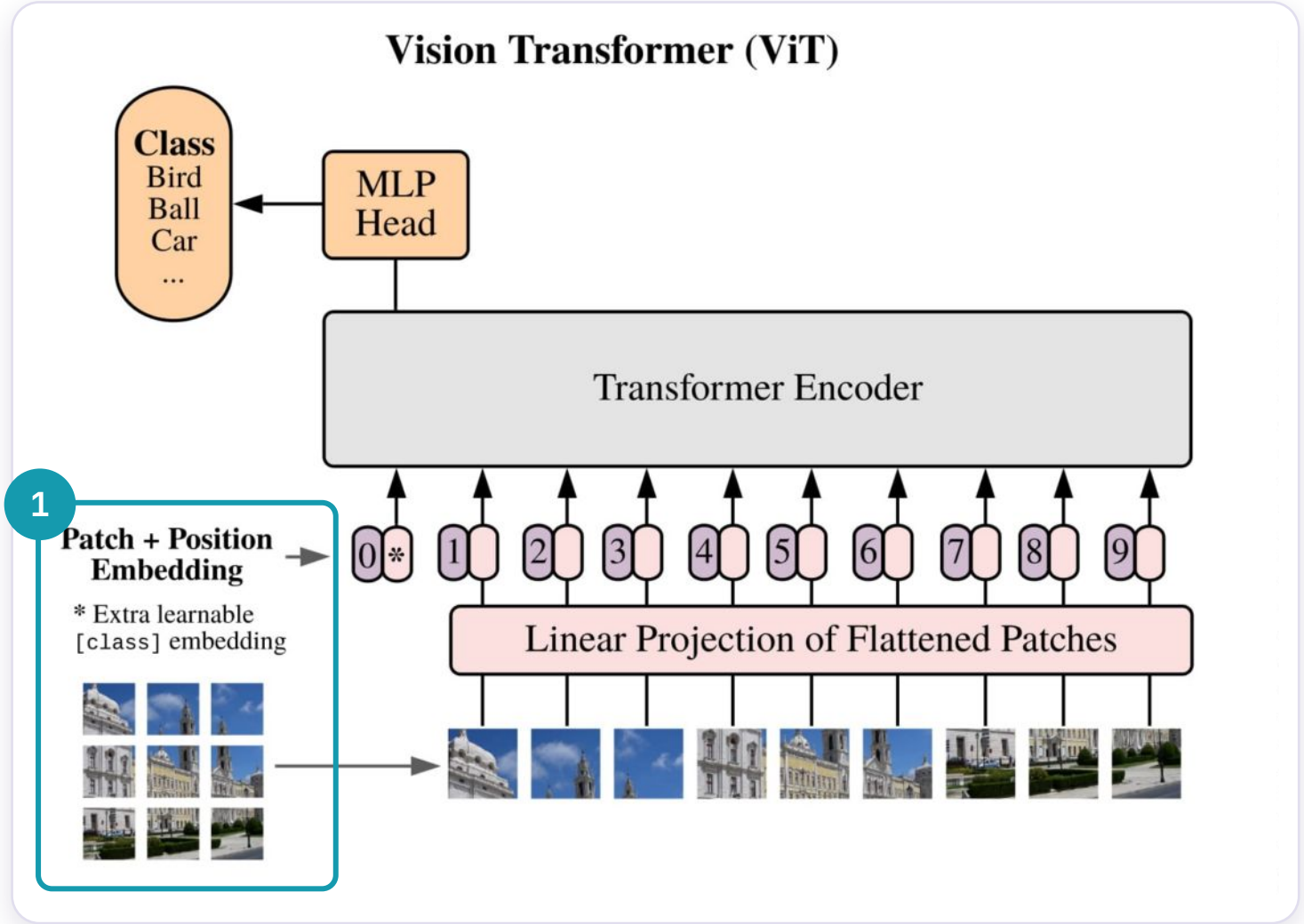
Vision Transformer (ViT)



An image is a sequence of patches: the same transformer encoder as in NLP, with no convolutions

paper →

Vision Transformer (ViT)



1 Patchify

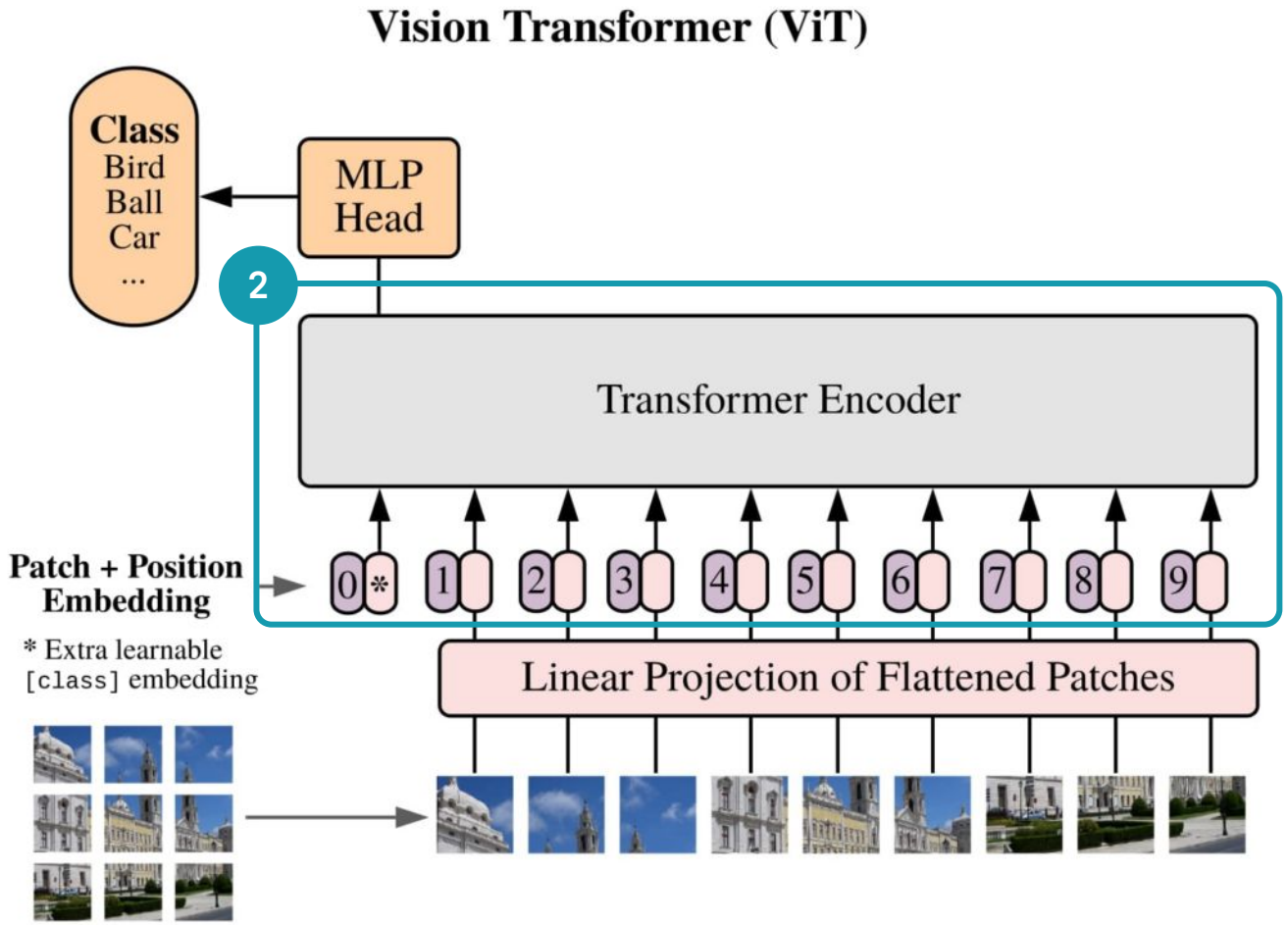
The image is split into fixed-size patches (e.g. 16×16). Each patch is flattened, linearly projected to a token and gets a position embedding.

2 Transformer encoder

Image tokens are processed as an ordinary sequence by standard transformer blocks. An extra [class] token is used for classification.

paper →

Vision Transformer (ViT)



1 Patchify

The image is split into fixed-size patches (e.g. 16×16). Each patch is flattened, linearly projected to a token and gets a position embedding.

2 Transformer encoder

Image tokens are processed as an ordinary sequence by standard transformer blocks. An extra [class] token is used for classification.

Diffusion transformers

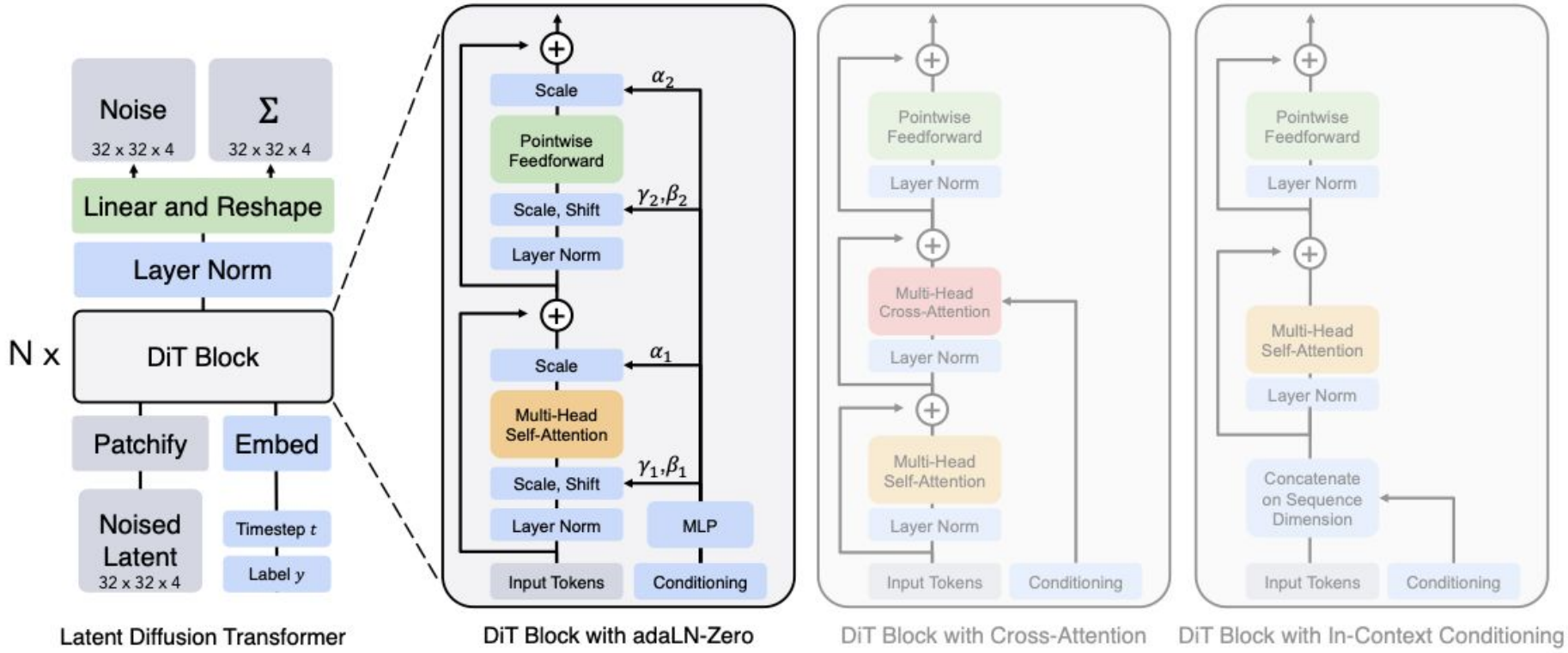
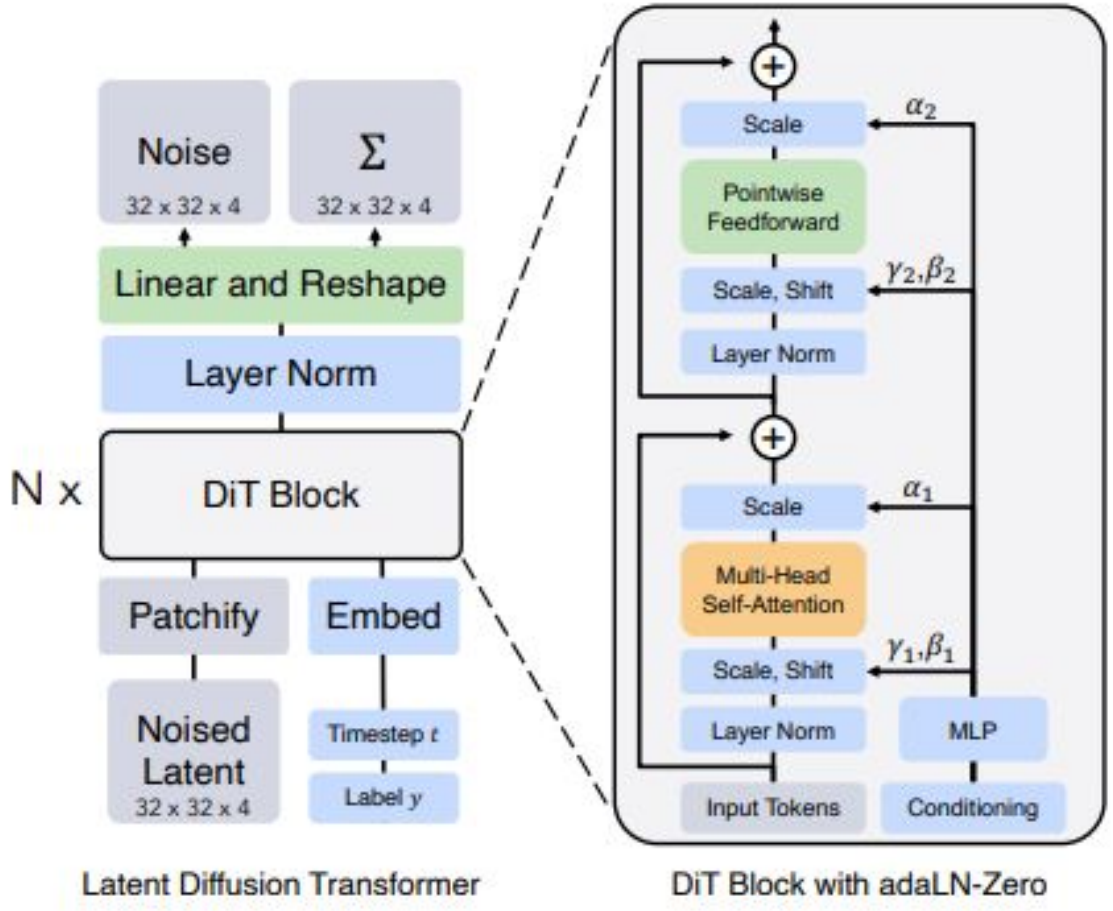


Figure 3. **The Diffusion Transformer (DiT) architecture.** *Left:* We train conditional latent DiT models. The input latent is decomposed into patches and processed by several DiT blocks. *Right:* Details of our DiT blocks. We experiment with variants of standard transformer blocks that incorporate conditioning via adaptive layer norm, cross-attention and extra input tokens. Adaptive layer norm works best.

paper →

Diffusion transformers: adaLN

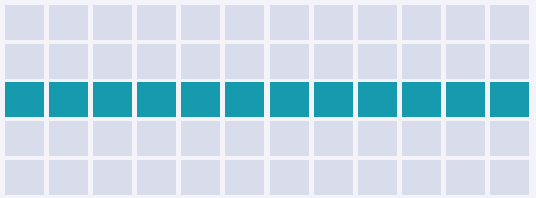


LAYER NORM, PER TOKEN

$$\mu_{b,n} = \frac{1}{D}\sum_{d=1}^{D} x_{b,n,d}$$

$$\sigma^2_{b,n} = \frac{1}{D}\sum_{d=1}^{D}(x_{b,n,d} - \mu_{b,n})^2$$

$$\mathrm{adaLN}(x, c) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot \gamma(c) + \beta(c)$$

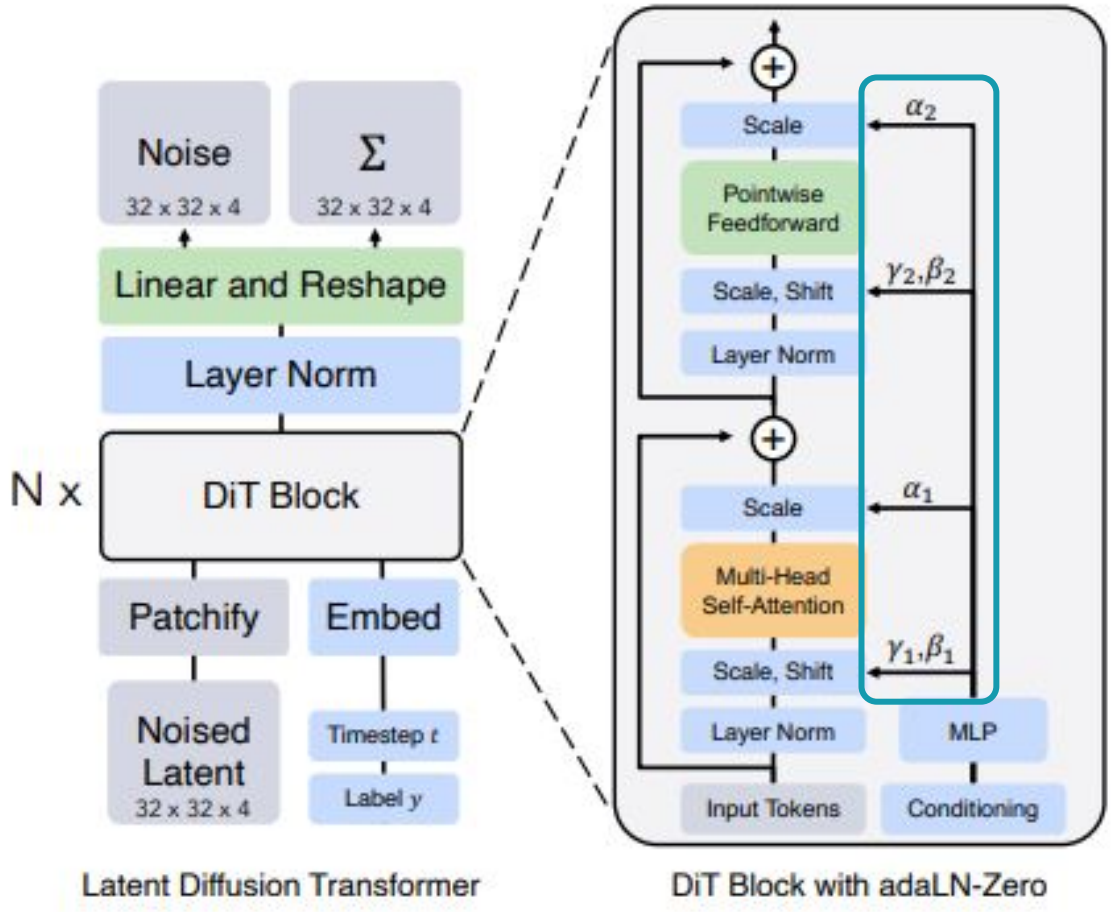


N tokens × D channels
μ, σ over the D channels of each token

Takeaway
The condition (timestep + class) only predicts γ and β.

paper →

Diffusion transformers: adaLN

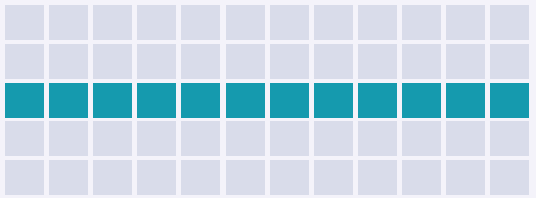


LAYER NORM, PER TOKEN

$$\mu_{b,n} = \frac{1}{D}\sum_{d=1}^{D} x_{b,n,d}$$

$$\sigma^2_{b,n} = \frac{1}{D}\sum_{d=1}^{D}(x_{b,n,d} - \mu_{b,n})^2$$

$$\mathrm{adaLN}(x, c) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot \gamma(c) + \beta(c)$$

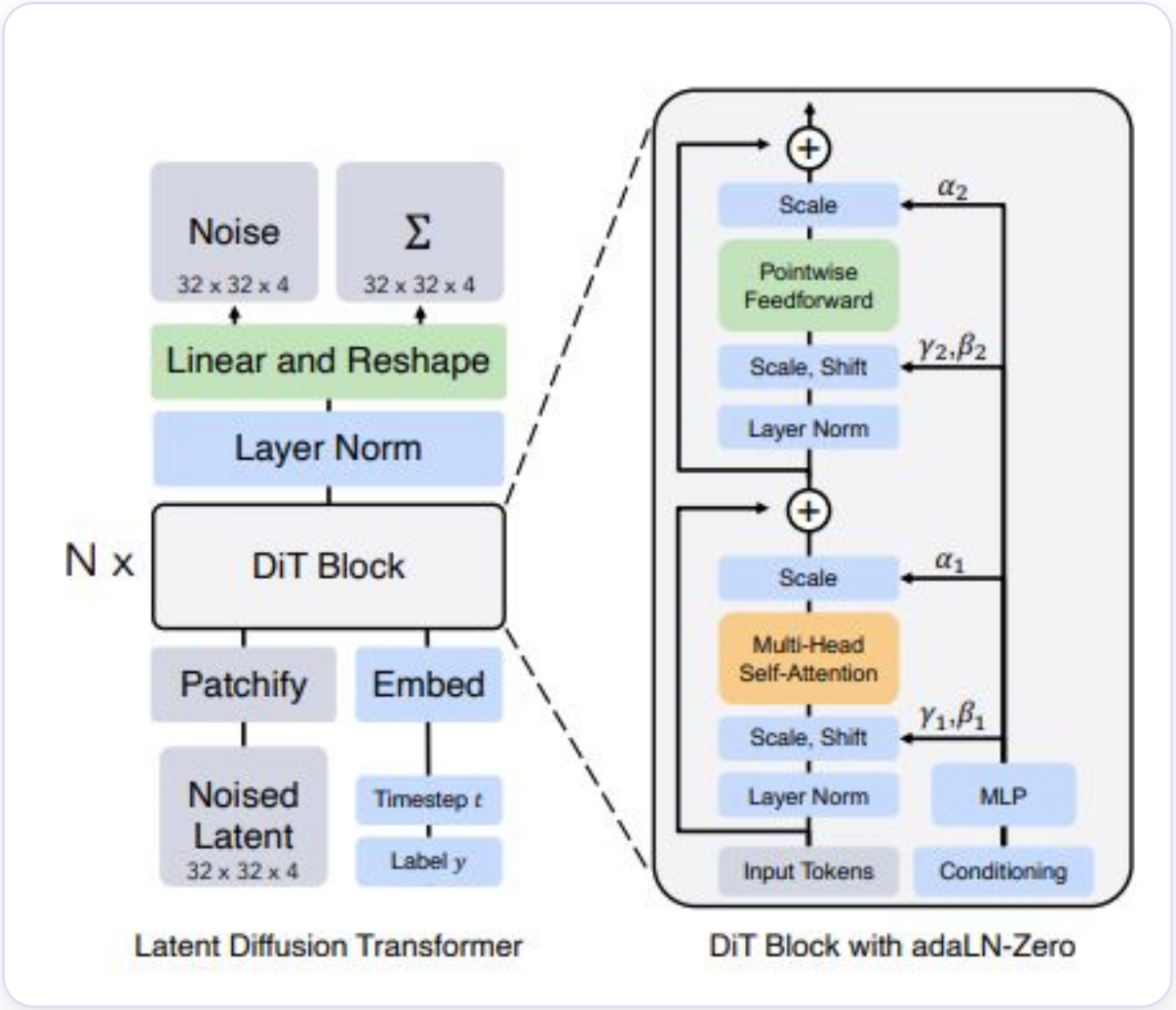


N tokens × D channels
μ, σ over the D channels of each token

Takeaway
The condition (timestep + class) only predicts γ and β.

paper →

Diffusion transformers: adaLN-Zero



$$\mathrm{AdaLN}(x, c) = \gamma(c) \cdot \mathrm{LayerNorm}(x) + \beta(c)$$

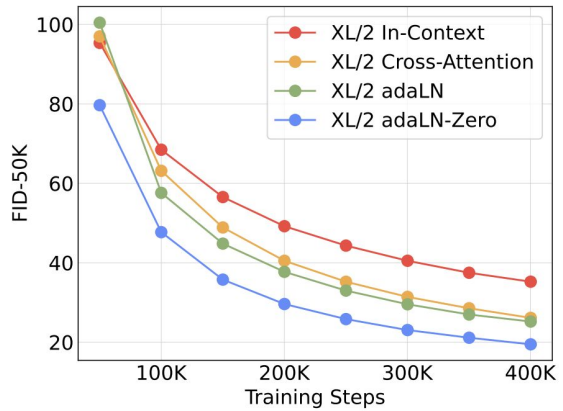


Figure 5. **Comparing different conditioning strategies.** adaLN-Zero outperforms cross-attention and in-context conditioning at all stages of training.

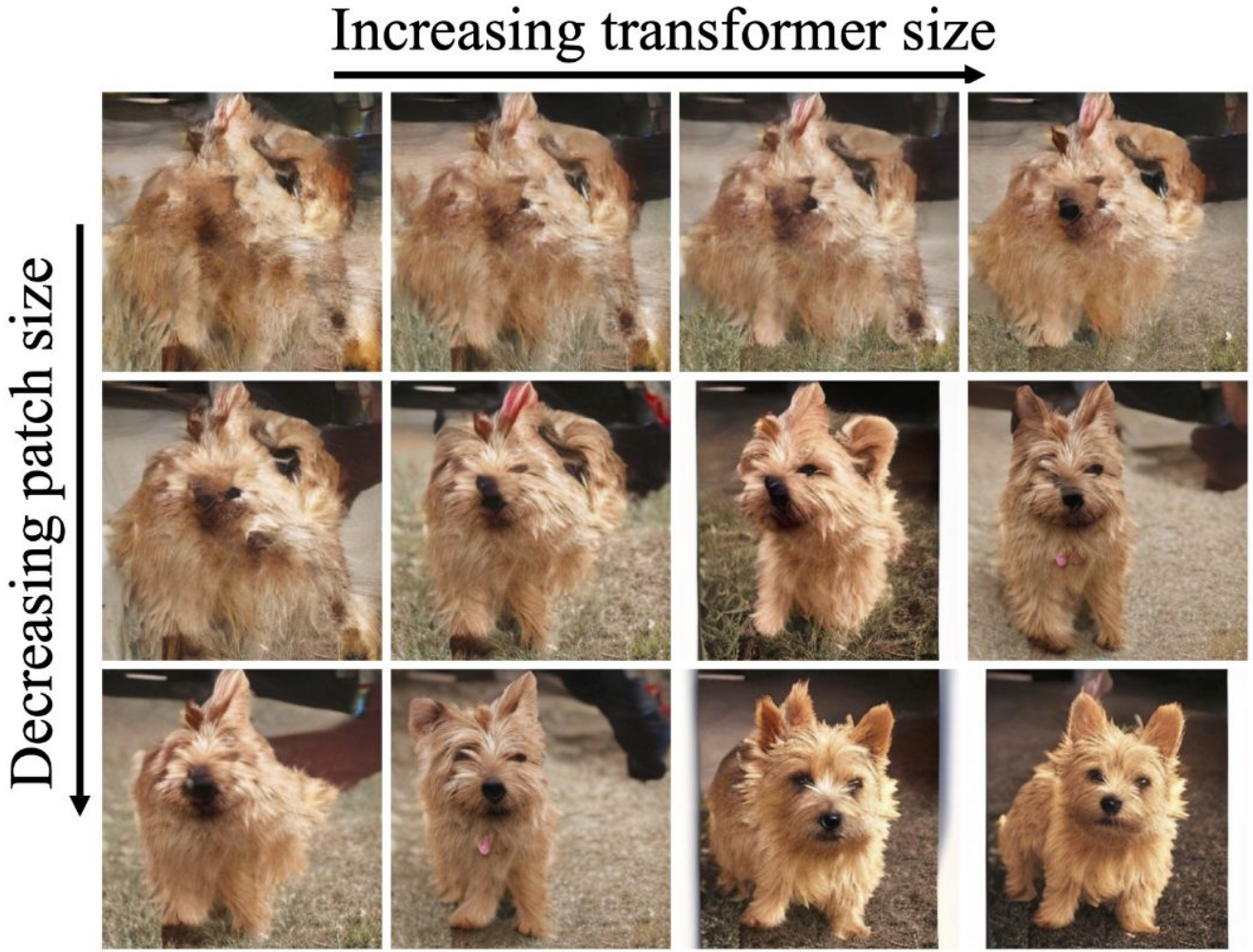
Takeaway

- The MLP that predicts γ, β, α is zero-initialized,
- Best FID among in-context, cross-attention and adaLN conditioning

Diffusion transformers results

paper →

Diffusion transformers: scaling



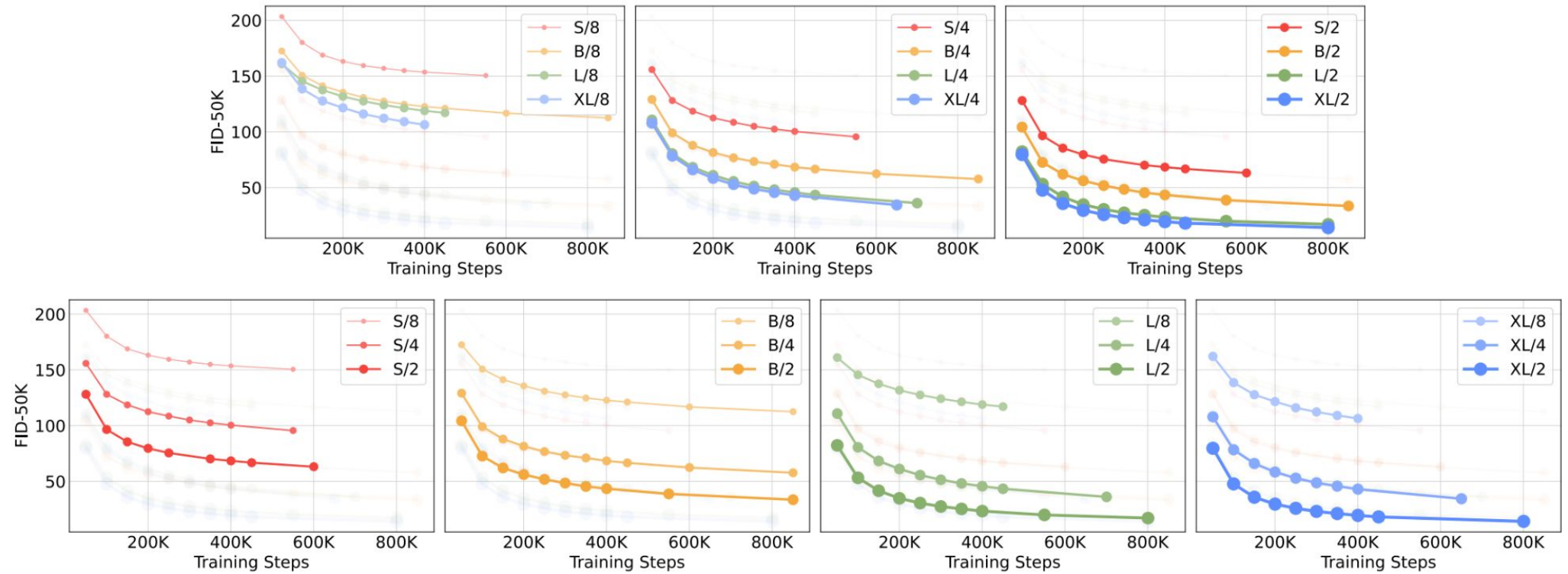
Two axes of scaling

- Bigger transformer: S → B → L → XL
- Smaller patches: more tokens for the same image

Takeaway

Sample quality improves along both axes: FID tracks model compute (Gflops), not just the number of parameters

Diffusion transformers scaling



DLS

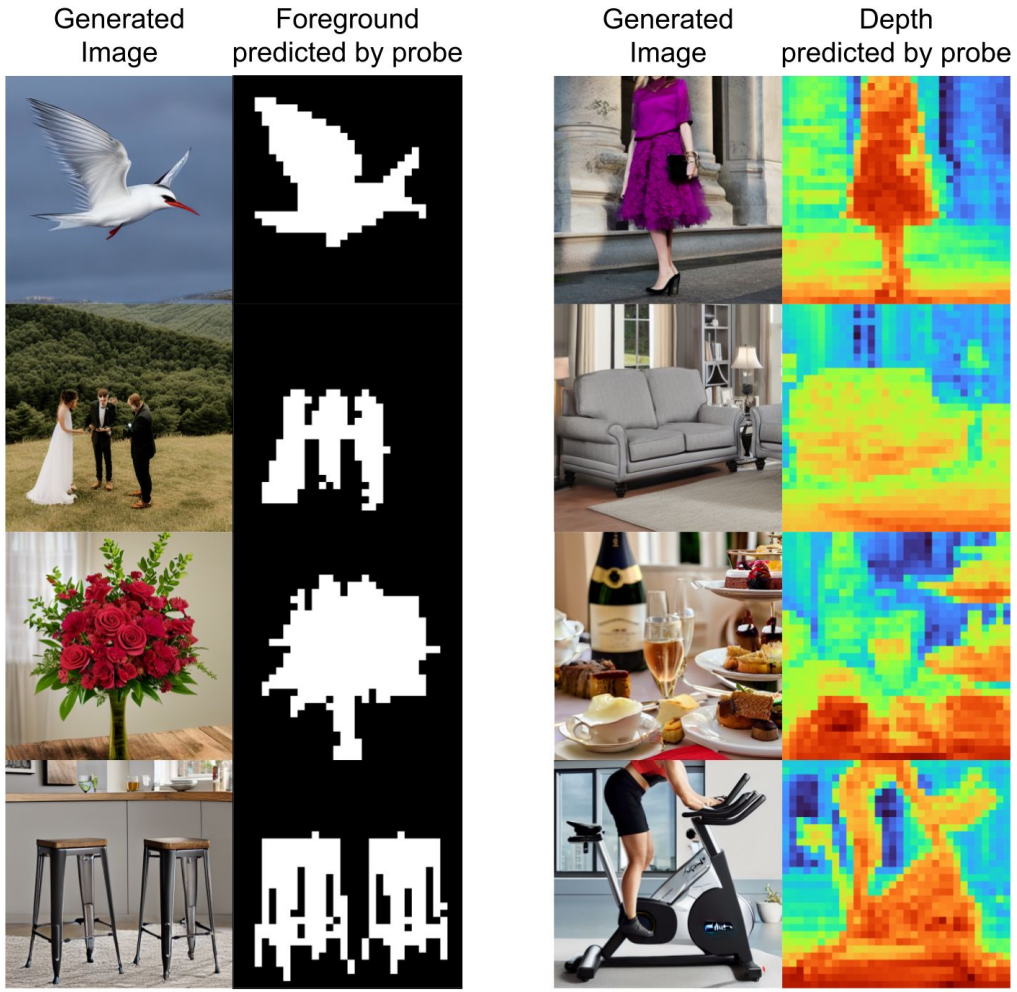
05

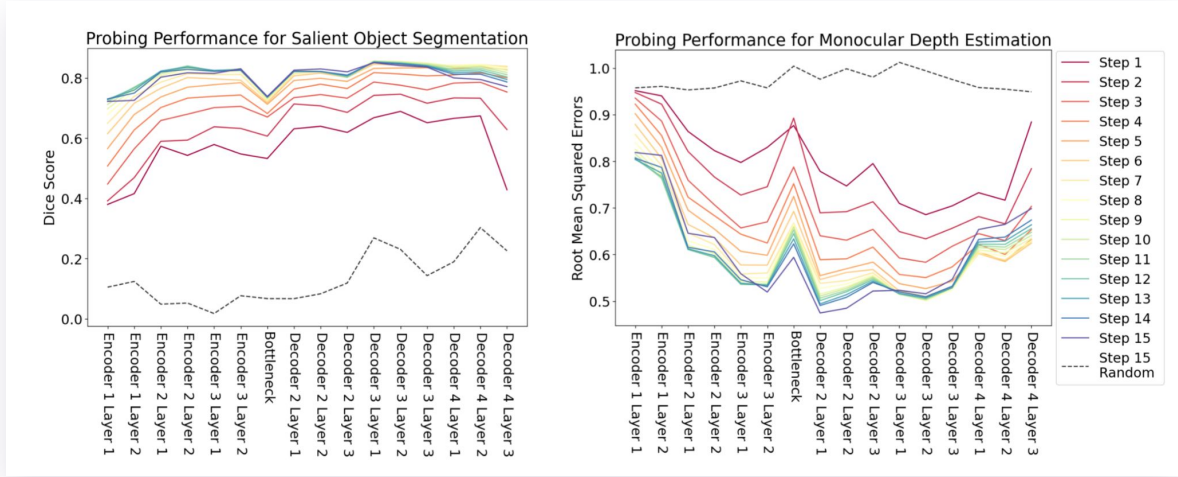
Improved training techniques

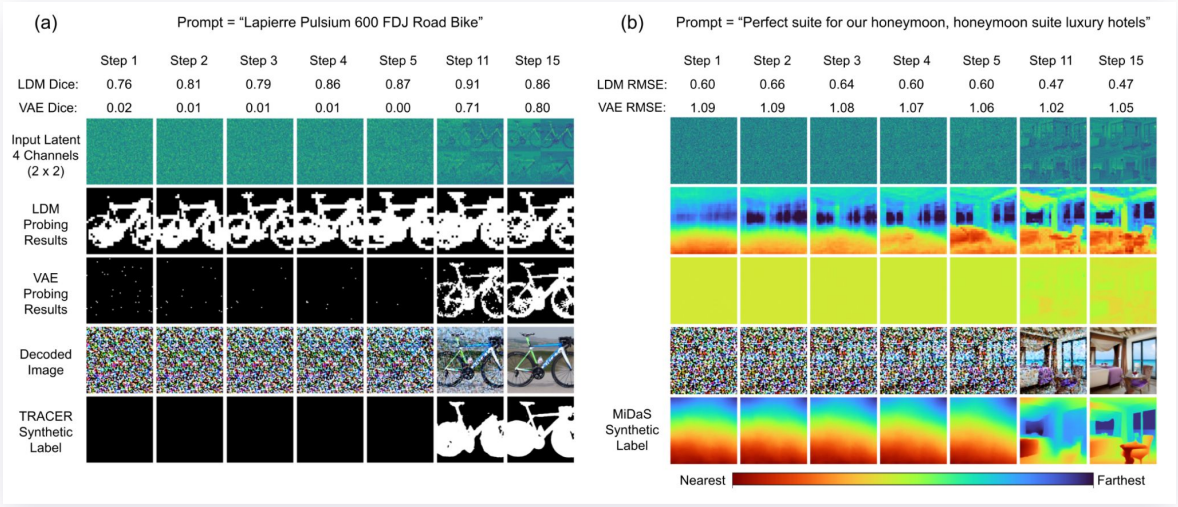
REPA, REG, Self-flow

paper →

Beyond Surface Statistics



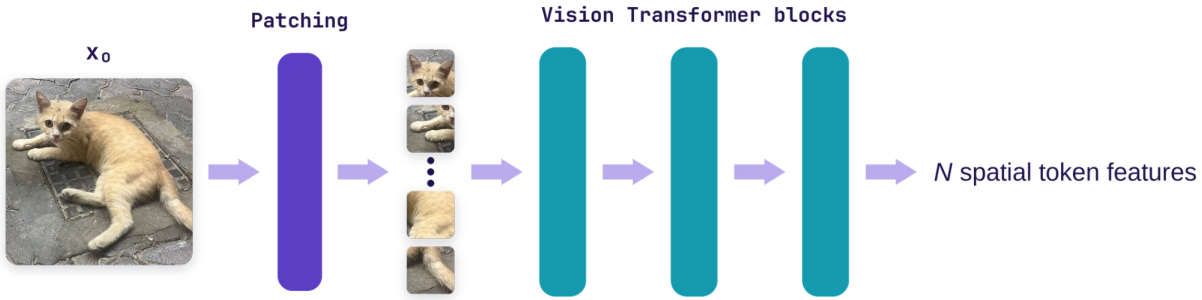


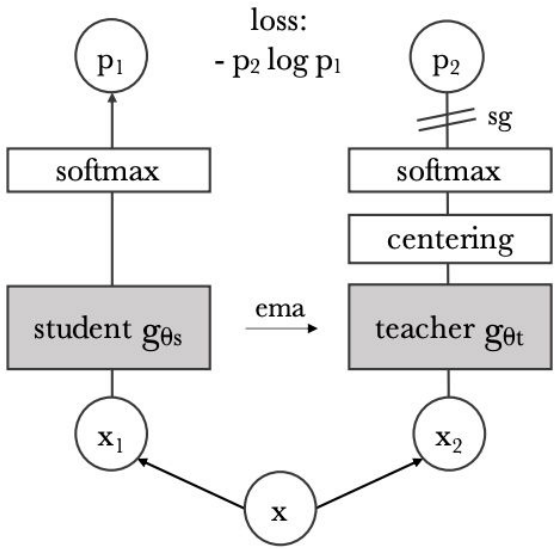


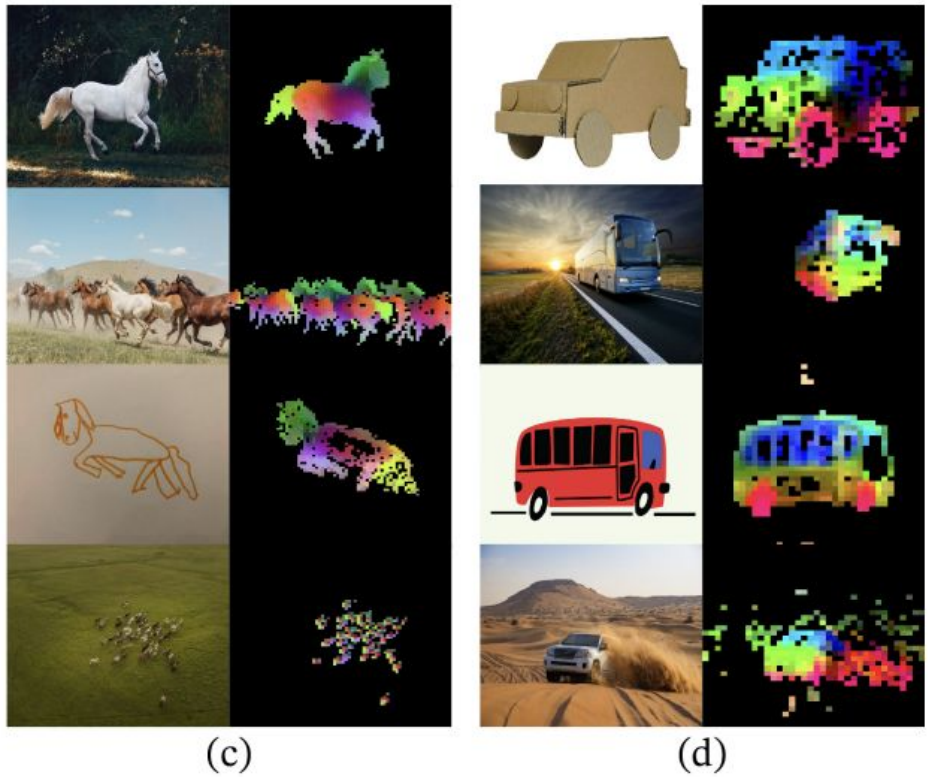
paper →

paper →

DINOv2 and beyond

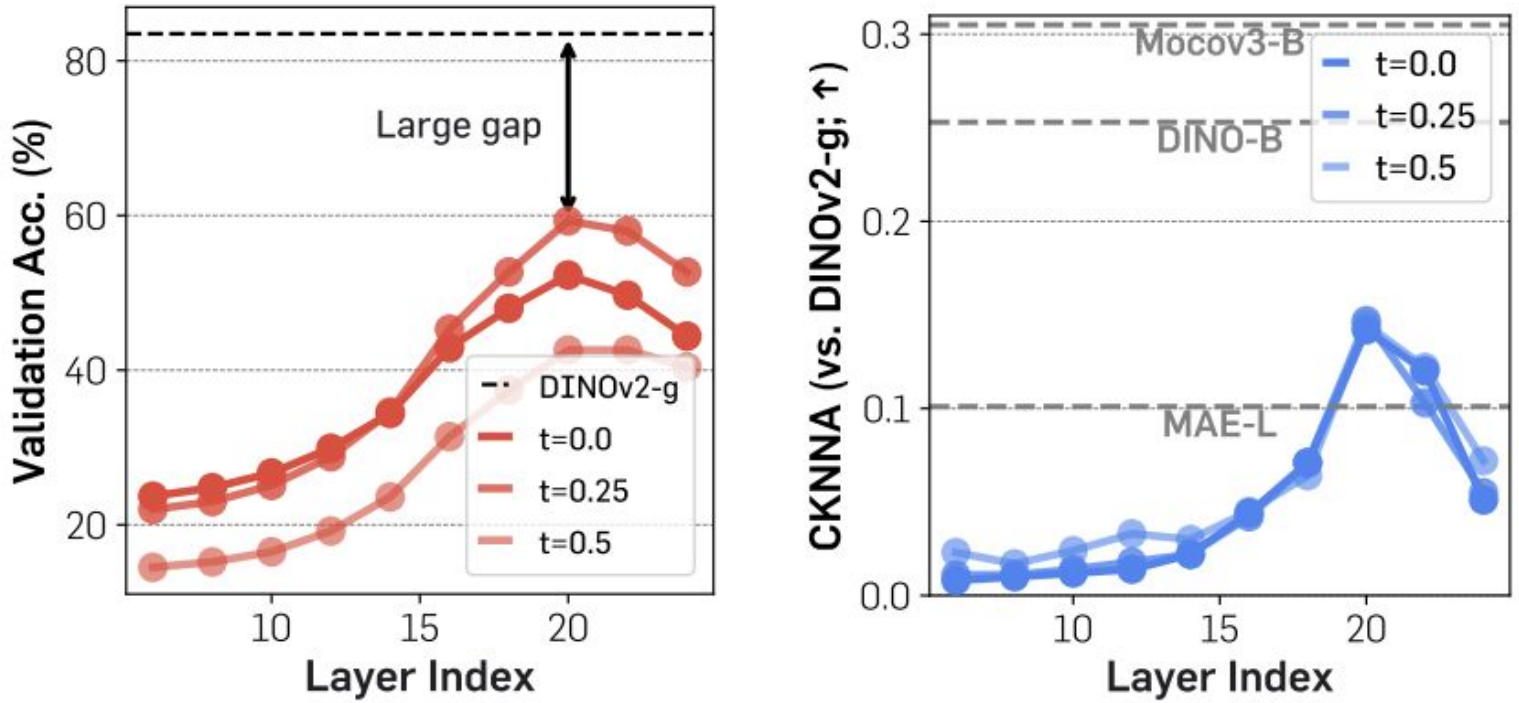






paper →

REPA: representation alignment



(a) Semantic gap: Linear probing

(b) Alignment to DINOv2-g

paper →

REPA: representation alignment

Idea

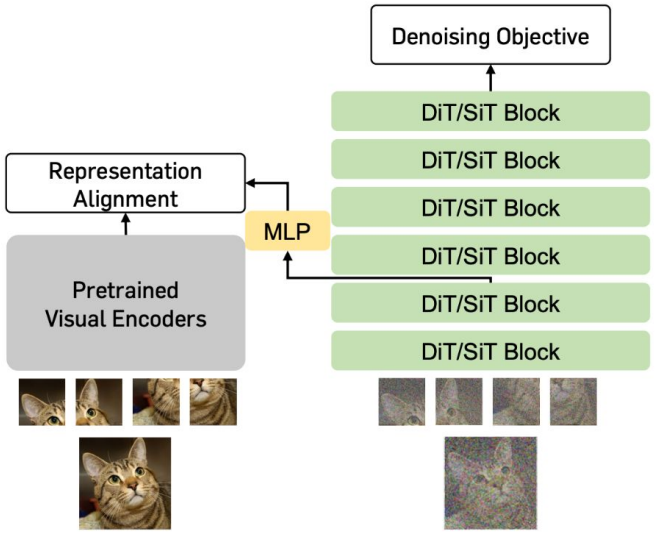
- Project an early hidden state of the noisy input with a small MLP
- Maximize patch-wise similarity to DINOv2 features of the clean image
- Encoder is frozen and used only during training

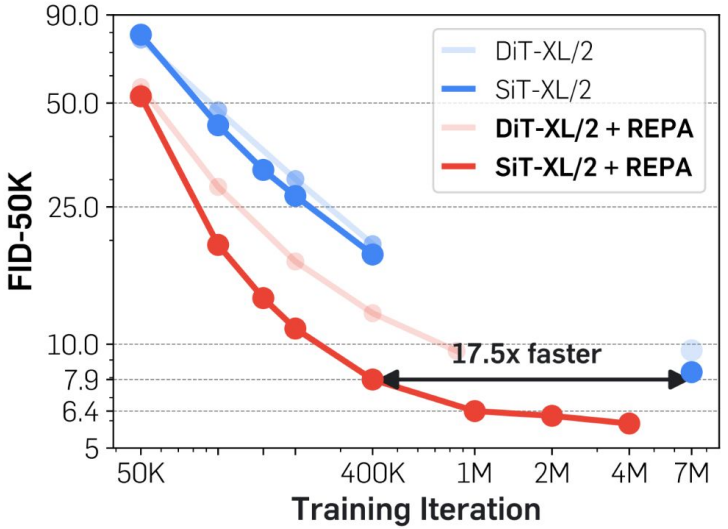
Objective

$$\mathcal{L} = \mathcal{L}_{\text{velocity}} + \lambda \mathcal{L}_{\text{REPA}}$$

$$\mathcal{L}_{\text{REPA}} = -\mathbb{E}\left[\frac{1}{N}\sum_{n=1}^{N}\text{sim}\left(y_*^{[n]},\, h_\phi(h_t^{[n]})\right)\right]$$

sim = cosine similarity, N = number of patches





paper →

REPA: representation alignment

Idea

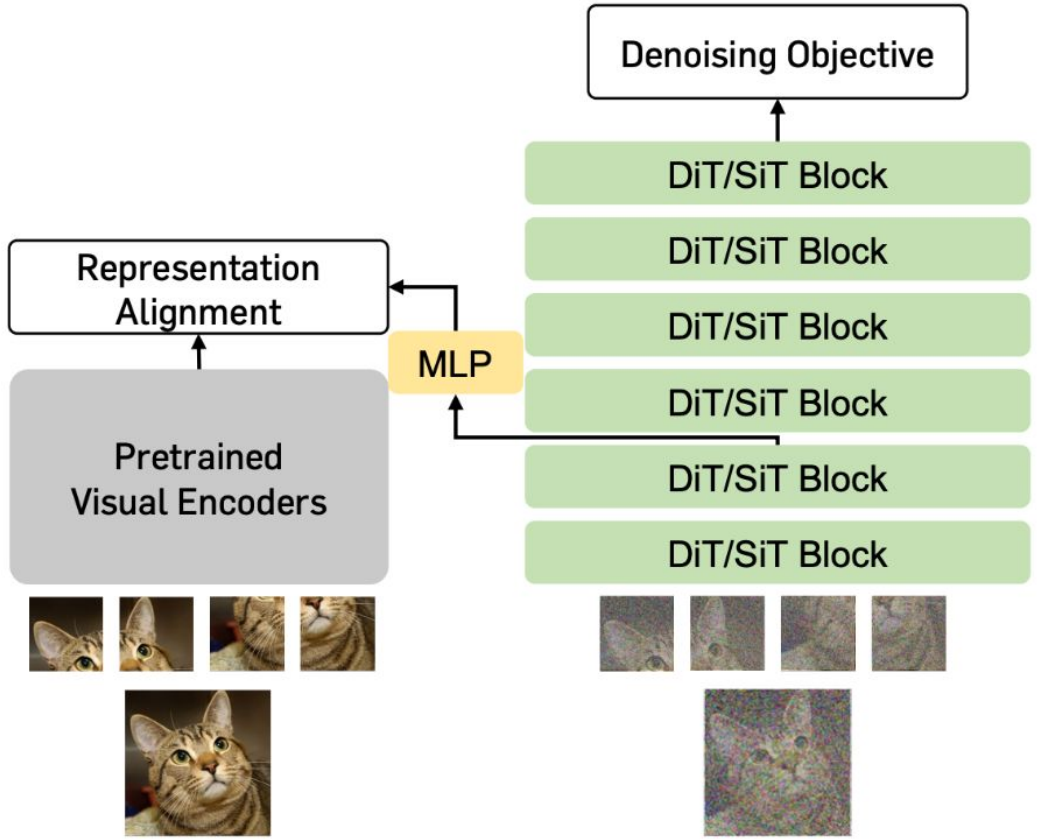
- Project an early hidden state of the noisy input with a small MLP
- Maximize patch-wise similarity to DINOv2 features of the clean image
- Encoder is frozen and used only during training

Objective

$$\mathcal{L} = \mathcal{L}_{\text{velocity}} + \lambda \mathcal{L}_{\text{REPA}}$$

$$\mathcal{L}_{\text{REPA}} = -\mathbb{E}\left[\frac{1}{N}\sum_{n=1}^{N}\text{sim}\left(y_*^{[n]}, h_\phi(h_t^{[n]})\right)\right]$$

sim = cosine similarity, N = number of patches



paper →

REPA: representation alignment

Idea

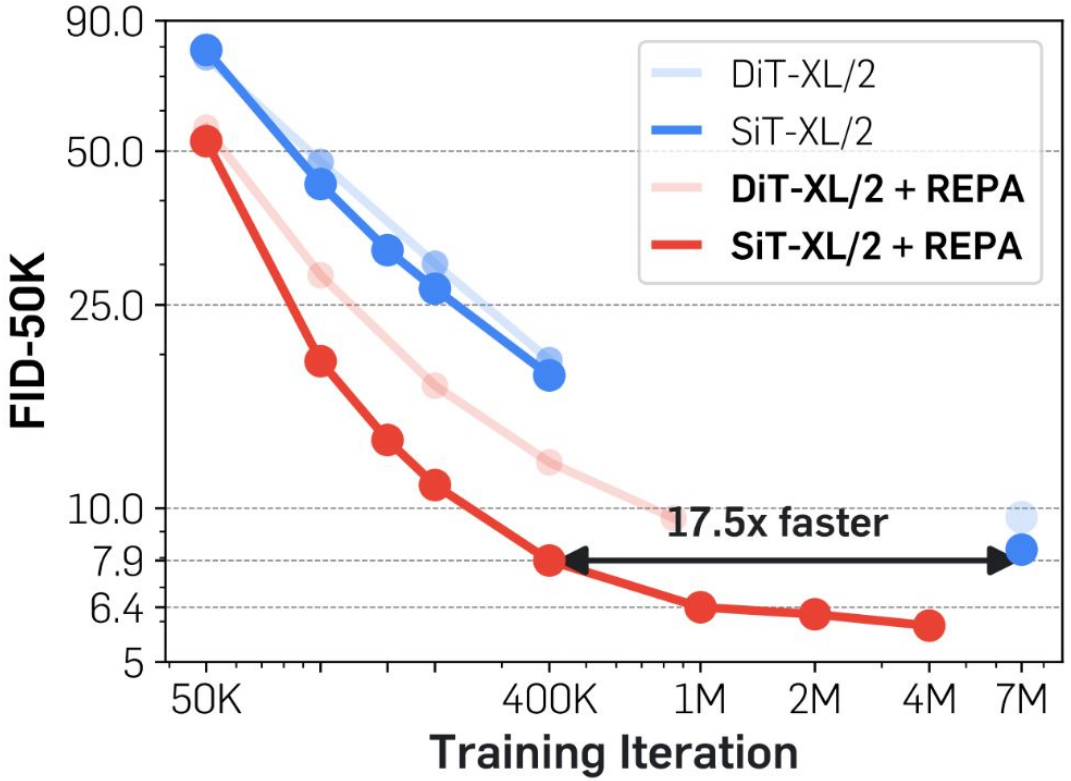
- Project an early hidden state of the noisy input with a small MLP
- Maximize patch-wise similarity to DINOv2 features of the clean image
- Encoder is frozen and used only during training

Objective

$$\mathcal{L} = \mathcal{L}_{\text{velocity}} + \lambda \mathcal{L}_{\text{REPA}}$$

$$\mathcal{L}_{\text{REPA}} = -\mathbb{E}\left[\frac{1}{N}\sum_{n=1}^{N}\text{sim}\left(y_*^{[n]}, h_\phi(h_t^{[n]})\right)\right]$$

sim = cosine similarity, N = number of patches



paper →

REPA: which encoder to align to?

Setup: SiT-L/2, 400K iterations, ImageNet 256

Encoders tried

MAE-L · DINO-B · MoCov3-L · I-JEPA-H · CLIP-L · SigLIP-L · DINOv2

Every encoder helps: FID 18.8 → 9.7–12.5.
DINOv2 is the best.

Findings

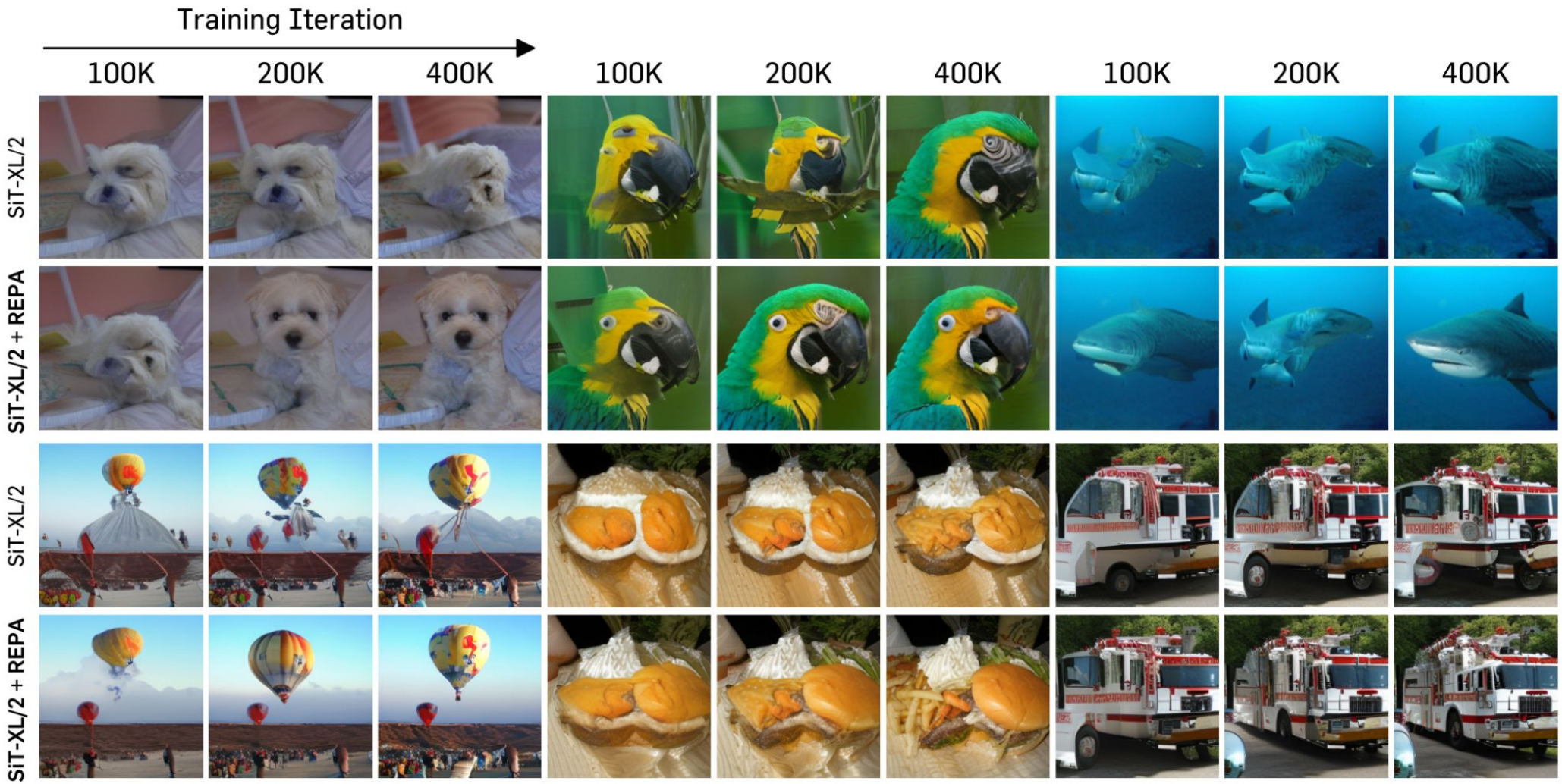
- Better encoder → better FID
- Align early: **depth ~8 works best,** deeper alignment hurts FID

Table 2: **Component-wise analysis** on ImageNet 256×256. All models are SiT-L/2 trained for 400K iterations. All metrics except accuracy (Acc.) are measured with the SDE Euler-Maruyama sampler with NFE=250 and without classifier-free guidance. For Acc., we report linear probing results on the ImageNet validation set using the latent features aligned with the target representation. We fix $\lambda = 0.5$ here. ↓ and ↑ indicate whether lower or higher values are better, respectively.

Iter.	Target Repr.	Depth	Objective	FID↓	sFID↓	IS↑	Pre.↑	Rec.↑	Acc.↑
400K	Vanilla SiT-L/2 (Ma et al., 2024a)			18.8	5.29	72.0	0.64	0.64	N/A
400K	MAE-L	8	NT-Xent	12.5	4.89	90.7	0.68	0.63	57.3
400K	DINO-B	8	NT-Xent	11.9	5.00	92.9	0.68	0.63	59.3
400K	MoCov3-L	8	NT-Xent	11.9	5.06	92.2	0.68	0.64	63.0
400K	I-JEPA-H	8	NT-Xent	11.6	5.21	98.0	0.68	0.64	62.1
400K	CLIP-L	8	NT-Xent	11.0	5.25	100.4	0.67	0.66	67.2
400K	SigLIP-L	8	NT-Xent	10.2	5.15	107.0	0.69	0.64	68.8
400K	DINOv2-L	8	NT-Xent	10.0	5.09	106.6	0.68	0.65	68.1
400K	DINOv2-B	8	NT-Xent	9.7	5.13	107.5	0.69	0.64	65.7
400K	DINOv2-L	8	NT-Xent	10.0	5.09	106.6	0.68	0.65	68.1
400K	DINOv2-g	8	NT-Xent	9.8	5.22	108.9	0.69	0.64	65.7
400K	DINOv2-L	6	NT-Xent	10.3	5.23	106.5	0.69	0.65	66.2
400K	DINOv2-L	8	NT-Xent	10.0	5.09	106.6	0.68	0.65	68.1
400K	DINOv2-L	10	NT-Xent	10.5	5.50	105.0	0.68	0.65	68.6
400K	DINOv2-L	12	NT-Xent	11.2	5.14	100.2	0.68	0.64	69.4
400K	DINOv2-L	14	NT-Xent	11.6	5.61	99.5	0.67	0.65	70.0
400K	DINOv2-L	16	NT-Xent	12.1	5.34	96.1	0.67	0.64	71.1
400K	DINOv2-L	8	NT-Xent	10.0	5.09	106.6	0.68	0.65	68.1
400K	DINOv2-L	8	Cos. sim.	9.9	5.34	111.9	0.68	0.65	68.2

paper →

REPA: results



paper →

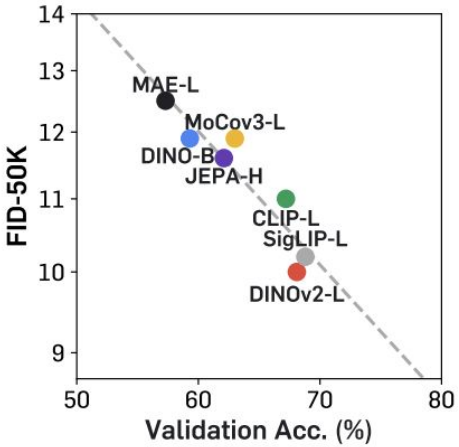
REPA: results

Training speeds up significantly

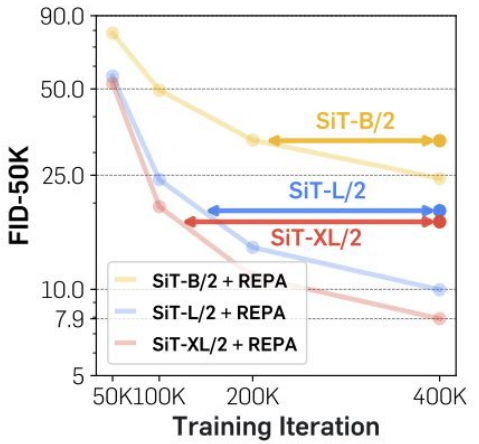
- SiT-XL/2 + REPA reaches in < 400K iterations the quality that plain SiT-XL/2 gets after 7M
- Holds for all sizes: SiT-B/2, L/2, XL/2
- Samples become sharp and coherent much earlier in training

In practice

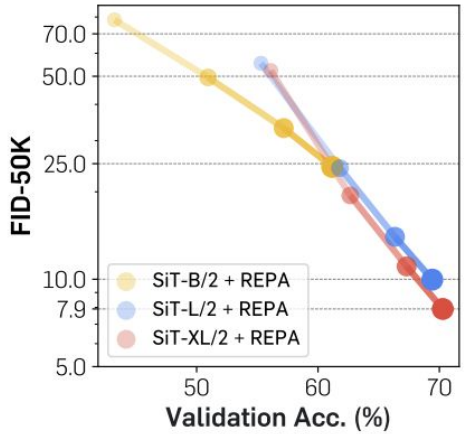
The REPA loss is used in the early phase of training and then switched off: later the fixed external target starts to limit the model



(a) Different visual encoders



(b) Relative convergence



(c) Validation acc. vs. FID

paper →

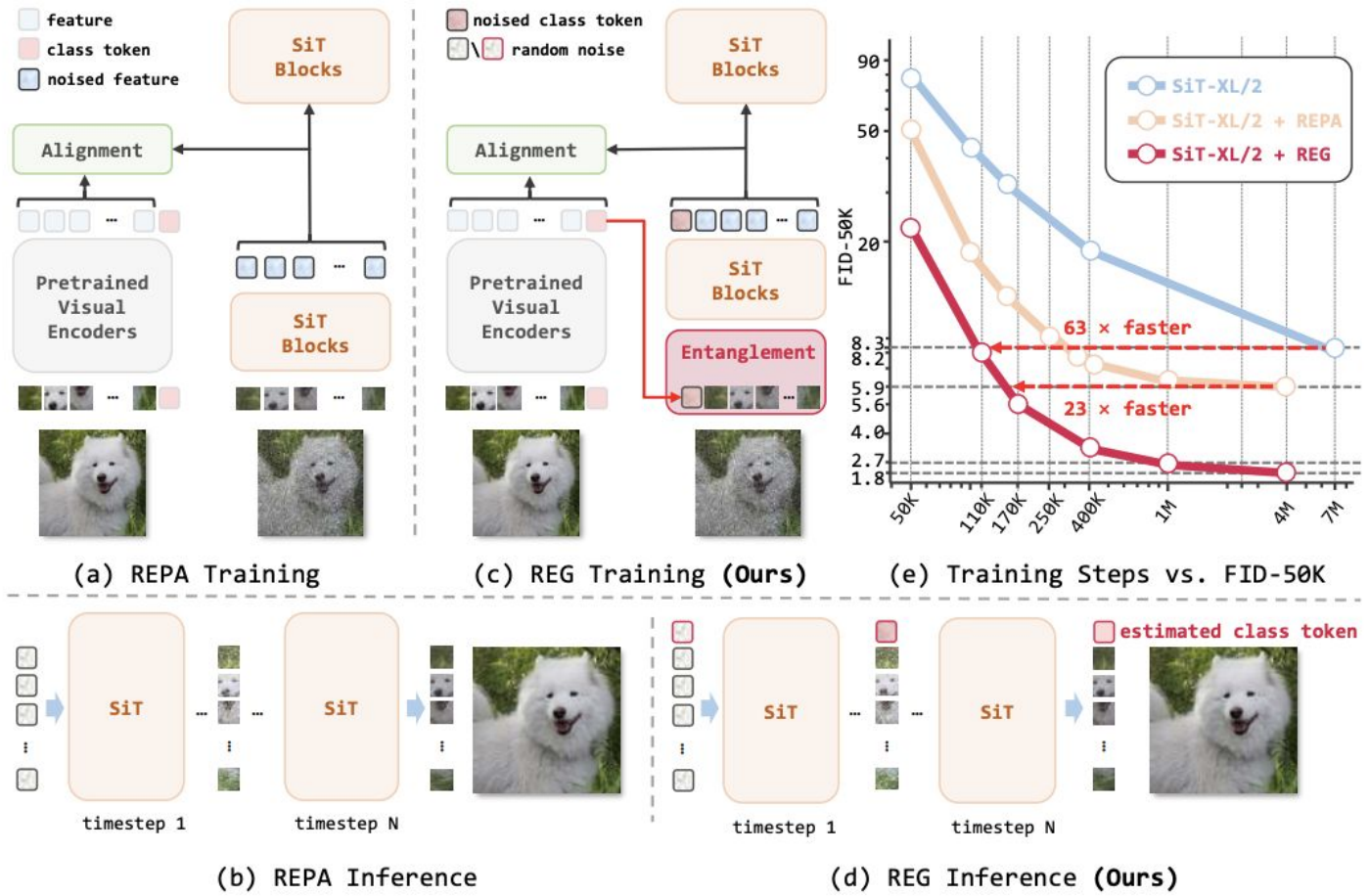
REG: representation entanglement

Idea

- Concatenate one class token from a pretrained encoder with the image latent tokens
- Noise both and denoise them jointly
- Keeps the REPA-style alignment loss

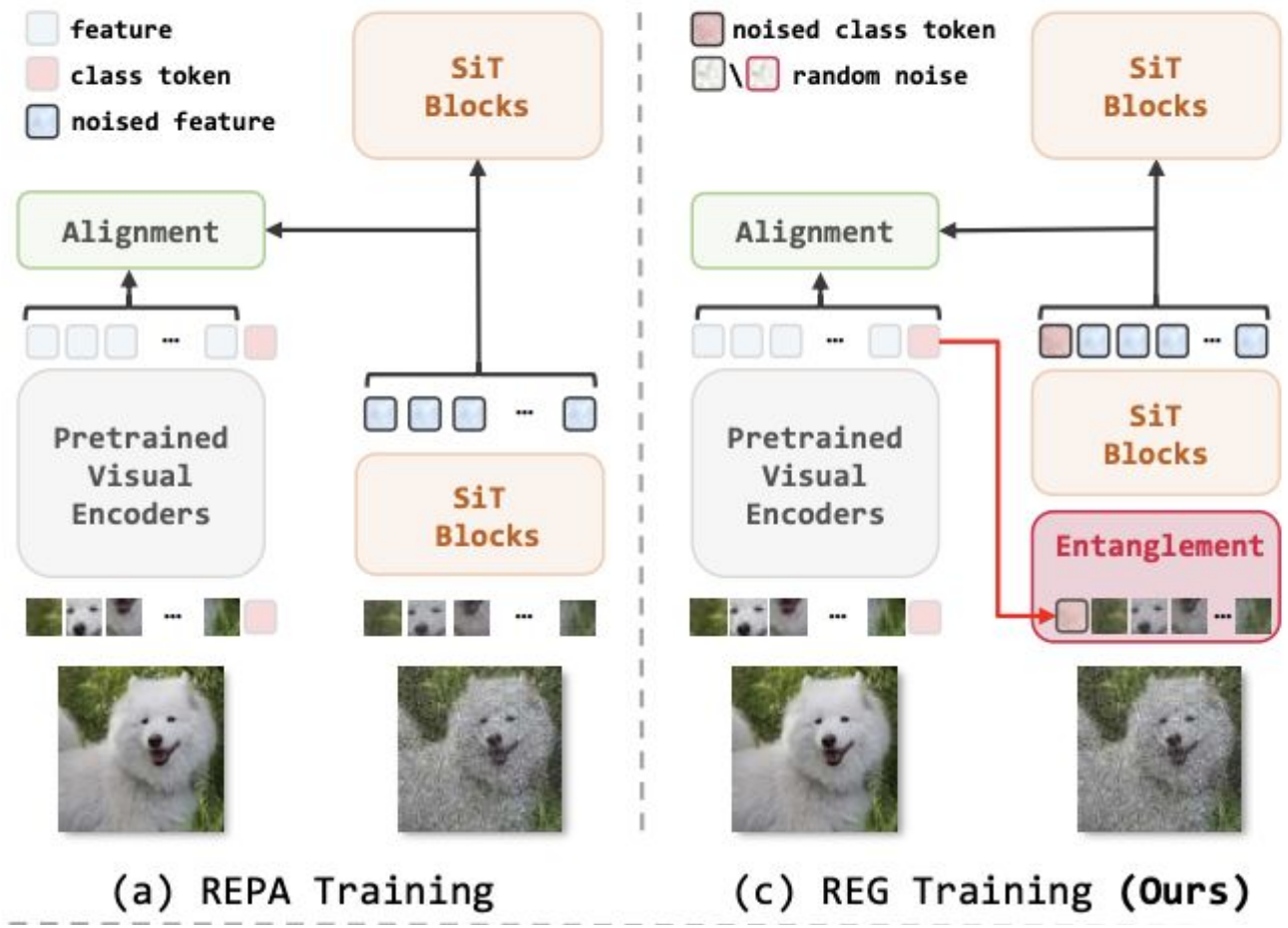
At inference

- The class token is generated from pure noise together with the image
- It provides global semantics that guide image generation
- Cost: one extra token, < 0.5% FLOPs and latency



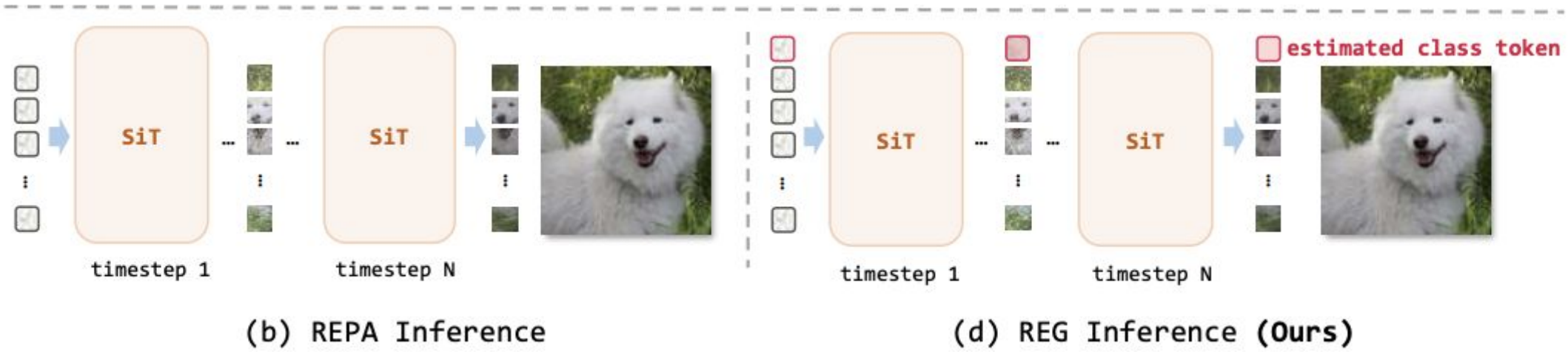
paper →

REG: representation entanglement



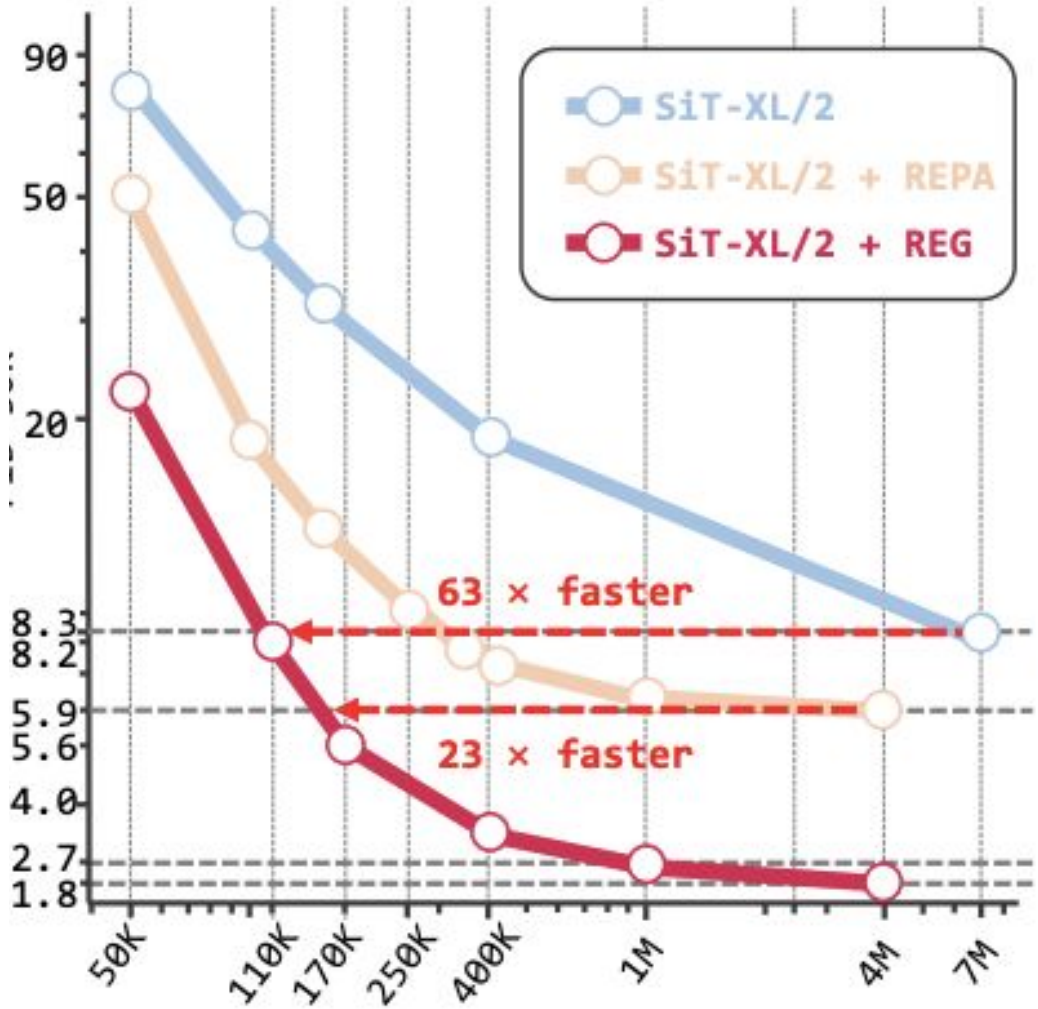
paper →

REG: representation entanglement



paper →

REG: representation entanglement



paper →

HASTE: REPA works until it doesn't

Observation

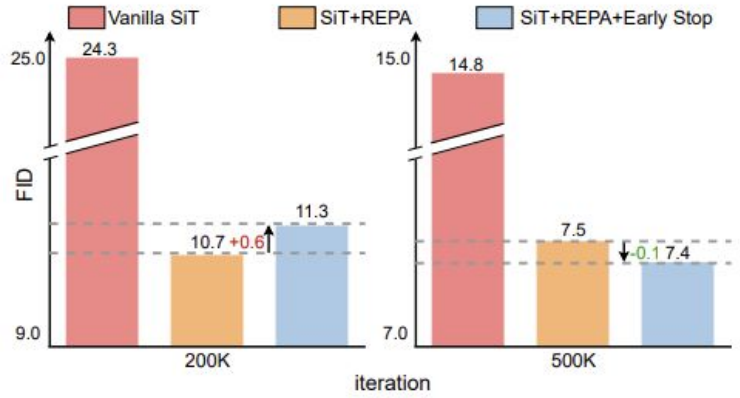
REPA slashes FID early, but its benefit fades: stopping it pays off later in training

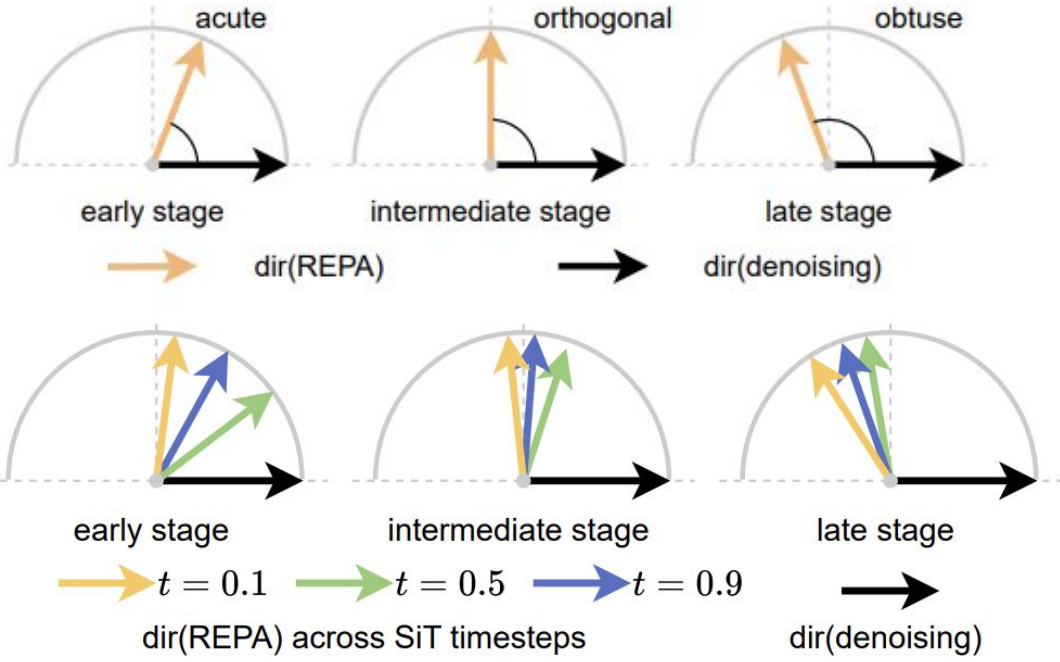
WHY: GRADIENT ANGLE

$$\rho_t = \cos\left(\nabla_\theta \mathcal{L}_{\text{diff}},\ \nabla_\theta \mathcal{L}_{\text{REPA}}\right)$$

acute	orthogonal	obtuse
REPA helps	no effect	conflict

early → intermediate → late training; same for all timesteps t





paper →

HASTE: alignment depends on the noise level

GRADIENT ANGLE VS. NOISE LEVEL

small t	larger t
fine details conflict appears	global structure REPA keeps helping longer

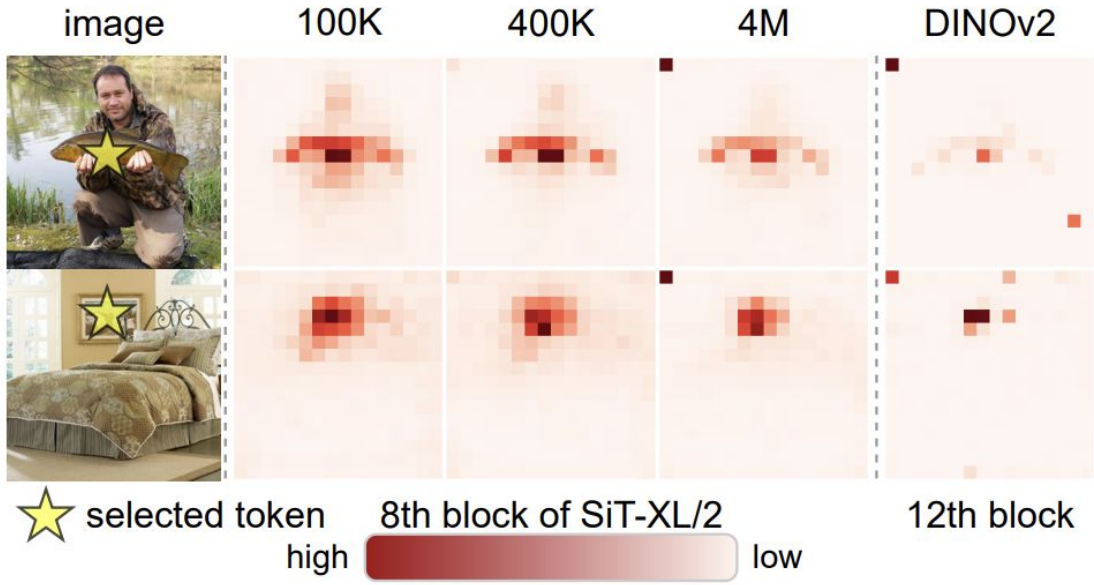
t = 0 clean → t = 1 pure noise

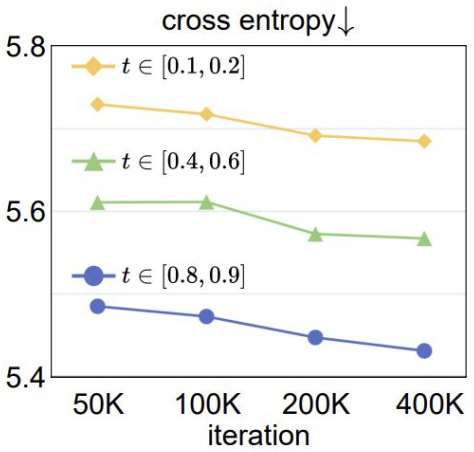
Where the conflict starts

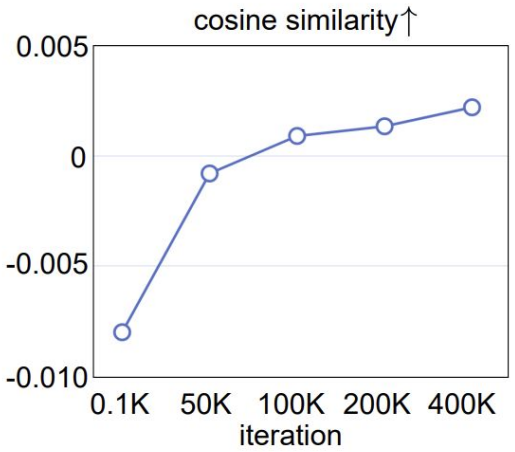
REPA and denoising gradients turn obtuse first at small t (≈ 0.1), where the input is almost clean

Why

Near t = 0 the model must learn fine details that DINOv2 features discard: the teacher becomes a straitjacket

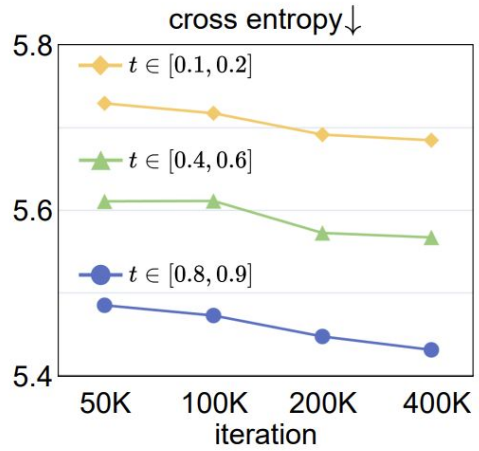


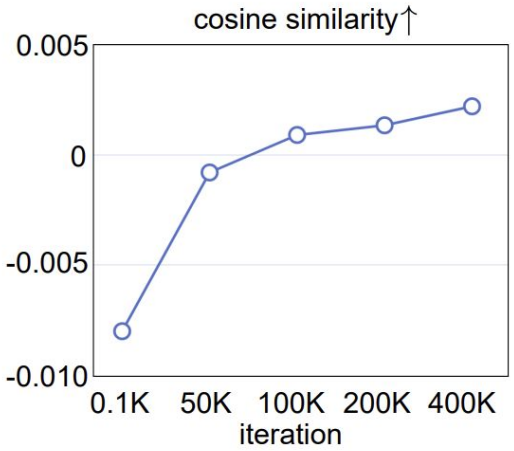




paper →

HASTE: alignment depends on the noise level





paper →

HASTE: the recipe

PHASE I · HOLISTIC ALIGNMENT

$$\mathcal{L} = \mathcal{L}_{\text{diff}} + \lambda_R \mathcal{L}_{\text{REPA}} + \lambda_A \mathcal{L}_{\text{ATTA}}$$

- **REPA:** features of one mid-level block are projected onto DINOv2 patch features (semantic anchors)
- **ATTA:** DINOv2 attention maps are distilled into mid-depth blocks (relational priors)

PHASE II · PLAIN DENOISING

$$\mathcal{L} = \mathcal{L}_{\text{diff}}$$

All auxiliary losses are dropped: the DiT is free to use its full generative capacity

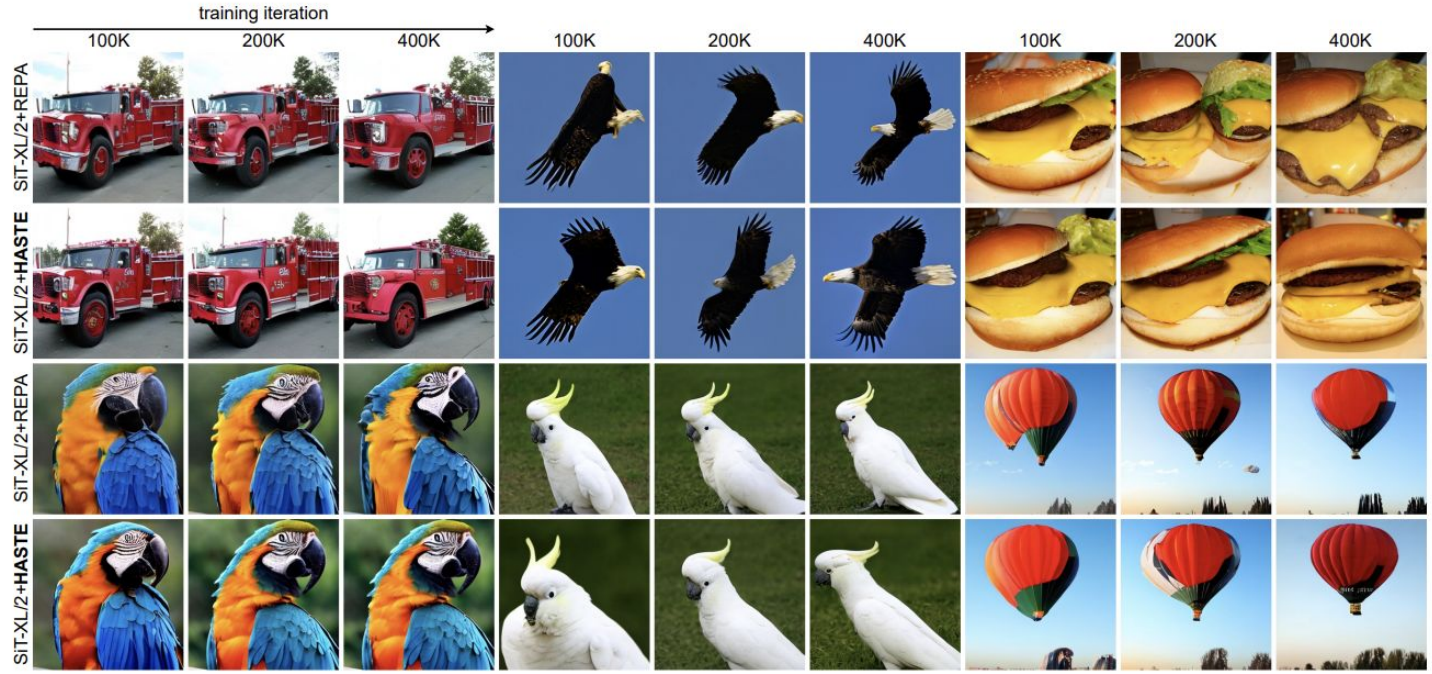


Figure 8: HASTE improves visual scaling. We compare images generated by SiT-XL/2+REPA and SiT-XL/2+HASTE (ours) at different training iterations. For both models, we use the same seed, noise, and sampling method with a classifier-free guidance scale of 4.0.

paper →

REPA, REG: main problems

Doesn't scale to large text-to-image models

- A stronger encoder does not mean better generation: swapping DINOv2-B for bigger encoders can make results worse
- The target is fixed: as the generator grows, the external features start to limit it

Requires an additional big vision encoder

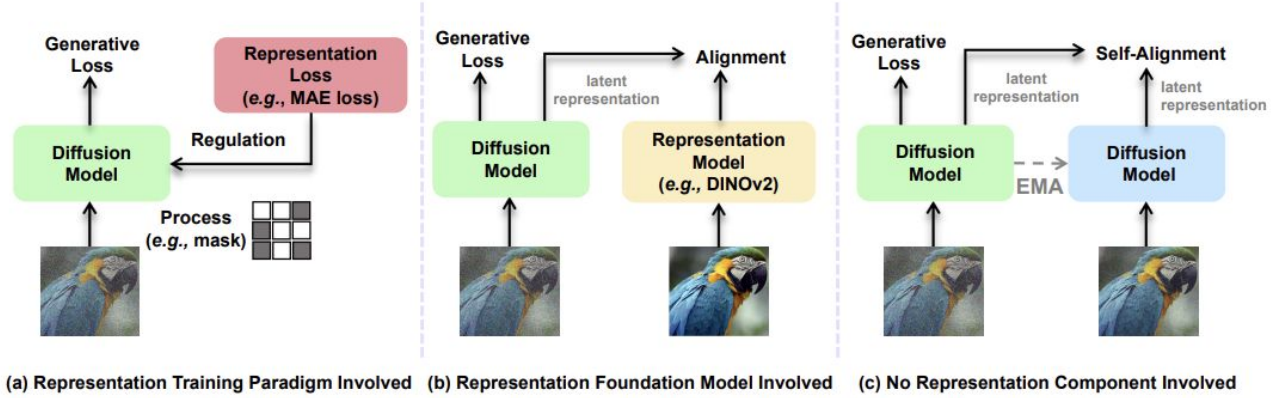
- Extra memory and compute: a frozen ViT runs on every training batch
- Trained separately, with a different objective than generation

paper →

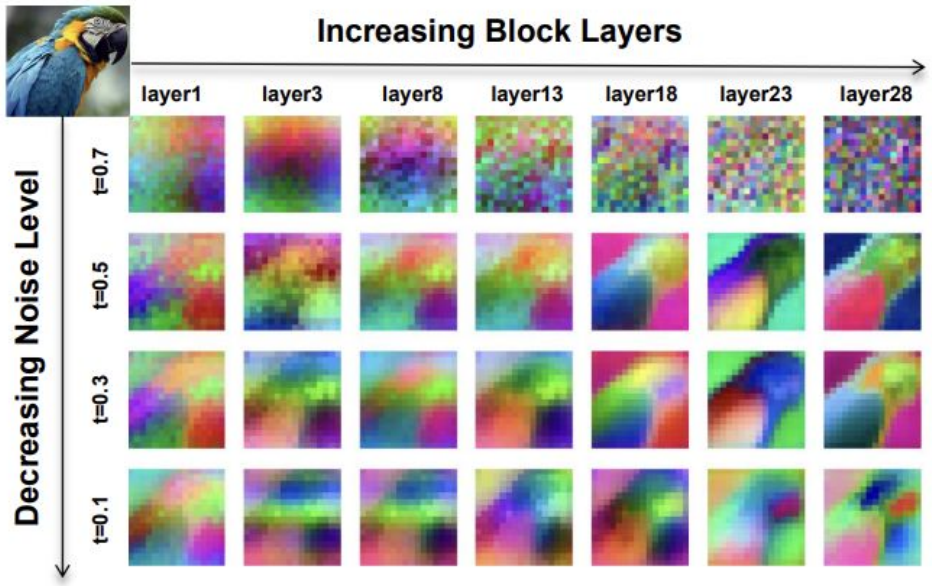
SRA: the DiT guides itself

Idea

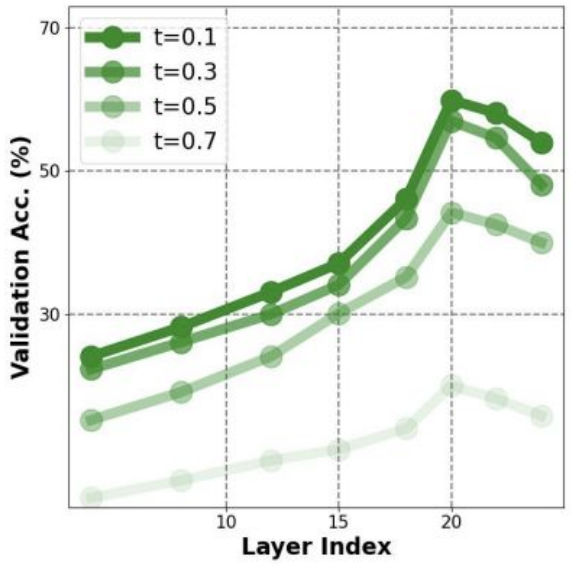
No external encoder: the model's own later layers at lower noise serve as the teacher



(a) Representation Training Paradigm Involved (b) Representation Foundation Model Involved (c) No Representation Component Involved



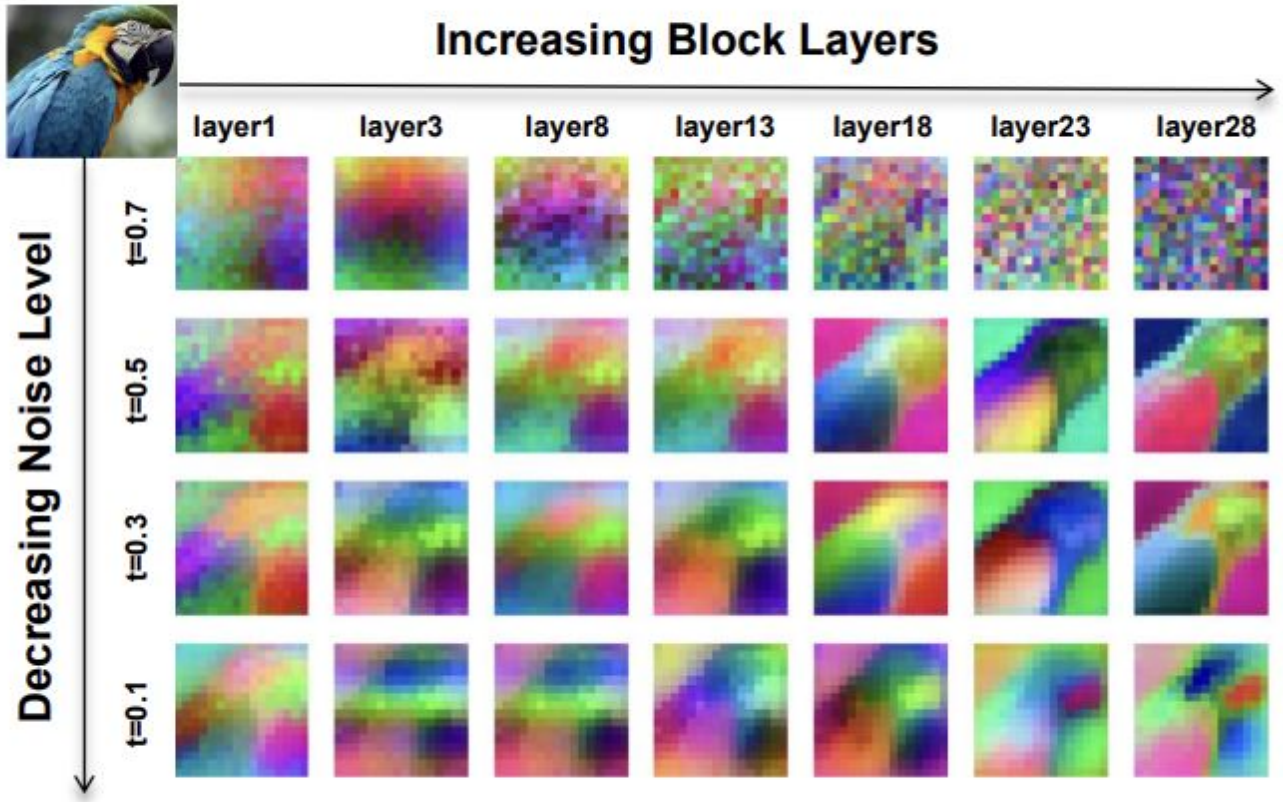
(a) PCA Visualization Result



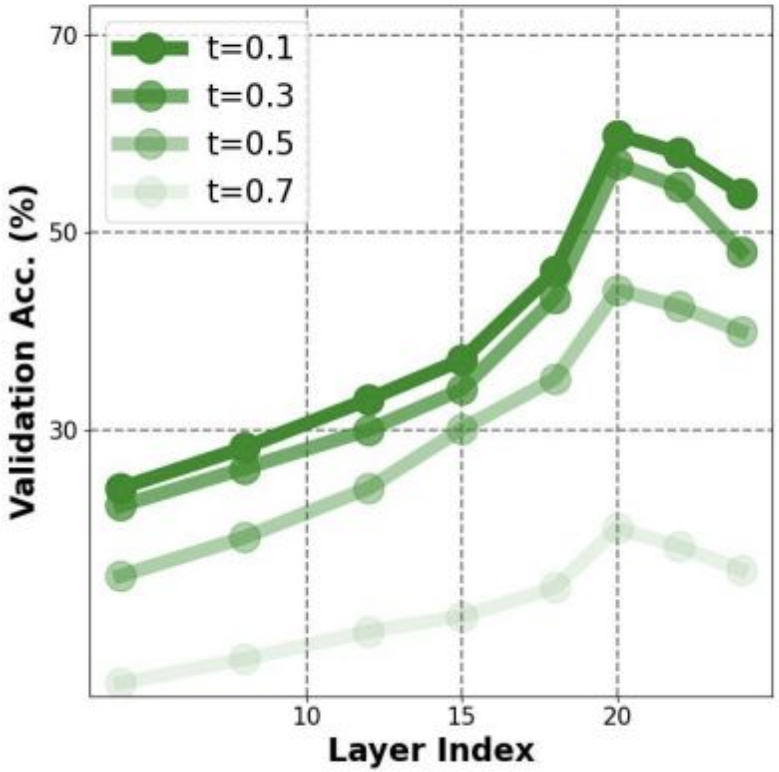
(b) Linear Probing Result

paper →

SRA: the DiT guides itself



(a) PCA Visualization Result



(b) Linear Probing Result

paper →

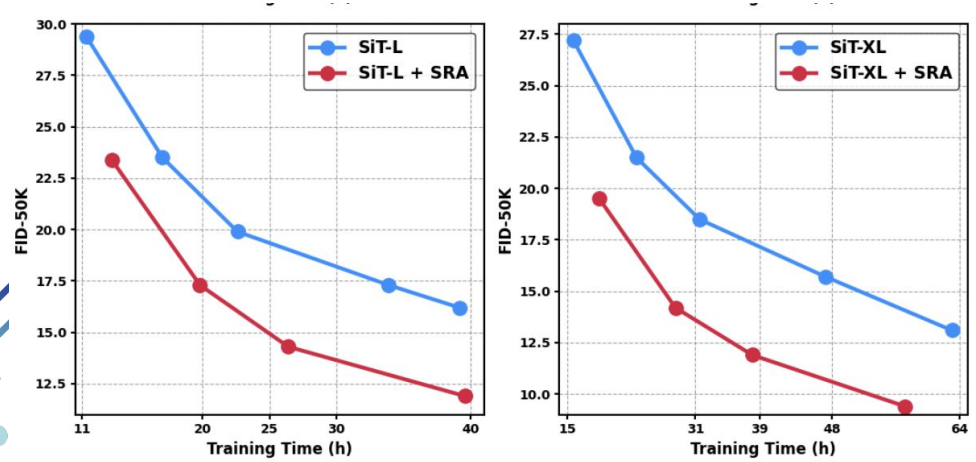
SRA: the DiT guides itself

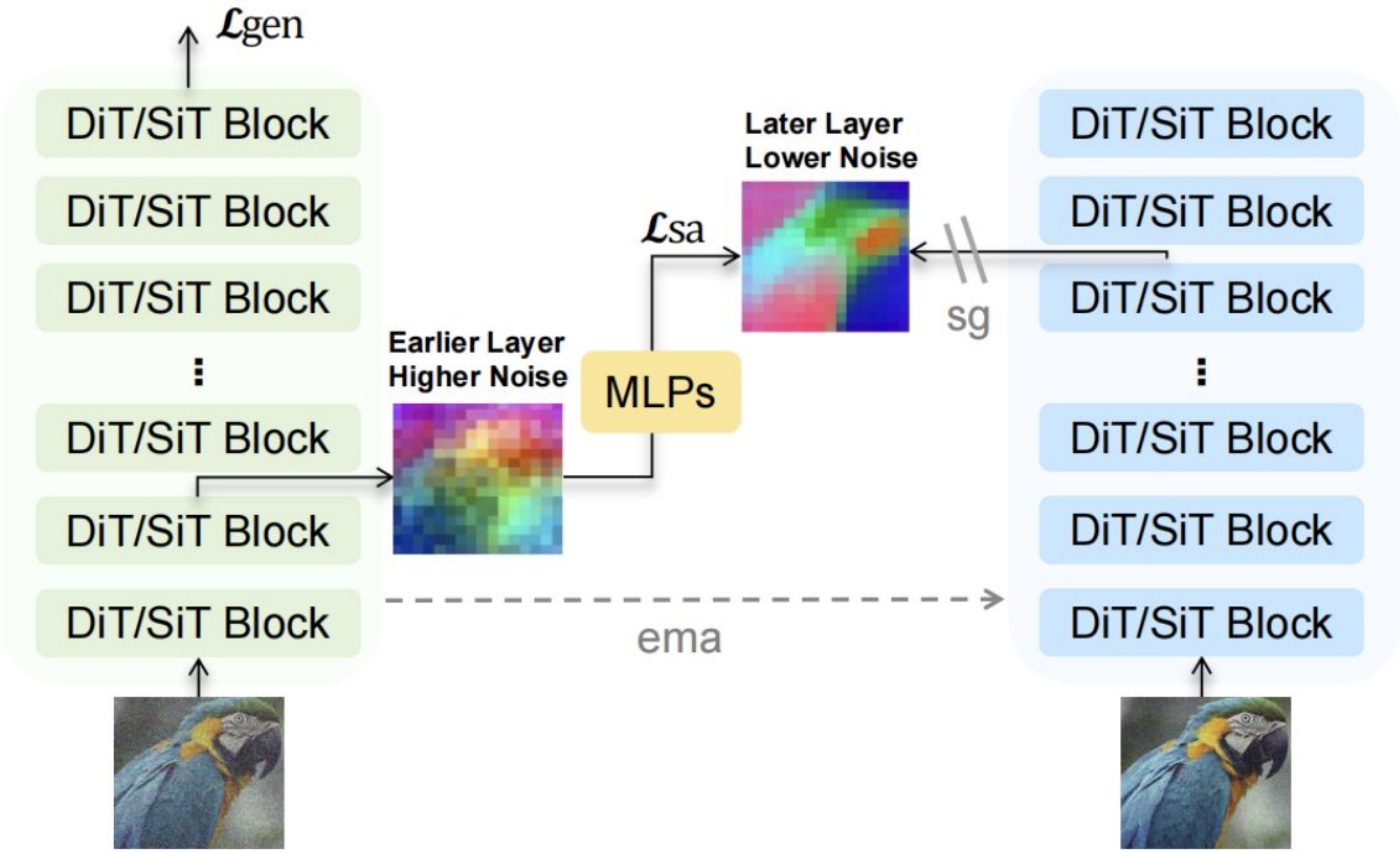
Idea

No external encoder: the model's own later layers at lower noise serve as the teacher

Details

- Patch-wise distance between student and teacher
- Teacher = EMA of the same DiT
- Sensitive to the choice of layers and of the noise gap





paper →

SRA: the DiT guides itself

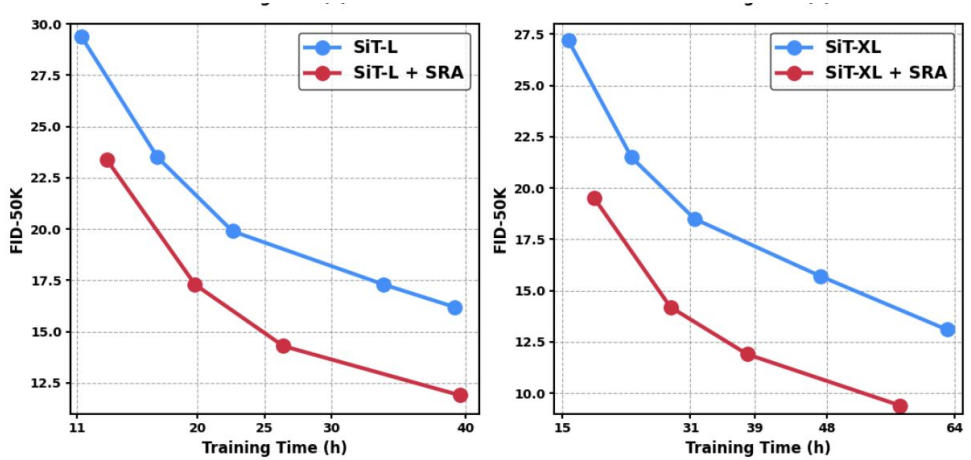


Table 2: FID and Pickscore [43] comparisons with vanilla MMDiT and REPA on COCO2014.

Method	FID↓	PickScore↑
ODE, NFE=50, Trained for 150K iter		
MMDiT	5.86	20.05
MMDiT + REPA	4.60	20.88
MMDiT + SRA	4.85	21.14

Block Layers	Time Interval	λ	PH.	FID↓	IS↑
SiT-B/2 Baseline [59]				33.02	43.71
$6 \rightarrow 10$	$[0, 0.2)$	0.2	✓	34.85	40.28
$4 \rightarrow 8$	$[0, 0.2)$	0.2	✓	30.00	47.78
$4 \rightarrow 10$	$[0, 0.2)$	0.2	✓	30.65	46.69
$4 \rightarrow 12$	$[0, 0.2)$	0.2	✓	33.19	43.30
$2 \rightarrow 6$	$[0, 0.2)$	0.2	✓	32.14	46.36
$2 \rightarrow 8$	$[0, 0.2)$	0.2	✓	29.31	50.13
$3 \rightarrow 8$	$[0, 0.2)$	0.2	✓	**29.10**	**50.20**
$3 \rightarrow 3$	$[0, 0.2)$	0.2	✓	37.08	41.54
$3 \rightarrow 8$	0.0	0.2	✓	31.07	47.32
$3 \rightarrow 8$	0.1	0.2	✓	29.55	49.01
$3 \rightarrow 8$	0.2	0.2	✓	30.70	47.72
$3 \rightarrow 8$	$[0, 0.1)$	0.2	✓	29.38	49.32
$3 \rightarrow 8$	$[0, 0.2)$	0.2	✓	**29.10**	**50.20**
$3 \rightarrow 8$	$[0, 0.3)$	0.2	✓	29.15	50.01
$3 \rightarrow 8$	$[0, 0.2)$	0.1	✓	30.65	48.31
$3 \rightarrow 8$	$[0, 0.2)$	0.2	✓	**29.10**	**50.20**
$3 \rightarrow 8$	$[0, 0.2)$	0.3	✓	29.28	49.72
$3 \rightarrow 8$	$[0, 0.2)$	0.4	✓	29.75	49.30
$3 \rightarrow 8$	$[0, 0.2)$	0.2	×	34.23	41.07
$3 \rightarrow 8$	$[0, 0.2)$	0.2	✓	**29.10**	**50.20**

paper →

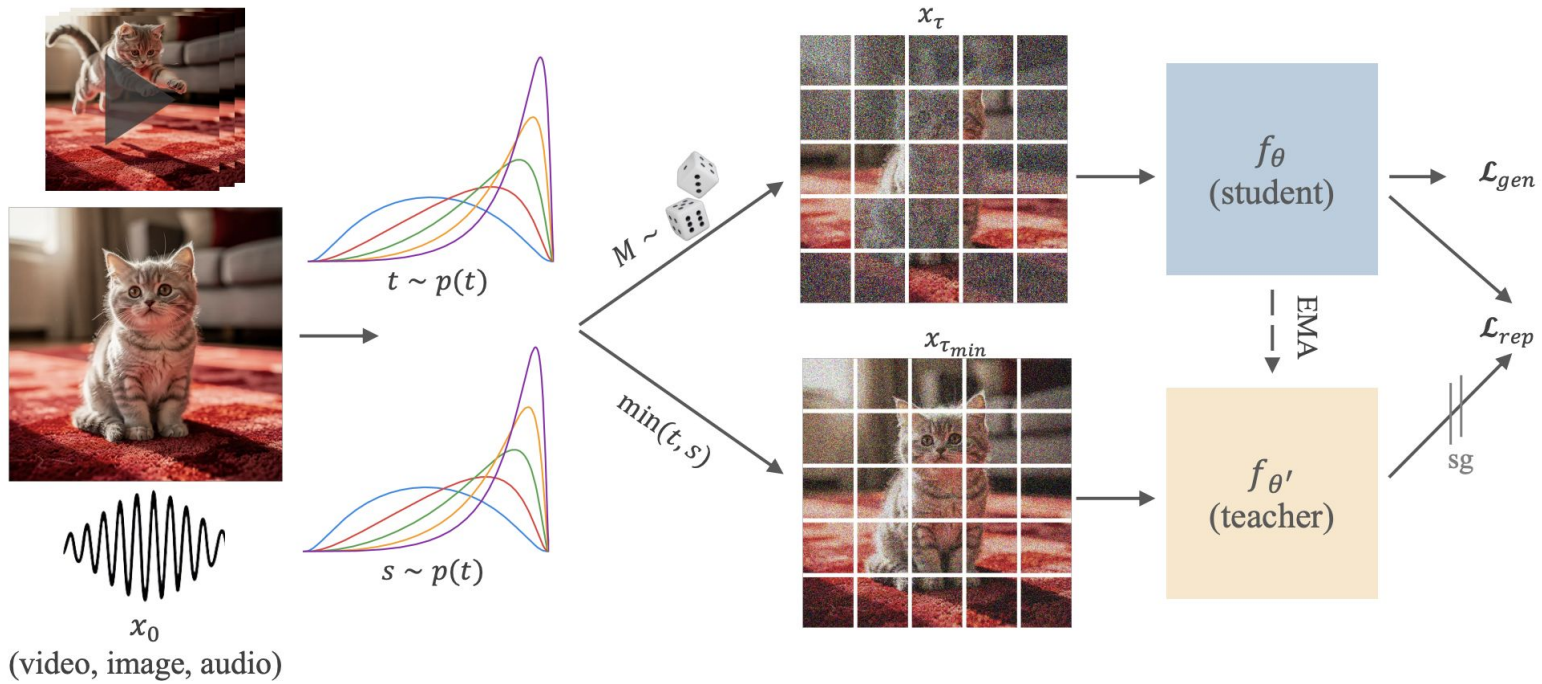
Self-Flow: self-supervised flow matching

Dual-timestep scheduling

- Sample two noise levels t and s
- A random mask M assigns them to different tokens
- Information asymmetry: noisy tokens must be inferred from cleaner context

Self-distillation

- EMA teacher sees the cleaner input, noised with τ_min = min(t, s)
- The student predicts the teacher's internal features
- Trained jointly with the usual flow-matching loss

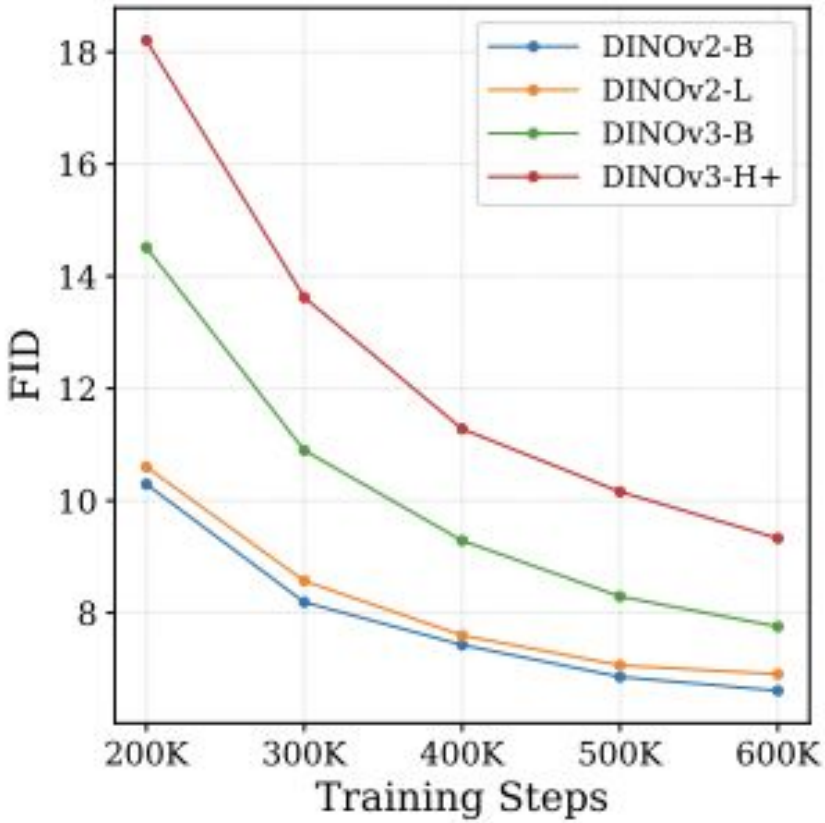


paper →

Self-Flow: results

Scaling

- Swapping DINOv2-B for stronger encoders makes REPA worse
- Self-Flow keeps improving with model size



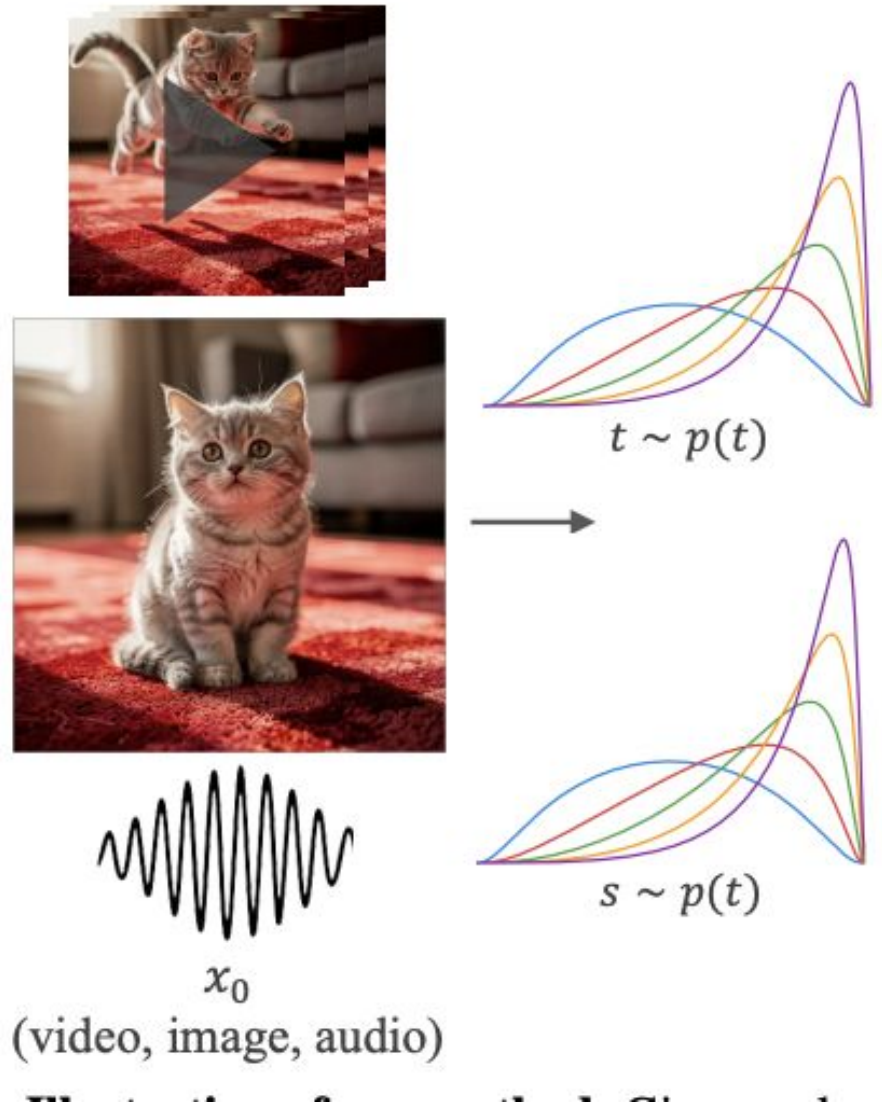
(a) REPA scaling

paper →

Self-Flow: results

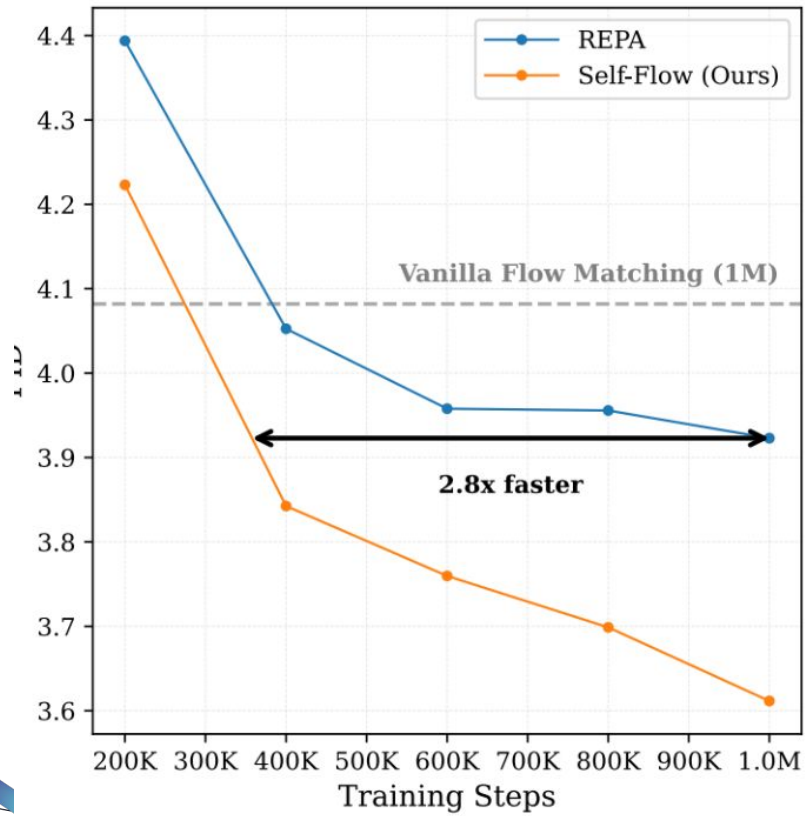
Beyond images

- Consistent gains for text-to-image, video and audio; video encoders used for alignment even hurt



paper →

Self-Flow: results



(a) Text-to-image FID

(b) Image and video generations by our multi-modal model

paper →

Self-Flow: results

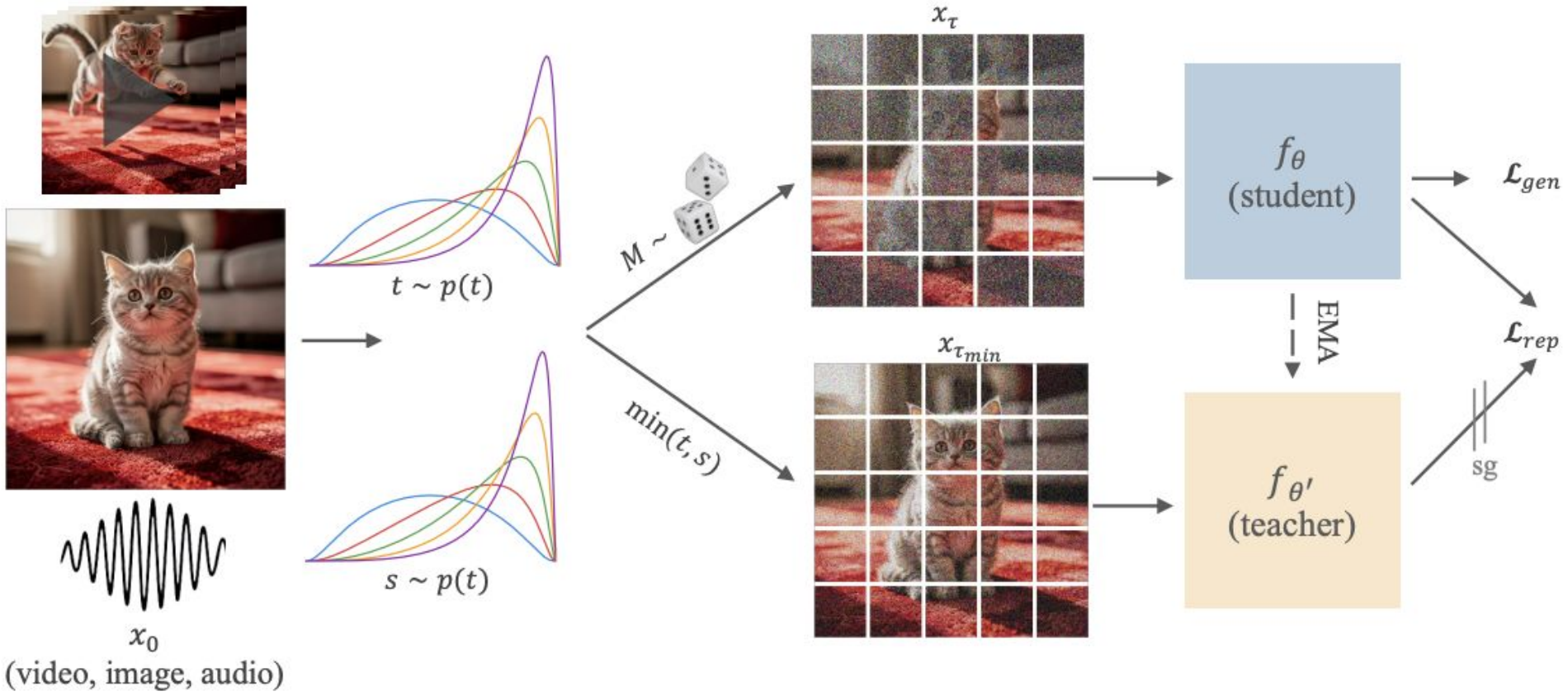


Figure 3. **Illustration of our method.** Given a clean input x_0, we draw two timesteps t, s, and a random mask M, then noise each token according to its assigned timestep. The teacher input is noised with $\tau_{min} = \min\{t, s\}$, creating an information asymmetry compared to the student. The student is trained to simultaneously denoise the input and reconstruct the teacher's features given its mixed-noised view.

paper →

TREAD: token routing

Idea

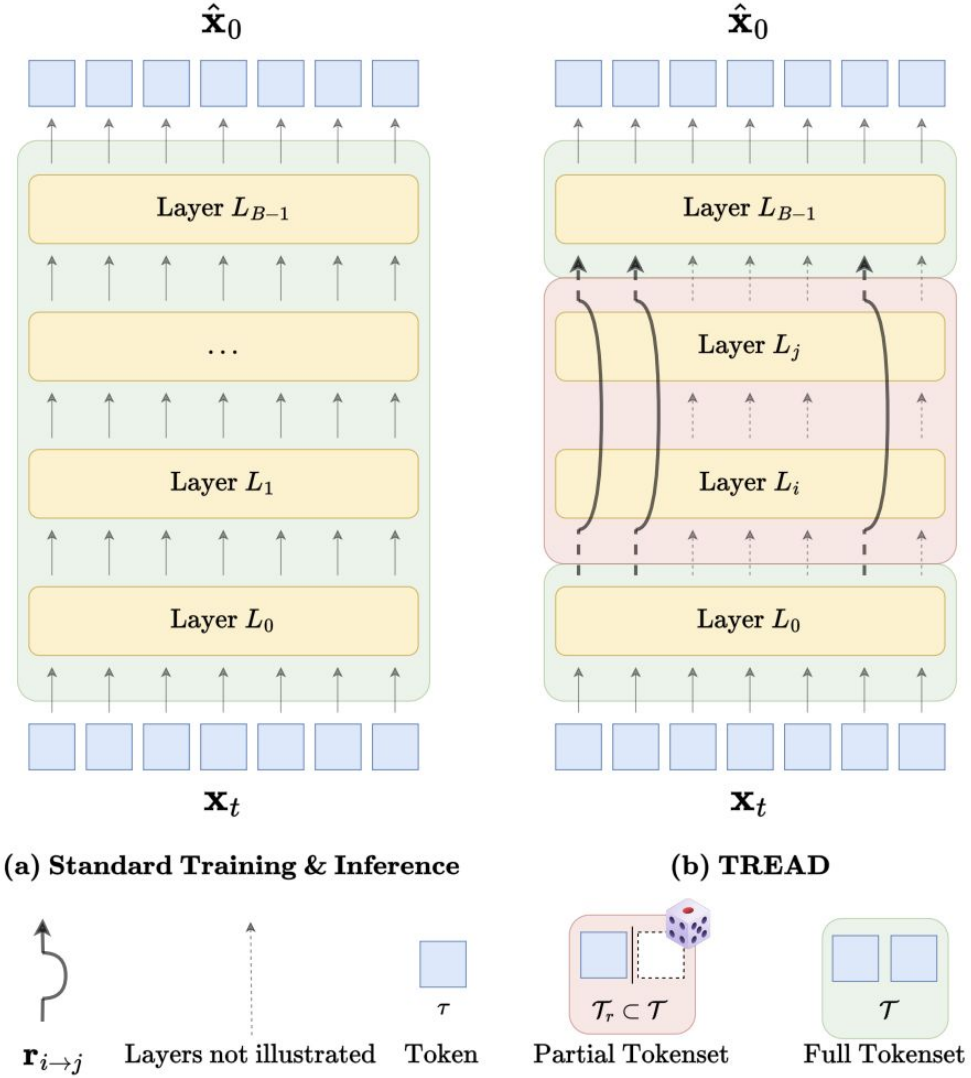
- At each training step a random subset of tokens is routed from an early layer i directly to a deeper layer j
- These tokens skip attention and MLP of all blocks in between

Why it works

- Fewer FLOPs per step → higher training throughput
- Routed tokens re-enter the network, so no information is lost

Architecture-agnostic

- Works for transformers and for state-space models (Mamba, RWKV) with care for spatial bias
- Used only during training



paper →

TREAD: results

Notes

- Gains multiply: faster iterations and better convergence per iteration
- Routing hurts the response to classifier-free guidance, so the model is briefly fine-tuned without routing
- Follow-up: "Guiding Token-Sparse Diffusion Models"

Method	Epochs	FID↓	sFID↓	IS↑	Prec.↑	Rec.↑
Unguided						
SD-DiT-XL/2 [78]†	480	7.21	5.17	144.68	0.72	0.61
MDT-XL/2 [18]†	1300	6.23	5.23	143.02	0.71	0.65
SiT-XL/2 [41]	1400	8.61	6.32	131.65	0.68	0.67
MaskDiT-XL/2 [77]†	1600	5.69	10.34	177.99	0.74	0.60
DiT-XL/2$_{+\text{REPA}}$[74]†	170	9.60	-	-	-	-
DiT-XL/2 [49]	1400	9.62	6.85	121.50	0.67	0.67
+TREAD	160	4.96	6.83	192.49	0.79	0.54
+TREAD	680	**3.93**	7.32	211.35	0.76	0.60
Guided						
SD-DiT-XL/2 [78]†	480	3.23	-	-	-	-
MDT-XL/2 [18]†	1300	1.79	4.57	283.01	0.81	0.61
MaskDiT-XL/2 [77]†	1600	2.28	5.67	276.56	0.80	0.61
SiT-XL/2 [41]	1400	2.06	4.50	270.30	0.82	0.59
DiT-XL/2 [49]	1400	2.27	4.60	278.24	0.83	0.57
+TREAD(F)	220	2.76	4.61	252.78	0.83	0.57
+TREAD(F)	740	2.09	4.46	269.07	0.81	0.62
+TREAD(F)‡	740	**1.69**	4.73	292.66	0.81	0.63

(†) alters the architecture or uses additional parameters. (‡) uses interval guidance [34].

DLS

05

Stable Diffusion 3

Flow matching, MM-DiT and joint attention

Stable Diffusion 3

Figure 1. High-resolution samples from our 8B rectified flow model, showcasing its capabilities in typography, precise prompt following and spatial reasoning, attention to fine details, and high image quality across a wide variety of styles.

paper →

From diffusion to flow matching

Rectified flow `rf`

straight line between data and noise

$$x_t = (1-t)\,x_0 + t\,\epsilon$$

EDM `edm`

variance exploding, σ(t) from a log-normal

$$x_t = x_0 + \sigma(t)\,\epsilon$$

Cosine schedule `v/cos`

v-prediction

$$x_t = \cos(\tfrac{\pi}{2}t)x_0 + \sin(\tfrac{\pi}{2}t)\epsilon$$

DDPM / LDM-linear `eps/linear`

ε-prediction

$$x_t = \sqrt{\bar{\alpha}_t}\,x_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon$$

x_0 — data, ϵ — noise, $t \in [0, 1]$

variant	ImageNet CLIP	ImageNet FID	CC12M CLIP	CC12M FID
rf	0.247	49.70	0.217	94.90
edm(-1.20, 1.20)	0.236	63.12	0.200	116.60
eps/linear	0.245	48.42	0.222	*90.34*
v/cos	0.244	50.74	0.209	97.87
v/linear	0.246	51.68	0.217	100.76
rf/lognorm(0.50, 0.60)	**0.256**	80.41	0.233	120.84
rf/mode(1.75)	*0.253*	**44.39**	0.218	94.06
rf/lognorm(1.00, 0.60)	0.254	114.26	**0.234**	147.69
rf/lognorm(-0.50, 1.00)	0.248	45.64	0.219	**89.70**
rf/lognorm(0.00, 1.00)	0.250	*45.78*	*0.224*	89.91

Table 2. **Metrics for different variants.** FID and CLIP scores of different variants with 25 sampling steps. We highlight the **best**, second best, and *third best* entries.

paper →

From diffusion to flow matching

1 Straight path

interpolate data and noise

$$x_t = (1 - t)\, x_0 + t\, \epsilon$$

2 Velocity field

the network predicts the direction of motion

$$\frac{dx_t}{dt} = v_\theta(x_t, t)$$

3 Conditional flow matching loss

regress the target velocity

$$\mathcal{L}_{CFM} = \mathbb{E}_{t,\, x_0,\, \epsilon} \left\| v_\theta(x_t, t) - u_t \right\|^2$$

4 Target velocity

constant along the straight line

$$u_t = \frac{dx_t}{dt} = \epsilon - x_0$$

x_0 — data, ε — noise, $t \in [0, 1]$

variant	ImageNet CLIP	ImageNet FID	CC12M CLIP	CC12M FID
rf	0.247	49.70	0.217	94.90
edm(-1.20, 1.20)	0.236	63.12	0.200	116.60
eps/linear	0.245	48.42	0.222	*90.34*
v/cos	0.244	50.74	0.209	97.87
v/linear	0.246	51.68	0.217	100.76
rf/lognorm(0.50, 0.60)	**0.256**	80.41	<u>0.233</u>	120.84
rf/mode(1.75)	*0.253*	**44.39**	0.218	94.06
rf/lognorm(1.00, 0.60)	<u>0.254</u>	114.26	**0.234**	147.69
rf/lognorm(-0.50, 1.00)	0.248	<u>45.64</u>	0.219	**89.70**
rf/lognorm(0.00, 1.00)	0.250	*45.78*	*0.224*	<u>89.91</u>

Table 2. **Metrics for different variants.** FID and CLIP scores of different variants with 25 sampling steps. We highlight the **best**, <u>second best</u>, and *third best* entries.

paper →

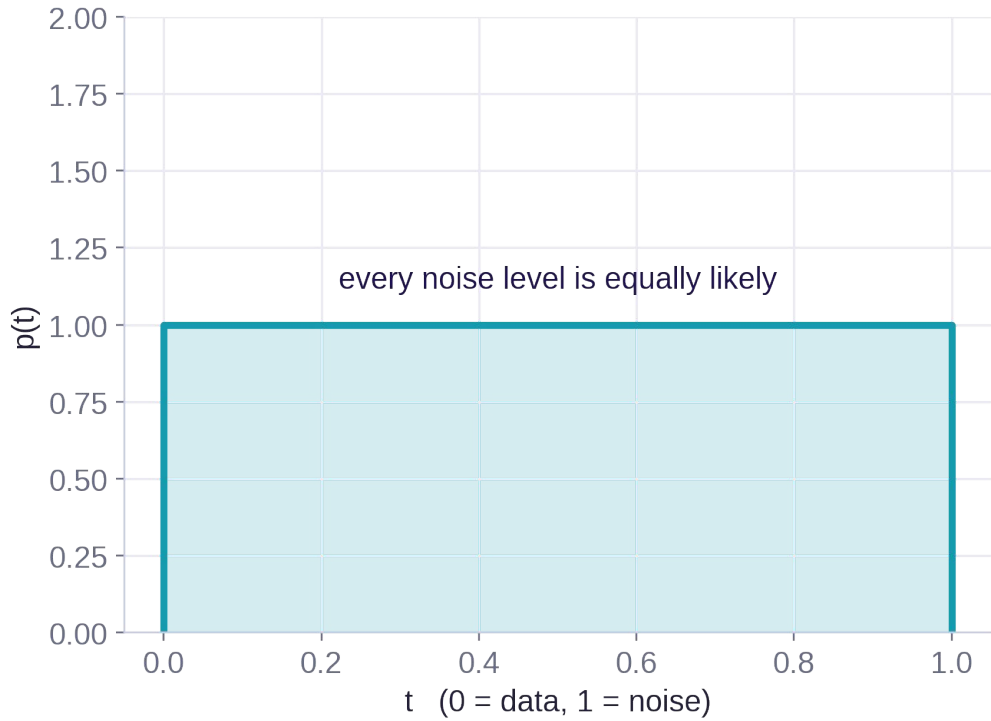
Timestep sampling: uniform

How do we choose t during training?

DEFAULT

$$t \sim \mathcal{U}[0, 1]$$

Every noise level gets the same share of training compute



paper →

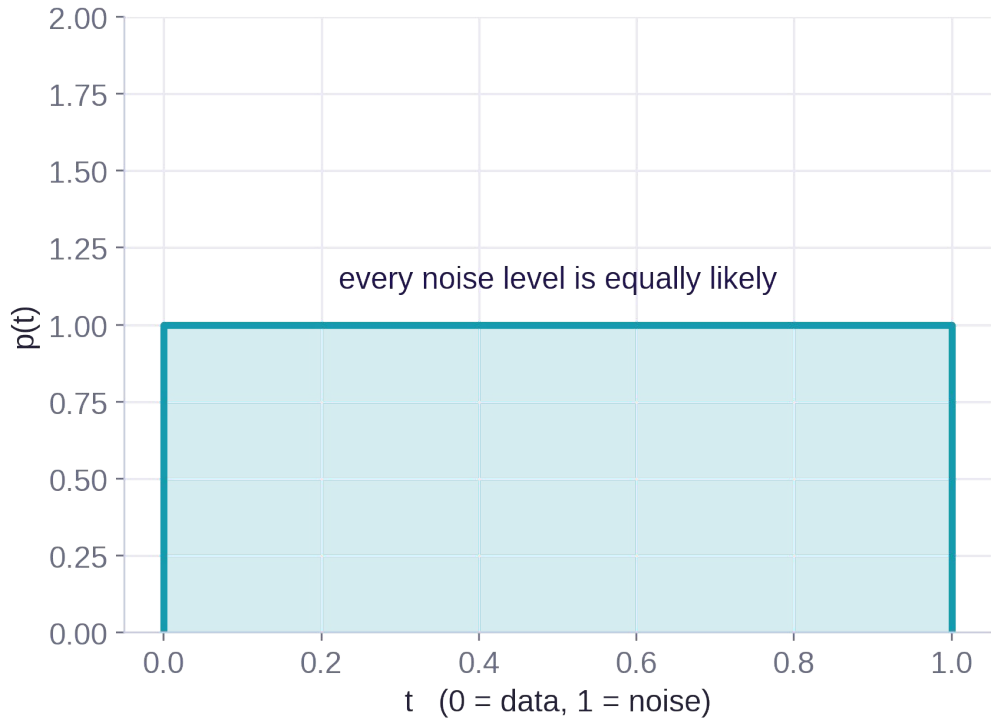
Timestep sampling: uniform

How do we choose t during training?

DEFAULT

$$t \sim \mathcal{U}[0, 1]$$

Every noise level gets the same share of training compute



Is it a good idea?

Are all noise levels equally hard and equally important?

paper →

Coarse-to-fine generation

Generation goes from coarse to fine: at t ≈ 1 the model only sets a blurry layout, at t ≈ 0 it polishes details. The real decisions — composition, objects, shapes — are made at intermediate t.

t ≈ 1
relatively easy

intermediate t
difficult and important

t ≈ 0
relatively easy

paper →

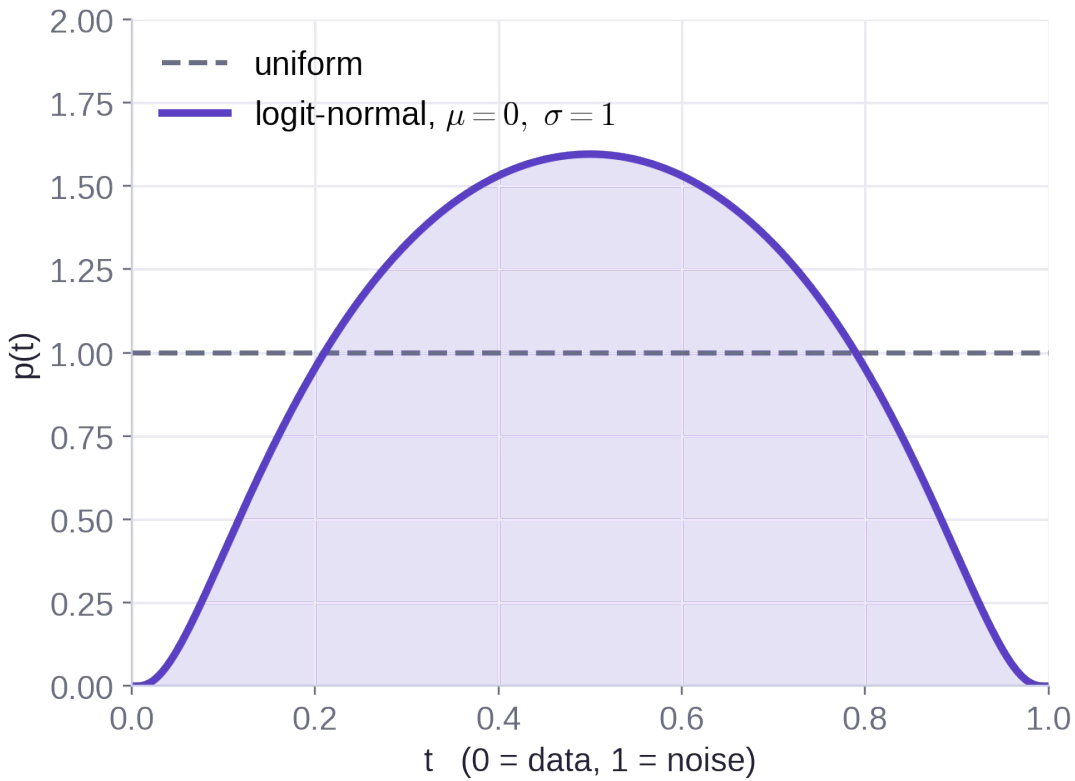
Logit-normal timestep sampling

Let's make p(t) focus on intermediate timesteps

LOGIT-NORMAL DISTRIBUTION

$$p_{\ln}(t;\mu,\sigma) = \frac{1}{\sigma\sqrt{2\pi}\,t(1-t)}\exp\left(-\frac{(\operatorname{logit}(t)-\mu)^2}{2\sigma^2}\right)$$

$$\operatorname{logit}(t) = \log\frac{t}{1-t}, \qquad \mu = \log\alpha, \quad \sigma = 1$$



paper →

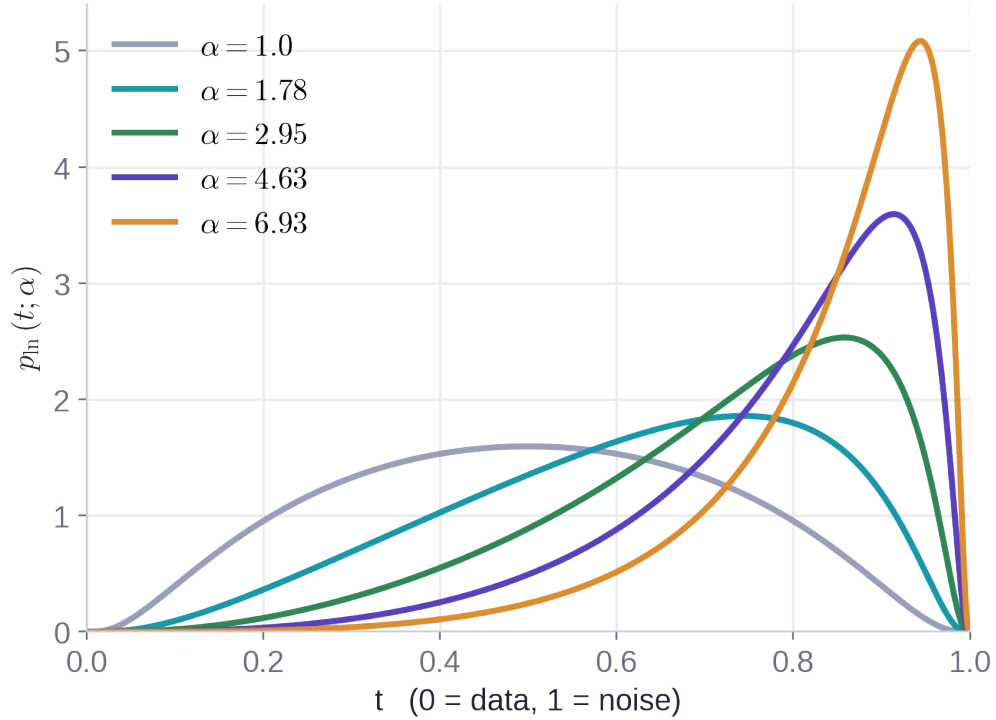
Logit-normal in practice: shift to high noise

Logit-normal distribution in practice

WITH A SHIFT PARAMETER

$$p_{\ln}(t;\alpha) = \frac{1}{\sqrt{2\pi}\, t(1-t)} \exp\left(-\frac{(\operatorname{logit}(t) - \log\alpha)^2}{2}\right)$$

Often it is beneficial to shift p(t) to high noise levels: **α > 1**



paper →

Same t, different resolution

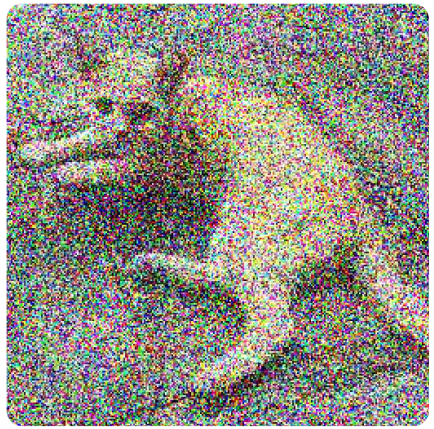
The same noise level t destroys the image less at higher resolution

t = 0.5

same image, same t, shown at the same size

128 × 128

256 × 256



512 × 512

1024 × 1024

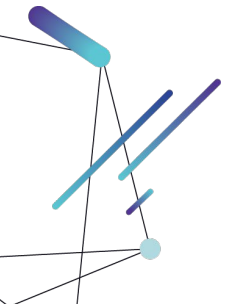
paper →

Why shift p(t)? A toy estimate

How much does noise at level t really hide?

1 **Toy model of one pixel** c — clean value (proxy for x_0), ε — noise

$$Y(t) = (1 - t)\, c + t\, \epsilon, \qquad c \in \mathbb{R}, \quad \epsilon \sim \mathcal{N}(0, 1)$$



paper →

Why shift p(t)? A toy estimate

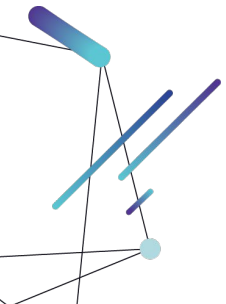
How much does noise at level t really hide?

1 **Toy model of one pixel** c — clean value (proxy for x_0), ε — noise

$$Y(t) = (1-t)\,c + t\,\epsilon, \qquad c \in \mathbb{R}, \quad \epsilon \sim \mathcal{N}(0,1)$$

2 **N noisy observations $y_1 \ldots y_N$** estimate c by averaging them

$$\mathbb{E}[Y(t)] = (1-t)\,c \quad \Rightarrow \quad \hat{c} = \frac{1}{1-t}\,\frac{1}{N}\sum_{i=1}^{N} y_i$$



paper →

Why shift p(t)? A toy estimate

How much does noise at level t really hide?

1 **Toy model of one pixel** c — clean value (proxy for x_0), ε — noise

$$Y(t) = (1-t)\,c + t\,\epsilon, \qquad c \in \mathbb{R}, \quad \epsilon \sim \mathcal{N}(0,1)$$

2 **N noisy observations $y_1 \ldots y_N$** estimate c by averaging them

$$\mathbb{E}[Y(t)] = (1-t)\,c \quad \Rightarrow \quad \hat{c} = \frac{1}{1-t}\,\frac{1}{N}\sum_{i=1}^{N} y_i$$

3 **Uncertainty of the estimate** variance shrinks with N

$$\mathrm{Var}[Y(t)] = t^2 \quad \Rightarrow \quad \mathrm{Var}[\hat{c}] = \frac{t^2}{(1-t)^2}\,\frac{1}{N}$$

paper →

Why shift p(t)? A toy estimate

How much does noise at level t really hide?

1 **Toy model of one pixel** c — clean value (proxy for x_0), ε — noise

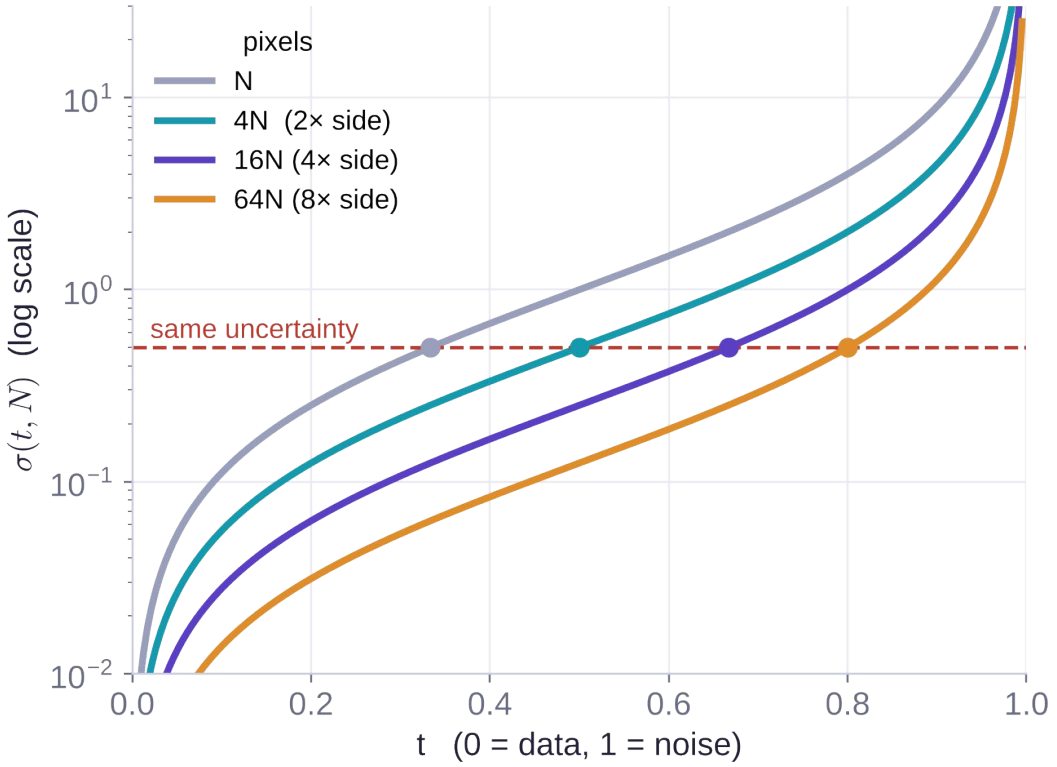
$$Y(t) = (1-t)\,c + t\,\epsilon, \qquad c \in \mathbb{R}, \quad \epsilon \sim \mathcal{N}(0,1)$$

2 **N noisy observations $y_1 \ldots y_N$** estimate c by averaging them

$$\mathbb{E}[Y(t)] = (1-t)\,c \quad \Rightarrow \quad \hat{c} = \frac{1}{1-t}\,\frac{1}{N}\sum_{i=1}^{N} y_i$$

3 **Uncertainty of the estimate** variance shrinks with N

$$\mathrm{Var}[Y(t)] = t^2 \quad \Rightarrow \quad \mathrm{Var}[\hat{c}] = \frac{t^2}{(1-t)^2}\,\frac{1}{N}$$



paper →

Same t, different resolution

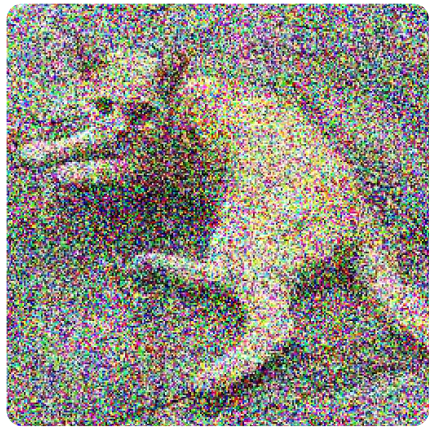
The same noise level t destroys the image less at higher resolution

t = 0.5

same image, same t, shown at the same size

128 × 128

256 × 256



512 × 512

1024 × 1024

paper →

Same t, different resolution

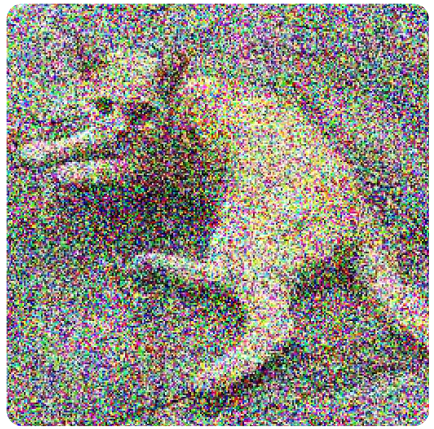
The same noise level t destroys the image less at higher resolution

t = 0.5

same image, same t, shown at the same size

128 × 128

256 × 256



512 × 512

1024 × 1024

KEEP THE UNCERTAINTY FIXED WHEN RESOLUTION GROWS

$$\sigma(t', m) = \sigma(t, n) \quad \Rightarrow \quad \frac{t'}{1-t'} = \sqrt{\frac{m}{n}} \cdot \frac{t}{1-t}$$

$$t' = \frac{\alpha\, t}{1 + (\alpha - 1)\, t}, \qquad \alpha = \sqrt{m/n}$$

This is exactly the timestep shift

m, n — pixel counts at the new and base resolution. Same as moving the logit-normal by log α

paper →

Same t, different resolution

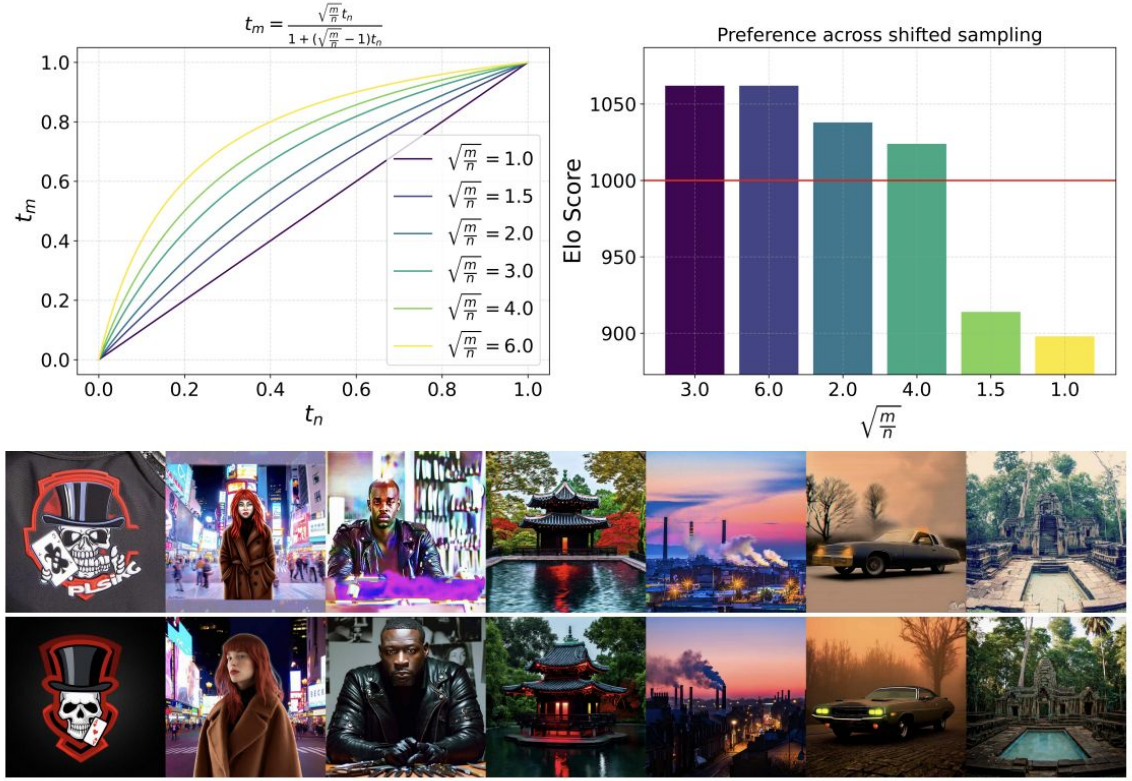


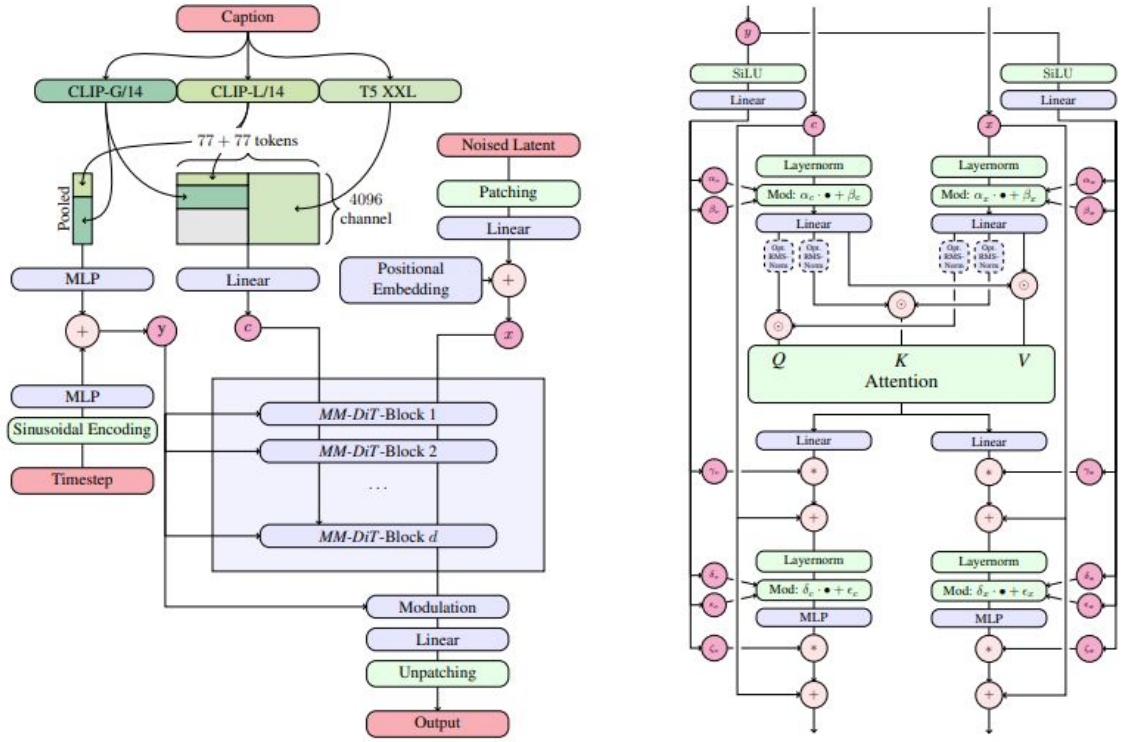
Figure 6. **Timestep shifting at higher resolutions.** *Top right:* Human quality preference rating when applying the shifting based on Equation (23). *Bottom row:* A 512^2 model trained and sampled with $\sqrt{m/n} = 1.0$ (*top*) and $\sqrt{m/n} = 3.0$ (*bottom*). See Section 5.3.2.

KEEP THE UNCERTAINTY FIXED WHEN RESOLUTION GROWS

$$\sigma(t', m) = \sigma(t, n) \quad \Rightarrow \quad \frac{t'}{1 - t'} = \sqrt{\frac{m}{n}} \cdot \frac{t}{1 - t}$$

$$t' = \frac{\alpha\, t}{1 + (\alpha - 1)\, t}, \qquad \alpha = \sqrt{m/n}$$

Stable Diffusion 3



(a) Overview of all components.

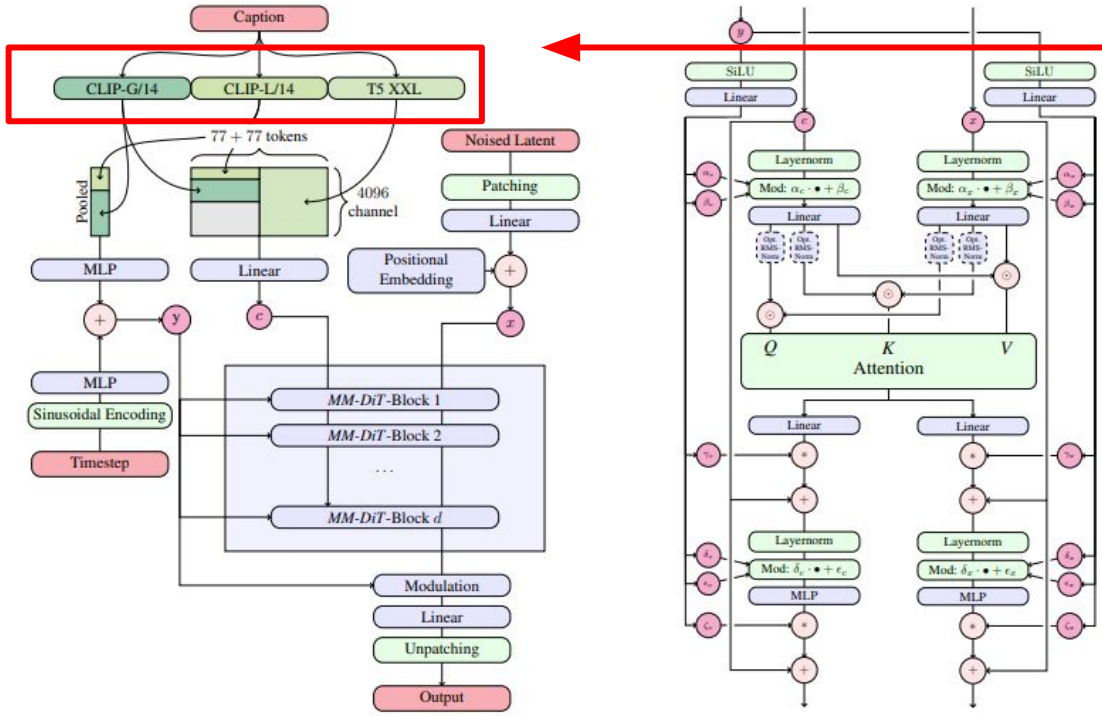
(b) One *MM-DiT* block

MM-DiT 8B parameters

3 text encoders (CLIP ViT-L, OpenCLIP ViT-bigG & T5 XXL)

16-channel VAE

Stable Diffusion 3

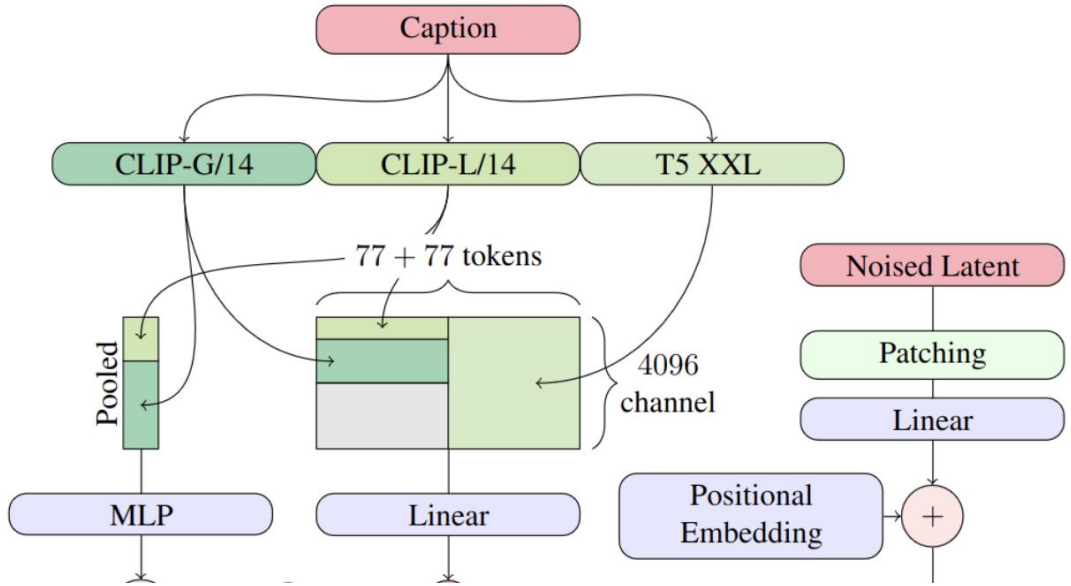


MM-DiT 8B parameters

3 text encoders (CLIP ViT-L, OpenCLIP ViT-bigG & T5 XXL)

16-channel VAE

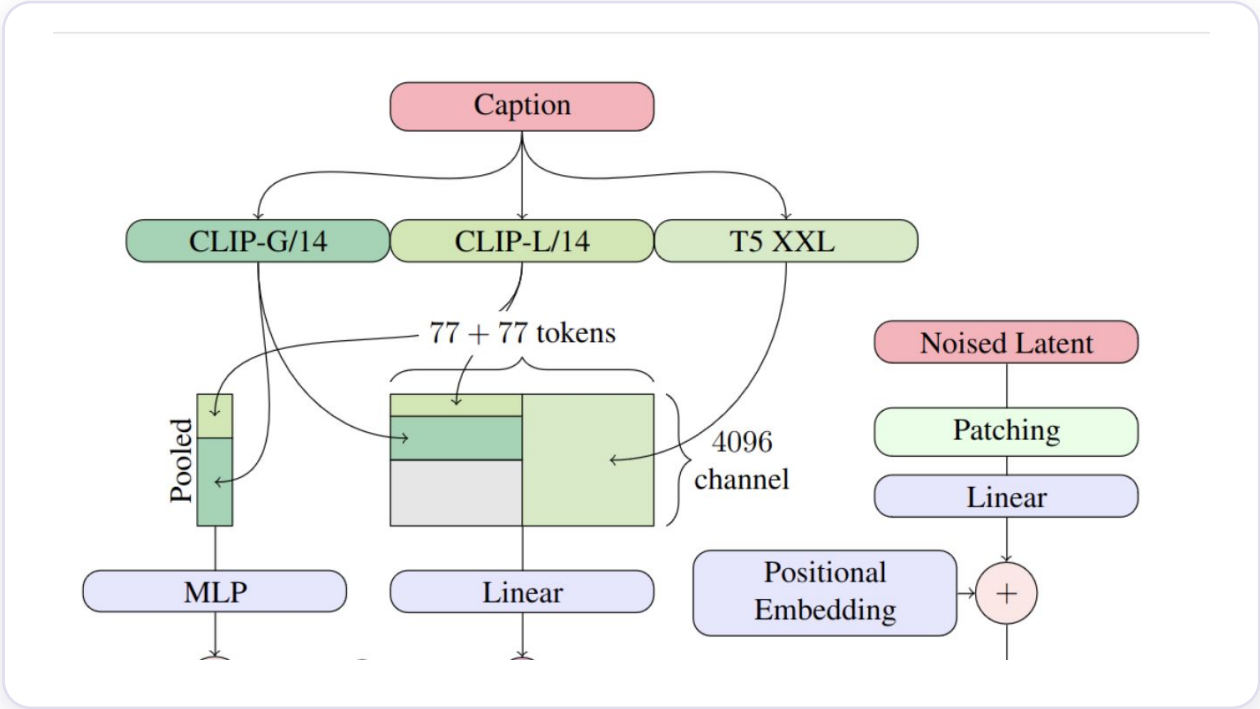
Stable Diffusion 3

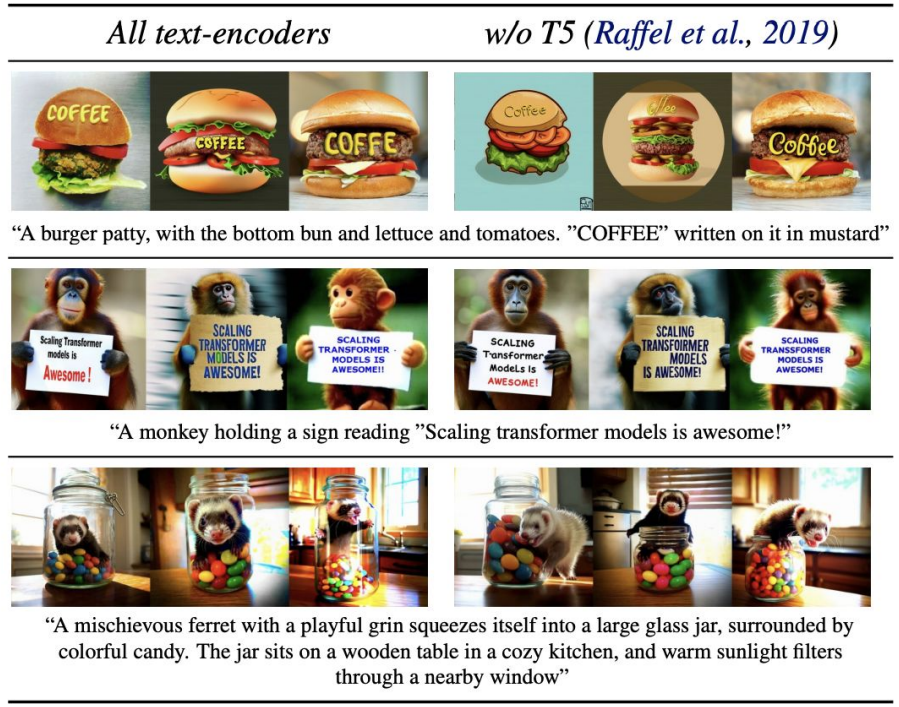


Text embeddings are used as follows:

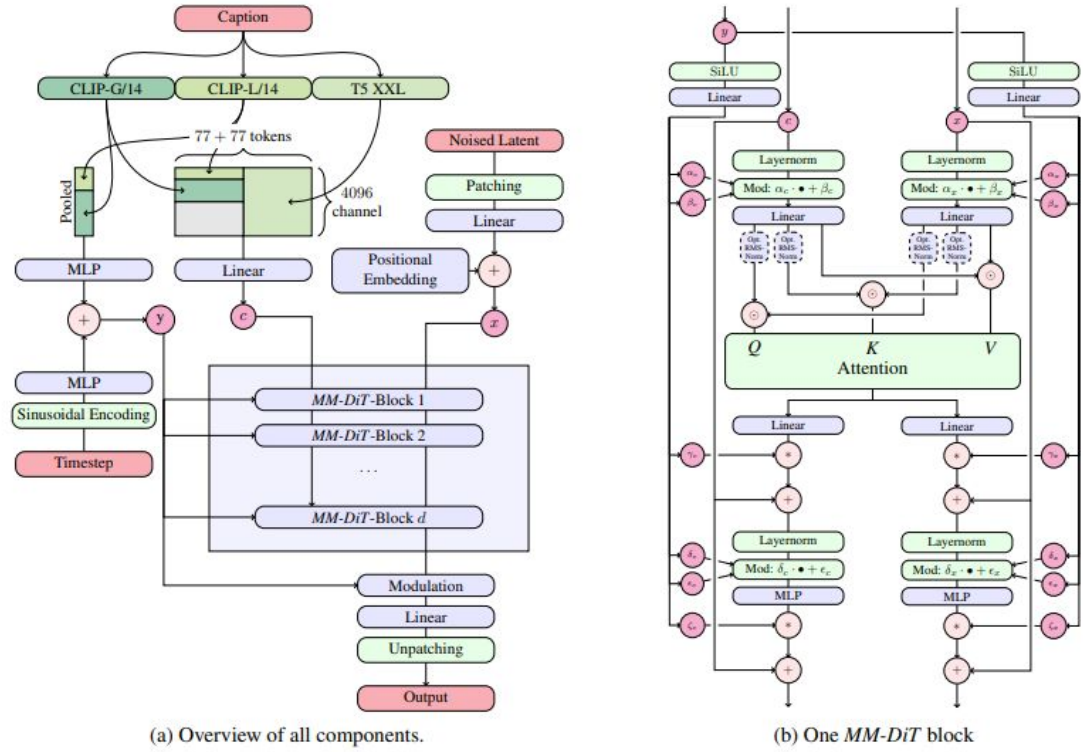
- The pooled CLIP embeddings are added to the timestep embedding and used in AdaLN
- CLIP + T5 embeddings are concatenated and fed into the MM-DiT block

Stable Diffusion 3





Stable Diffusion 3



(a) Overview of all components.

(b) One *MM-DiT* block

MM-DiT 8B parameters

3 text encoders (CLIP ViT-L, OpenCLIP ViT-bigG & T5 XXL)

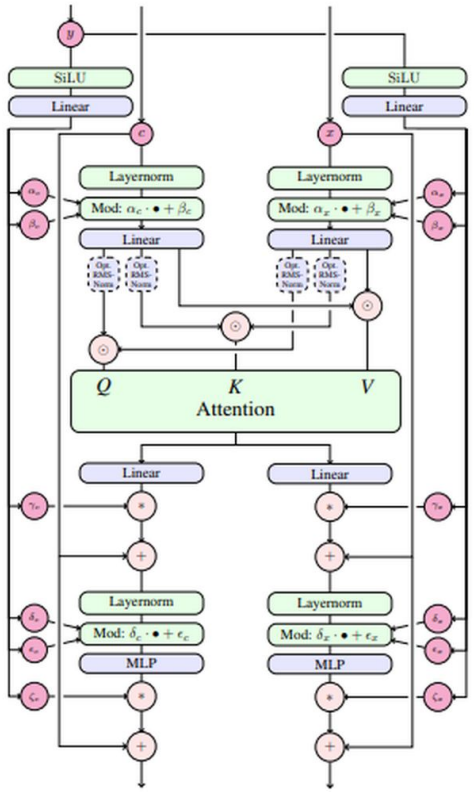
16-channel VAE

Metric	4 chn	8 chn	16 chn
FID (↓)	2.41	1.56	**1.06**
Perceptual Similarity (↓)	0.85	0.68	**0.45**
SSIM (↑)	0.75	0.79	**0.86**
PSNR (↑)	25.12	26.40	**28.62**

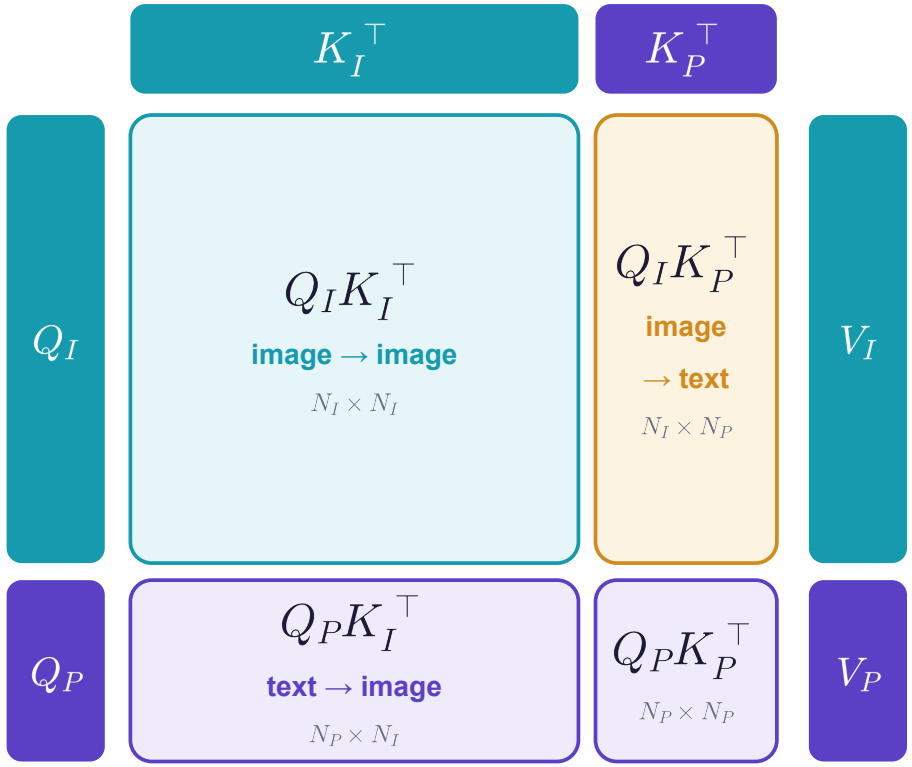
Table 3. **Improved Autoencoders.** Reconstruction performance metrics for different channel configurations. The downsampling factor for all models is $f = 8$.

paper →

Joint attention (MM-DiT)



One MM-DiT block: two streams, shared attention



self-attention: already in SD 1.x

cross-attention: already in SD 1.x

new: text tokens attend to image and text

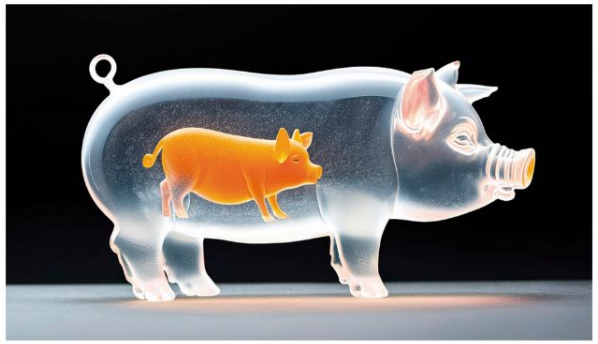
One attention for both

Image and text tokens are concatenated into one sequence: every query sees all image and text keys

Separate weights

Each modality keeps its own LayerNorm, adaLN modulation, QKV projections and MLP. Only the attention is shared

Stable Diffusion 3, results



Prompt: Translucent pig, inside is a smaller pig.

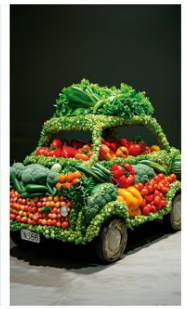
Prompt: A massive alien space ship that is shaped like a pretzel.

Prompt: A kangaroo holding a beer, wearing ski goggles and passionately singing silly songs.

Prompt: An entire universe inside a bottle sitting on the shelf at walmart on sale.

Prompt: A cheeseburger with juicy beef patties and melted cheese sits on top of a toilet that looks like a throne and stands in the middle of the royal chamber.

Prompt: This dreamlike digital art captures a vibrant, kaleidoscopic bird in a lush rainforest



Prompt: A car made out of vegetables.

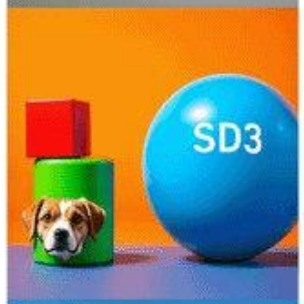
Prompt: Heat death of the universe line art

But…

blue sphere on right, green cylinder on left

blue sphere on right with text "SD3", green cylinder on left with red cube on top

blue sphere on right with text "SD3", green cylinder on left with red cube on top, orange background, dog face at the bottom



man sitting on beach

SD3 → SD3.5

SD 3.5 Large

~8 миллиарда параметров, базовая модель самая мощная в семействе Stable Diffusion

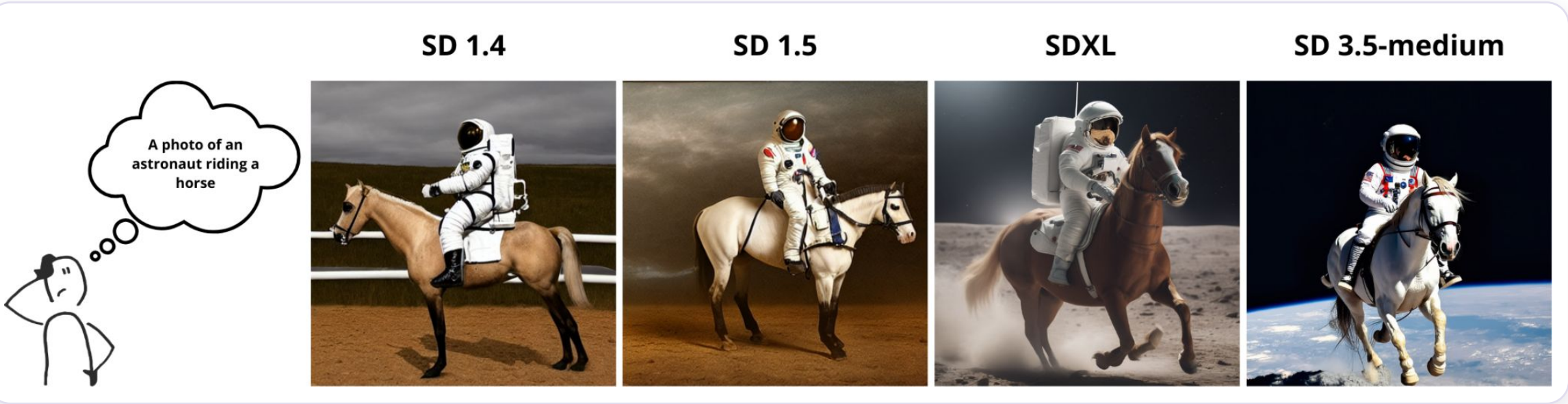
SD 3.5 Large Turbo

та же архитектура, но использует Adversarial Diffusion Distillation (ADD) для уменьшения числа шагов инференса (4 шага)

SD 3.5 Medium

2.5 миллиарда параметров

Stable diffusion history



DLS

06

FLUX

FLUX.1, FLUX Kontext and FLUX.2

code →

FLUX.1

DLS

Rectified-flow transformer, 12B parameters

`[pro]` closed, API only
`[dev]` open weights, guidance-distilled
`[schnell]` open, fast: 1–4 steps

WAIT… SAME PEOPLE?

R. Rombach · A. Blattmann · P. Esser

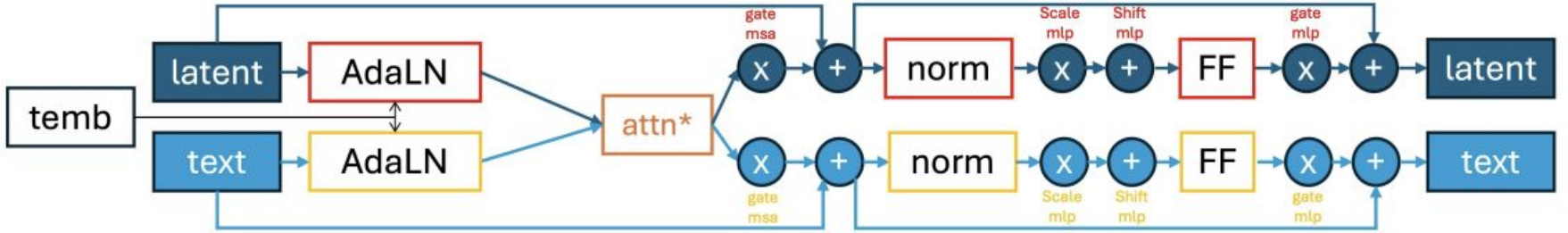
LDM 2022 → SD3 2024 → **FLUX.1** Aug 2024

left Stability AI and founded Black Forest Labs

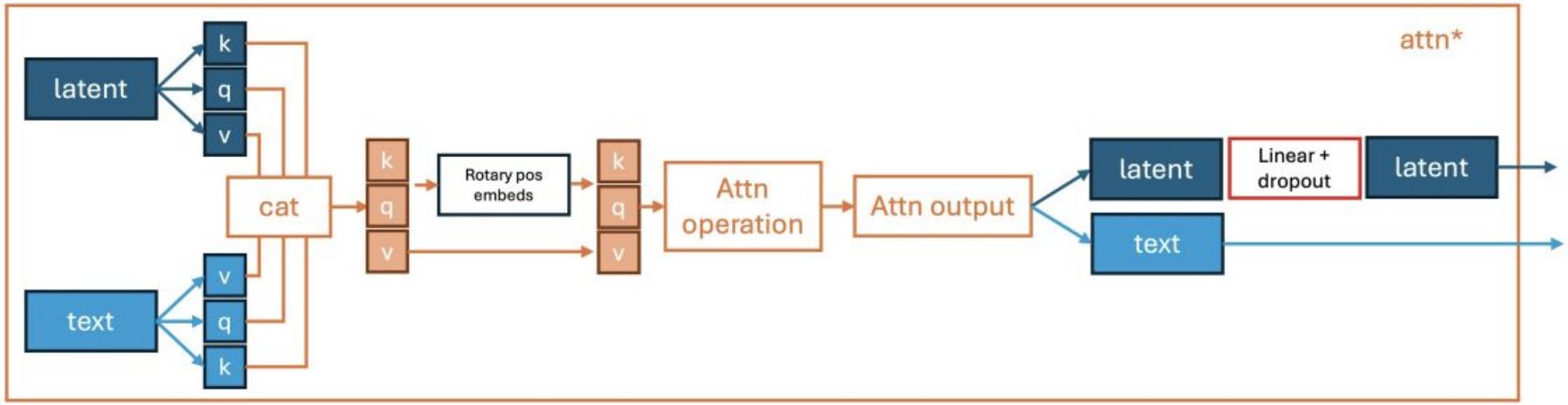
Same team, next step: the authors of Latent Diffusion and SD3 built FLUX on the same recipe: rectified flow + MM-DiT-style transformer

paper →

Double-Stream Attention Block

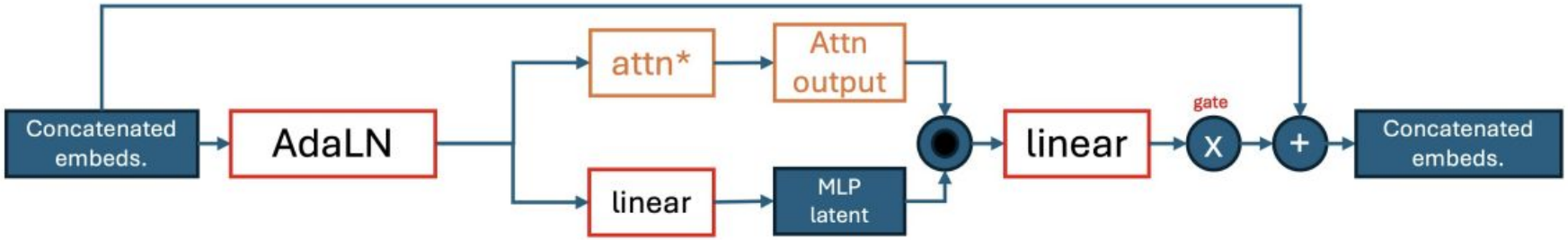


(a) Double-Stream Attention Block: latent and prompt embeddings are processed separately, in a standard multi-modal attention scheme.

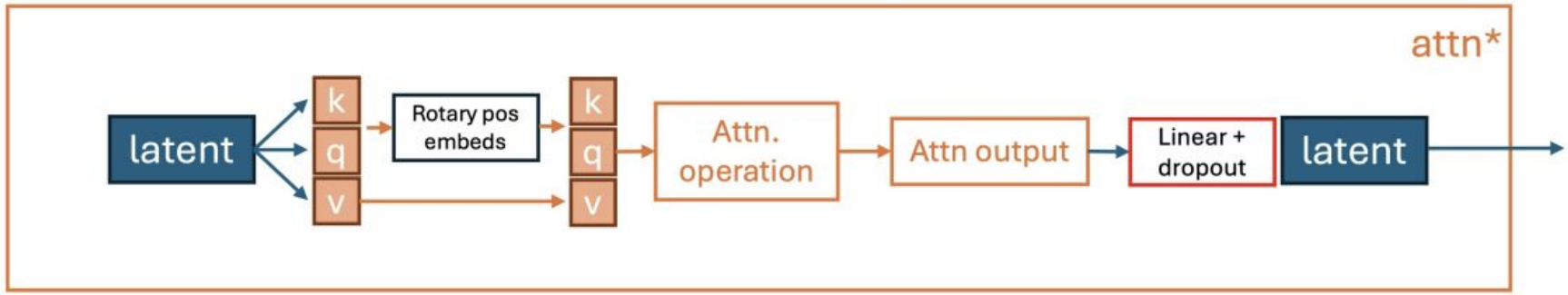


paper →

Single-Stream Attention Block



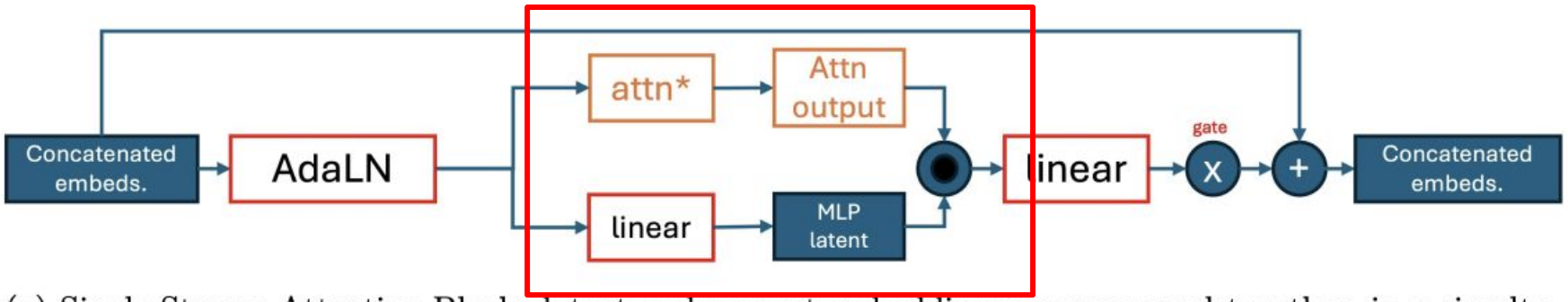
(a) Single-Stream Attention Block: latent and prompt embeddings are processed together, in a simultaneous attention scheme.



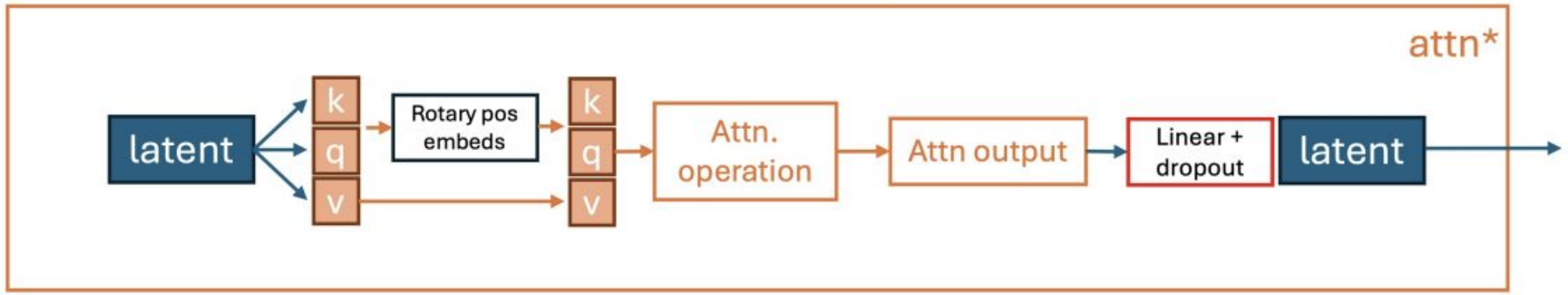
(b) The K, Q and V metrices are computed directly from the concatenated representation.

paper →

Single-Stream Attention Block



(a) Single-Stream Attention Block: latent and prompt embeddings are processed together, in a simultaneous attention scheme.



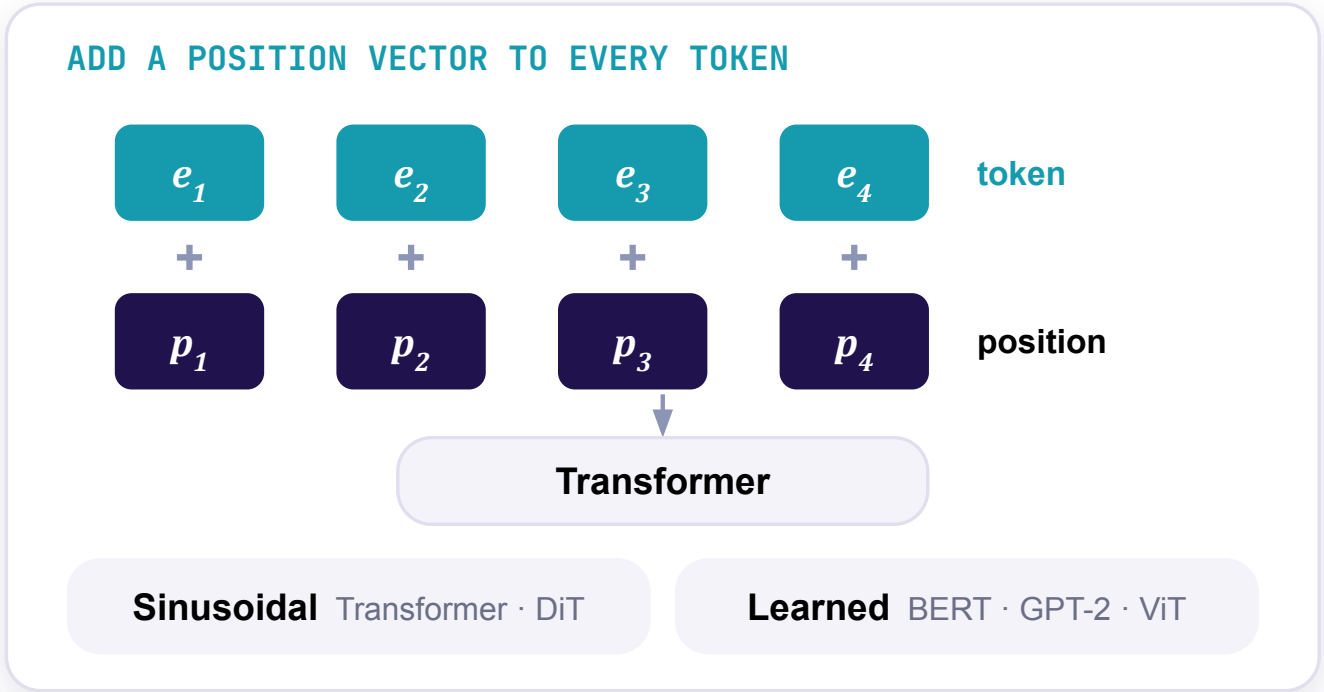
(b) The K, Q and V metrices are computed directly from the concatenated representation.

paper →

DLS

Positional embeddings: the classic way

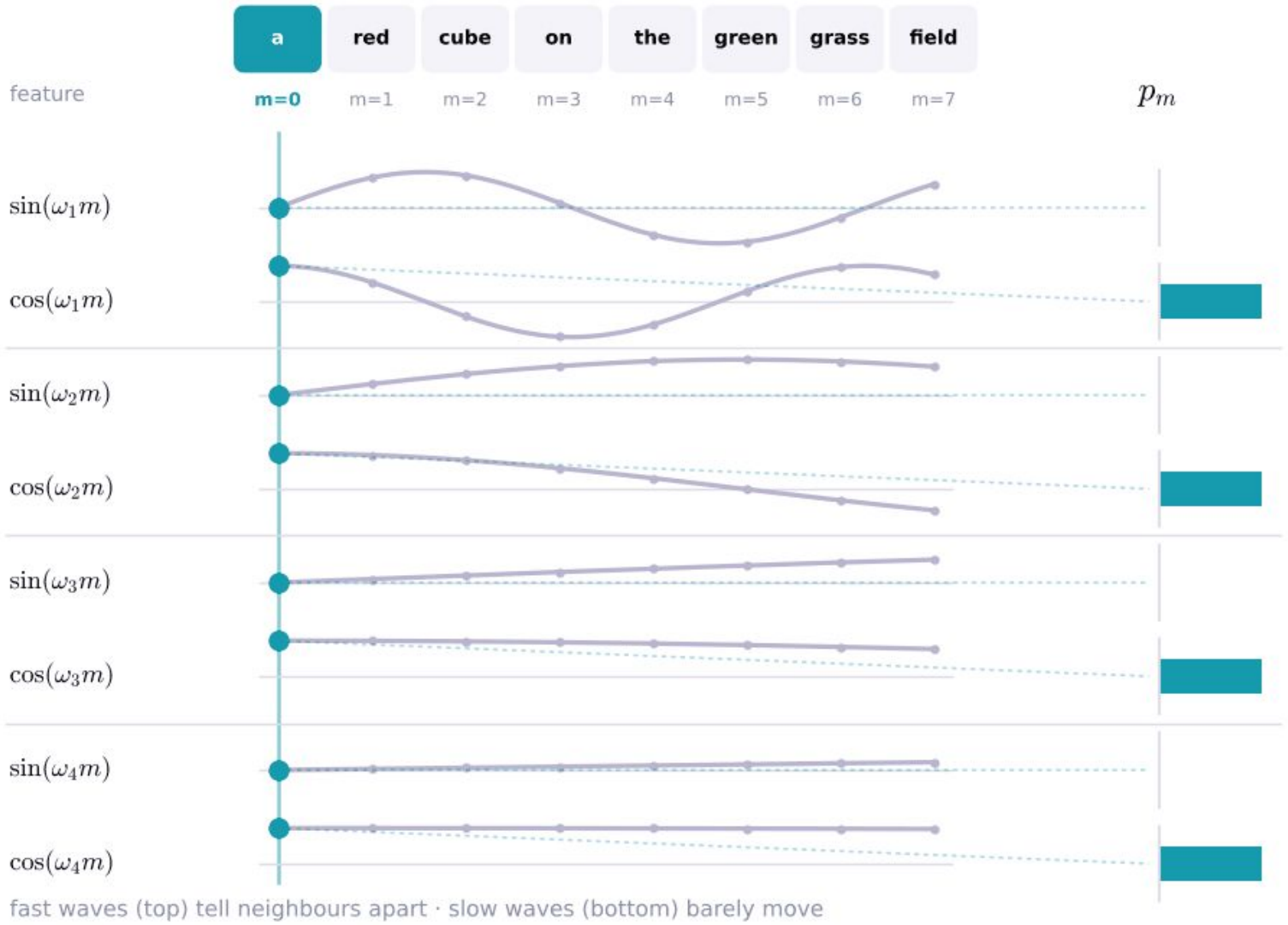
Why at all? self-attention is permutation-invariant: shuffle the tokens (or image patches) and the outputs are just shuffled too. The model has no idea of order unless we inject it.



$$p_{m,\,2i} = \sin\left(\frac{m}{10000^{2i/d}}\right), \qquad p_{m,\,2i+1} = \cos\left(\frac{m}{10000^{2i/d}}\right)$$

paper →

Absolute PE: what happens to the features



fast waves (top) tell neighbours apart · slow waves (bottom) barely move

ADD, DON'T ROTATE

$$x_m = e_m + p_m$$

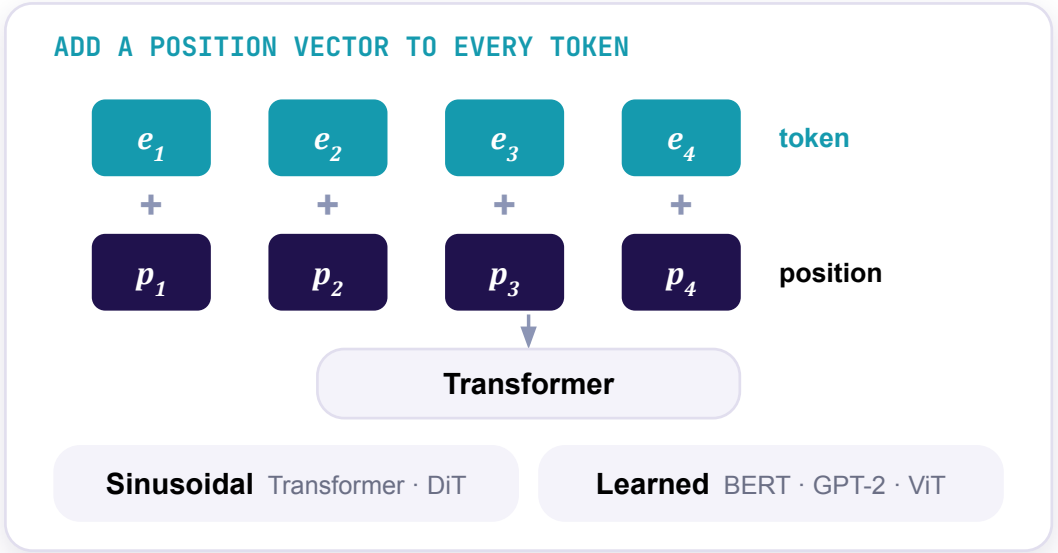
$$p_{m,\,2i} = \sin\left(m \,/\, 10000^{2i/d}\right)$$

$$p_{m,\,2i+1} = \cos\left(m \,/\, 10000^{2i/d}\right)$$

paper →

Positional embeddings: the classic way

Why at all? self-attention is permutation-invariant: shuffle the tokens (or image patches) and the outputs are just shuffled too. The model has no idea of order unless we inject it.



$$p_{m,\,2i} = \sin\left(\frac{m}{10000^{2i/d}}\right), \qquad p_{m,\,2i+1} = \cos\left(\frac{m}{10000^{2i/d}}\right)$$

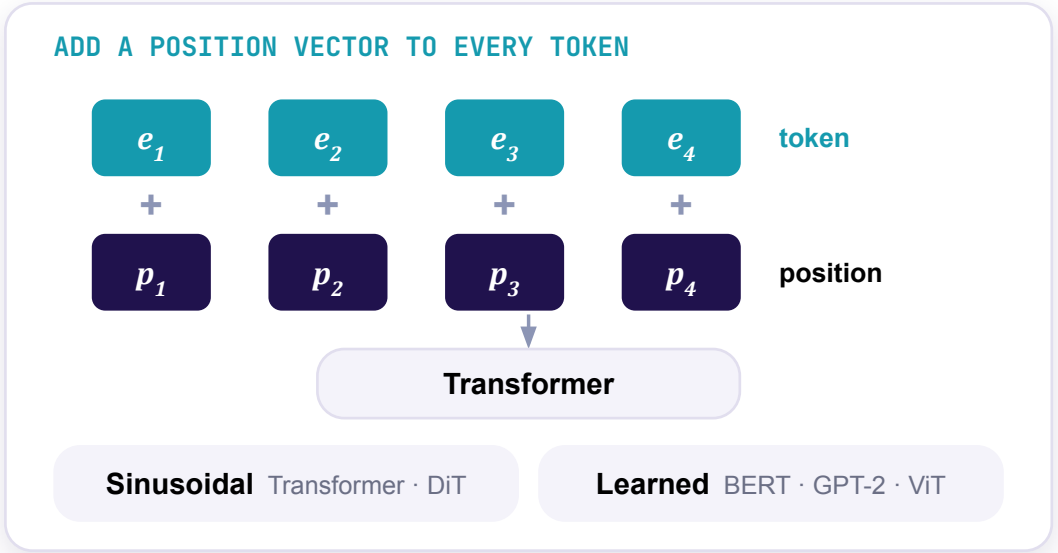
1 **Absolute, not relative**
attention cares how far apart two tokens are

paper →

DLS

Positional embeddings: the classic way

Why at all? self-attention is permutation-invariant: shuffle the tokens (or image patches) and the outputs are just shuffled too. The model has no idea of order unless we inject it.



$$p_{m,\,2i} = \sin\left(\frac{m}{10000^{2i/d}}\right), \qquad p_{m,\,2i+1} = \cos\left(\frac{m}{10000^{2i/d}}\right)$$

1 **Absolute, not relative**
attention cares how far apart two tokens are

2 **Fixed length / resolution**
a learned table stops at N; ViT and DiT trained on one grid need interpolation for other sizes and aspect ratios

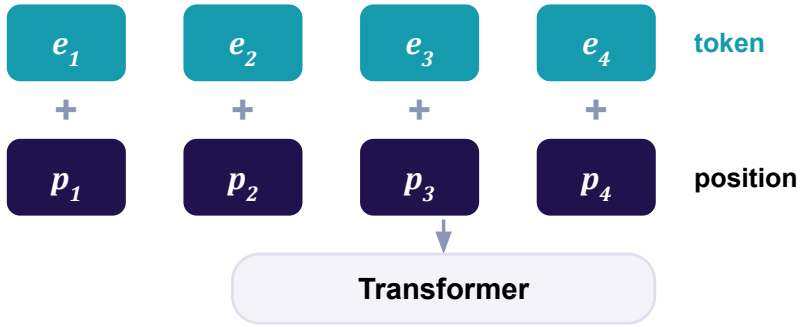
paper →

DLS

Positional embeddings: the classic way

Why at all? self-attention is permutation-invariant: shuffle the tokens (or image patches) and the outputs are just shuffled too. The model has no idea of order unless we inject it.

ADD A POSITION VECTOR TO EVERY TOKEN



Sinusoidal Transformer · DiT

Learned BERT · GPT-2 · ViT

$$p_{m,\,2i} = \sin\left(\frac{m}{10000^{2i/d}}\right), \qquad p_{m,\,2i+1} = \cos\left(\frac{m}{10000^{2i/d}}\right)$$

1. **Absolute, not relative**
   attention cares how far apart two tokens are

2. **Fixed length / resolution**
   a learned table stops at N; ViT and DiT trained on one grid need interpolation for other sizes and aspect ratios

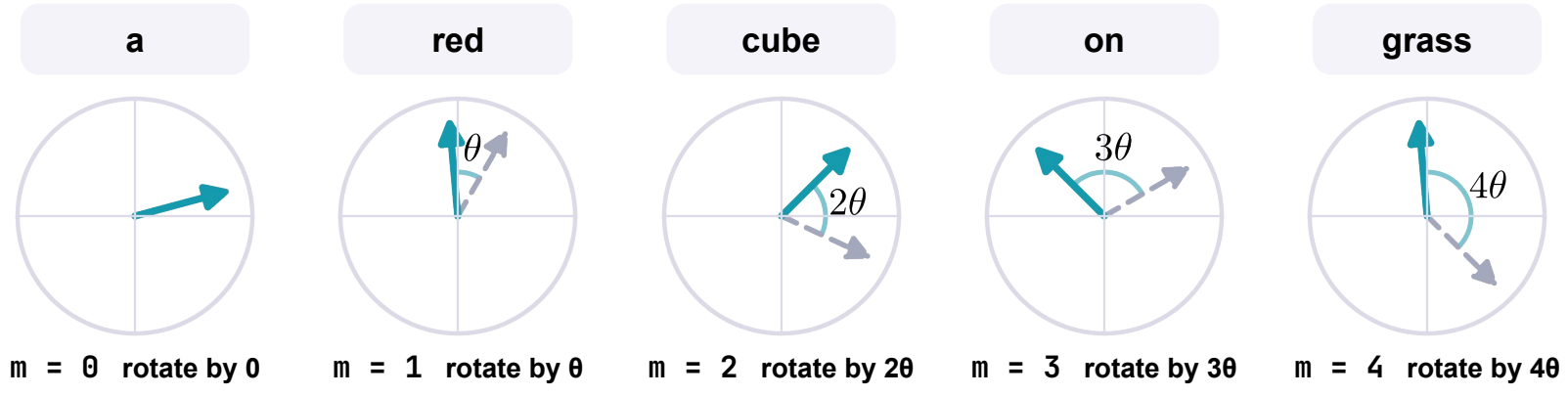
3. **Added once, at the input**
   position is mixed into the content and must survive through all layers

paper →

RoPE: every token is rotated

DLS

Q AND K OF THE TOKEN AT POSITION M ARE ROTATED BY THE ANGLE M·θ



ONE PAIR OF DIMENSIONS

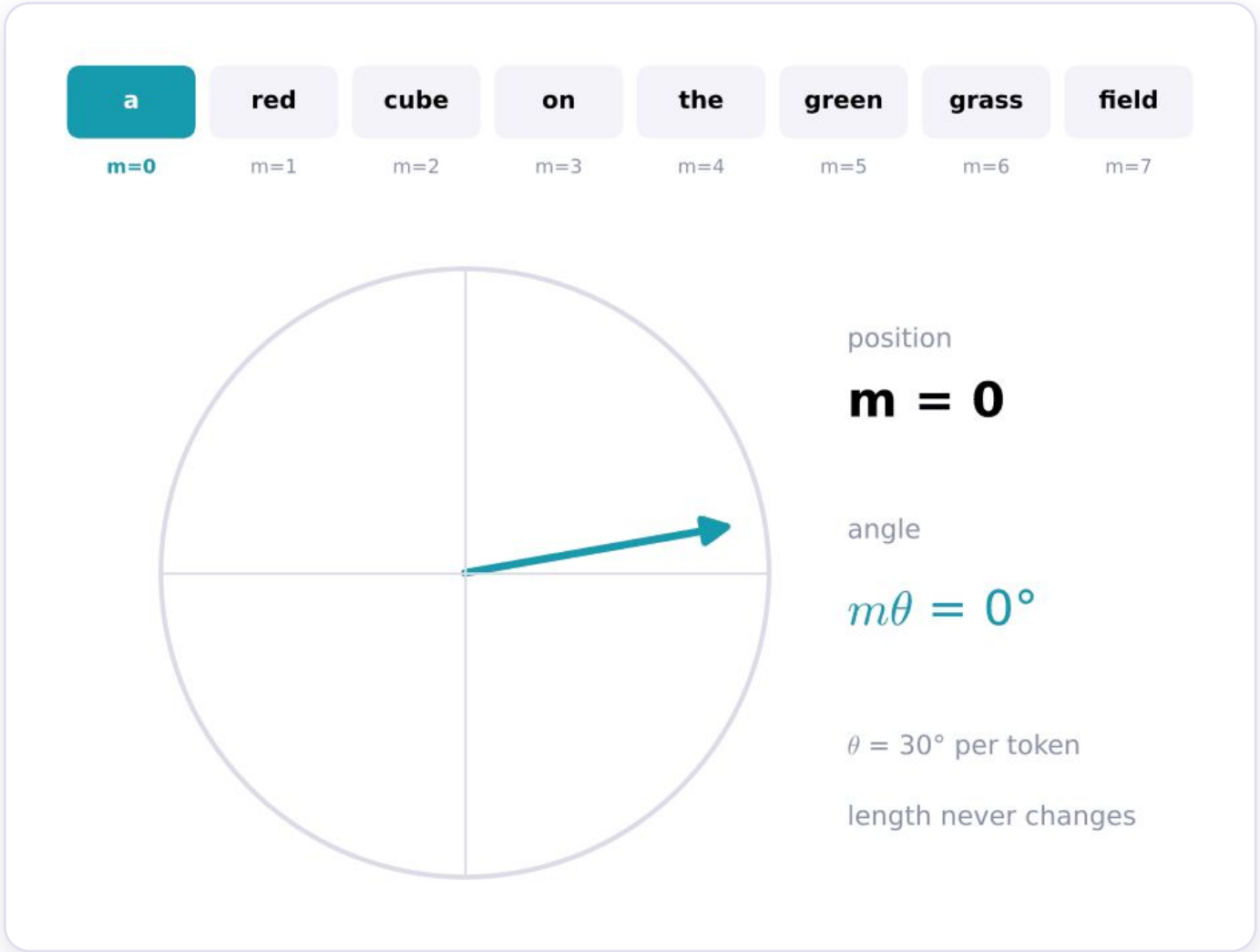
$$\begin{bmatrix} x'_{2i} \\ x'_{2i+1} \end{bmatrix} = \begin{bmatrix} \cos m\theta_i & -\sin m\theta_i \\ \sin m\theta_i & \cos m\theta_i \end{bmatrix} \begin{bmatrix} x_{2i} \\ x_{2i+1} \end{bmatrix}$$

q is split into d/2 pairs; each pair is a 2D point rotated by its own speed

$$\theta_i = 10000^{-2i/d}$$

paper →

RoPE in 2D: walking along the sequence



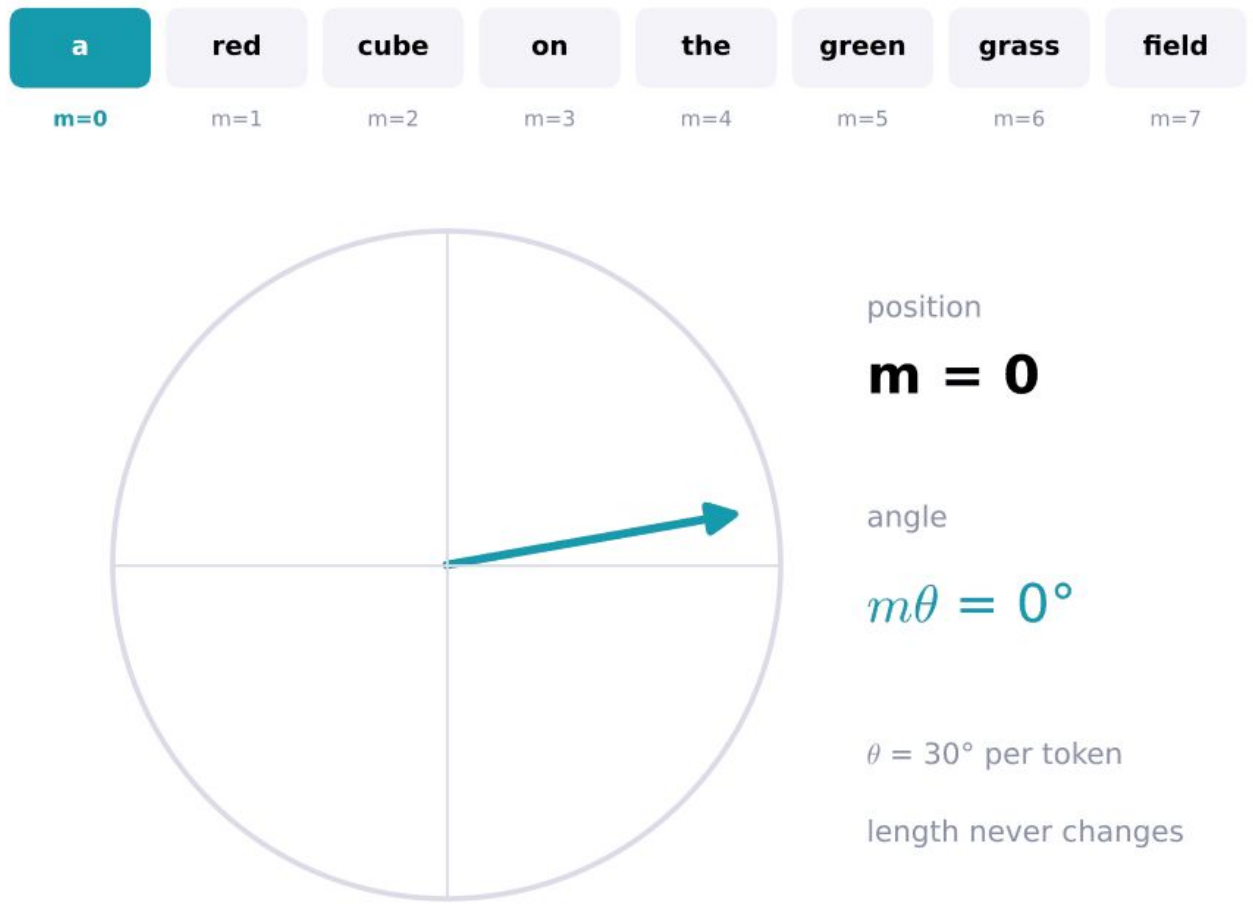
D = 2: ONE PAIR

the whole vector q = (x, y) is just rotated

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

paper →

RoPE in 2D: walking along the sequence



D = 2: ONE PAIR

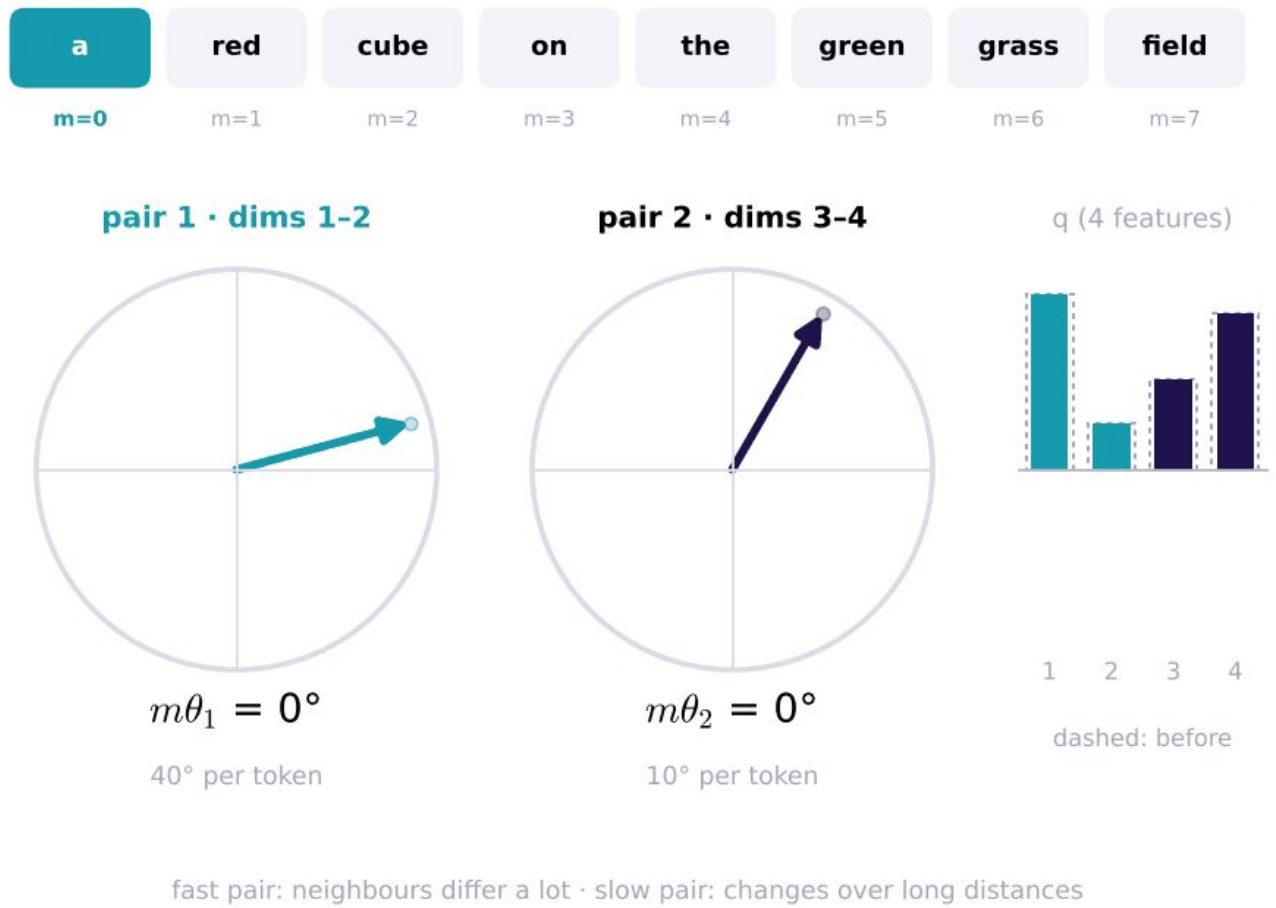
the whole vector q = (x, y) is just rotated

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

- token at position m → angle mθ
- the length never changes
- for two tokens the angle between them is (n − m)θ: this is all attention sees

paper →

RoPE in 4D: two pairs, two speeds



BLOCK-DIAGONAL ROTATION

$$R_m = \begin{bmatrix} R(m\theta_1) & 0 \\ 0 & R(m\theta_2) \end{bmatrix}$$

ATTENTION SCORE

$$(R_m q_m)^\top (R_n k_n) = q_m^\top R_{n-m} k_n$$

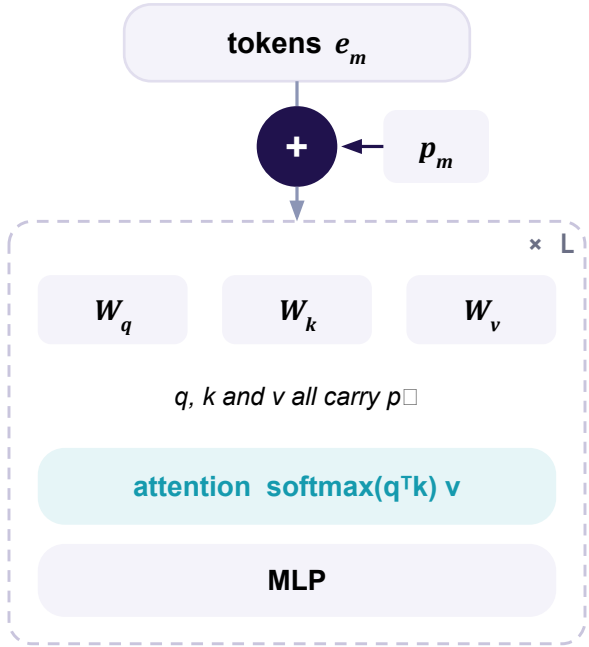
depends only on the distance n − m

paper →

DLS

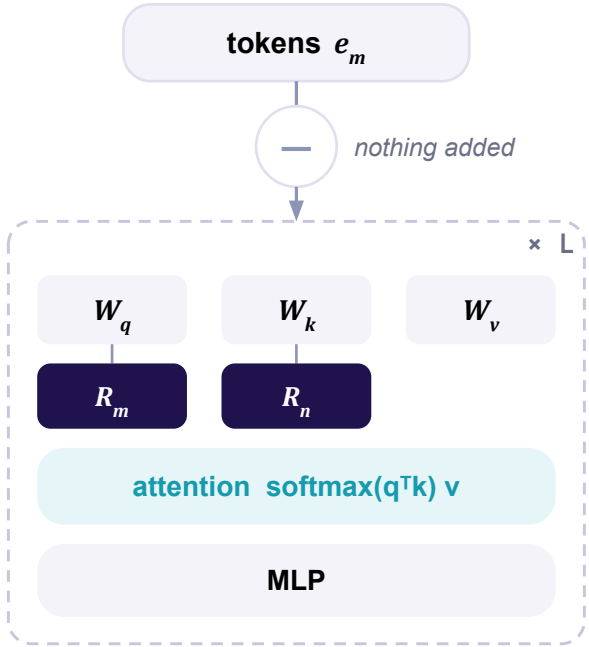
RoPE: where the position enters

ABSOLUTE PE · TRANSFORMER, VIT, DIT



- added once, at the input
- goes into q, k and v
- a learned or fixed table

ROPE · LLAMA, FLUX, QWEN-IMAGE



- rotate q and k in every layer
- only q and k; v stays clean
- no learned parameters

ATTENTION SCORE

$$q_m^\top k_n = (W_q(e_m + p_m))^\top W_k(e_n + p_n)$$

mixes content with absolute positions m and n

ATTENTION SCORE

$$(R_m q_m)^\top (R_n k_n) = q_m^\top R_{n-m} k_n$$

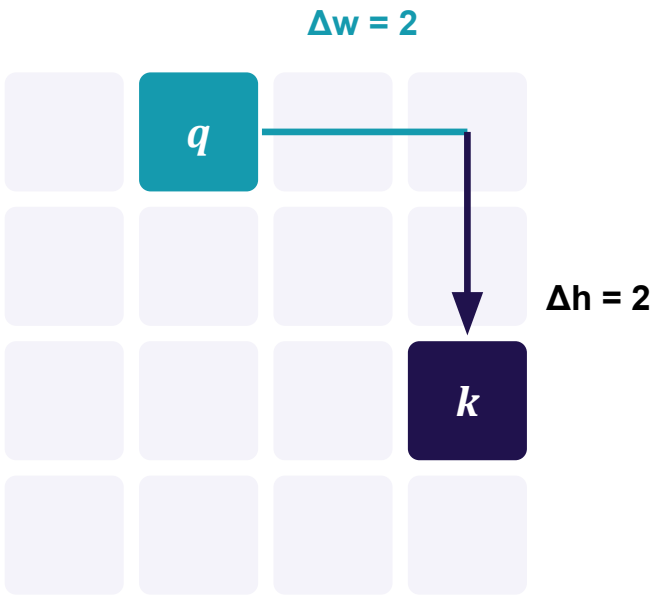
depends only on the distance n − m

paper →

Axial RoPE: attention sees both offsets

DLS

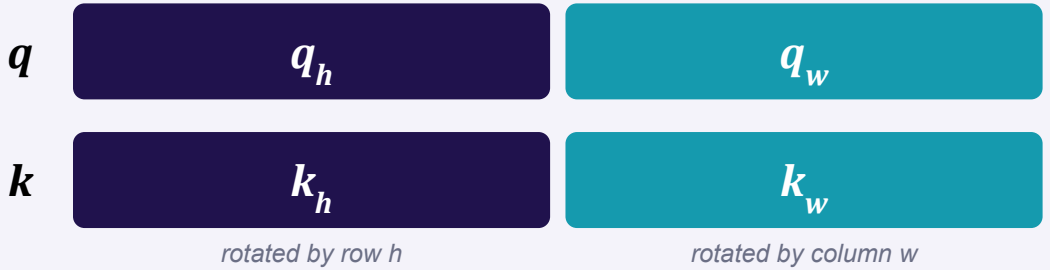
TWO TOKENS ON THE GRID



q at (0, 1) · k at (2, 3)

2 columns right and 2 rows down

EACH VECTOR IS SPLIT INTO TWO HALVES

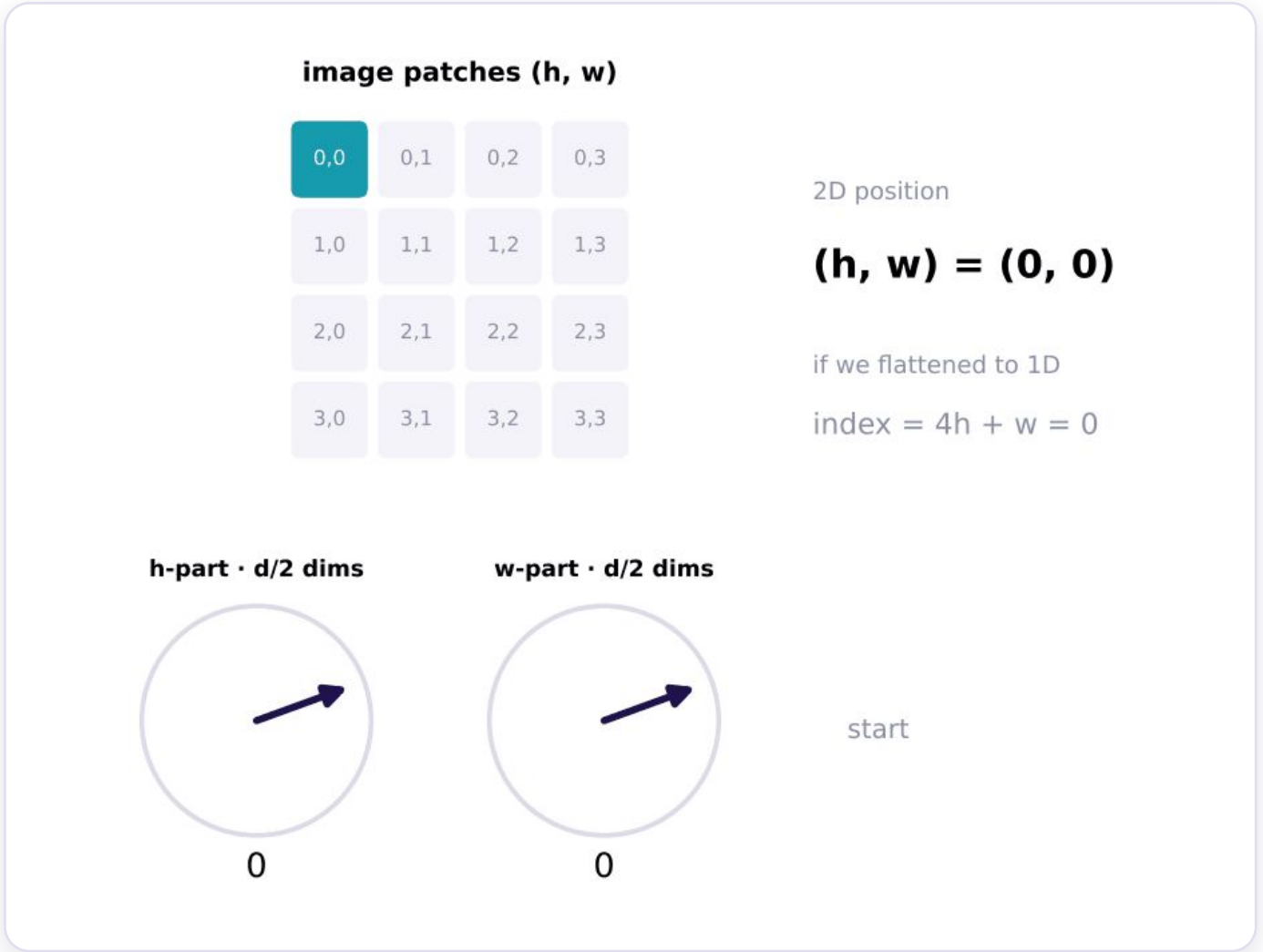


$$\text{score} = \underbrace{\langle q_h,\ R(\Delta h)\, k_h \rangle}_{\text{rows apart}} + \underbrace{\langle q_w,\ R(\Delta w)\, k_w \rangle}_{\text{columns apart}}$$

The two halves add up: attention takes both the vertical and the horizontal offset into account

paper →

2D RoPE: one rotation per axis

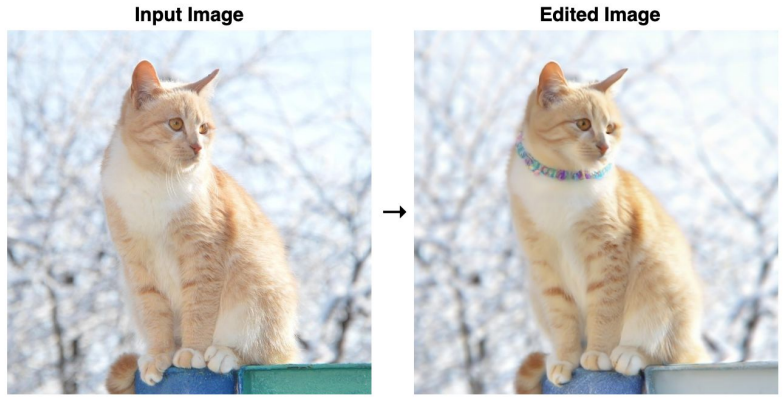


DLS

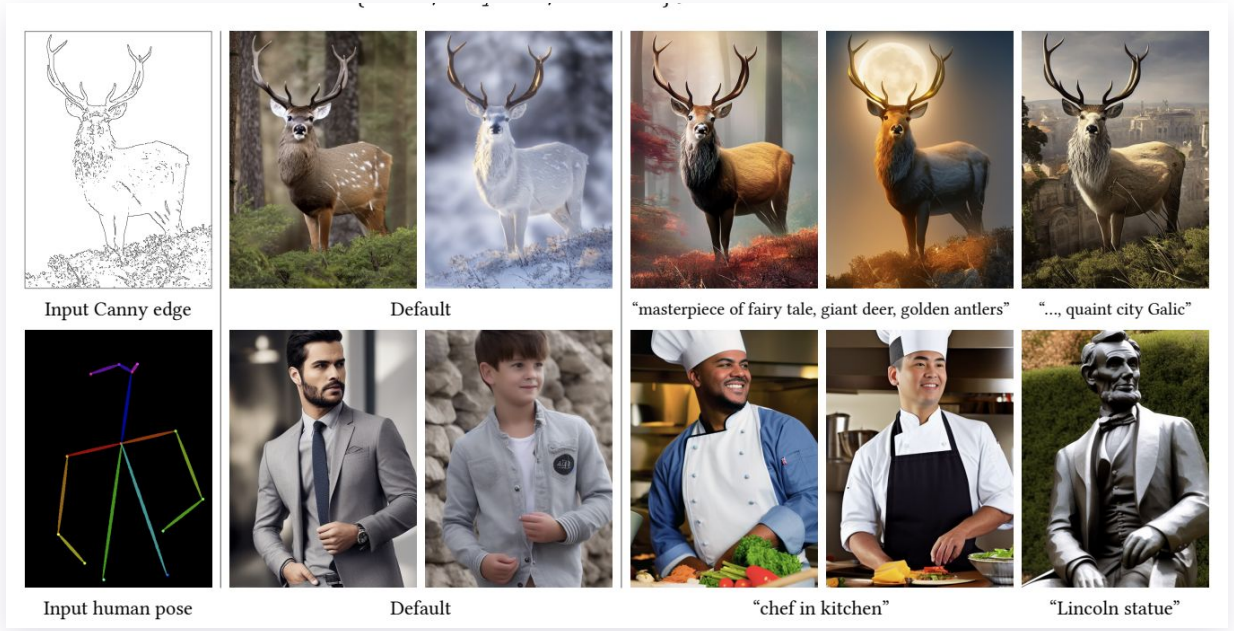
06

In-context models

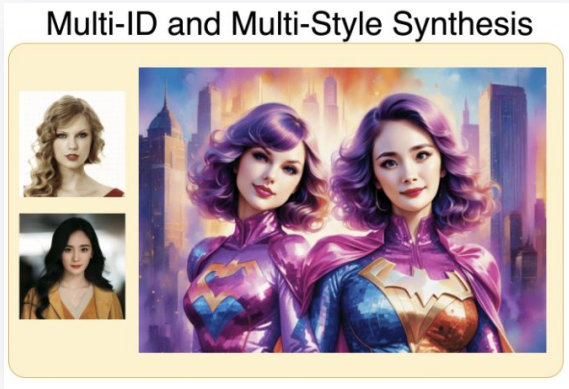
Other tasks


Input Image
Edited Image
"A cat wearing a hat an apron a necklace."

Input Image
Edited Image
"A person giving the thumbs up with crossed arms with their hands on hips."

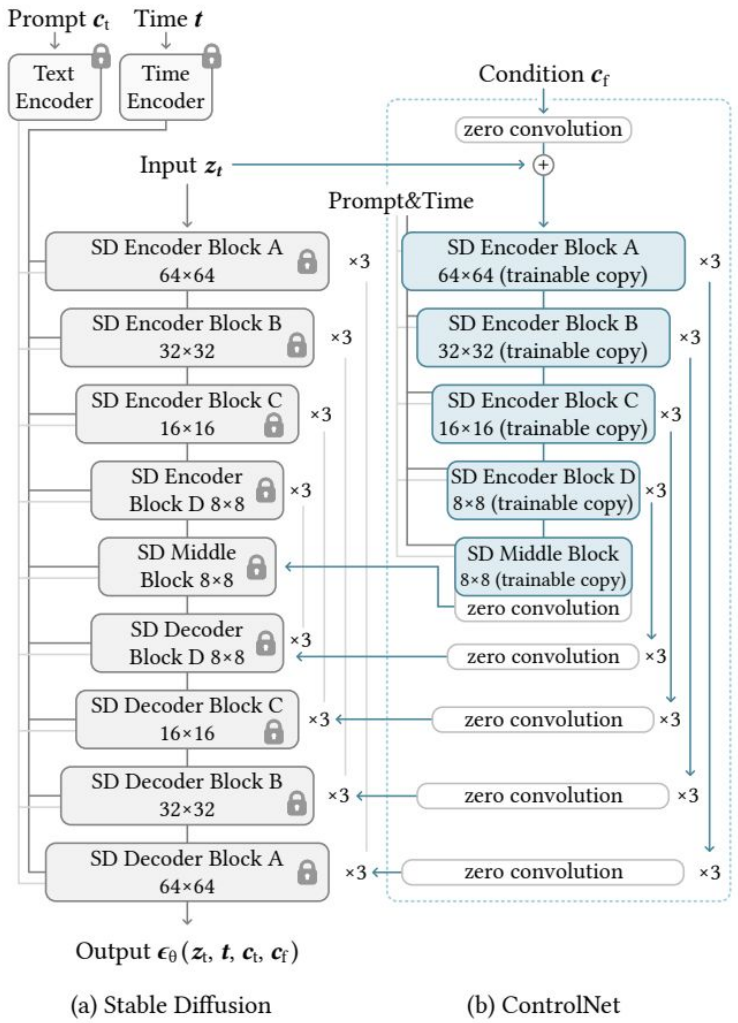

Input Canny edge
Default
"masterpiece of fairy tale, giant deer, golden antlers"
"..., quaint city Galic"
Input human pose
Default
"chef in kitchen"
"Lincoln statue"

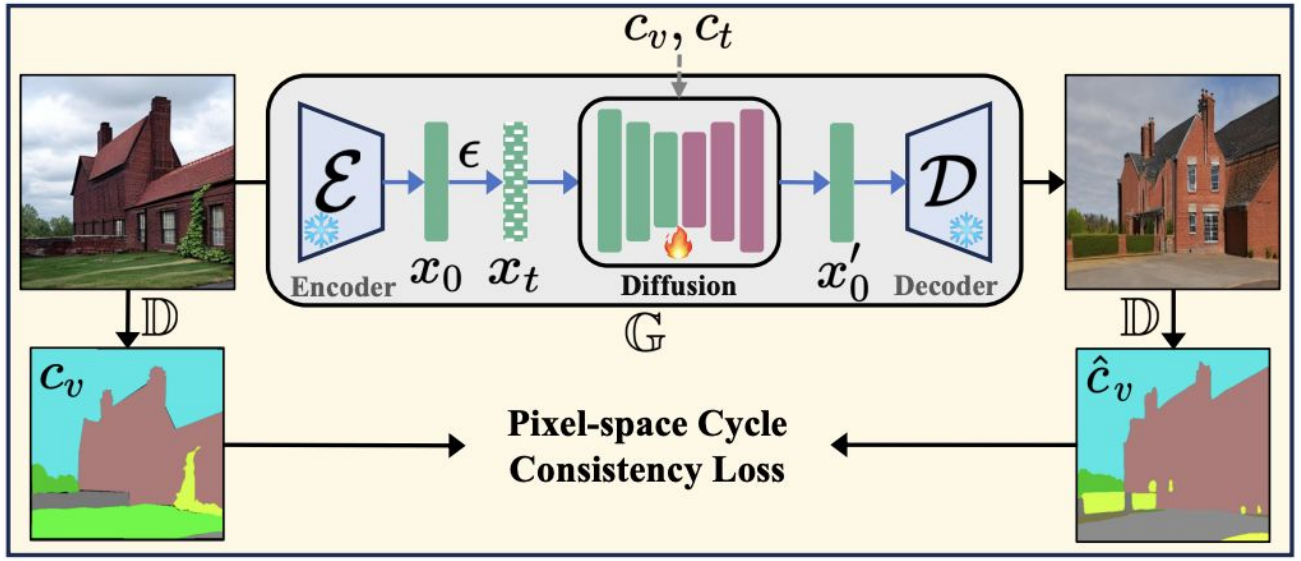
Stylized Synthesis


Multi-ID and Multi-Style Synthesis

paper → paper → paper →

ControlNet and others





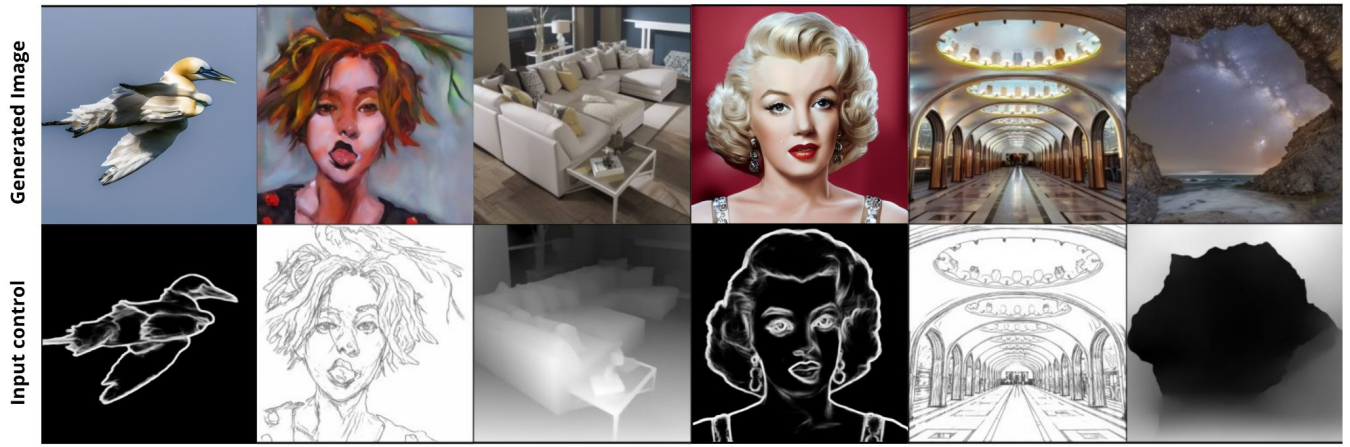


IMAGE EDITING

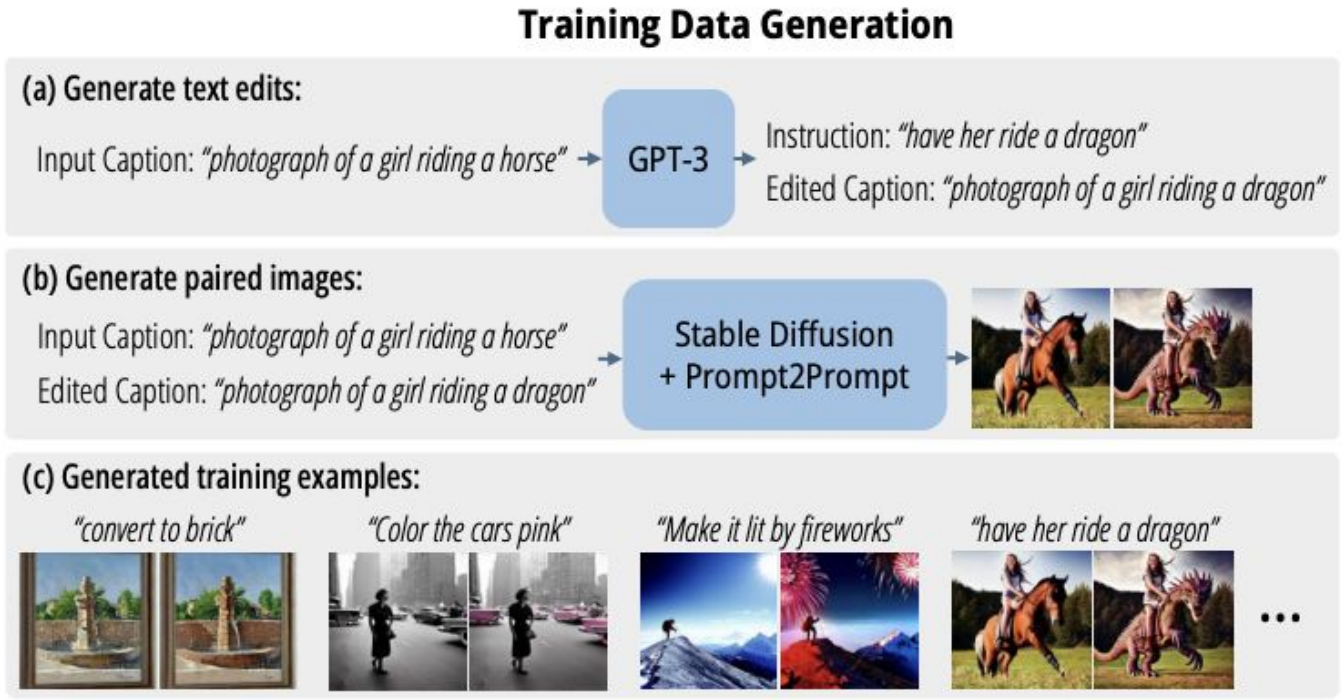
paper →

InstructPix2Pix

1. **Synthetic training pairs**
   GPT-3 writes edits, SD + Prompt-to-Prompt draws before / after

2. **Image as extra input channels**
   input latent concatenated to the noisy latent

3. **Two guidance scales**
   s_I keeps the input image, s_T pushes the edit



paper →

FLUX.1 Kontext: text + reference images

GOAL

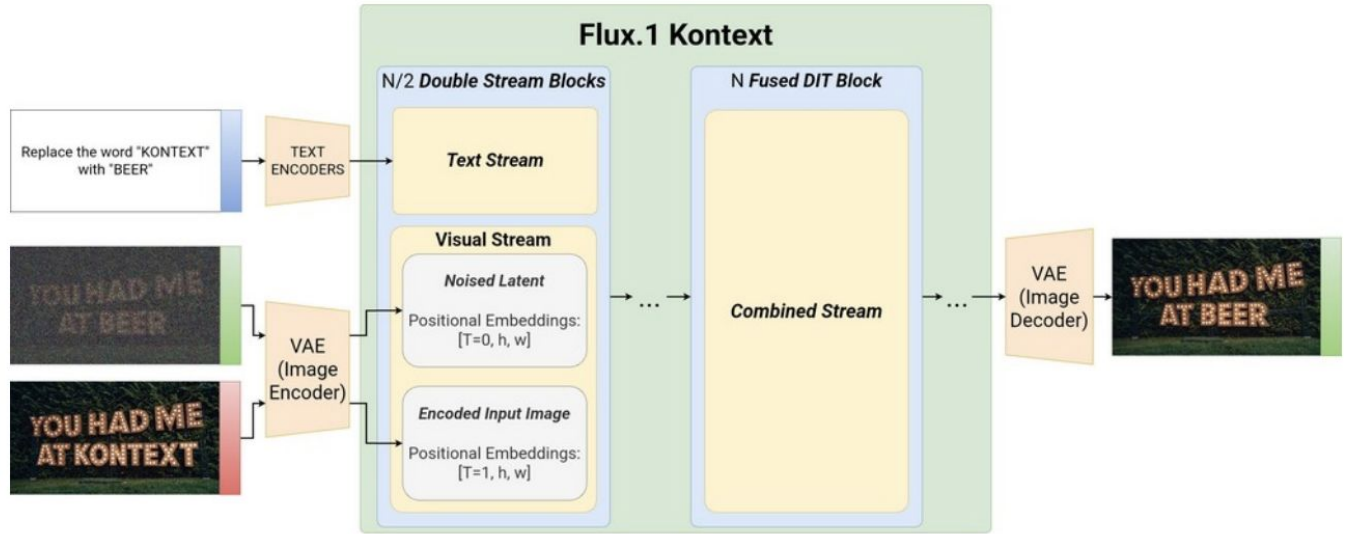
$$p_\theta(x \mid y,\ c)$$

x — target image, y — reference image(s) or ∅, c — text instruction

Key idea: sequence concatenation

Reference image → frozen VAE → tokens appended to the noisy target tokens in the visual stream.

Start: FLUX.1 text-to-image checkpoint, then joint fine-tuning on image-to-image and text-to-image



local editing
change one object, keep the rest

in-context generation
same character / style in a new scene

text-to-image
no reference: y = ∅

FLUX Kontext generation

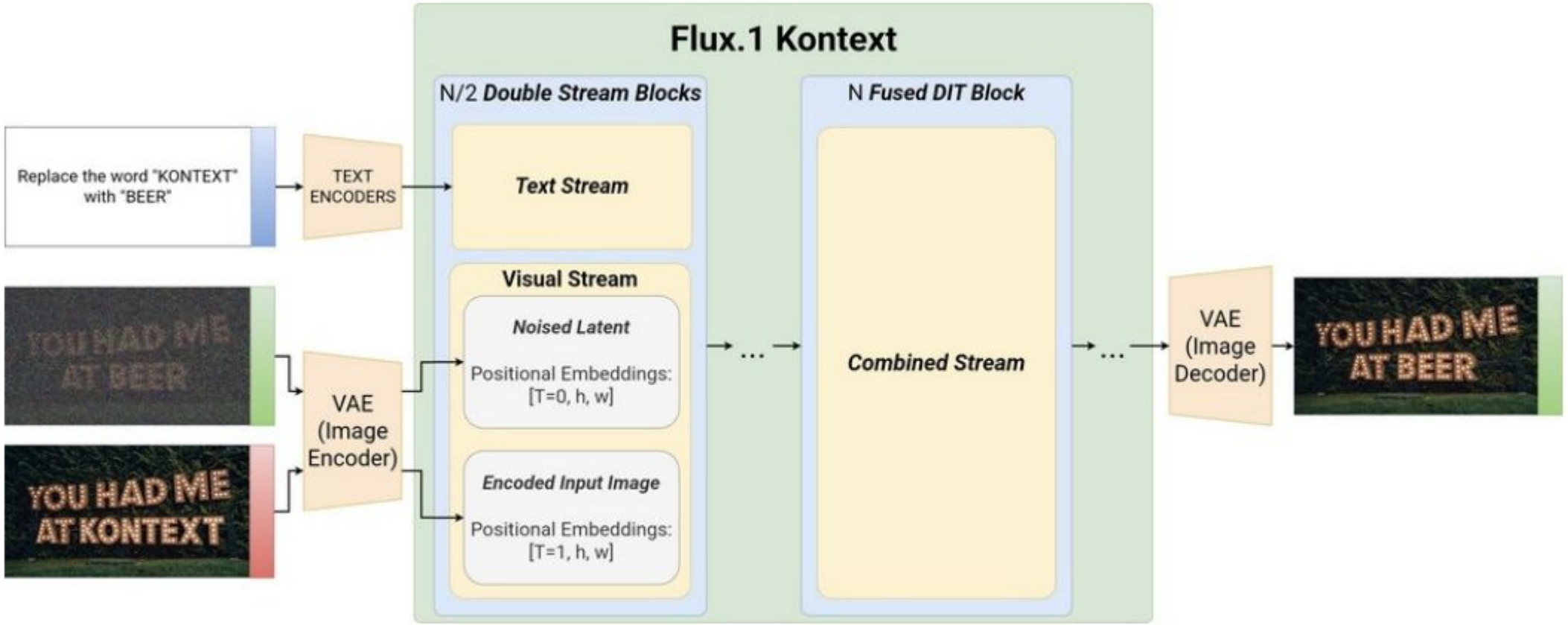
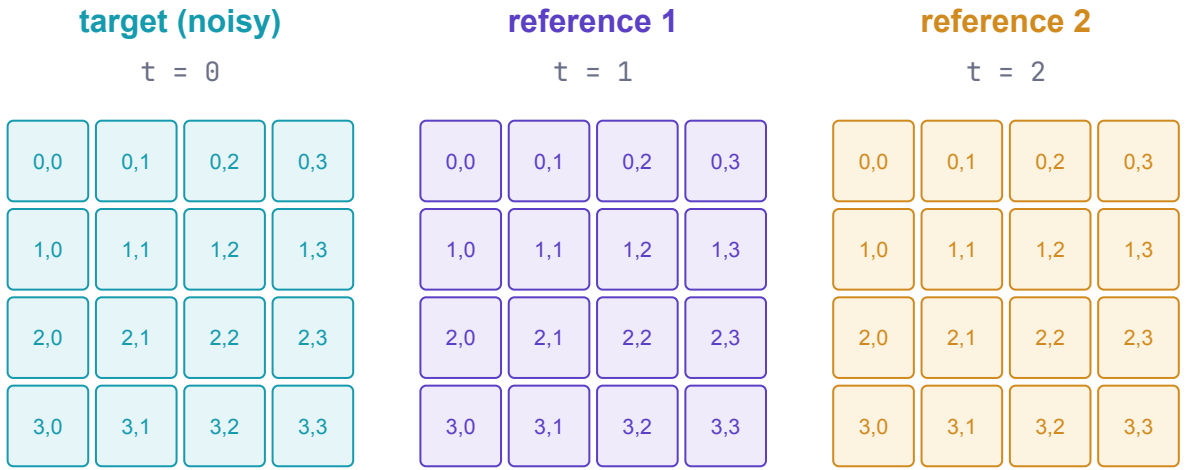


Figure 4: High-level overview of *FLUX.1 Kontext*, with input and context image on the left. Details in Section 3.

paper →

FLUX.1 Kontext: 3D RoPE

Every image token gets a position **(t, h, w)** — t tells the images apart, (h, w) is the place inside the image



same (h, w) grid in every image · only the index t changes

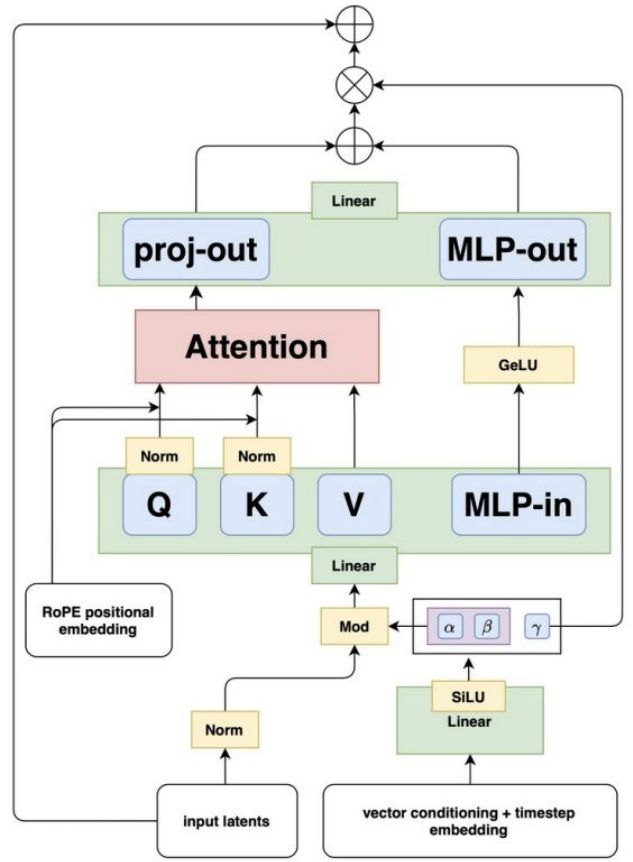
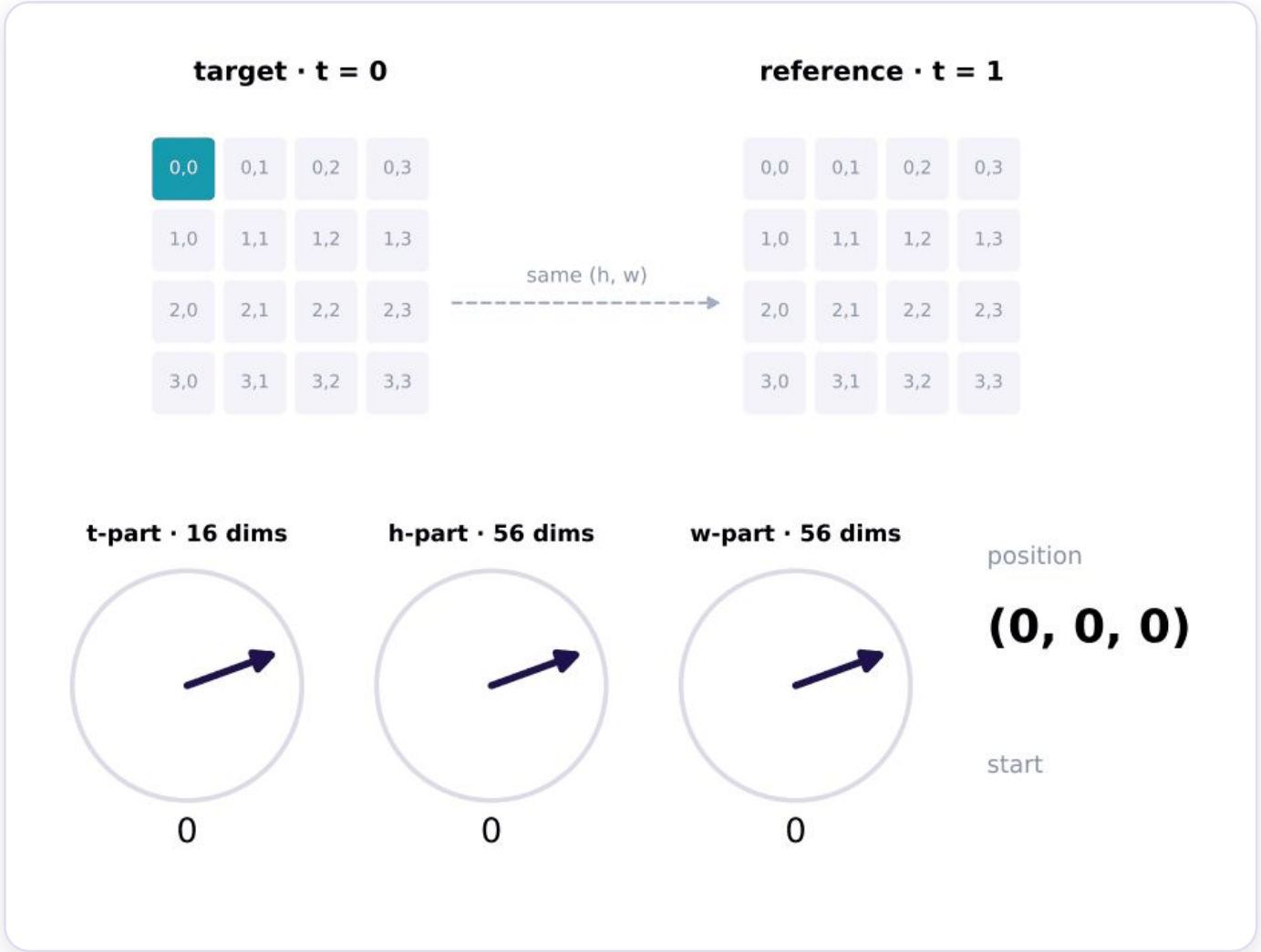


Figure 3: A fused DiT block equipped with rotary positional embeddings

paper →

3D RoPE: one rotation per axis



EACH HEAD IS SPLIT INTO 3 PARTS

FLUX: head dim 128

- each part is an ordinary 1D RoPE that rotates by its own coordinate
- moving inside an image turns only the h- or w-clock
- jumping to the reference turns only the t-clock: same (h, w), other image

Attention sees **(Δt, Δh, Δw)**: which image and how far apart

FLUX Kontext generation (Results)

(a) Input image

(b) *"extract only the skirt over a white background, product photography style"*

(c) *"show me an extreme closeup of the fabric"*

Figure 6: **Product Photography.** (a) Input image showing the full outfit. (b) Extracted skirt on a white background in a product-photography style. (c) Extreme close-up of the skirt's fabric, highlighting texture and pattern details.

(a) Input image

(b) *"Using this style, a kid on a bicycles rolls through desert ruins, spotlights scanning ancient scrolls projected as holographic sandstorms."*

(c) *"Using this style, a grand piano made of shifting mirrors performs itself for an audience of empty velvet chairs in zero-gravity."*

(e) Input image

(f) *"Using this style, a half-folded metropolis hangs from steel strings over an ink-wash ocean while cranes of light sketch new streets in mid-air."*

(g) *"Using this style, a dapper octopus conducts a jazz duo of owls on a shimmering moonlit bandstand."*

FLUX Context generation (Results)

(a) Input image

(b) *"make me a matching flower vase, product photography set against a white wall, sitting on a wooden desk, put some nice flowers in it"*

(c) *"change the vase base color to black"*

Figure 10: **Iterative, product-style editing.** Starting from the reference bowl (a), our model first generates a matching flower vase in a tabletop studio setting with fresh flowers (b), and subsequently changes the vase's base color to black while preserving the floral pattern, lighting, and composition (c).

Failure cases

(d) Input image

(e) *"move the coffee to the left"*

(f) Input image

(g) After six iterative edits.

Figure 15: Editing failure cases. *Top row:* An example for identity degradation: While the center image shows a good edit, preserving character identity the right one (using a slightly modified prompt) comes with significant identity loss. *Middle row:* Scene modification instead of object movement: the model adds milk foam rather than repositioning the mug. *Bottom row:* After six iterative edits, samples can exhibit visible artifacts.

From FLUX.1 to FLUX.2

From FLUX.1 to FLUX.2

- **Multi-Reference Support:** Reference up to 10 images simultaneously with the best character / product / style consistency available today.
- **Image Detail & Photorealism:** Greater detail, sharper textures, and more stable lighting suitable for product shots, visualization, and photography-like use cases.
- **Text Rendering:** Complex typography, infographics, memes and UI mockups with legible fine text now work reliably in production.
- **Enhanced Prompt Following:** Improved adherence to complex, structured instructions, including multi-part prompts and compositional constraints.
- **World Knowledge:** Significantly more grounded in real-world knowledge, lighting, and spatial logic, resulting in more coherent scenes with expected behavior.
- **Higher Resolution & Flexible Input/Output Ratios:** Image editing on resolutions up to 4MP.

code →

FLUX.2: architecture and model family

ARCHITECTURE

Mistral-3 · 24B VLM
reads the prompt and the reference images

Rectified flow transformer
32B in [dev]

new VAE · better reconstruction

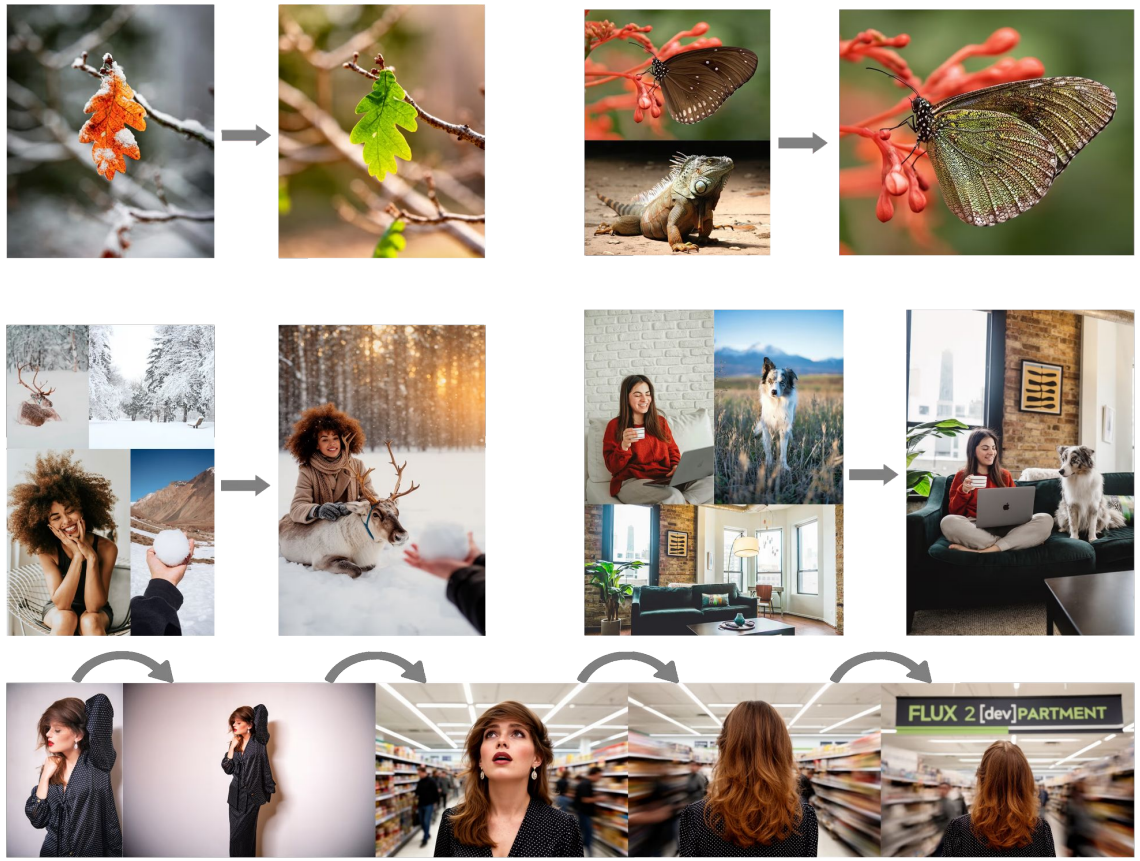
MODEL FAMILY

[pro]
closed · production

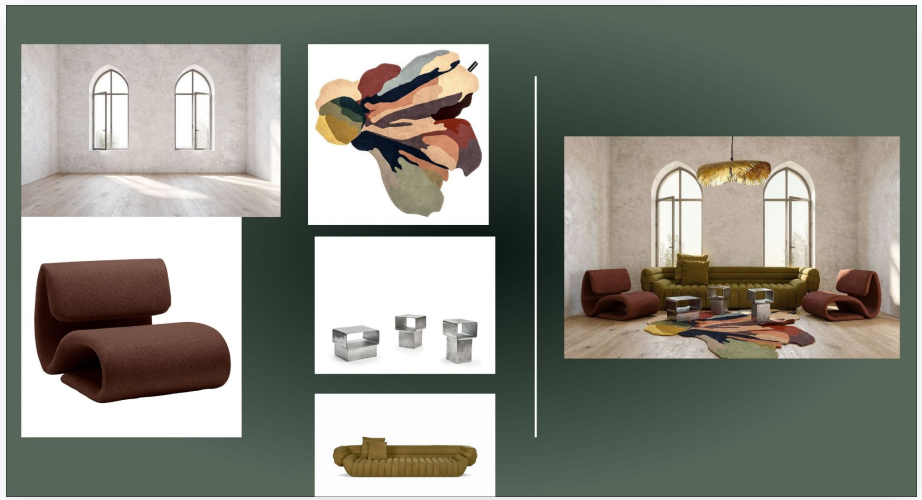
[flex]
closed · you set steps and guidance

[dev]
open weights · 32B

[klein]
distilled · 9B and 4B



From FLUX.1 to FLUX.2



But…

A photo of a horse riding an astronaut

DLS

07

Open-weight models

Arenas, Z-Image, Qwen-Image

Text-to-image arena

Summer 2026

	Range	Creator	Model
1	1–3	Fal	FLUX.2 [dev] Turbo · Open Weights
2	1–3	Black Forest Labs	FLUX.2 [dev] · Open Weights
3	1–3	Alibaba	Qwen Image Max 2512 · Open Weights
4	4	Fal	FLUX.2 [dev] Flash · Open Weights
5	5–6	Tencent	HunyuanImage 3.0 (Fal) · Open Weights
6	5–6	Black Forest Labs	FLUX.2 [klein] 9B · Open Weights
7	7–8	Tencent	HunyuanImage 3.0 Instruct (Fal) · Open Weights
8	7–8	Alibaba	Z-Image Turbo · Open Weights
9	9	Black Forest Labs	FLUX.2 [klein] Base 9B · Open Weights

October 2026

	Range	Model
1	1–2	Qwen-Image-2.1
2	1–2	Fibo 2.0
3	3	Ideogram 4.0 (Quality)
4	4–8	Ideogram 4.0
5	5	FLUX.2 [dev]
6	4–8	Ming-Image-0.1-Design
7	4–11	FLUX.2 [dev] Turbo
8	4–11	Cosmos3-Super-Text2Image (ag...
9	6–13	FLUX.2 [dev] Flash

artificialanalysis.ai/image/leaderboard/text…

Image editing arena

Summer 2026

	Range	Creator	Model
1	1	Tencent	HunyuanImage 3.0 Instruct (Fal) Open Weights
2	2–3	Black Forest Labs	FLUX.2 [klein] 9B Open Weights
3	2–4	Alibaba	Qwen Image Edit Plus 2511 Open Weights
4	3–4	Fal	FLUX.2 [dev] Turbo Open Weights
5	5–8	Alibaba	Qwen Image Edit Plus 2509 Open Weights
6	5–8	Black Forest Labs	FLUX.2 [klein] Base 9B Open Weights
7	5–8	Black Forest Labs	FLUX.2 [dev] Open Weights
8	5–8	Fal	FLUX.2 [dev] Flash Open Weights
9	9–10	Black Forest Labs	FLUX.2 [klein] 4B Open Weights

October 2026

	Range	Model
1	1–2	Fibo 2.0
2	1–3	Qwen-Image-2.1
3	2–4	FLUX 3 Image Klein
4	3–4	HunyuanImage 3.0 Instruct
5	5–7	FLUX.2 [dev] Turbo
6	5–8	FLUX.2 [dev] Flash
7	5–7	FLUX.2 [klein] 9B
8	8	FLUX.2 [dev]
9	8–9	Qwen Image Edit Plus 2511

artificialanalysis.ai/image/leaderboard/text…

Z-image

Figure 2 | Showcases of Z-Image-Turbo in bilingual text-rendering. All related prompts can be found in Appendix A.2.

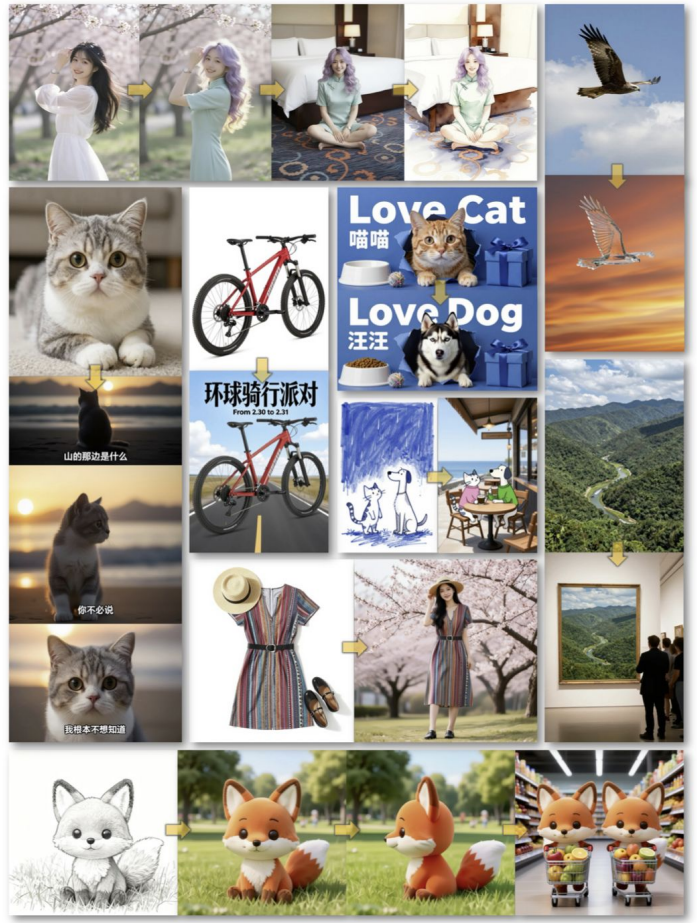


Figure 3 | Showcases of Z-Image-Edit in various image-to-image tasks. Each arrow represents an edit from the input to output images. All related prompts can be found in Appendix A.3.

paper →

Z-Image: efficiency everywhere

DLS

Data

efficient data infrastructure: curation at scale

Architecture

S3-DiT: single-stream multimodal DiT, only 6B

Training

low-res pre-training → omni pre-training → PE-aware SFT

Inference

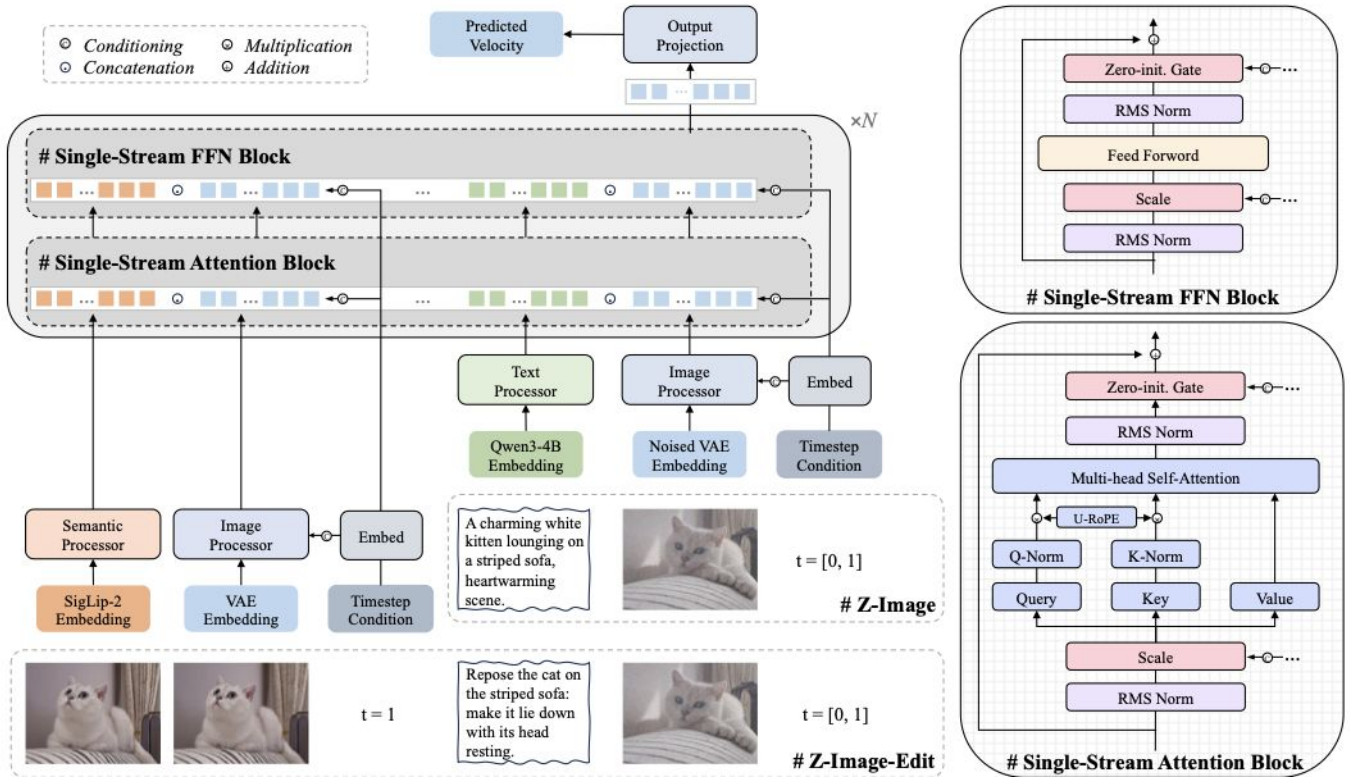
few-step distilled model: Z-Image-Turbo

Table 1 | Training costs of Z-Image, assuming the rental price of H800 is about $2 per GPU hour. The rental price refers from [37].

Training Costs	Low-res. Pre-Training	Omni-Pre-Training	Post-Training	Total
in H800 GPU Hours	147.5K	142.5K	24K	314K
in USD	$295K	$285K	$48K	$628K

paper →

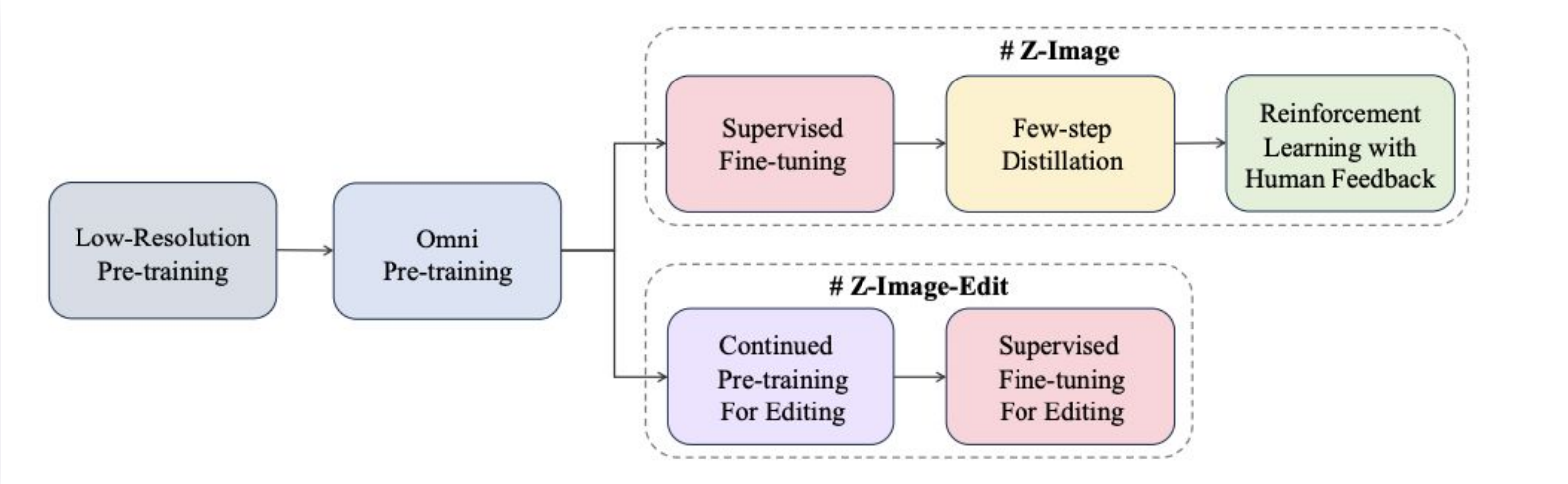
Z-Image: S3-DiT architecture



Single stream
text, image and condition tokens are concatenated into one sequence; every block is shared by all modalities

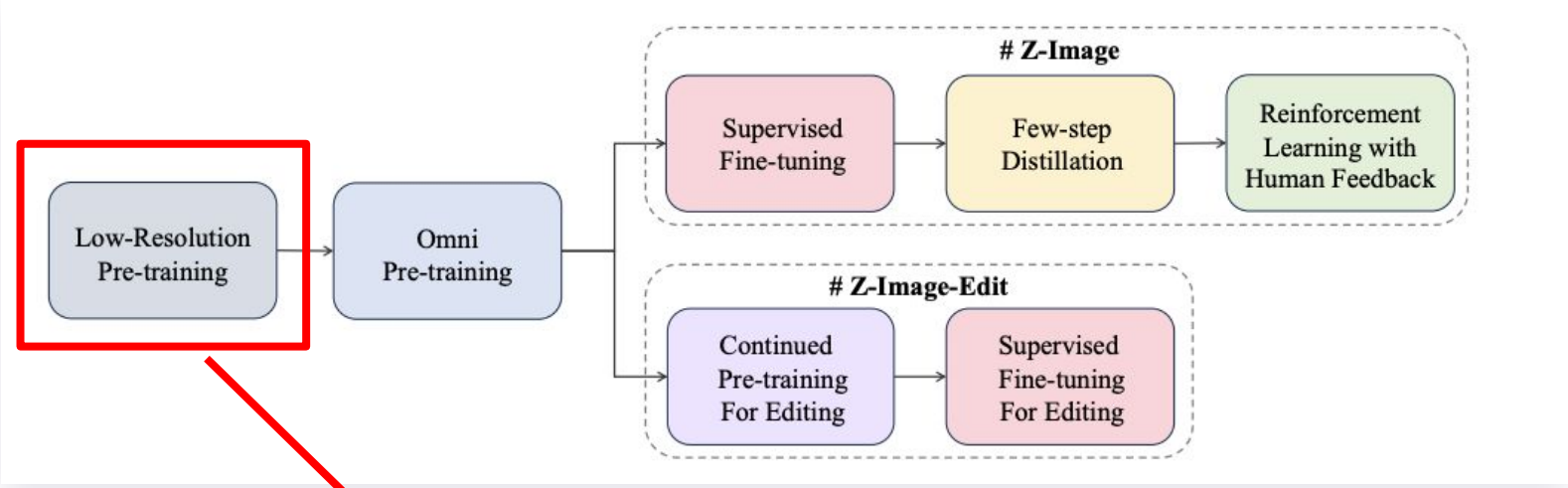
Light processors
small modality-specific processors: Qwen3-4B text, SigLIP-2 semantics, VAE latents

Z-image training



Training Costs	Low-res. Pre-Training
in H800 GPU Hours	147.5K
in USD	$295K

Z-image training

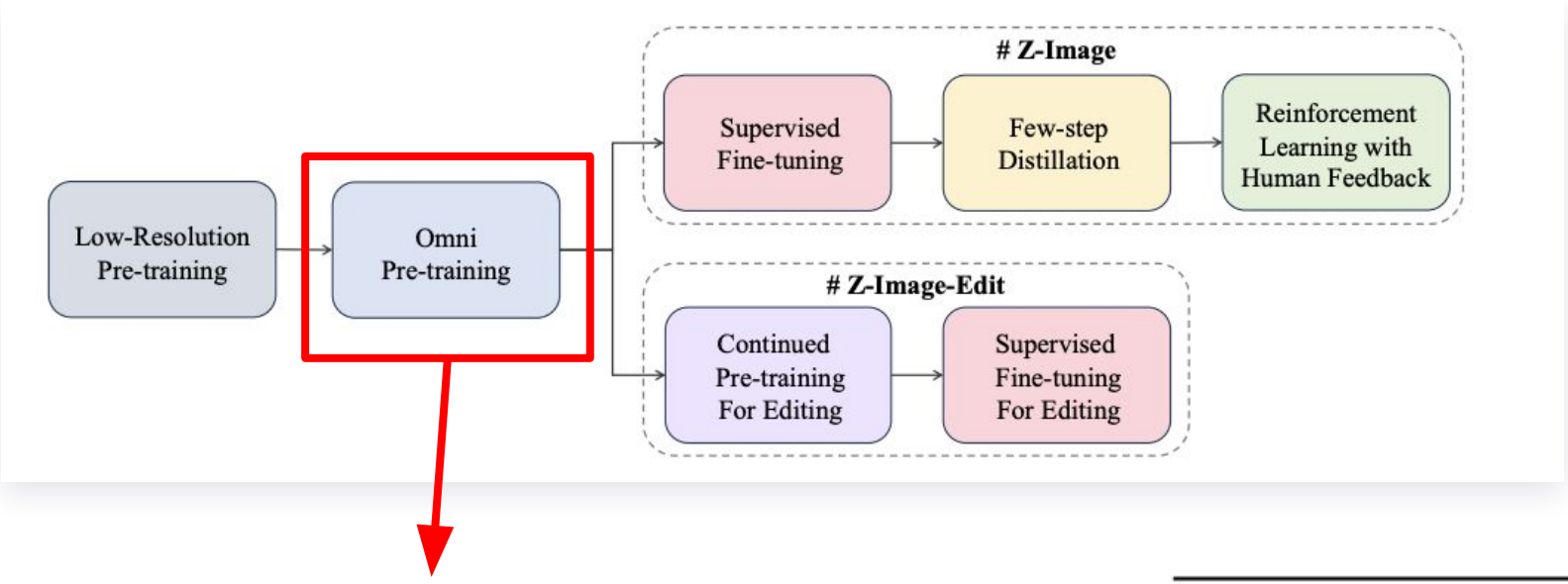


Training Costs	Low-res. Pre-Training
in H800 GPU Hours	147.5K
in USD	$295K

Learn the basics cheaply

- text-to-image only
- 256 × 256: short token sequences, so every step is cheap
- the model learns composition, objects and text–image alignment

Z-image training

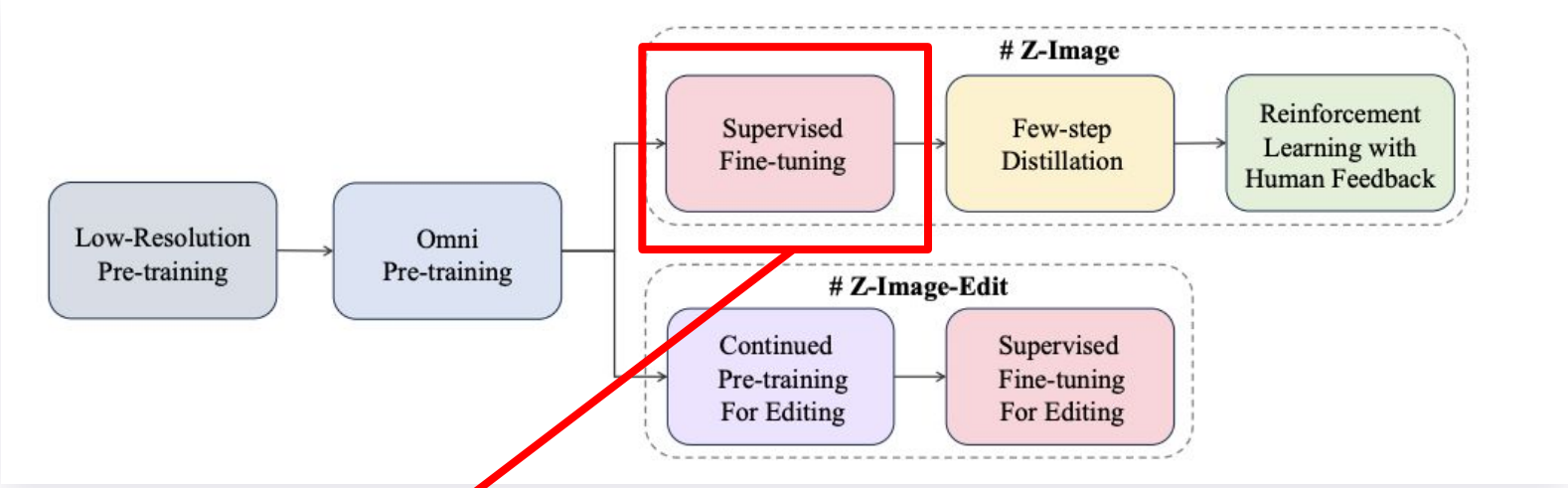


Training Costs	Omni-Pre-Training
in H800 GPU Hours	142.5K
in USD	$285K

One model for many tasks

- mixed resolutions up to 1.5K
- image-to-image tasks alongside text-to-image, incl. editing-difference pairs
- multi-level, bilingual captions: short to long, English and Chinese

Z-image training



Polish on curated data

- SFT on carefully curated data
- PE-aware: the model knows short prompts will be expanded by a prompt enhancer at inference
- model merging: several SFT checkpoints are averaged

Training Costs	Post-Training
in H800 GPU Hours	24K
in USD	$48K

Z-image, Data infrastructure

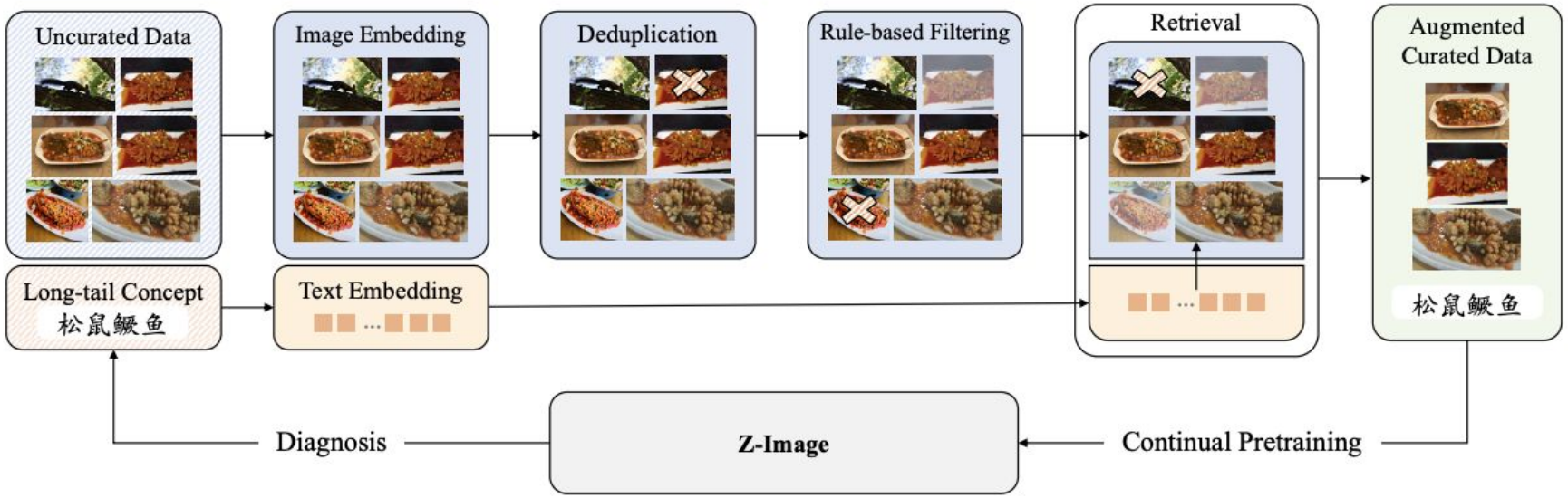


Figure 5 | Overview of the Active Curation Engine. The pipeline refines uncurated data through cross-modal embedding, deduplication, and rule-based filtering to construct a high-quality augmented dataset. A feedback mechanism leverages the Z-Image model to diagnose long-tail distribution deficiencies, dynamically guiding cross-modal retrieval to reinforce the data collection process. The "Squirrel Fish" (松鼠鳜鱼) case illustrates a classic long-tail challenge: it is actually the name of a Chinese cuisine but the model lacks the specific concept for this dish and may rely on compositional reasoning (combining "Squirrel" (松鼠) and "Fish" (鳜鱼)), leading to erroneous generations absent of domain-specific training data.

Z-image

Figure 12 | Intermediate generation results throughout Z-Image-Turbo's training process, echoing our analysis of each stage's contribution.

QWEN Image

Figure 3: Showcase of Qwen-Image in general image generation, supporting diverse artistic styles.

Figure 4: Showcase of Qwen-Image in general image editing, including style transfer, text editing, background change, object addition, removal, and replacement, pose manipulation, and more.

QWEN Image

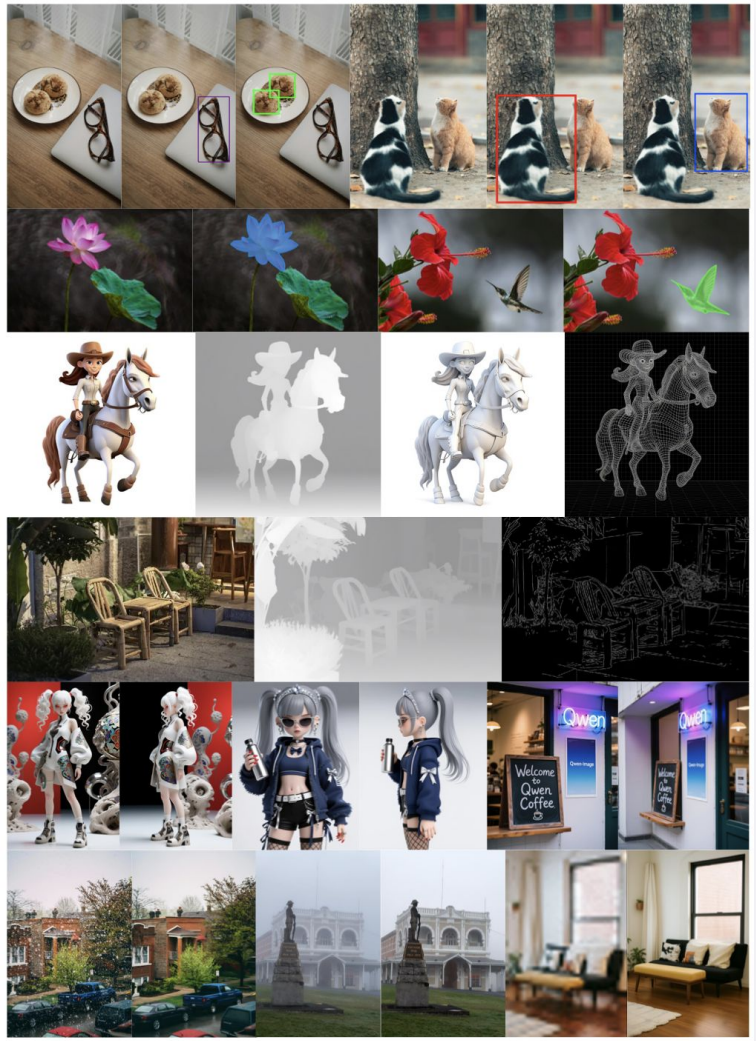


Figure 5: Showcase of Qwen-Image in general image understanding tasks, including detection, segmentation, depth/canny estimation, novel view synthesis, and super resolution—tasks that can all be viewed as specialized forms of image editing.

Figure 2: Showcase of Qwen-Image in complex text rendering, including multi-line layouts, paragraph-level semantics, and fine-grained details. Qwen-Image supports both alphabetic languages (e.g., English) and logographic languages (e.g., Chinese) with high fidelity.

QWEN Image dataset

paper →

Qwen-Image data: a 7-stage curriculum

Filtering gets stricter as training goes on — the data grows up together with the model

1

Initial curation

broken files, size, < 256p,, NSFW

paper →

Qwen-Image data: a 7-stage curriculum

Filtering gets stricter as training goes on — the data grows up together with the model

1 **Initial curation**

broken files, size, < 256p, dedup, NSFW

2 **Image quality**

EXIF rotation, clarity, saturation, entropy, texture

paper →

Qwen-Image data: a 7-stage curriculum

Filtering gets stricter as training goes on — the data grows up together with the model

1

Initial curation

broken files, size, < 256p, dedup, NSFW

2

Image quality

EXIF rotation, clarity, luma, saturation, entropy, texture

3

Image–text alignment

re-captioning + Chinese CLIP / SigLIP 2 filters

paper →

Qwen-Image data: a 7-stage curriculum

Filtering gets stricter as training goes on — the data grows up together with the model

+ synthetic data

1 Initial curation

broken files, size, < 256p, dedup, NSFW

2 Image quality

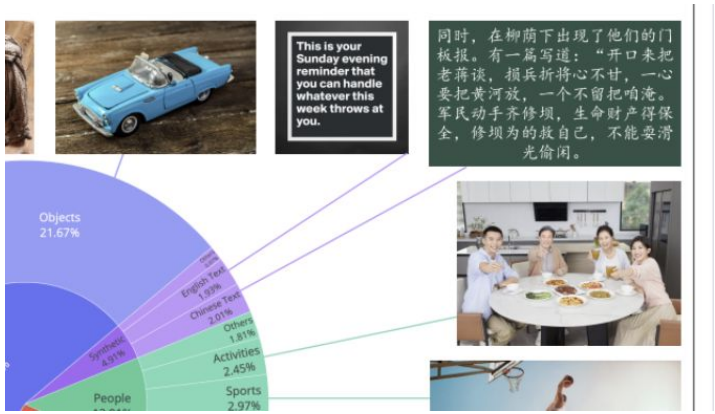
EXIF rotation, clarity, luma, saturation, entropy, texture

3 Image–text alignment

re-captioning + Chinese CLIP / SigLIP 2 filters

4 Text rendering

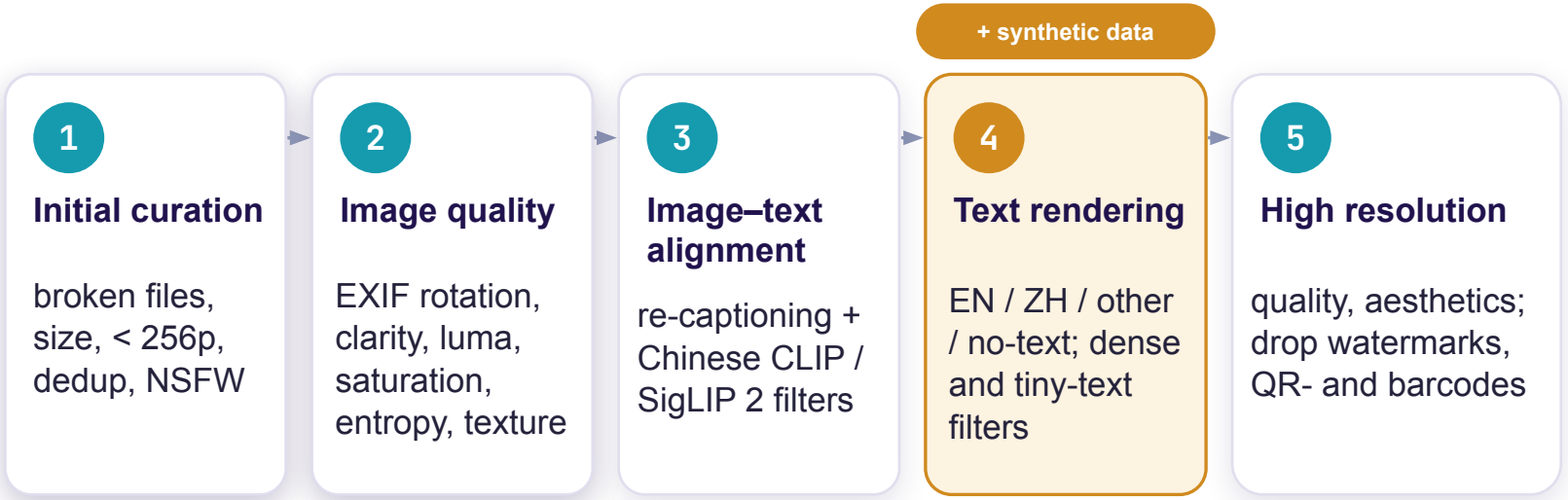
EN / ZH / other / no-text; dense and tiny-text filters



paper →

Qwen-Image data: a 7-stage curriculum

Filtering gets stricter as training goes on — the data grows up together with the model



paper →

Qwen-Image data: a 7-stage curriculum

Filtering gets stricter as training goes on — the data grows up together with the model

+ synthetic data

1 Initial curation

broken files, size, < 256p, dedup, NSFW

2 Image quality

EXIF rotation, clarity, luma, saturation, entropy, texture

3 Image–text alignment

re-captioning + Chinese CLIP / SigLIP 2 filters

4 Text rendering

EN / ZH / other / no-text; dense and tiny-text filters

5 High resolution

quality, aesthetics; drop watermarks, QR- and barcodes

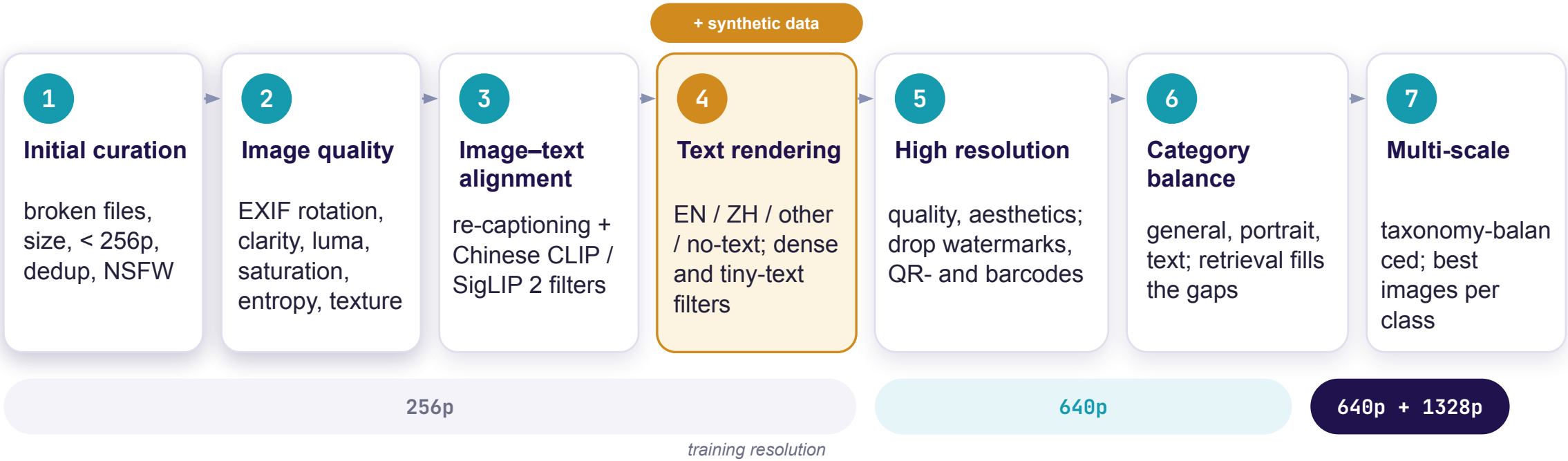
6 Category balance

general, portrait, text; retrieval fills the gaps

paper →

Qwen-Image data: a 7-stage curriculum

Filtering gets stricter as training goes on — the data grows up together with the model



paper →

Qwen-Image data: what is inside

DLS

DATA COMPOSITION

Nature 55%

objects, landscapes, cities, animals, food

Design 27%

posters, UI, slides, art

People 13%

portraits, sports, activities

Synthetic 5%

rendered text only, no AI images

paper →

Qwen-Image data: what is inside

DATA COMPOSITION

Nature 55%

objects, landscapes, cities, animals, food

Design 27%

posters, UI, slides, art

People 13%

portraits, sports, activities

Synthetic 5%

rendered text only, no AI images

SYNTHETIC TEXT RENDERING

1 · Pure

paragraphs on clean backgrounds; drop it if any character fails to render

2 · Compositional

text on paper or boards, pasted into real scenes

3 · Complex

slides and UI templates filled in by rules

paper →

Qwen-Image data: what is inside

DATA COMPOSITION

Nature 55%

objects, landscapes, cities, animals, food

Design 27%

posters, UI, slides, art

People 13%

portraits, sports, activities

Synthetic 5%

rendered text only, no AI images

SYNTHETIC TEXT RENDERING

1 · Pure

paragraphs on clean backgrounds; drop it if any character fails to render

2 · Compositional

text on paper or boards, pasted into real scenes

3 · Complex

slides and UI templates filled in by rules

easy → hard: why Qwen-Image is strong at text

ANNOTATION

Qwen2.5-VL writes a caption and structured metadata in one pass

```
{
  "caption": "…",
  "image_type": "photo",
  "image_style": "…",
  "watermark_list": [],
  "abnormal_element": …
}
```

- visible text is copied verbatim, in quotes
- rules + small classifiers double-check watermarks and content

paper →

Qwen-Image: architecture

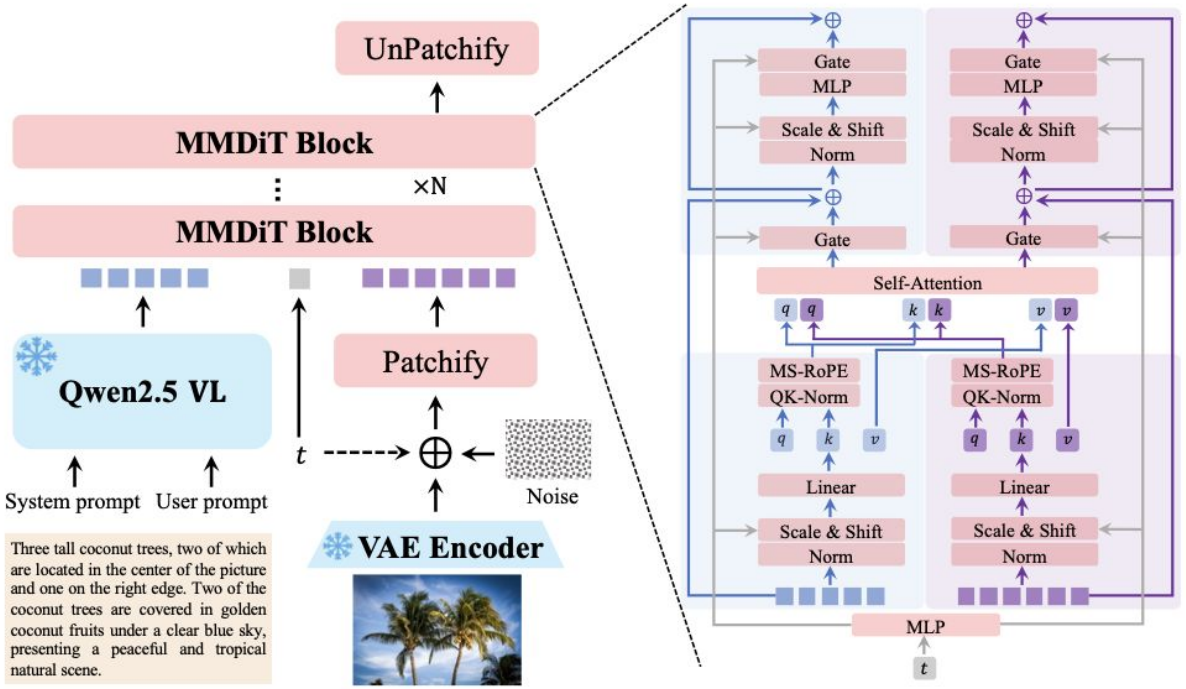


Figure 6: Overview of the Qwen-Image architecture. It adopts a standard double-stream MMDiT architecture. The input representations are provided by a frozen Qwen2.5-VL and a VAE encoder. The model employs RMSNorm (Zhang &Sennrich, 2019) for QK-Norm, while all other normalization layers use LayerNorm. Additionally, we design a new positional encoding scheme, MSRoPE (Multimodal Scalable RoPE), to jointly encode positional information for both image and text modalities.

paper →

Qwen-Image: architecture

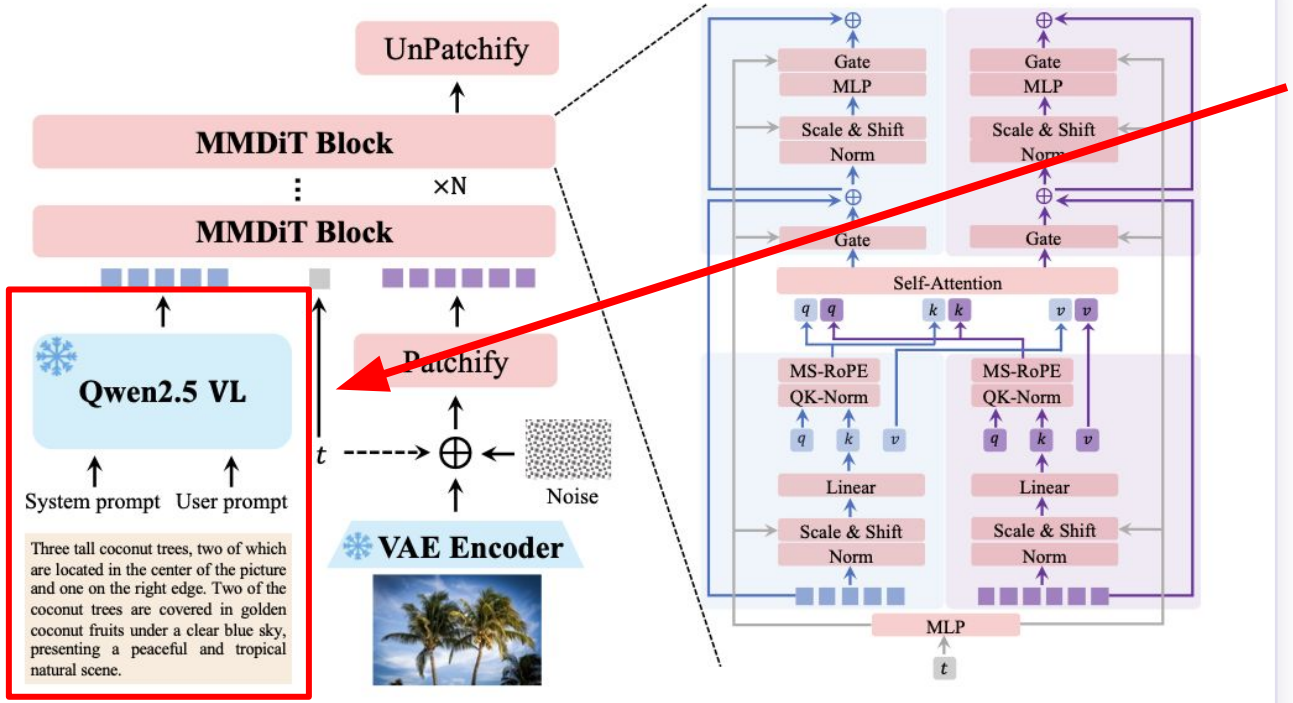


Figure 6: Overview of the Qwen-Image architecture. It adopts a standard double-stream MMDiT architecture. The input representations are provided by a frozen Qwen2.5-VL and a VAE encoder. The model employs RMSNorm (Zhang &Sennrich, 2019) for QK-Norm, while all other normalization layers use LayerNorm. Additionally, we design a new positional encoding scheme, MSRoPE (Multimodal Scalable RoPE), to jointly encode positional information for both image and text modalities.

MLLM · Qwen2.5-VL 7B

frozen condition encoder: text (and images for editing)

paper →

Qwen-Image: architecture

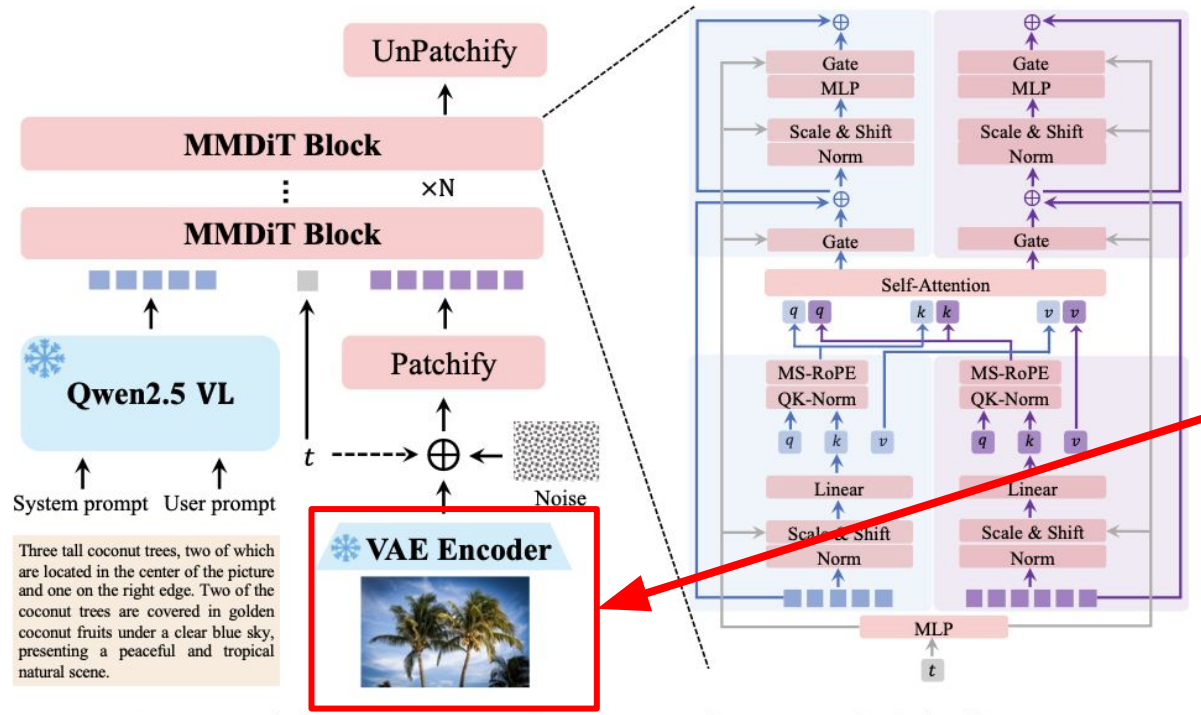


Figure 6: Overview of the Qwen-Image architecture. It adopts a standard double-stream MMDiT architecture. The input representations are provided by a frozen Qwen2.5-VL and a VAE encoder. The model employs RMSNorm (Zhang &Sennrich, 2019) for QK-Norm, while all other normalization layers use LayerNorm. Additionally, we design a new positional encoding scheme, MSRoPE (Multimodal Scalable RoPE), to jointly encode positional information for both image and text modalities.

MLLM · Qwen2.5-VL 7B

frozen condition encoder: text (and images for editing)

VAE · 16 channels, 8×8

single encoder (from Wan-2.1) + image-tuned decoder

paper →

Qwen-Image: architecture

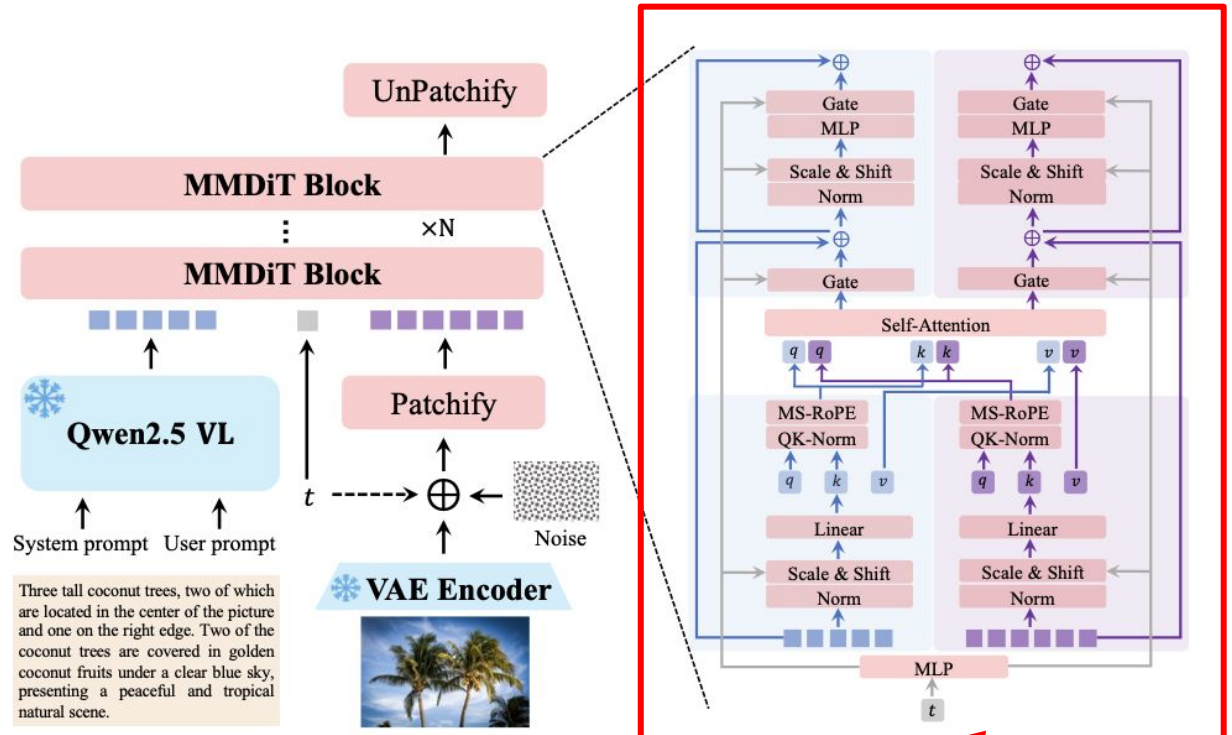


Figure 6: Overview of the Qwen-Image architecture. It adopts a standard double-stream MMDiT architecture. The input representations are provided by a frozen Qwen2.5-VL and a VAE encoder. The model employs RMSNorm (Zhang &Sennrich, 2019) for QK-Norm, while all other normalization layers use LayerNorm. Additionally, we design a new positional encoding scheme, MSRoPE (Multimodal Scalable RoPE), to jointly encode positional information for both image and text modalities.

MLLM · Qwen2.5-VL 7B

frozen condition encoder: text (and images for editing)

VAE · 16 channels, 8×8

single encoder (from Wan-2.1) + image-tuned decoder

MMDiT · 20B, 60 layers

double-stream blocks with MSRoPE, flow matching

paper →

Qwen-Image: MSRoPE

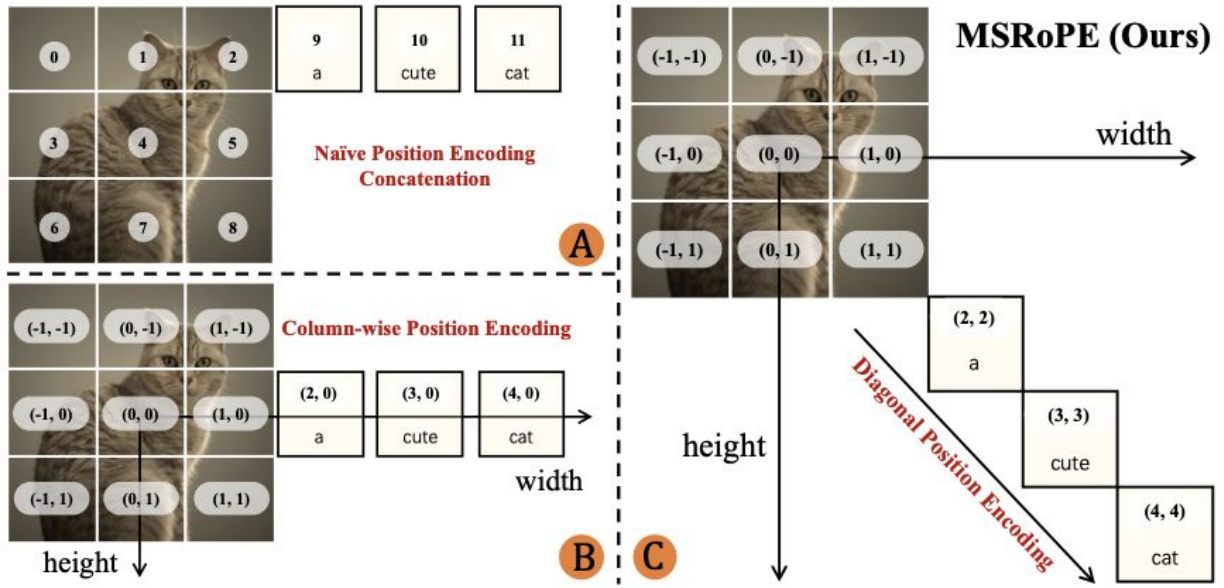


Figure 8: Comparison of different image-text joint positional encoding strategies. We design the Multimodal Scalable RoPE (MSRoPE) strategy, which starts encoding from the image center and positions textual encodings along the diagonal of the grid, enabling better image resolution scaling and improved text-image alignment.

A · Naive

text positions simply continue after the flattened image positions

B · Column-wise

text tokens are placed as an extra column next to the image grid

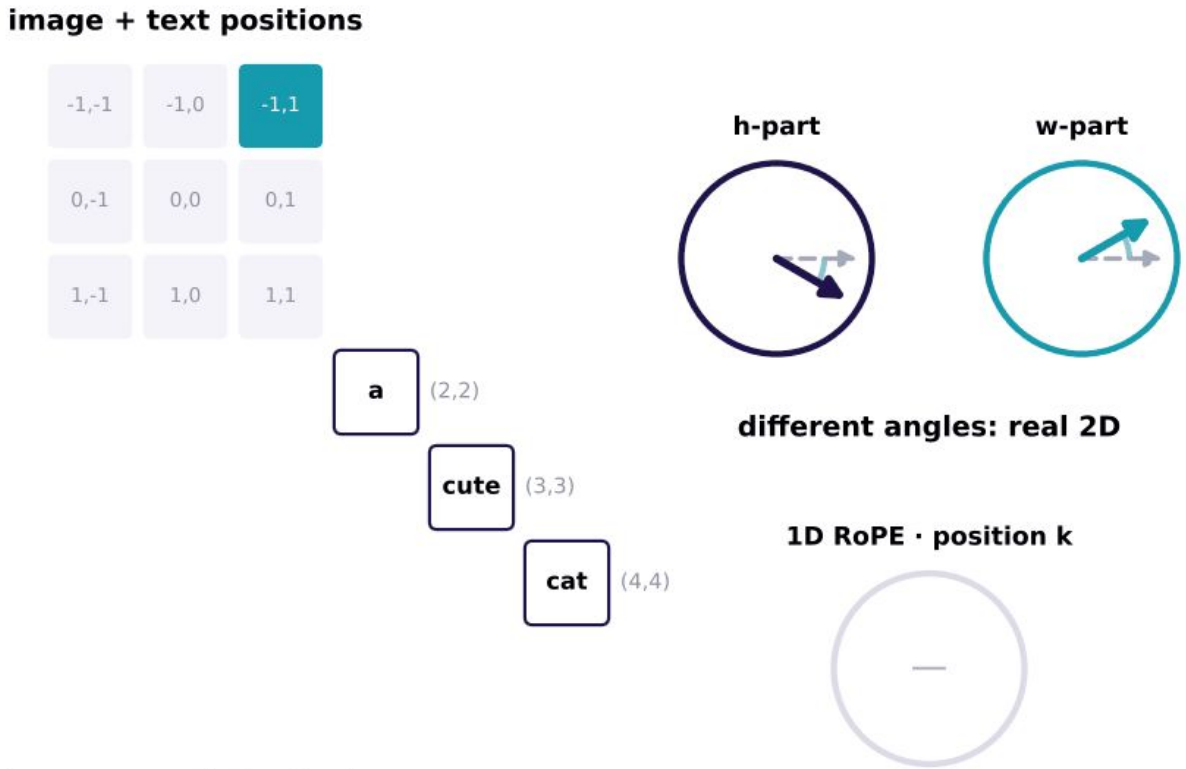
C · MSRoPE

image positions start from the centre, text goes along the diagonal

For text, MSRoPE is equivalent to ordinary 1D RoPE

paper →

MSRoPE: text on the diagonal = 1D RoPE



text token at (k, k):

$$R_{(k,\,k)} = \mathrm{diag}\big(R(k\,\theta^{(h)}),\ R(k\,\theta^{(w)})\big) = R_k^{\mathrm{1D}}$$

both halves rotate by the same k

Text stays an ordinary 1D sequence,

images get real 2D positions.

Image generation metrics

WHAT DO WE WANT TO MEASURE?

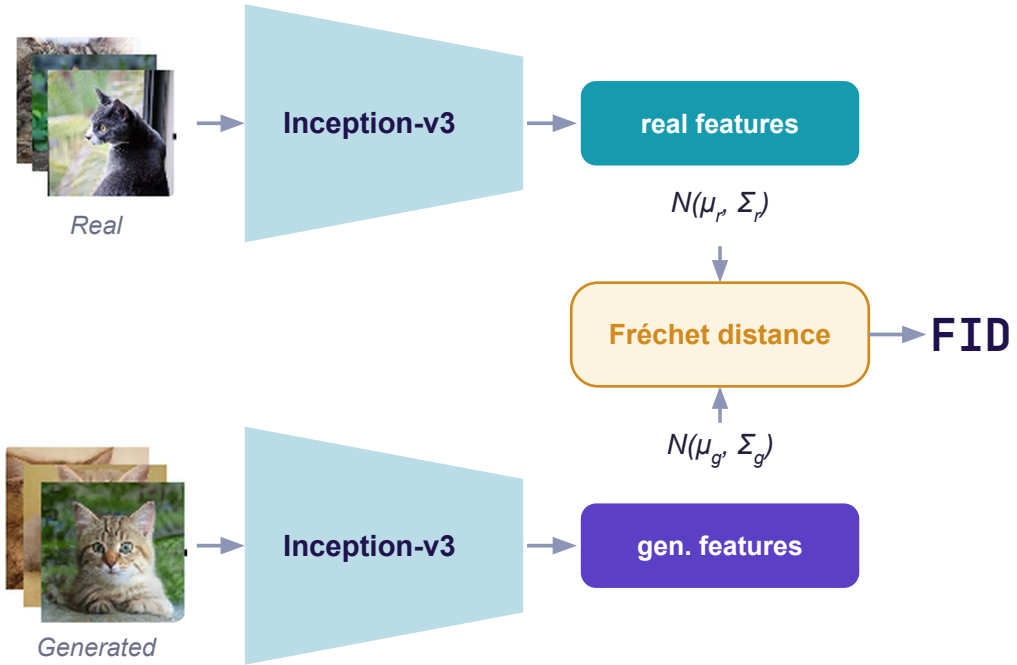
Quality & diversity	FID, FD-DINOv2, KID, CMMD
Prompt alignment	CLIP score, BLIP score
Human preference	ImageReward, PickScore
Gold standard	User study

Text rendering? Diversity per prompt? Safety? What else?

paper →

Fréchet Inception Distance (FID)

$$\mathrm{FID} = \|\mu_r - \mu_g\|_2^2 + \mathrm{Tr}\left(\Sigma_r + \Sigma_g - 2\left(\Sigma_r \Sigma_g\right)^{1/2}\right)$$



IDEA

- embed real and generated images with Inception-v3 (an ImageNet classifier)
- fit a Gaussian to each set and compare them: lower is better

PROBLEMS

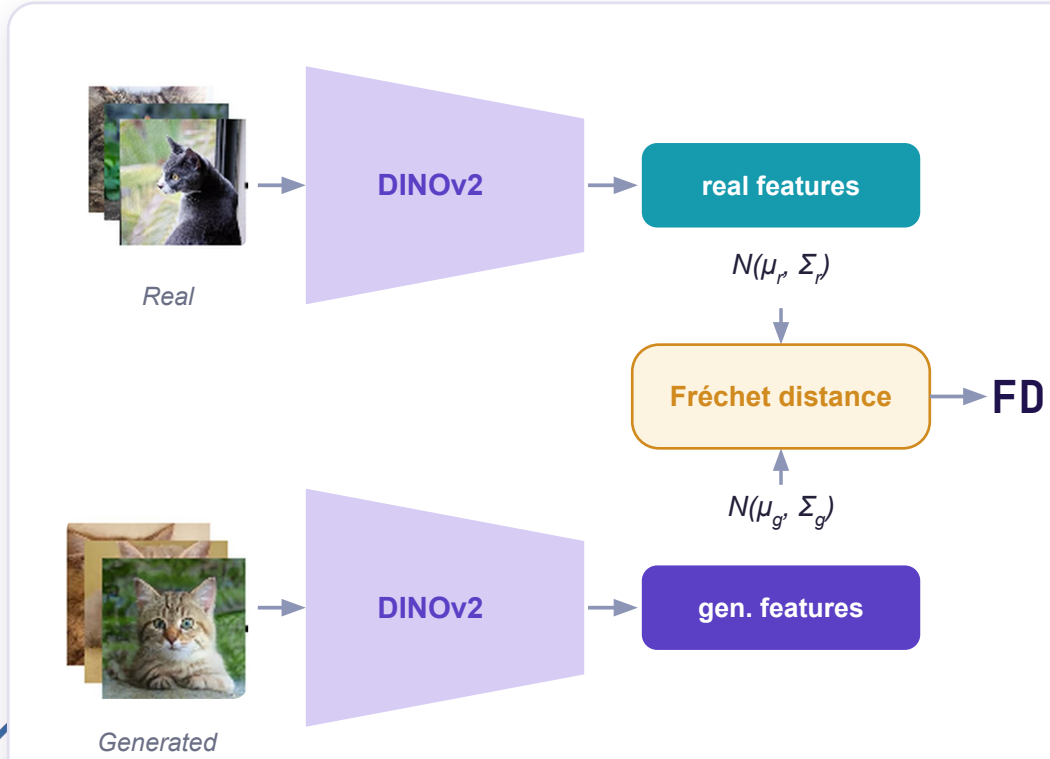
- needs many samples (~10K+, usually 50K)
- old, ImageNet-centric features
- the Gaussian assumption is weak

…but still the default metric in most papers

paper →

Fréchet Distance on DINOv2 (FD-DINOv2)

$$\mathrm{FD}_{\mathrm{DINOv2}} = \|\mu_r - \mu_g\|_2^2 + \mathrm{Tr}\left(\Sigma_r + \Sigma_g - 2\left(\Sigma_r \Sigma_g\right)^{1/2}\right)$$



IDEA

- same formula, but DINOv2 ViT-L/14 features instead of Inception-v3
- self-supervised features match human judgement much better

PROBLEMS

- still needs many samples
- still the Gaussian assumption

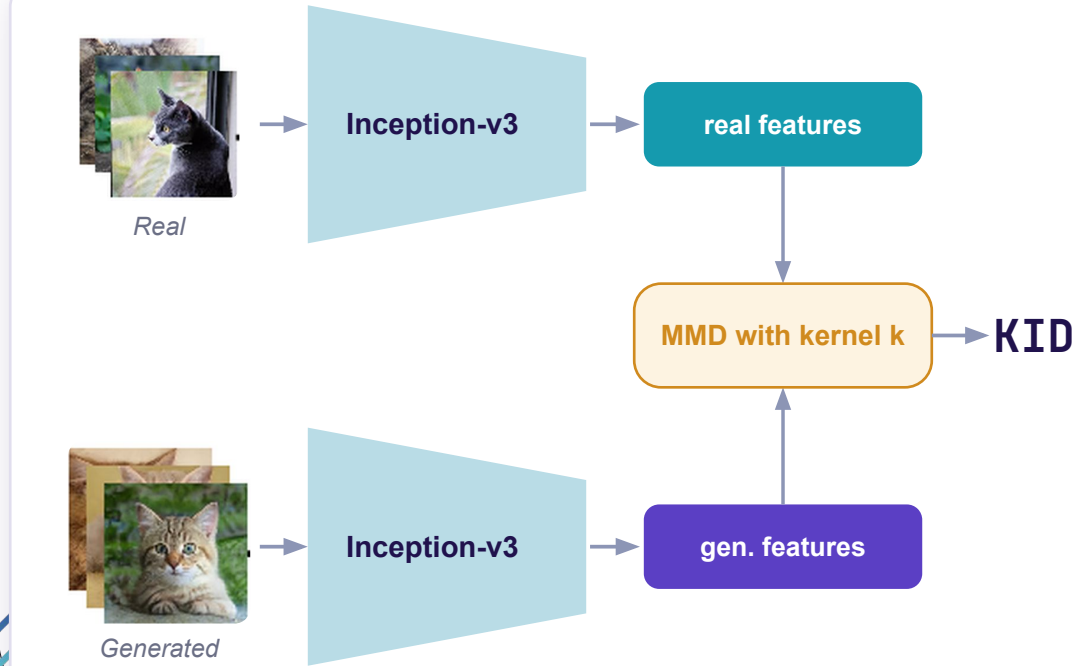
reported next to FID in newer papers, e.g. EDM2

paper →

Kernel Inception Distance (KID)

$$k(x, y) = \left(\frac{1}{d} x^\top y + 1\right)^3$$

$$\mathrm{MMD}^2 = \frac{1}{m(m-1)} \sum_{i \neq j} k(x_i, x_j) + \frac{1}{n(n-1)} \sum_{i \neq j} k(y_i, y_j) - \frac{2}{mn} \sum_{i,j} k(x_i, y_j)$$



PROS

- no Gaussian assumption
- unbiased: stable with ~1K samples

CONS

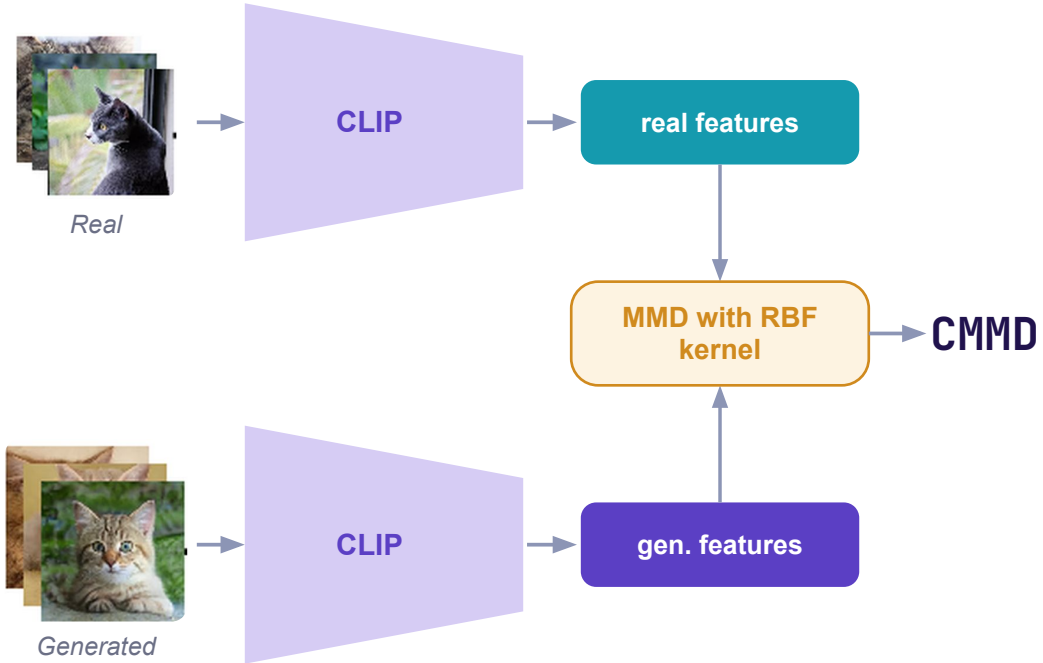
- still Inception-v3 features
- rarely reported in papers

paper →

CLIP Maximum Mean Discrepancy (CMMD)

$$k(x, y) = \exp\left(-\frac{\|x - y\|^2}{2\sigma^2}\right)$$

$$\mathrm{MMD}^2 = \frac{1}{m(m-1)}\sum_{i \neq j} k(x_i, x_j) + \frac{1}{n(n-1)}\sum_{i \neq j} k(y_i, y_j) - \frac{2}{mn}\sum_{i,j} k(x_i, y_j)$$



PROS

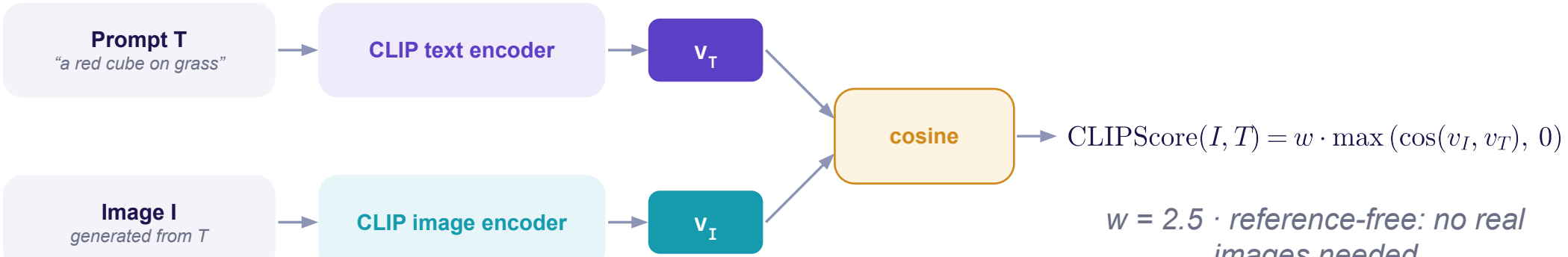
- CLIP knows far more than ImageNet classes
- no Gaussian assumption, sample-efficient

CONS

- inherits CLIP's own biases
- still not widely adopted

paper →

CLIP score



USED FOR

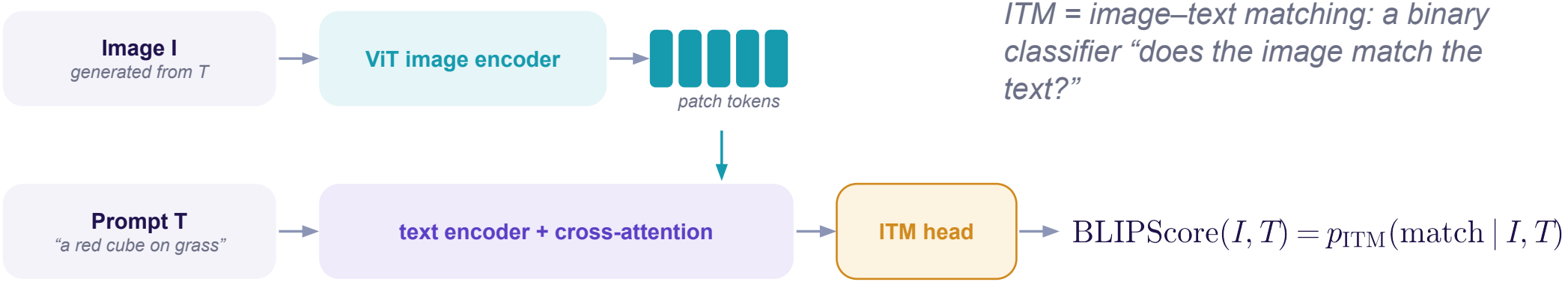
- prompt alignment in almost every T2I paper
- filtering and re-ranking samples
- quick sanity check during training

PROBLEMS

- bag-of-words: ignores word order and relations
- weak at counting, spatial relations, attribute binding
- saturates: strong models get similar scores

paper →

BLIP score



ITM = image–text matching: a binary classifier “does the image match the text?”

WHY BETTER THAN CLIP

- text tokens attend to image patches, not to one global vector
- trained with hard negatives: checks details

PROBLEMS

- still weak at counting and complex scenes
- not standardised: ITM probability or ITC cosine — check what a paper reports

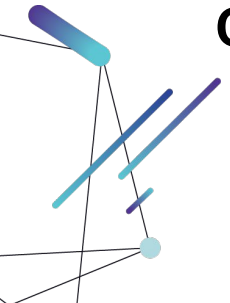
From CLIP score to human preference

CLIP SCORE ASKS ONE QUESTION

“How similar is the image to the prompt, according to CLIP?”

PEOPLE LOOK AT MANY THINGS

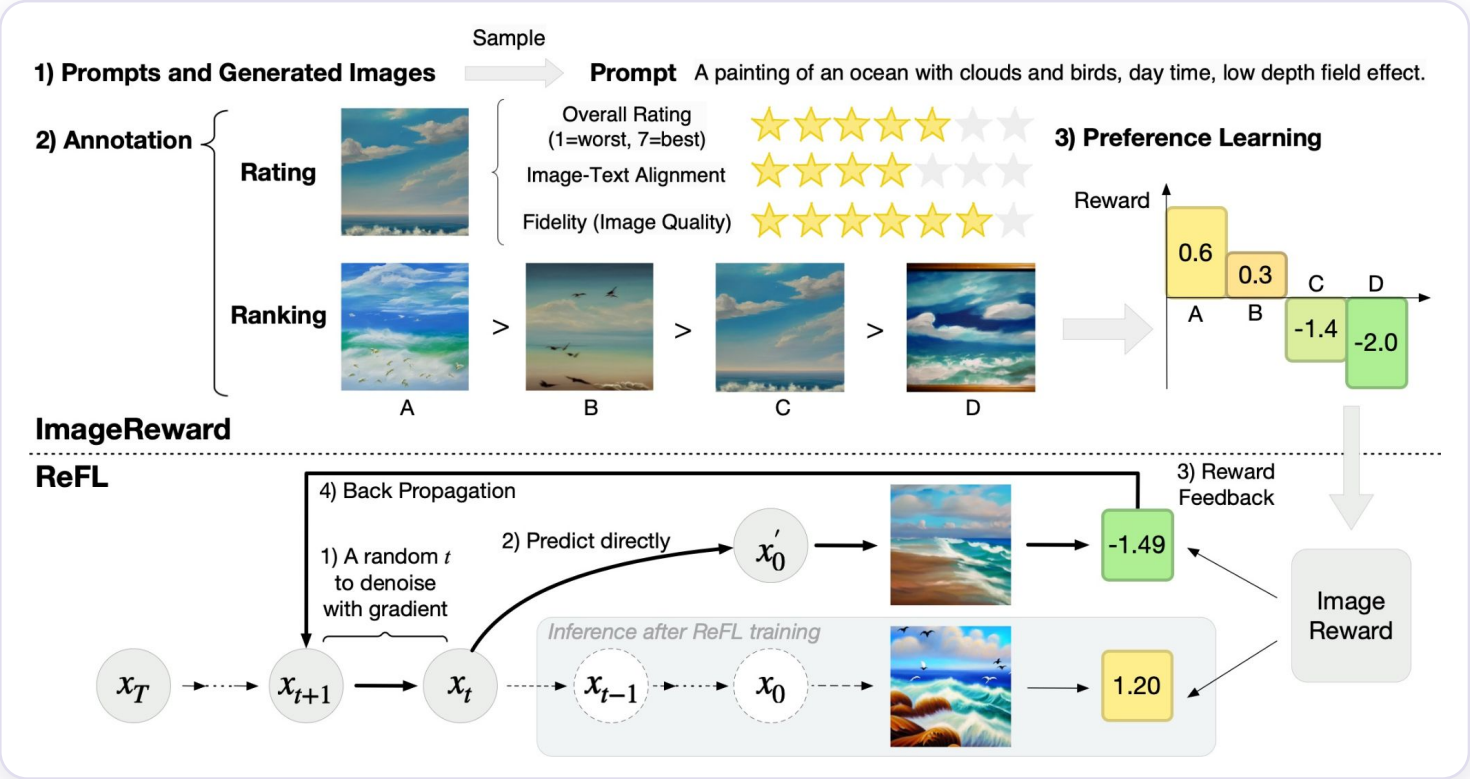
1. **Text alignment** is everything from the prompt there?
2. **Aesthetics** is it beautiful?
3. **Artifacts** hands, faces, broken text
4. **Toxicity** is it safe to show?



Can we train a model that answers like people?

paper →

ImageReward



1 **Prompts & images**
real user prompts from DiffusionDB, several images each

2 **Expert annotation**
ratings + ranking: 137K comparisons

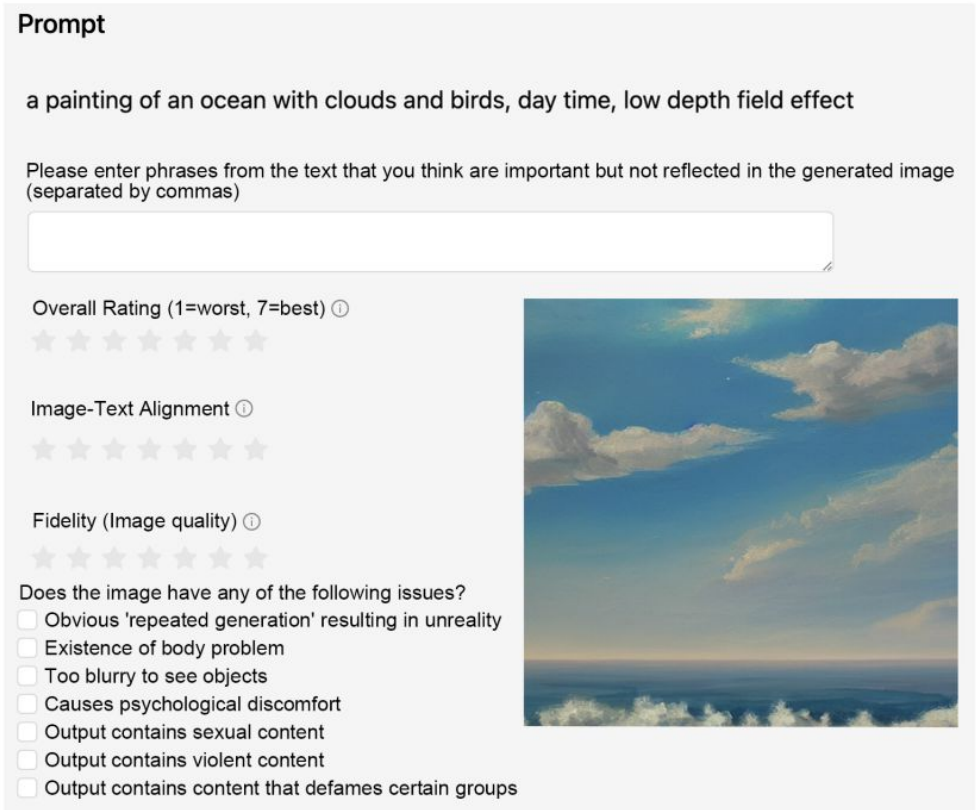
3 **Reward model**
BLIP backbone + MLP head, trained on pairs

ImageReward: collect human preferences → train a model to predict them → use it as a metric and as a reward for fine-tuning (ReFL)

paper →

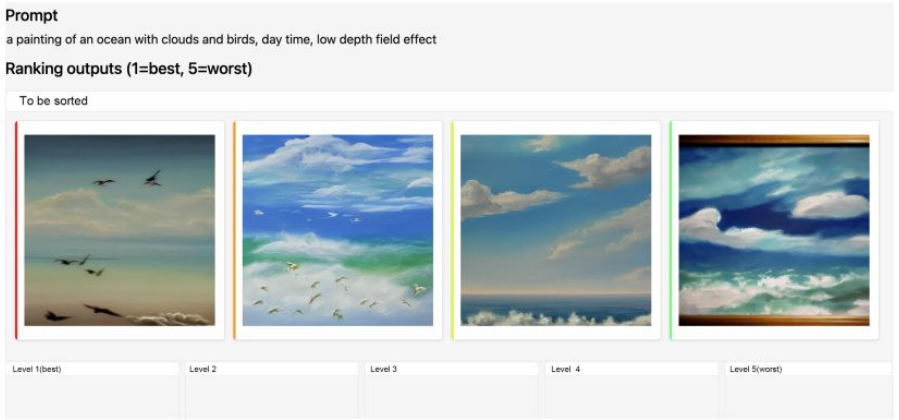
ImageReward: dataset creation

STAGE 1 · RATING



(a) **Stage 1: Text-Image Rating.** Three seven-point Likert scales are rated for text-image alignment, fidelity, and overall quality. Several options are to be checked and filled in to identify problems.

STAGE 2 · RANKING



(b) **Stage 2: Image Ranking.** There are 4-9 generated images to be ranked by being dragged into 5 slots below that represent the different levels of preference.

- 7-point scales: overall, image–text alignment, fidelity
- checklist of issues: body problems, repeats, NSFW, violence…
- then 4–9 images per prompt are ranked by drag-and-drop

paper →

ImageReward: how good is it?

(a) Agreement between different annotators, researchers, and models. Especially, "annotator ensemble" means, for each pair of images, we use the image considered better by most people as the better one.

	researcher	annotator	annotator ensemble	CLIP Score	Aesthetic	BLIP Score	Ours
researcher	71.2% ± 11.1%	65.3% ± 8.5%	73.4% ± 6.2%	57.8% ± 3.6%	55.6% ± 3.1%	57.0% ± 3.0%	**64.5% ± 2.5%**
annotator	65.3% ± 8.5%	65.3% ± 5.6%	53.9% ± 5.8%	54.3% ± 3.2%	55.9% ± 3.1%	57.4% ± 2.7%	**65.3% ± 3.7%**
annotator ensemble	73.4% ± 6.2%	53.9% ± 5.8%	-	54.4% ± 21.1%	57.5% ± 15.9%	62.0% ± 16.1%	**70.5% ± 18.6%**

70.5%

ImageReward agreement with the annotator ensemble

54–58%

CLIP, Aesthetic and BLIP scores on the same pairs

PART 2 · METRICS

paper →

PickScore

DLS

PICK-A-PIC: DATA FROM REAL USERS

1. **Write a prompt**
   in a free web app
2. **Get two images**
   from different models / settings
3. **Pick the better one**
   or declare a tie

Intrinsically motivated users instead of paid annotators

MODEL: FINE-TUNED CLIP-H

$$s(x, y) = T \cdot E_{\text{txt}}(x) \cdot E_{\text{img}}(y)$$

$$\hat{p}_i = \text{softmax}(s(x, y_1), s(x, y_2))$$

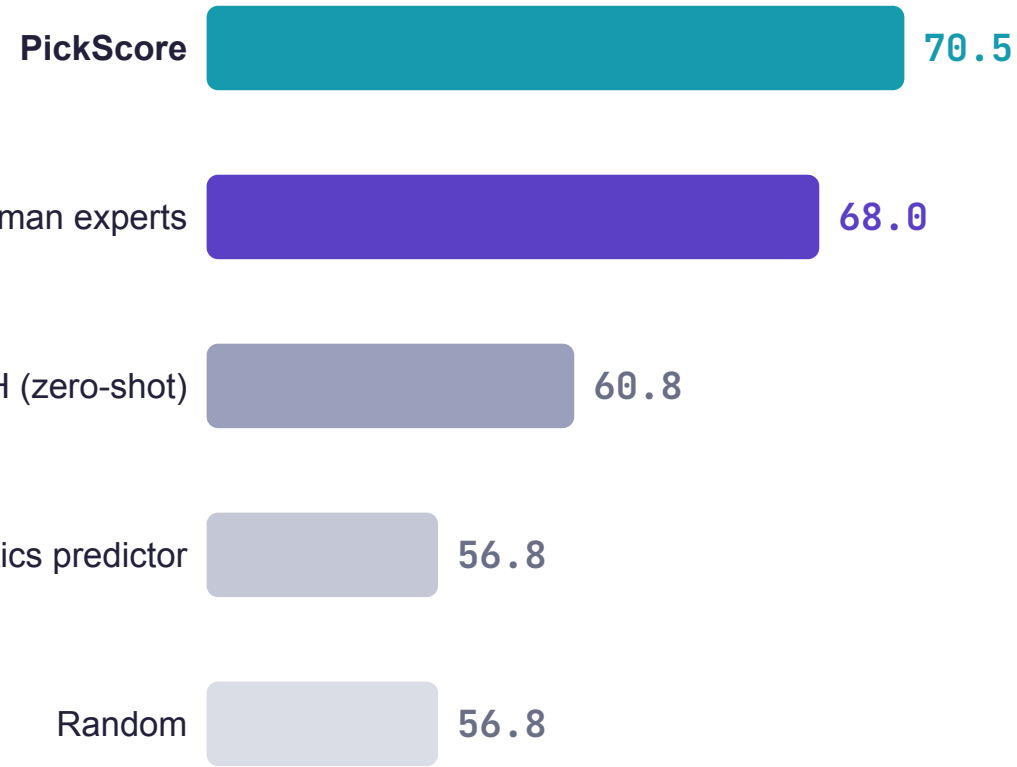
- Loss: KL between the user preference p and p̂ (as in InstructGPT reward models)
- Preferred image: p = [1, 0]; tie: p = [0.5, 0.5]
- Score = how much an image fits the prompt and pleases users

PART 2 · METRICS

paper →

PickScore: results

ACCURACY OF PREDICTING USER PREFERENCES, %



axis starts at 50%

0.917

correlation with expert ranking of models

-0.900

FID correlation with the same ranking

Takeaways

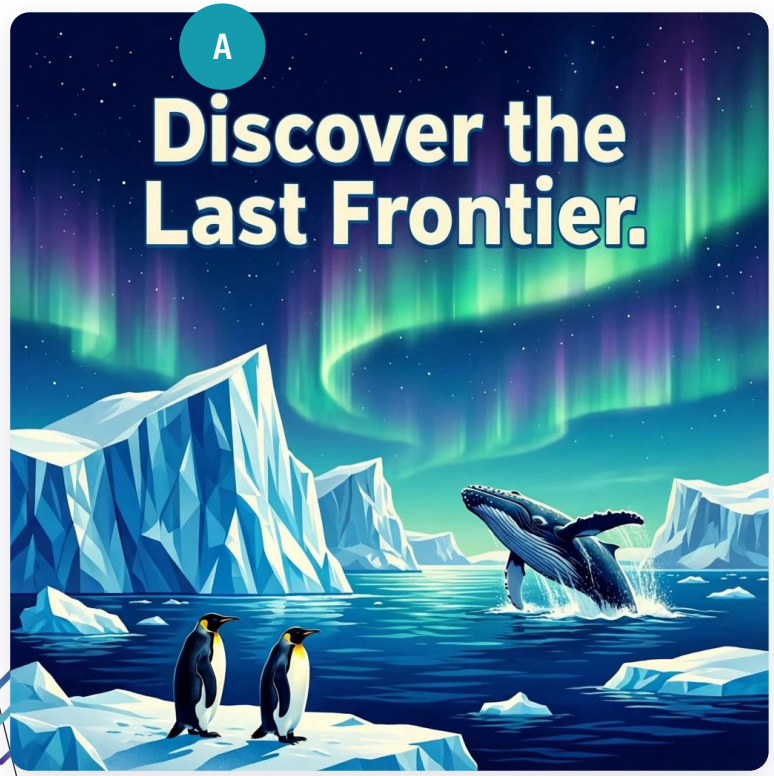
- PickScore beats expert annotators: they lack the user's own context
- Lower FID does not mean images people prefer
- Used as a metric and as a reward for fine-tuning

User study

DLS

PROMPT

Design a retro-inspired travel poster enticing visitors to experience the unique wonders of Antarctica, with a stylized illustration showcasing iconic wildlife, icebergs, and the aurora australis, accompanied by bold, sans-serif text proclaiming "Discover the Last Frontier."



WHICH ONE IS BETTER?

- Prompt alignment
- Aesthetics / quality
- Defects
- Complexity
- **Overall**

PART 2 · METRICS

paper →

GenEval: object-focused prompt following

553 prompts
6 tasks · COCO objects

→

T2I model
4 images per prompt

→

Mask2Former
objects, count, boxes

→

CLIP ViT-L/14
colour of each crop

→

correct / wrong
score = share correct

Single object 80

“a photo of a giraffe”

detector finds the object

Two objects 99

“a photo of a cat and a laptop”

both objects are found

Counting 80

“a photo of three apples”

exact count, threshold 0.9

Colours 94

“a photo of a purple umbrella”

CLIP classifies the crop colour

Position 100

“a photo of a cup left of a laptop”

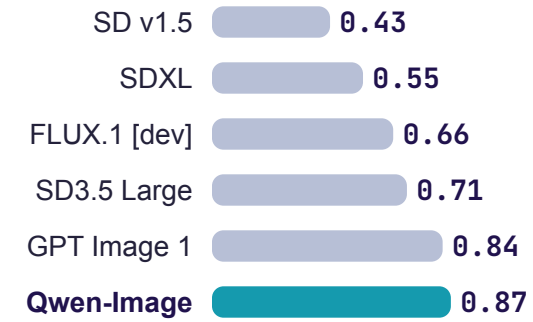
box centroids: left / right / above / below

Attribute binding 100

“a photo of a red car and a blue bird”

each object has its own colour

GENEVAL OVERALL, 0–1



GenEval and Qwen-Image papers

83% agreement with humans (CLIPScore: 80%)

Position and attribute binding are the hard tasks; top models now get ~0.9

PART 2 · METRICS

paper →

DPG-Bench: dense prompts

DLS

A DENSE PROMPT, ~67 WORDS

“A cozy reading nook: an orange cat sleeps on a knitted blue blanket on the windowsill, next to a stack of three old books and a steaming mug; warm evening light, rain on the glass…”

PROMPT → QUESTIONS WITH DEPENDENCIES

- Is there a cat? — entity
  - Is the cat orange? — attribute
  - Is the cat on the windowsill? — relation
- Are there three books? — entity
- Is the light warm? — global

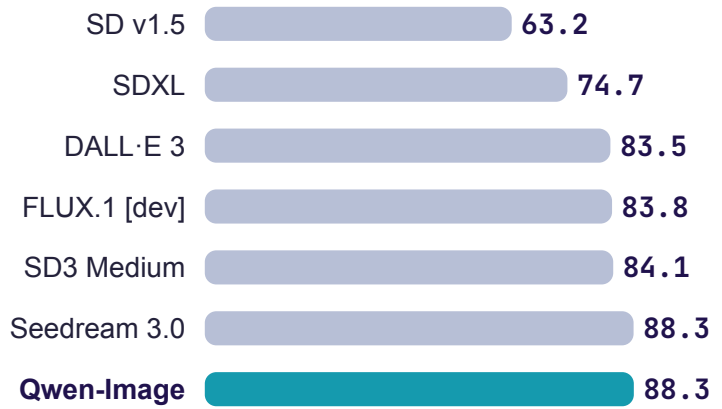
mPLUG-large answers each question; if the parent fails, its children fail too

1,065	~67	4
prompts	words per prompt	images per prompt

QUESTIONS BY CATEGORY

“Other” = global 2% + other 2%

DPG-BENCH OVERALL, %

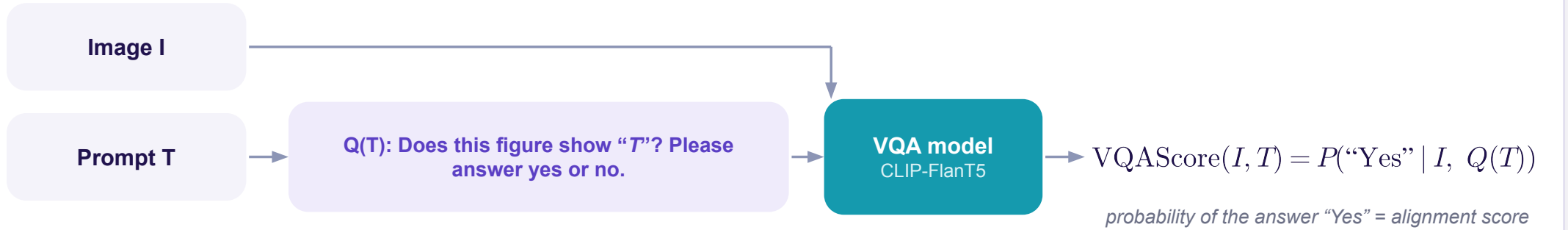


ELLA and Qwen-Image papers

PART 2 · METRICS

paper →

VQAScore: ask a VLM instead of CLIP



WHY CLIP FAILS HERE

“the moon is over the cow”

“the cow is over the moon”

schematic: CLIP sees the same bag of words, the VLM reads the relation

RESULTS

- SOTA on 8 alignment benchmarks; also works for video and 3D
- GenAI-Bench: 1,600 compositional prompts, 15K+ human ratings

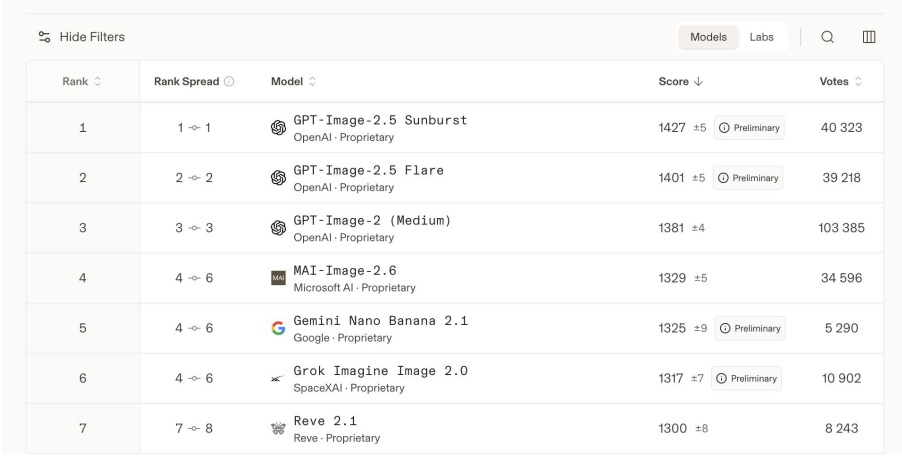
CONS

- a VLM forward pass per image: slower than CLIP
- inherits the VLM’s biases; can be hacked if used as an RL reward

Bradley-Terry

Arenas: from votes to ratings

1. **One vote = one row** +1 for A, −1 for B; label: A won (tie = ½)
2. **Logistic regression** fit all votes at once, not one by one
3. **Rescale to Elo** coefficients × 400 / ln 10, anchor one model
4. **Bootstrap votes** refit many times → ± interval, rank spread



MAXIMUM LIKELIHOOD

$$\max_{R_1,\ldots,R_n} \sum_{\text{votes}} \log P(\text{winner} \succ \text{loser})$$

$$P(i \succ j) = \frac{1}{1 + 10^{(R_j - R_i)/400}}$$

ONLY DIFFERENCES MATTER

Elo is a log-strength, not a probability:

$$P(A \succ B) = \frac{q_A}{q_A + q_B}, \quad q = 10^{R/400}$$

Shift all ratings by a constant → same predictions.

CHESS ELO VS ARENAS

Chess: online update after each game

$$R \leftarrow R + K\,(S - E)$$

depends on vote order and K.
Arenas: one joint fit over all votes.

DLS

Nina Konovalova

@AfeliaN

@ReadingAI

Thank you for your attention!

@READING_AI